

AKSHARA

Complete Code Companion

A file-by-file, workflow-by-workflow explanation of the implementation completed through commit `de9d4fc`

Scope: 99 changed files · approximately 7,259 additions · backend, frontend, security, persistence and tests

Prepared for: Ayush Thakur and the Akshara project team

Generated: 3 August 2026

How to Use This Companion

This is not a marketing report. It is a teaching document intended to let you explain the implementation in a viva, debug it without guessing, and safely extend it later. Read Chapters 1–15 first. They explain why the system is shaped this way and trace complete requests. Then use the annotated appendix when opening a particular source file in the editor.

Exact scope. The appendix covers every file added or modified between commits `48cdd34` and `de9d4fc`. That is the implementation phase that connected hybrid import, catalogue operations, fulfilment, My Books, private uploads, review management and discussions. Existing authentication/cart foundations are explained where they participate in these flows.

Three questions to ask about every line

1. **Which layer owns this decision?** UI, controller, service, repository, entity or configuration?
2. **What state can it change?** Browser state, HTTP state, database state, inventory, order history or a private file?
3. **What protects it?** Validation, JWT role, ownership query, transaction, uniqueness constraint, lock, test or error handler?

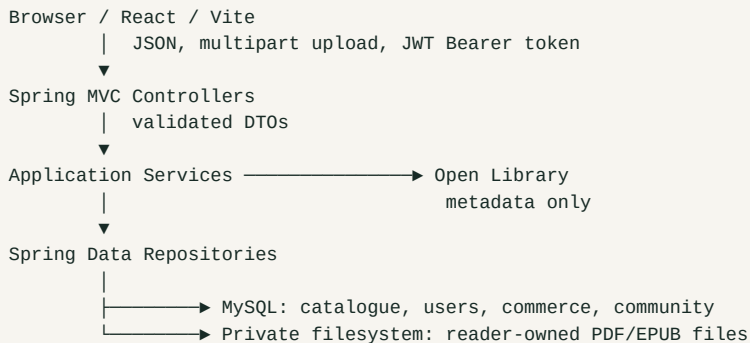
Reading Java syntax

Syntax	Meaning
<code>@Annotation</code>	Metadata interpreted by Spring, Jakarta Validation, JPA or JUnit.
<code>record</code>	Immutable request/response carrier with generated accessors.
<code>final</code>	The reference cannot be reassigned after construction.
<code>Optional<T></code>	A value may be present or absent; callers must handle both.
<code>stream().filter().map()</code>	A pipeline that selects and transforms collection elements.
<code>throw</code>	Stops normal flow and delegates error translation upward.

Reading React syntax

Syntax	Meaning
<code>useState</code>	Stores a value between renders and supplies a setter.
<code>useEffect</code>	Runs synchronization work such as an API request after rendering.
<code>useMemo</code>	Recomputes derived data only when dependencies change.
<code>{condition && <UI />}</code>	Renders UI only when the condition is truthy.
<code>await</code>	Pauses the async function until a promise settles.
<code>setState(previous => ...)</code>	Safely calculates the next state from the previous state.

1. System Architecture



The browser is deliberately not trusted with business decisions. React decides what to display and which request to send. Spring Security decides whether the caller may enter an endpoint. Controllers translate HTTP into Java inputs. Services enforce rules. Repositories load and persist state. Entities model relational data. DTOs prevent internal entity graphs from leaking into JSON.

Why the frontend does not call Open Library

If React called Open Library directly, provider JSON would leak into the UI, duplicate detection would be impossible before import, provider errors would vary, and a malicious caller could bypass Akshara's normalization. The backend provider isolates all of this. The public catalogue never depends on Open Library availability because imported books are read from MySQL.

Dependency direction

Controllers depend on services; services depend on repositories and provider interfaces; repositories depend on JPA. The provider interface is important because a later Google Books adapter can be added without changing controller contracts. React pages depend on service modules rather than duplicating Axios calls.

2. Security and Identity

Akshara uses stateless JWT authentication. Login produces a token whose subject is the user ID and whose role claim is converted to `ROLE_READER` or `ROLE_ADMIN`. Every protected request sends `Authorization: Bearer ...`. The shared Axios interceptor attaches it automatically.

Endpoint family	Authority	Reason
<code>/api/public/**</code> , <code>/api/auth/**</code>	Anyone	Discovery and authentication must be reachable before login.
<code>/api/admin/**</code>	Admin	Price, stock, import, deletion and fulfilment affect the entire store.
Cart, wishlist, checkout, orders, library, review writes	Reader	These resources belong to a signed-in reader.

Frontend route guards improve navigation but are not security boundaries. A user can manually send HTTP requests, so Spring Security and ownership-aware repository queries must enforce the rule again. Personal books use `findByIdAndUser_Id`; orders are queried with both order ID and user ID; review/reply mutation checks the author.

Why IDs come from the token

A request must not say “perform this operation for user 7” and expect trust. Services parse the authenticated JWT subject, load that enabled user and derive ownership internally. This prevents horizontal privilege escalation.

3. Hybrid Catalogue Import

Search sequence

```
Admin types title/author/ISBN
→ BookImportPage.handleSearch
→ adminBookService.searchExternalBooks
→ GET /api/admin/book-import/search
→ AdminBookImportController.search
→ ExternalBookSearchService.search
→ OpenLibraryBookMetadataProvider.search
→ GET https://openlibrary.org/search.json
→ normalize each work/edition
→ query MySQL for ISBN duplicates
→ return normalized results to React
```

The provider requests only selected fields to reduce response size. Offset is calculated as $(\text{page} - 1) \times \text{size}$. A custom User-Agent includes the configured contact because external services need identifiable responsible clients. Connect/read timeouts prevent a slow provider from holding a server thread indefinitely.

Normalization

Open Library may place ISBN, publisher, page count or publication date on either a work or an edition. Mapping prefers edition data and falls back to work data. ISBN characters are cleaned; the first 10- and 13-character candidates are selected. Subjects are bounded so a single noisy provider record cannot create hundreds of local categories. Cover IDs become Cover API URLs.

Permanent import

```
Selected external metadata
+
Admin-owned format, SKU, price, stock, availability
↓
POST /api/admin/book-import
↓
@Transactional BookImportService
├─ find/create Authors
├─ find/create Categories
├─ find/create Publisher
├─ validate ISBN and SKU uniqueness
├─ create Book
├─ create BookEdition
└─ create Inventory
↓
Public catalogue reads the stored book from MySQL
```

The external API never decides price or stock. The transaction is essential: a book without its edition/inventory is unusable, so partial persistence must roll back. Manual entry calls the same save path but does not require an external identity and may relax ISBN requirements for rare books.

Seed operation

The seed endpoint is explicit and Admin-only. It searches eight broad genres, balances results across them, skips entries without ISBNs, derives predictable demonstration prices/stock/SKUs, and marks an initial subset featured. It does not

run automatically at startup; therefore normal boots never scrape or unexpectedly mutate the catalogue.

4. Catalogue and Inventory Administration

A Book represents the conceptual title. A BookEdition represents a specific format/publication/ISBN. Inventory belongs to an edition because different editions can have different price, stock and availability. This separation is the reason an “Atomic Habits” paperback and hardcover can be managed independently.

Exact update versus restock

- **Inventory update** replaces price, total quantity, availability and active status with the submitted values.
- **Restock** adds units to the existing quantity while holding a database lock. It never accidentally replaces 10 units with 5 when the intention was “add five.”

Availability normalization prevents contradictory state. Zero stock cannot remain `IN_STOCK`; adding stock to `OUT_OF_STOCK` restores `IN_STOCK`; deliberately `UNAVAILABLE` inventory stays unavailable until an admin changes it.

Safe deletion

Catalogue deletion must consider carts, wishlists, reviews and historical orders. Temporary references may be removed. Historical order items instead preserve immutable snapshots such as title, ISBN, format and price. Their optional live edition link is detached so deleting a catalogue entry cannot erase or corrupt an old receipt.

5. Cart, Checkout and Orders

The cart stores an edition and quantity, not merely a book ID. Adding checks that the edition is active, sellable and sufficiently stocked. Checkout locks/validates inventory, creates an address snapshot, order, order items and payment, reduces stock and clears the cart in one transaction.

Why order snapshots exist

Catalogue values can change after purchase. An order must continue showing what the customer actually bought at the original price. OrderItem therefore stores title, cover, format, ISBN, publisher and unit-price snapshots rather than rendering an old order from mutable catalogue data.

Fulfilment state machine

PLACED → CONFIRMED → PROCESSING → SHIPPED → DELIVERED
└────────── cancellation allowed before shipping ─────────┘

The backend owns allowed transitions. React only presents valid next actions. Cancellation restores reserved stock. Delivering a cash-on-delivery order marks payment paid. Skipping directly from PLACED to SHIPPED or reversing a delivered order produces a 400 response.

6. My Books and Private Uploads

My Books combines two different ownership concepts without confusing them. Purchased physical editions come from non-cancelled order items and show fulfilment status. Personal uploads are reader-owned files stored privately and served only after an ownership check.

Upload safety

- Maximum size is enforced by Spring multipart configuration and the service.
- The extension alone is not trusted: PDF must begin with `%PDF-`; EPUB must be a ZIP whose first entry is the correct mimetype.
- The original filename is sanitized for display only.
- The stored filename is a generated UUID.
- The resolved path must remain inside the configured storage root.
- Database rows are queried by both file ID and owner ID.
- Failed persistence cleans up the copied file.

A physical purchase deliberately does not unlock a copyrighted digital file. PDF uploads open using an authenticated Blob response. EPUB currently downloads because a complete EPUB reader is outside this implementation.

7. Reviews, My Reviews and Discussions

A review is a rated reflection attached to one user and one book. My Reviews makes ownership visible by providing edit/delete controls. Community displays recent reviews and can expand a reply thread under each one.

Replies are separate entities because a review remains the stable root of conversation. Each reply records its review, author, content and creation time. Creating and deleting replies requires a Reader JWT; deletion verifies that the current user wrote the reply. Public users can read conversations without acquiring mutation rights.

Frontend behavior

Community loads a bounded recent feed, derives counts/average locally, and supports client-side search/sort. DiscussionThread loads replies on demand, posts a new reply, updates local state and allows the author to delete their own reply. Empty/loading/error states prevent the earlier “section that does nothing” experience.

8. React Application Structure

`App.jsx` is the route table. `Navbar` and `Footer` remain outside the route switch. `RouteErrorBoundary` wraps routed content. `ProtectedRoute` requires authentication; `AdminRoute` requires the `Admin` role.

Route	Responsibility
<code>/, /books/:id</code>	Public discovery and book evaluation.
<code>/wishlist, /cart, /checkout</code>	Reader commerce preparation.
<code>/orders, /orders/:id</code>	Reader receipts and physical delivery journey.
<code>/my-books, /my-reviews</code>	Personal library and owned contributions.
<code>/community</code>	Public reader reviews and conversations.
<code>/admin/books/import</code>	External metadata import and manual entry.
<code>/admin/books</code>	Catalogue/inventory operations.
<code>/admin/orders</code>	Order confirmation and fulfilment.

State placement

Local page state holds form fields, loading/error flags and selected records. `AuthProvider` owns the session. `ReaderDataProvider` owns cart/wishlist data that is shared across navigation. Service modules own HTTP syntax, which keeps pages focused on user interaction.

9. API Client Design

The Axios instance uses relative `/api`

Admin and reader service modules are intentionally small: each function describes one backend operation. Normalization in `adminBookService` tolerates different external-result shapes at one boundary rather than scattering nullish checks across the import page.

10. Error Handling and Validation

Validation occurs at several levels because each level knows different facts:

1. HTML attributes give immediate usability feedback.
2. React prevents empty actions and displays backend messages.
3. DTO annotations validate structure and limits.
4. Services validate database-dependent rules such as duplicate ISBN or legal state transitions.
5. Database constraints provide the final concurrency-safe boundary.

GlobalExceptionHandler produces stable statuses: 400 malformed/invalid input, 401 invalid authentication, 403 denied access, 404 missing resource, 409 duplicate/conflict, 413 upload too large, 500 storage/internal failure and 503 external catalogue unavailability.

11. Persistence Model

```
AppUser — Cart — CartItem — BookEdition — Inventory
|
|— WishlistItem — Book
|— Review — ReviewReply
|— PersonalBook
|— Order — OrderItem snapshots — optional live BookEdition
|
|— Publisher
|— Book — Authors / Categories
```

Relationships express ownership. Join entities model many-to-many book-author and book-category links explicitly. Inventory is one-to-one with an edition. Order snapshots deliberately duplicate selected values because historical truth is more important than normalization for receipts.

12. Automated Tests

The verified backend suite contains 74 passing tests. Controller tests use MockMvc to prove HTTP status, JSON shape, validation and role rules. Service unit tests mock repositories/providers to isolate business branches. BookImportServiceIntegrationTest uses Spring/JPA with H2 to prove transactional persistence and uniqueness behavior. Provider tests use a local mock HTTP server rather than relying on Open Library during the test run.

The essential testing pattern is Arrange–Act–Assert: create controlled state; call one behavior; inspect result and side effects. Mockito `verify` proves that invalid requests do not reach dangerous collaborators. Integration tests complement mocks by checking real mappings and constraints.

13. Configuration and Operational Boundaries

Secrets come from environment variables. Database passwords and JWT secrets are intentionally absent from source control. Open Library configuration exposes contact identity and timeouts. Private file storage has a configurable root and multipart limits. The ignored storage directory prevents personal files entering Git.

`ddl-auto=update`, local filesystem storage, cash-on-delivery simulation and the lack of a Google Books adapter are deliberate current limitations, not hidden completed features.

14. How to Explain the Project in a Viva

1. Start with the problem: discovery, commerce, ownership and conversation are fragmented.
2. Explain the roles and one complete Reader/Admin journey.
3. Draw the layered architecture and say why business rules live in services.
4. Demonstrate metadata versus inventory using Open Library and Akshara-owned fields.
5. Show one transaction: import or checkout.
6. Show one ownership rule: personal file, order or reply.
7. Show order snapshots and state transitions.
8. Show tests and state the honest limitations.

Core distinction: Open Library describes books. Akshara decides what it sells, at what price, in which format, with how much stock, and how that purchase continues into the reader's private shelf and community participation.

15. Current Gaps and Safe Extension Points

- Add Flyway migrations before treating the schema as production-managed.
- Add React component and end-to-end workflow tests.
- Add reading status, progress, notes and goals to complete “Discover · Read · Grow.”
- Add a Google Books provider through BookMetadataProvider rather than modifying controllers.
- Add community pagination, notifications and moderation.
- Move private files to managed object storage for deployment.
- Add real payment webhooks only behind a provider interface and idempotent transaction handling.

These are extension points, not reasons to rewrite working modules. Preserve endpoint contracts and move new rules into focused services.

Annotated Source Appendix

The following sections contain the exact current content of every file added or modified in the implementation commit. Code is split into readable line ranges. Explanations identify syntax, responsibility, state changes and safeguards. Repeated imports/closing braces are still shown so you can match this companion directly to the editor.

.gitignore

File 1 of 99 · Added or modified · Project support

Purpose: Supports project support in the current Akshara implementation.

Current file length: 75 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–3

```
1 # Compiled class file
2 *.class
3
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 4–6

```
4 # Log file
5 *.log
6
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 7–9

```
7 # BlueJ files
8 *.ctxt
9
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 10–12

```
10 # Mobile Tools for Java (J2ME)
11 .mtj.tmp/
12
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 13–21

```
13 # Package Files #
14 *.jar
15 *.war
16 *.nar
17 *.ear
```



```
18 *.zip
19 *.tar.gz
20 *.rar
21
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 22–25

```
22 # virtual machine crash logs, see http://www.java.com/en/download/help/error\_hotspot.xml
23 hs_err_pid*
24 replay_pid*
25
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 26–32

```
26
27 # Maven / Spring Boot
28 target/
29 *.class
30 *.log
31 logs/
32
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 33–41

```
33 # React / Node.js
34 node_modules/
35 dist/
36 build/
37 .vite/
38 npm-debug.log*
39 yarn-debug.log*
40 yarn-error.log*
41
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 42–48

```
42 # IntelliJ IDEA
43 .idea/
44 *.iml
45 *.iws
46 *.ipr
47 out/
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 49–59

```
49 # Eclipse / Spring Tool Suite
50 .classpath
51 .project
52 .settings/
53 bin/
54 .factorypath
55 .apt_generated/
56 .sts4-cache/
57 .springBeans
58 .recommenders/
59
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 60–62

```
60 # VS Code
61 .vscode/
62
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 63–70

```
63 # Local configuration and secrets
64 .env
65 .env.*
66 !.env.example
67 application-local.properties
68 application-local.yml
69 application-local.yaml
70
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 71–75

```
71 # Operating-system files
72 .DS_Store
73 Thumbs.db
74 backend/data/personal-library/
75
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

README.md

File 2 of 99 · Added or modified · Project support

Purpose: Documents setup, security, catalogue import, administration, order transitions, API contracts and scope boundaries.

Current file length: 139 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–3

```
1 # akshara-bookstore
2 Akshara - a full-stack online bookstore built with React, Spring Boot, JWT and MySQL.
3
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 4–13

```
4 ## Current features
5
6 - Public editorial catalogue, search, book details, ratings and reviews
7 - JWT Reader/Admin authentication
8 - Wishlist, cart, shipping checkout, orders and stock management
9 - Admin-only Open Library metadata search and transactional catalogue import
10 - Admin-only manual entry for rare or missing books
11 - Admin catalogue dashboard for restocking, pricing, availability and curation
12 - ISBN and SKU duplicate protection
13
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 14–16

```
14 External services supply descriptive metadata only. Akshara owns price, stock,
15 availability, SKU, reviews, wishlist, cart, users and orders.
16
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 17–21

```
17 ## Local configuration
18
19 Run MySQL 8.4 with database `akshara`, then provide secrets through the process
20 environment:
21
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because

formatting has split one logical construct across line ranges.

Lines 22–29

```
22  ```bash
23  export DB_URL='jdbc:mysql://localhost:3306/akshara'
24  export DB_USERNAME='akshara_app'
25  export DB_PASSWORD='<local-password>'
26  export JWT_SECRET='<long-random-local-secret>'
27  export BOOK_IMPORT_CONTACT='your-contact@example.com'
28  ```
29
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 30–32

```
30  `BOOK_IMPORT_CONTACT` is only Akshara's Open Library contact identifier. It is not
31  an administrator login. No Google Books key is required for the current flow.
32
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 33–39

```
33  ## Run and verify
34
35  ```bash
36  cd backend
37  ./mvnw spring-boot:run
38  ```
39
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 40–45

```
40  ```bash
41  cd frontend
42  npm ci
43  npm run dev
44  ```
45
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 46–52

```
46  Verification:
47
```

```
48 ```bash
49 cd backend
50 ./mvnw test
51 ```
52
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 53–58

```
53 ```bash
54 cd frontend
55 npm run lint
56 npm run build
57 ```
58
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 59–62

```
59 Health check: `GET http://localhost:8080/api/public/health`.
60
61 ## Administrator account
62
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 63–65

```
63 Public registration always creates a `READER`. For local development, register a
64 normal account and deliberately promote it in MySQL:
65
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 66–69

```
66 ```sql
67 UPDATE users SET role = 'ADMIN' WHERE email = 'your-email@example.com';
68 ```
69
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 70–72

```
70 Log out and back in afterward so the new JWT contains the Admin role. Never commit
71 a default administrator password.
72
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 73–81

```
73 ## Hybrid catalogue workflow
74
75 1. Log in as Admin and open `/admin/books/import`.
76 2. Search Open Library by title, author or ISBN.
77 3. Select the correct edition and review the normalized metadata.
78 4. Enter Akshara format, SKU, price, stock, availability and active state.
79 5. Import and follow the success link to `/books/{bookId}`.
80 6. Use Manual Entry when an external record is unsuitable or missing.
81
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 82–85

```
82 Akshara never contacts Open Library during public catalogue requests or startup.
83
84 ## Catalogue operations
85
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 86–91

```
86 Open `/admin/books` as an administrator to manage existing catalogue records. The
87 dashboard supports searching by title, author, ISBN or SKU; reviewing stock
88 totals; adding stock; changing the exact stock quantity and price; changing
89 availability; activating or pausing an edition; and curating the featured shelf.
90 Unreferenced books can also be permanently deleted after confirmation.
91
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 92–97

```
92 Quick restocking adds units under a database inventory lock and does not replace
93 the current quantity. Adding stock to an `OUT_OF_STOCK` edition automatically
94 returns it to `IN_STOCK`; an explicitly `UNAVAILABLE` edition stays unavailable.
95 Setting total stock to zero while requesting `IN_STOCK` is normalized to
96 `OUT_OF_STOCK`.
97
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 98–105

```
98  ## Order operations
99
100 1. Sign in with an `ADMIN` account and open `/admin/orders`.
101 2. Select a newly placed order and choose Confirm order.
102 3. Advance fulfilment in order: `CONFIRMED` → `PROCESSING` → `SHIPPED` → `DELIVERED`.
103 4. Orders may be cancelled before shipping; cancellation restores reserved inventory.
104 5. Marking a cash-on-delivery order delivered marks its payment as paid.
105
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 106–108

```
106 Readers see these changes in Order History and My Books. Status transitions are
107 validated by Spring Boot and cannot be skipped or reversed.
108
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 109–122

```
109  ## Admin API
110
111 | Method | Endpoint | Purpose |
112 | --- | --- | --- |
113 | `GET` | `/api/admin/book-import/search?query=...&source=OPEN_LIBRARY&page=1&size=10` | Search normalized metadata |
114 | `POST` | `/api/admin/book-import` | Import reviewed metadata and store data |
115 | `POST` | `/api/admin/books` | Create a manual catalogue book |
116 | `GET` | `/api/admin/books` | List catalogue records |
117 | `PUT` | `/api/admin/books/{id}` | Update a catalogue record |
118 | `PATCH` | `/api/admin/books/{id}/featured` | Curate featured status |
119 | `PUT` | `/api/admin/editions/{editionId}/inventory` | Update price, total stock, availability and active state |
120 | `POST` | `/api/admin/editions/{editionId}/inventory/restock` | Add units safely to current stock |
121 | `DELETE` | `/api/admin/books/{id}` | Delete a catalogue record |
122 | `GET` | `/api/admin/orders` | List customer orders for fulfilment |
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 123–125

```
123 | `GET` | `/api/admin/orders/{id}` | View customer, delivery, payment and book details |
124 | `PATCH` | `/api/admin/orders/{id}/status` | Advance or cancel an order using validated transitions |
125
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 126–129

```
126 Creation returns `201`. Invalid input returns `400`, access failures return
127 `401`/`403`, duplicate ISBN/SKU returns `409`, and provider unavailability returns
128 `503`. Zero matches return `200` with an empty `results` list.
129
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 130–134

```
130 Deletion removes the book, its editions, inventory, author/category links, cart
131 items, wishlist entries and reviews. Historical order items keep their immutable
132 title, ISBN, format and price snapshots; their optional live-edition link is
133 detached before catalogue deletion so order history remains readable.
134
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 135–139

```
135 ## Deferred scope
136
137 Seller accounts, external price comparison, AI recommendations, real payment
138 gateways, subscriptions and copyrighted eBook distribution are outside the MVP.
139
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/pom.xml

File 3 of 99 · Added or modified · Backend dependencies and build

Purpose: Supports backend dependencies and build in the current Akshara implementation.

Current file length: 110 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–14

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <parent>
6         <groupId>org.springframework.boot</groupId>
7         <artifactId>spring-boot-starter-parent</artifactId>
8         <version>4.1.0</version>
9         <relativePath/> <!-- lookup parent from repository -->
10    </parent>
11    <groupId>com.akshara</groupId>
12    <artifactId>akshara-api</artifactId>
13    <version>0.0.1-SNAPSHOT</version>
14    <name/>
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 15–28

```
15    <description/>
16    <url/>
17    <licenses>
18        <license/>
19    </licenses>
20    <developers>
21        <developer/>
22    </developers>
23    <scm>
24        <connection/>
25        <developerConnection/>
26        <tag/>
27        <url/>
28    </scm>
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 29–42

```
29    <properties>
30        <java.version>17</java.version>
31    </properties>
32    <dependencies>
33        <dependency>
34            <groupId>org.springframework.boot</groupId>
35            <artifactId>spring-boot-starter-security</artifactId>
36        </dependency>
37        <dependency>
38            <groupId>org.springframework.boot</groupId>
39            <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
```

```
40         </dependency>
41         <dependency>
42             <groupId>org.springframework.boot</groupId>
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 43–49

```
43             <artifactId>spring-boot-starter-validation</artifactId>
44         </dependency>
45         <dependency>
46             <groupId>org.springframework.boot</groupId>
47             <artifactId>spring-boot-starter-webmvc</artifactId>
48         </dependency>
49
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 50–63

```
50         <dependency>
51             <groupId>org.springframework.boot</groupId>
52             <artifactId>spring-boot-devtools</artifactId>
53             <scope>runtime</scope>
54             <optional>true</optional>
55         </dependency>
56         <dependency>
57             <groupId>org.springframework.boot</groupId>
58             <artifactId>spring-boot-starter-security-test</artifactId>
59             <scope>test</scope>
60         </dependency>
61         <dependency>
62             <groupId>com.h2database</groupId>
63             <artifactId>h2</artifactId>
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 64–77

```
64             <scope>test</scope>
65         </dependency>
66         <dependency>
67             <groupId>org.springframework.boot</groupId>
68             <artifactId>spring-boot-starter-validation-test</artifactId>
69             <scope>test</scope>
70         </dependency>
71         <dependency>
72             <groupId>org.springframework.boot</groupId>
73             <artifactId>spring-boot-starter-webmvc-test</artifactId>
74             <scope>test</scope>
75         </dependency>
76         <dependency>
77             <groupId>net.bytebuddy</groupId>
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 78–85

```
78         <artifactId>byte-buddy-agent</artifactId>
79         <scope>test</scope>
80     </dependency>
81     <dependency>
82         <groupId>org.springframework.boot</groupId>
83         <artifactId>spring-boot-starter-data-jpa</artifactId>
84     </dependency>
85
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 86–92

```
86     <dependency>
87         <groupId>com.mysql</groupId>
88         <artifactId>mysql-connector-j</artifactId>
89         <scope>runtime</scope>
90     </dependency>
91 </dependencies>
92
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 93–106

```
93 <build>
94     <plugins>
95         <plugin>
96             <groupId>org.apache.maven.plugins</groupId>
97             <artifactId>maven-surefire-plugin</artifactId>
98             <configuration>
99                 <argLine>-javaagent:${settings.localRepository}/net/bytebuddy/byte-buddy-agent/${
100 {byte-buddy.version}/byte-buddy-agent-${byte-buddy.version}.jar</argLine>
101             </configuration>
102         </plugin>
103         <plugin>
104             <groupId>org.springframework.boot</groupId>
105             <artifactId>spring-boot-maven-plugin</artifactId>
106         </plugin>
107     </plugins>
108
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 107–110

```
107 </build>
108
109 </project>
110
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/book/controller/

AdminBookController.java

File 4 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines HTTP routes for the catalogue and inventory module and delegates business decisions to services.

Current file length: 86 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.book.controller;
2
3 import com.akshara.api.book.dto.BookRequest;
4 import com.akshara.api.book.dto.BookResponse;
5 import com.akshara.api.book.dto.FeaturedRequest;
6 import com.akshara.api.book.service.BookService;
7 import com.akshara.api.bookimport.dto.ManualBookRequest;
8 import com.akshara.api.bookimport.service.BookImportService;
9 import jakarta.validation.Valid;
10 import org.springframework.http.HttpStatus;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.web.bind.annotation.DeleteMapping;
13 import org.springframework.web.bind.annotation.GetMapping;
14 import org.springframework.web.bind.annotation.PathVariable;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–21

```
15 import org.springframework.web.bind.annotation.PatchMapping;
16 import org.springframework.web.bind.annotation.PostMapping;
17 import org.springframework.web.bind.annotation.PutMapping;
18 import org.springframework.web.bind.annotation.RequestBody;
19 import org.springframework.web.bind.annotation.RequestMapping;
20 import org.springframework.web.bind.annotation.RestController;
21
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 22–27

```
22 import java.util.List;
23
24 @RestController
25 @RequestMapping("/api/admin/books")
26 public class AdminBookController {
27
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create

objects or execute code.

- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

Lines 28–30

```
28     private final BookService bookService;
29     private final BookImportService bookImportService;
30
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 31–38

```
31     public AdminBookController(
32         BookService bookService,
33         BookImportService bookImportService
34     ) {
35         this.bookService = bookService;
36         this.bookImportService = bookImportService;
37     }
38
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 39–47

```
39     @PostMapping
40     public ResponseEntity<BookResponse> create(
41         @Valid @RequestBody ManualBookRequest request
42     ) {
43         return ResponseEntity
44             .status(HttpStatus.CREATED)
45             .body(bookImportService.createManualBook(request));
46     }
47
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props

carry data and event handlers into elements or child components.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 48–52

```
48     @GetMapping
49     public ResponseEntity<List<BookResponse>> getAll() {
50         return ResponseEntity.ok(bookService.getAll());
51     }
52
```

How to read this block

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 53–59

```
53     @GetMapping("/{id}")
54     public ResponseEntity<BookResponse> getById(
55         @PathVariable Long id
56     ) {
57         return ResponseEntity.ok(bookService.getById(id));
58     }
59
```

How to read this block

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 60–69

```
60     @PutMapping("/{id}")
61     public ResponseEntity<BookResponse> update(
62         @PathVariable Long id,
63         @Valid @RequestBody BookRequest request
64     ) {
65         return ResponseEntity.ok(
66             bookService.update(id, request)
67         );
68     }
69
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 70–79

```

70     @PatchMapping("/{id}/featured")
71     public ResponseEntity<BookResponse> updateFeatured(
72         @PathVariable Long id,
73         @Valid @RequestBody FeaturedRequest request
74     ) {
75         return ResponseEntity.ok(
76             bookService.updateFeatured(id, request.featured())
77         );
78     }
79 
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 80–86

```

80     @DeleteMapping("/{id}")
81     public ResponseEntity<Void> delete(@PathVariable Long id) {
82         bookService.delete(id);
83         return ResponseEntity.noContent().build();
84     }
85 }
86 
```

How to read this block

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/book/controller/ AdminEditionController.java

File 5 of 99 · Added or modified · Catalogue and inventory

Purpose: Provides admin-only inventory replacement and additive restocking endpoints for an edition.

Current file length: 46 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.book.controller;
2
3 import com.akshara.api.book.dto.InventoryRequest;
4 import com.akshara.api.book.dto.InventoryResponse;
5 import com.akshara.api.book.dto.RestockRequest;
6 import com.akshara.api.book.service.BookService;
7 import jakarta.validation.Valid;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.web.bind.annotation.PathVariable;
10 import org.springframework.web.bind.annotation.PostMapping;
11 import org.springframework.web.bind.annotation.PutMapping;
12 import org.springframework.web.bind.annotation.RequestBody;
13 import org.springframework.web.bind.annotation.RequestMapping;
14 import org.springframework.web.bind.annotation.RestController;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–19

```
15
16 @RestController
17 @RequestMapping("/api/admin/editions")
18 public class AdminEditionController {
19
```

How to read this block

- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

Lines 20–25

```
20     private final BookService bookService;
21
22     public AdminEditionController(BookService bookService) {
23         this.bookService = bookService;
24     }
25
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 26–35

```
26     @PutMapping("/{editionId}/inventory")
27     public ResponseEntity<InventoryResponse> updateInventory(
28         @PathVariable Long editionId,
29         @Valid @RequestBody InventoryRequest request
30     ) {
31         return ResponseEntity.ok(
32             bookService.updateInventory(editionId, request)
33         );
34     }
35
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 36–46

```
36     @PostMapping("/{editionId}/inventory/restock")
37     public ResponseEntity<InventoryResponse> restock(
38         @PathVariable Long editionId,
39         @Valid @RequestBody RestockRequest request
40     ) {
41         return ResponseEntity.ok(
42             bookService.restockInventory(editionId, request.quantity())
43         );
44     }
45 }
46
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final`` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/main/java/com/akshara/api/book/dto/

BookEditionRequest.java

File 6 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines a validated API boundary type used by the catalogue and inventory module.

Current file length: 50 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–10

```
1 package com.akshara.api.book.dto;
2
3 import com.akshara.api.book.entity.BookFormat;
4 import jakarta.validation.Valid;
5 import jakarta.validation.constraints.NotNull;
6 import jakarta.validation.constraints.NotBlank;
7 import jakarta.validation.constraints.PastOrPresent;
8 import jakarta.validation.constraints.Positive;
9 import jakarta.validation.constraints.Size;
10
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 11–14

```
11 import java.time.LocalDate;
12
13 public record BookEditionRequest(
14
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 15–17

```
15         @Positive(message = "Publisher ID must be positive")
16         Long publisherId,
17
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 18–20

```
18         @NotNull(message = "Book format is required")
19         BookFormat format,
20
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 21–26

```
21         @Size(
22             max = 100,
23             message = "Edition name must not exceed 100 characters"
24         )
25         String editionName,
26
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 27–29

```
27         @Size(max = 20, message = "ISBN-10 input must not exceed 20 characters")
28         String isbn10,
29
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 30–32

```
30         @Size(max = 25, message = "ISBN-13 input must not exceed 25 characters")
31         String isbn13,
32
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 33–37

```
33         @PastOrPresent(
34             message = "Publication date cannot be in the future"
35         )
36         LocalDate publicationDate,
37
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 38–40

```
38         @Positive(message = "Page count must be greater than zero")
39         Integer pageCount,
40
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 41–44

```
41         @NotBlank(message = "SKU is required")
42         @Size(max = 100, message = "SKU must not exceed 100 characters")
43         String sku,
44
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 45–50

```
45         @NotNull(message = "Inventory information is required")
46         @Valid
47         InventoryRequest inventory
48     ) {
49     }
50
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

backend/src/main/java/com/akshara/api/book/dto/

BookEditionResponse.java

File 7 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines a validated API boundary type used by the catalogue and inventory module.

Current file length: 24 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.book.dto;
2
3 import com.akshara.api.book.entity.BookFormat;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–7

```
5 import java.time.Instant;
6 import java.time.LocalDate;
7
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 8–21

```
8 public record BookEditionResponse(
9     Long id,
10     PublisherResponse publisher,
11     BookFormat format,
12     String editionName,
13     String isbn10,
14     String isbn13,
15     LocalDate publicationDate,
16     Integer pageCount,
17     String sku,
18     String externalEditionId,
19     InventoryResponse inventory,
20     Instant createdAt,
21     Instant updatedAt
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic

metadata.

Lines 22–24

```
22  ) {  
23  }  
24
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/book/dto/

BookRequest.java

File 8 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines a validated API boundary type used by the catalogue and inventory module.

Current file length: 77 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–10

```
1 package com.akshara.api.book.dto;
2
3 import jakarta.validation.Valid;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.NotEmpty;
6 import jakarta.validation.constraints.NotNull;
7 import jakarta.validation.constraints.Pattern;
8 import jakarta.validation.constraints.Positive;
9 import jakarta.validation.constraints.Size;
10
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 11–13

```
11 import java.util.List;
12 import java.util.Set;
13
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 14–22

```
14 public record BookRequest(
15
16     @NotBlank(message = "Book title is required")
17     @Size(
18         max = 255,
19         message = "Book title must not exceed 255 characters"
20     )
21     String title,
22
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 23–28

```
23         @Size(  
24             max = 255,  
25             message = "Subtitle must not exceed 255 characters"  
26         )  
27         String subtitle,  
28
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 29–34

```
29         @Size(  
30             max = 20000,  
31             message = "Description must not exceed 20000 characters"  
32         )  
33         String description,  
34
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 35–44

```
35         @Size(  
36             max = 1000,  
37             message = "Cover image URL must not exceed 1000 characters"  
38         )  
39         @Pattern(  
40             regexp = "^$|^https?:/.+$$",  
41             message = "Cover image URL must begin with http:// or https://"   
42         )  
43         String coverImageUrl,  
44
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 45–54

```
45         @Size(  
46             max = 10,  
47             message = "Language code must not exceed 10 characters"  
48         )  
49         @Pattern(  
50             regexp = "^$|^[a-z]{2,3}(:-[A-Z]{2})?$",  
51             message = "Language code must use a format such as en, hi or en-US"  
52         )  
53         String languageCode,  
54
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 55–57

```
55         @NotNull(message = "Featured status is required")
56         Boolean featured,
57
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 58–64

```
58         @NotEmpty(message = "At least one author is required")
59         Set<
60             @NotNull(message = "Author ID must not be null")
61             @Positive(message = "Author ID must be positive")
62             Long
63             > authorIds,
64
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 65–71

```
65         @NotEmpty(message = "At least one category is required")
66         Set<
67             @NotNull(message = "Category ID must not be null")
68             @Positive(message = "Category ID must be positive")
69             Long
70             > categoryIds,
71
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 72–77

```
72         @NotEmpty(message = "At least one book edition is required")
73         @Valid
74         List<BookEditionRequest> editions
75     ) {
76     }
77
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

backend/src/main/java/com/akshara/api/book/dto/

BookResponse.java

File 9 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines a validated API boundary type used by the catalogue and inventory module.

Current file length: 23 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.book.dto;
2
3 import java.time.Instant;
4 import java.util.List;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–19

```
6 public record BookResponse(
7     Long id,
8     String title,
9     String subtitle,
10    String description,
11    String coverImageUrl,
12    String languageCode,
13    boolean featured,
14    String metadataSource,
15    String externalWorkId,
16    List<AuthorResponse> authors,
17    List<CategoryResponse> categories,
18    List<BookEditionResponse> editions,
19    Instant createdAt,
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 20–23

```
20         Instant updatedAt
21     ) {
22     }
23
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/book/dto/

FeaturedRequest.java

File 10 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines a validated API boundary type used by the catalogue and inventory module.

Current file length: 10 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.book.dto;
2
3 import jakarta.validation.constraints.NotNull;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–10

```
5 public record FeaturedRequest(
6     @NotNull(message = "Featured status is required")
7     Boolean featured
8 ) {
9 }
10
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/book/dto/InventoryRequest.java

File 11 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines a validated API boundary type used by the catalogue and inventory module.

Current file length: 38 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–8

```
1 package com.akshara.api.book.dto;
2
3 import com.akshara.api.book.entity.AvailabilityStatus;
4 import jakarta.validation.constraints.DecimalMin;
5 import jakarta.validation.constraints.Digits;
6 import jakarta.validation.constraints.NotNull;
7 import jakarta.validation.constraints.PositiveOrZero;
8
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 9–12

```
9 import java.math.BigDecimal;
10
11 public record InventoryRequest(
12
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 13–24

```
13     @NotNull(message = "Price is required")
14     @DecimalMin(
15         value = "0.01",
16         message = "Price must be greater than zero"
17     )
18     @Digits(
19         integer = 8,
20         fraction = 2,
21         message = "Price must contain at most 8 integer digits and 2 decimal places"
22     )
23     BigDecimal price,
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 25–30

```

25         @NotNull(message = "Stock quantity is required")
26         @PositiveOrZero(
27             message = "Stock quantity must not be negative"
28         )
29         Integer stockQuantity,
30     
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 31–33

```

31         @NotNull(message = "Availability status is required")
32         AvailabilityStatus availabilityStatus,
33     
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 34–38

```

34         @NotNull(message = "Active status is required")
35         Boolean active
36     ) {
37     }
38 
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

backend/src/main/java/com/akshara/api/book/dto/

RestockRequest.java

File 12 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines a validated API boundary type used by the catalogue and inventory module.

Current file length: 14 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 package com.akshara.api.book.dto;
2
3 import jakarta.validation.constraints.Max;
4 import jakarta.validation.constraints.NotNull;
5 import jakarta.validation.constraints.Positive;
6
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 7–14

```
7 public record RestockRequest(
8     @NotNull(message = "Restock quantity is required")
9     @Positive(message = "Restock quantity must be greater than zero")
10    @Max(value = 1_000_000, message = "Restock quantity must not exceed 1000000")
11    Integer quantity
12 ) {
13 }
14
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/main/java/com/akshara/api/book/entity/Book.java

File 13 of 99 · Added or modified · Catalogue and inventory

Purpose: Models persisted domain state for the catalogue and inventory module using JPA mappings.

Current file length: 166 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–13

```
1 package com.akshara.api.book.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.Index;
9 import jakarta.persistence.PrePersist;
10 import jakarta.persistence.PreUpdate;
11 import jakarta.persistence.Table;
12 import jakarta.persistence.UniqueConstraint;
13
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 14–27

```
14 import java.time.Instant;
15
16 @Entity
17 @Table(
18     name = "books",
19     uniqueConstraints = @UniqueConstraint(
20         name = "uk_books_metadata_source_work",
21         columnNames = {"metadata_source", "external_work_id"}
22     ),
23     indexes = {
24         @Index(
25             name = "idx_books_title",
26             columnList = "title"
27         ),

```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.

Lines 28–35

```
28         @Index(
29             name = "idx_books_language",
30             columnList = "language_code"
31         )
32     }
```

```
33 )
34 public class Book {
35
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 36–39

```
36 @Id
37 @GeneratedValue(strategy = GenerationType.IDENTITY)
38 private Long id;
39
```

How to read this block

- This field is the database identity. Its value is generated when a new entity is inserted.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 40–42

```
40 @Column(nullable = false, length = 255)
41 private String title;
42
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 43–45

```
43 @Column(length = 255)
44 private String subtitle;
45
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 46–48

```
46 @Column(columnDefinition = "TEXT")
47 private String description;
48
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 49–51

```
49 @Column(name = "cover_image_url", length = 1000)
50 private String coverImageUrl;
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 52–54

```
52     @Column(name = "language_code", length = 10)
53     private String languageCode;
54
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 55–57

```
55     @Column(nullable = false)
56     private boolean featured;
57
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 58–60

```
58     @Column(name = "metadata_source", length = 30)
59     private String metadataSource;
60
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 61–63

```
61     @Column(name = "external_work_id", length = 255)
62     private String externalWorkId;
63
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 64–66

```
64     @Column(name = "created_at", nullable = false, updatable = false)
65     private Instant createdAt;
66
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 67–69

```
67     @Column(name = "updated_at", nullable = false)
68     private Instant updatedAt;
69
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 70–73

```
70     protected Book() {
71         // Required by JPA
72     }
73
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 74–77

```
74     public Book(String title) {
75         this.title = title;
76     }
77
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 78–84

```
78     @PrePersist
79     void beforeInsert() {
80         Instant now = Instant.now();
81         this.createdAt = now;
82         this.updatedAt = now;
83     }
84
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 85–89

```
85     @PreUpdate
86     void beforeUpdate() {
87         this.updatedAt = Instant.now();
88     }
89
```

```
88     }
89
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 90–93

```
90     public Long getId() {
91         return id;
92     }
93
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 94–97

```
94     public String getTitle() {
95         return title;
96     }
97
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 98–101

```
98     public String getSubtitle() {
99         return subtitle;
100    }
101
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 102–105

```
102    public String getDescription() {
103        return description;
104    }
105
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 106–109

```
106     public String getCoverImageUrl() {  
107         return coverImageUrl;  
108     }  
109
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 110–113

```
110     public String getLanguageCode() {  
111         return languageCode;  
112     }  
113
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 114–117

```
114     public boolean isFeatured() {  
115         return featured;  
116     }  
117
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 118–121

```
118     public String getMetadataSource() {  
119         return metadataSource;  
120     }  
121
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 122–125

```
122     public String getExternalWorkId() {  
123         return externalWorkId;  
124     }  
125
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 126–129

```
126     public Instant getCreatedAt() {  
127         return createdAt;  
128     }  
129
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 130–133

```
130     public Instant getUpdatedAt() {  
131         return updatedAt;  
132     }  
133
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 134–137

```
134     public void setTitle(String title) {  
135         this.title = title;  
136     }  
137
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 138–141

```
138     public void setSubtitle(String subtitle) {  
139         this.subtitle = subtitle;  
140     }  
141
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 142–145

```
142     public void setDescription(String description) {  
143         this.description = description;  
144     }  
145
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 146–149

```
146     public void setCoverImageUrl(String coverImageUrl) {  
147         this.coverImageUrl = coverImageUrl;  
148     }  
149
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 150–153

```
150     public void setLanguageCode(String languageCode) {  
151         this.languageCode = languageCode;  
152     }  
153
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 154–157

```
154     public void setFeatured(boolean featured) {  
155         this.featured = featured;  
156     }  
157
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 158–161

```
158     public void setMetadataSource(String metadataSource) {  
159         this.metadataSource = metadataSource;  
160     }  
161
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 162–166

```
162     public void setExternalWorkId(String externalWorkId) {  
163         this.externalWorkId = externalWorkId;  
164     }  
165 }  
166
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

dependencies.

backend/src/main/java/com/akshara/api/book/entity/

BookEdition.java

File 14 of 99 · Added or modified · Catalogue and inventory

Purpose: Models persisted domain state for the catalogue and inventory module using JPA mappings.

Current file length: 212 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.book.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.EnumType;
6 import jakarta.persistence.Enumerated;
7 import jakarta.persistence.FetchType;
8 import jakarta.persistence.GeneratedValue;
9 import jakarta.persistence.GenerationType;
10 import jakarta.persistence.Id;
11 import jakarta.persistence.Index;
12 import jakarta.persistence.JoinColumn;
13 import jakarta.persistence.ManyToOne;
14 import jakarta.persistence.PrePersist;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–18

```
15 import jakarta.persistence.PreUpdate;
16 import jakarta.persistence.Table;
17 import jakarta.persistence.UniqueConstraint;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 19–21

```
19 import java.time.Instant;
20 import java.time.LocalDate;
21
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 22–35

```
22 @Entity
```

```

23 @Table(
24     name = "book_editions",
25     uniqueConstraints = {
26         @UniqueConstraint(
27             name = "uk_book_editions_isbn10",
28             columnNames = "isbn10"
29         ),
30         @UniqueConstraint(
31             name = "uk_book_editions_isbn13",
32             columnNames = "isbn13"
33         ),
34         @UniqueConstraint(
35             name = "uk_book_editions_sku",

```

How to read this block

- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 36–49

```

36         columnNames = "sku"
37     )
38 },
39     indexes = {
40         @Index(
41             name = "idx_book_editions_book",
42             columnList = "book_id"
43         ),
44         @Index(
45             name = "idx_book_editions_publisher",
46             columnList = "publisher_id"
47         ),
48         @Index(
49             name = "idx_book_editions_format",

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 50–55

```

50         columnList = "format"
51     )
52 }
53 )
54 public class BookEdition {
55

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 56–59

```

56     @Id
57     @GeneratedValue(strategy = GenerationType.IDENTITY)
58     private Long id;
59

```

How to read this block

- This field is the database identity. Its value is generated when a new entity is inserted.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 60–63

```
60     @ManyToOne(fetch = FetchType.LAZY, optional = false)
61     @JoinColumn(name = "book_id", nullable = false)
62     private Book book;
63
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 64–67

```
64     @ManyToOne(fetch = FetchType.LAZY)
65     @JoinColumn(name = "publisher_id")
66     private Publisher publisher;
67
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 68–71

```
68     @Enumerated(EnumType.STRING)
69     @Column(nullable = false, length = 20)
70     private BookFormat format;
71
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 72–74

```
72     @Column(name = "edition_name", length = 100)
73     private String editionName;
74
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 75–77

```
75     @Column(length = 10)
76     private String isbn10;
77
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 78–80

```
78     @Column(length = 13)
79     private String isbn13;
80
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 81–83

```
81     @Column(name = "publication_date")
82     private LocalDate publicationDate;
83
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 84–86

```
84     @Column(name = "page_count")
85     private Integer pageCount;
86
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 87–89

```
87     @Column(nullable = false, length = 100)
88     private String sku;
89
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 90–92

```
90     @Column(name = "external_edition_id", length = 255)
91     private String externalEditionId;
92
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 93–95

```
93     @Column(name = "created_at", nullable = false, updatable = false)
94     private Instant createdAt;
95
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 96–98

```
96     @Column(name = "updated_at", nullable = false)
97     private Instant updatedAt;
98
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 99–102

```
99     protected BookEdition() {
100         // Required by JPA
101     }
102
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 103–107

```
103     public BookEdition(Book book, BookFormat format) {
104         this.book = book;
105         this.format = format;
106     }
107
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 108–114

```
108     @PrePersist
109     void beforeInsert() {
110         Instant now = Instant.now();
111         this.createdAt = now;
112         this.updatedAt = now;
113     }
114
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 115–119

```
115     @PreUpdate
116     void beforeUpdate() {
117         this.updatedAt = Instant.now();
118     }
119
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 120–123

```
120     public Long getId() {
121         return id;
122     }
123
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 124–127

```
124     public Book getBook() {
125         return book;
126     }
127
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 128–131

```
128     public Publisher getPublisher() {
129         return publisher;
130     }
131
```

```
130     }  
131
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 132–135

```
132     public BookFormat getFormat() {  
133         return format;  
134     }  
135
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 136–139

```
136     public String getEditionName() {  
137         return editionName;  
138     }  
139
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 140–143

```
140     public String getIsbn10() {  
141         return isbn10;  
142     }  
143
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 144–147

```
144     public String getIsbn13() {  
145         return isbn13;  
146     }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 148–151

```
148     public LocalDate getPublicationDate() {
149         return publicationDate;
150     }
151
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 152–155

```
152     public Integer getPageCount() {
153         return pageCount;
154     }
155
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 156–159

```
156     public String getSku() {
157         return sku;
158     }
159
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 160–163

```
160     public String getExternalEditionId() {
```

```
161         return externalEditionId;
162     }
163
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 164–167

```
164     public Instant getCreatedAt() {
165         return createdAt;
166     }
167
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 168–171

```
168     public Instant getUpdatedAt() {
169         return updatedAt;
170     }
171
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 172–175

```
172     public void setBook(Book book) {
173         this.book = book;
174     }
175
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 176–179

```
176     public void setPublisher(Publisher publisher) {
177         this.publisher = publisher;
178     }
179
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 180–183

```
180     public void setFormat(BookFormat format) {
181         this.format = format;
182     }
183
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 184–187

```
184     public void setEditionName(String editionName) {
185         this.editionName = editionName;
186     }
187
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 188–191

```
188     public void setIsbn10(String isbn10) {
189         this.isbn10 = isbn10;
190     }
191
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 192–195

```
192     public void setIsbn13(String isbn13) {  
193         this.isbn13 = isbn13;  
194     }  
195
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 196–199

```
196     public void setPublicationDate(LocalDate publicationDate) {  
197         this.publicationDate = publicationDate;  
198     }  
199
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 200–203

```
200     public void setPageCount(Integer pageCount) {  
201         this.pageCount = pageCount;  
202     }  
203
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 204–207

```
204     public void setSku(String sku) {  
205         this.sku = sku;  
206     }  
207
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 208–212

```
208     public void setExternalEditionId(String externalEditionId) {  
209         this.externalEditionId = externalEditionId;  
210     }  
211 }  
212
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

backend/src/main/java/com/akshara/api/book/entity/

BookFormat.java

File 15 of 99 · Added or modified · Catalogue and inventory

Purpose: Models persisted domain state for the catalogue and inventory module using JPA mappings.

Current file length: 17 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–8

```
1 package com.akshara.api.book.entity;
2
3 public enum BookFormat {
4     PAPERBACK,
5     HARDCOVER,
6     PDF,
7     EPUB;
8
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 9–12

```
9     public boolean isPhysical() {
10         return this == PAPERBACK || this == HARDCOVER;
11     }
12
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 13–17

```
13     public boolean isDigital() {
14         return !isPhysical();
15     }
16 }
17
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/book/entity/ Inventory.java

File 16 of 99 · Added or modified · Catalogue and inventory

Purpose: Models persisted domain state for the catalogue and inventory module using JPA mappings.

Current file length: 207 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.book.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.EnumType;
6 import jakarta.persistence.Enumerated;
7 import jakarta.persistence.FetchType;
8 import jakarta.persistence.GeneratedValue;
9 import jakarta.persistence.GenerationType;
10 import jakarta.persistence.Id;
11 import jakarta.persistence.Index;
12 import jakarta.persistence.JoinColumn;
13 import jakarta.persistence.OneToOne;
14 import jakarta.persistence.PrePersist;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–18

```
15 import jakarta.persistence.PreUpdate;
16 import jakarta.persistence.Table;
17 import jakarta.persistence.UniqueConstraint;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 19–21

```
19 import java.math.BigDecimal;
20 import java.time.Instant;
21
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 22–35

```
22 import com.akshara.api.common.exception.InvalidRequestException;
```

```

23
24 @Entity
25 @Table(
26     name = "inventory",
27     uniqueConstraints = {
28         @UniqueConstraint(
29             name = "uk_inventory_book_edition",
30             columnNames = "book_edition_id"
31         )
32     },
33     indexes = {
34         @Index(
35             name = "idx_inventory_availability",

```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 36–41

```

36         columnList = "availability_status"
37     )
38 }
39 )
40 public class Inventory {
41

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 42–45

```

42     @Id
43     @GeneratedValue(strategy = GenerationType.IDENTITY)
44     private Long id;
45

```

How to read this block

- This field is the database identity. Its value is generated when a new entity is inserted.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 46–53

```

46     @OneToOne(fetch = FetchType.LAZY, optional = false)
47     @JoinColumn(
48         name = "book_edition_id",
49         nullable = false,
50         unique = true
51     )
52     private BookEdition bookEdition;
53

```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 54–60

```
54     @Column(  
55         nullable = false,  
56         precision = 10,  
57         scale = 2  
58     )  
59     private BigDecimal price;  
60
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 61–63

```
61     @Column(name = "stock_quantity", nullable = false)  
62     private Integer stockQuantity;  
63
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 64–71

```
64     @Enumerated(EnumType.STRING)  
65     @Column(  
66         name = "availability_status",  
67         nullable = false,  
68         length = 20  
69     )  
70     private AvailabilityStatus availabilityStatus;  
71
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 72–74

```
72     @Column(nullable = false)  
73     private boolean active = true;
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 75–77

```
75     @Column(name = "updated_at", nullable = false)
76     private Instant updatedAt;
77
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 78–81

```
78     protected Inventory() {
79         // Required by JPA
80     }
81
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 82–93

```
82     public Inventory(
83         BookEdition bookEdition,
84         BigDecimal price,
85         Integer stockQuantity,
86         AvailabilityStatus availabilityStatus
87     ) {
88         this.bookEdition = bookEdition;
89         this.price = price;
90         this.stockQuantity = stockQuantity;
91         this.availabilityStatus = availabilityStatus;
92     }
93
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 94–98

```
94     @PrePersist
95     void beforeInsert() {
96         this.updatedAt = Instant.now();
97     }
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 99–103

```
99     @PreUpdate
100     void beforeUpdate() {
101         this.updatedAt = Instant.now();
102     }
103
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 104–107

```
104     public Long getId() {
105         return id;
106     }
107
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 108–111

```
108     public BookEdition getBookEdition() {
109         return bookEdition;
110     }
111
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 112–115

```
112     public void setBookEdition(BookEdition bookEdition) {
113         this.bookEdition = bookEdition;
114     }
115
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous

value.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 116–119

```
116     public BigDecimal getPrice() {  
117         return price;  
118     }  
119
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 120–123

```
120     public void setPrice(BigDecimal price) {  
121         this.price = price;  
122     }  
123
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 124–127

```
124     public Integer getStockQuantity() {  
125         return stockQuantity;  
126     }  
127
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 128–131

```
128     public void setStockQuantity(Integer stockQuantity) {
129         this.stockQuantity = stockQuantity;
130     }
131
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 132–135

```
132     public AvailabilityStatus getAvailabilityStatus() {
133         return availabilityStatus;
134     }
135
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 136–141

```
136     public void setAvailabilityStatus(
137         AvailabilityStatus availabilityStatus
138     ) {
139         this.availabilityStatus = availabilityStatus;
140     }
141
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 142–145

```
142     public boolean isActive() {  
143         return active;  
144     }  
145
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 146–149

```
146     public void setActive(boolean active) {  
147         this.active = active;  
148     }  
149
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 150–153

```
150     public Instant getUpdatedAt() {  
151         return updatedAt;  
152     }  
153
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 154–160

```
154     public void decreaseStock(int requestedQuantity) {  
155         if (requestedQuantity <= 0) {  
156             throw new InvalidRequestException(  
157                 "Checkout quantity must be greater than zero"  
158             );  
159         }  
160
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 161–166

```
161         if (!active) {
162             throw new InvalidRequestException(
163                 "Book edition is inactive"
164             );
165         }
166     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 167–172

```
167         if (availabilityStatus != AvailabilityStatus.IN_STOCK) {
168             throw new InvalidRequestException(
169                 "Book edition is not available"
170             );
171         }
172     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 173–179

```
173         if (requestedQuantity > stockQuantity) {
174             throw new InvalidRequestException(
175                 "Insufficient stock. Available quantity: "
176                 + stockQuantity
177             );
178         }
179     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 180–186

```
180         stockQuantity -= requestedQuantity;
181
182         if (stockQuantity == 0) {
183             availabilityStatus = AvailabilityStatus.OUT_OF_STOCK;
184         }
185     }
186
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 187–193

```
187     public void increaseStock(int returnedQuantity) {
188         if (returnedQuantity <= 0) {
189             throw new InvalidRequestException(
190                 "Returned quantity must be greater than zero"
191             );
192         }
193
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 194–199

```
194         if (stockQuantity > Integer.MAX_VALUE - returnedQuantity) {
195             throw new InvalidRequestException(
196                 "Restock quantity exceeds the supported stock limit"
197             );
198         }
199
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 200–207

```
200         stockQuantity += returnedQuantity;
201
202         if (availabilityStatus == AvailabilityStatus.OUT_OF_STOCK) {
203             availabilityStatus = AvailabilityStatus.IN_STOCK;
204         }
205     }
206 }
207
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/main/java/com/akshara/api/book/repository/BookEditionRepository.java

File 17 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines Spring Data persistence queries required by the catalogue and inventory module.

Current file length: 24 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.book.repository;
2
3 import com.akshara.api.book.entity.BookEdition;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 6–8

```
6 import java.util.List;
7 import java.util.Optional;
8
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 9–11

```
9 public interface BookEditionRepository
10     extends JpaRepository<BookEdition, Long> {
11
```

How to read this block

- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 12–15

```
12     List<BookEdition> findAllByBook_Id(Long bookId);
```

```
13
14     Optional<BookEdition> findByIsbn10(String isbn10);
15
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 16–19

```
16     Optional<BookEdition> findByIsbn13(String isbn13);
17
18     Optional<BookEdition> findBySkuIgnoreCase(String sku);
19
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 20–24

```
20     boolean existsByIsbn10(String isbn10);
21
22     boolean existsByIsbn13(String isbn13);
23 }
24
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/book/repository/

BookRepository.java

File 18 of 99 · Added or modified · Catalogue and inventory

Purpose: Defines Spring Data persistence queries required by the catalogue and inventory module.

Current file length: 18 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 package com.akshara.api.book.repository;
2
3 import com.akshara.api.book.entity.Book;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
6
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 7–12

```
7 import java.util.Optional;
8
9 public interface BookRepository
10     extends JpaRepository<Book, Long>,
11         JpaSpecificationExecutor<Book> {
12
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 13–18

```
13     Optional<Book> findByMetadataSourceAndExternalWorkId(
14         String metadataSource,
15         String externalWorkId
16     );
17 }
18
```


How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

backend/src/main/java/com/akshara/api/book/service/

BookService.java

File 19 of 99 · Added or modified · Catalogue and inventory

Purpose: Owns catalogue CRUD, response mapping, inventory updates, locked restocking, featured curation and safe deletion while preserving historical orders.

Current file length: 974 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.book.service;
2
3 import com.akshara.api.book.dto.AuthorResponse;
4 import com.akshara.api.book.dto.BookEditionRequest;
5 import com.akshara.api.book.dto.BookEditionResponse;
6 import com.akshara.api.book.dto.BookRequest;
7 import com.akshara.api.book.dto.BookResponse;
8 import com.akshara.api.book.dto.CategoryResponse;
9 import com.akshara.api.book.dto.InventoryRequest;
10 import com.akshara.api.book.dto.InventoryResponse;
11 import com.akshara.api.book.dto.PublisherResponse;
12 import com.akshara.api.book.entity.Author;
13 import com.akshara.api.book.entity.AvailabilityStatus;
14 import com.akshara.api.book.entity.Book;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–28

```
15 import com.akshara.api.book.entity.BookAuthor;
16 import com.akshara.api.book.entity.BookCategory;
17 import com.akshara.api.book.entity.BookEdition;
18 import com.akshara.api.book.entity.BookFormat;
19 import com.akshara.api.book.entity.Category;
20 import com.akshara.api.book.entity.Inventory;
21 import com.akshara.api.book.entity.Publisher;
22 import com.akshara.api.book.repository.AuthorRepository;
23 import com.akshara.api.book.repository.BookAuthorRepository;
24 import com.akshara.api.book.repository.BookCategoryRepository;
25 import com.akshara.api.book.repository.BookEditionRepository;
26 import com.akshara.api.book.repository.BookRepository;
27 import com.akshara.api.book.repository.CategoryRepository;
28 import com.akshara.api.book.repository.PublisherRepository;
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 29–42

```
29 import com.akshara.api.common.exception.DuplicateResourceException;
30 import com.akshara.api.common.exception.ResourceNotFoundException;
31 import com.akshara.api.cart.repository.CartItemRepository;
32 import com.akshara.api.order.repository.OrderItemRepository;
33 import com.akshara.api.review.repository.ReviewRepository;
34 import com.akshara.api.wishlist.repository.WishlistItemRepository;
35 import org.springframework.data.domain.Sort;
36 import org.springframework.stereotype.Service;
37 import org.springframework.transaction.annotation.Transactional;
38 import com.akshara.api.book.dto.BookPageResponse;
39 import com.akshara.api.book.dto.BookSearchCriteria;
40 import com.akshara.api.book.dto.BookSort;
41 import com.akshara.api.book.specification.BookSpecifications;
42 import org.springframework.data.domain.Page;
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 43–46

```
43 import org.springframework.data.domain.PageRequest;
44 import org.springframework.data.domain.Pageable;
45 import com.akshara.api.book.repository.InventoryRepository;
46
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 47–55

```
47 import java.util.Comparator;
48 import java.util.HashSet;
49 import java.util.List;
50 import java.util.Locale;
51 import java.util.Map;
52 import java.util.Set;
53 import java.util.function.Function;
54 import java.util.stream.Collectors;
55
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 56–59

```
56 @Service
57 @Transactional(readOnly = true)
58 public class BookService {
59
```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.
- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.

Lines 60–72

```
60     private final BookRepository bookRepository;  
61     private final AuthorRepository authorRepository;  
62     private final CategoryRepository categoryRepository;  
63     private final PublisherRepository publisherRepository;  
64     private final BookAuthorRepository bookAuthorRepository;  
65     private final BookCategoryRepository bookCategoryRepository;  
66     private final BookEditionRepository bookEditionRepository;  
67     private final InventoryRepository inventoryRepository;  
68     private final CartItemRepository cartItemRepository;  
69     private final WishListItemRepository wishListItemRepository;  
70     private final ReviewRepository reviewRepository;  
71     private final OrderItemRepository orderItemRepository;  
72
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 73–86

```
73     public BookService(  
74         BookRepository bookRepository,  
75         AuthorRepository authorRepository,  
76         CategoryRepository categoryRepository,  
77         PublisherRepository publisherRepository,  
78         BookAuthorRepository bookAuthorRepository,  
79         BookCategoryRepository bookCategoryRepository,  
80         BookEditionRepository bookEditionRepository,  
81         InventoryRepository inventoryRepository,  
82         CartItemRepository cartItemRepository,  
83         WishListItemRepository wishListItemRepository,  
84         ReviewRepository reviewRepository,  
85         OrderItemRepository orderItemRepository  
86     ) {
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 87–100

```
87         this.bookRepository = bookRepository;  
88         this.authorRepository = authorRepository;  
89         this.categoryRepository = categoryRepository;  
90         this.publisherRepository = publisherRepository;
```

```

91         this.bookAuthorRepository = bookAuthorRepository;
92         this.bookCategoryRepository = bookCategoryRepository;
93         this.bookEditionRepository = bookEditionRepository;
94         this.inventoryRepository = inventoryRepository;
95         this.cartItemRepository = cartItemRepository;
96         this.wishlistItemRepository = wishlistItemRepository;
97         this.reviewRepository = reviewRepository;
98         this.orderItemRepository = orderItemRepository;
99     }
100

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 101–108

```

101     @Transactional
102     public BookResponse create(BookRequest request) {
103         validateSupportedFormats(request.editions());
104         List<Author> authors = findAuthors(request.authorIds());
105         List<Category> categories = findCategories(
106             request.categoryIds()
107         );
108

```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 109–112

```

109         Map<Long, Publisher> publishers = findPublishers(
110             request.editions()
111         );
112

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 113–115

```

113         validateIsbnUniqueness(request.editions(), null);
114         validateSkuUniqueness(request.editions(), null);
115

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 116–118

```
116         Book book = new Book(request.title().trim());
117         applyBookFields(book, request);
118     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 119–124

```
119         Book savedBook = bookRepository.save(book);
120
121         saveAuthorRelationships(savedBook, authors);
122         saveCategoryRelationships(savedBook, categories);
123         createEditions(savedBook, request.editions(), publishers);
124     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 125–127

```
125         return toResponse(savedBook);
126     }
127 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 128–141

```
128     @Transactional
129     public BookResponse createImported(
130         BookRequest request,
131         String metadataSource,
132         String externalWorkId,
133         String externalEditionId
134     ) {
135         validateSupportedFormats(request.editions());
136         List<Author> authors = findAuthors(request.authorIds());
137         List<Category> categories = findCategories(request.categoryIds());
138         Map<Long, Publisher> publishers = findPublishers(request.editions());
139         validateIsbnUniqueness(request.editions(), null);
140         validateSkuUniqueness(request.editions(), null);
141     }
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 142–149

```
142      String source = normalizeOptional(metadataSource);
143      String workId = normalizeOptional(externalWorkId);
144      Book book = source == null || workId == null
145          ? null
146          : bookRepository.findByMetadataSourceAndExternalWorkId(
147              source, workId
148          ).orElse(null);
149
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 150–163

```
150      if (book == null) {
151          book = new Book(request.title().trim());
152          book.setMetadataSource(source);
153          book.setExternalWorkId(workId);
154          applyBookFields(book, request);
155          book = bookRepository.save(book);
156          saveAuthorRelationships(book, authors);
157          saveCategoryRelationships(book, categories);
158      } else {
159          applyBookFields(book, request);
160          bookRepository.save(book);
161          replaceAuthorRelationships(book, authors);
162          replaceCategoryRelationships(book, categories);
163      }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 164–171

```
164
165      createEditions(
166          book, request.editions(), publishers,
167          normalizeOptional(externalEditionId)
168      );
169      return toResponse(book);
170  }
171
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 172–185

```
172     public List<BookResponse> getAll() {
173         return bookRepository
174             .findAll(
175                 Sort.by(Sort.Direction.ASC, "title")
176                     .and(Sort.by(
177                         Sort.Direction.ASC,
178                         "id"
179                     ))
180             )
181             .stream()
182             .map(this::toResponse)
183             .toList();
184     }
185
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 186–197

```
186     public BookPageResponse search(
187         BookSearchCriteria criteria,
188         int page,
189         int size,
190         BookSort sort
191     ) {
192         Pageable pageable = PageRequest.of(
193             page,
194             size,
195             createSort(sort)
196         );
197
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 198–202

```
198         Page<Book> result = bookRepository.findAll(
199             BookSpecifications.withCriteria(criteria),
200             pageable
201         );
202
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 203–207

```
203         List<BookResponse> content = result.getContent()
204             .stream()
205             .map(this::toResponse)
206             .toList();
207
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 208–218

```
208         return new BookPageResponse(
209             content,
210             result.getNumber(),
211             result.getSize(),
212             result.getTotalElements(),
213             result.getTotalPages(),
214             result.isFirst(),
215             result.isLast()
216         );
217     }
218
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 219–222

```
219     public BookResponse getById(Long id) {
220         return toResponse(findBookById(id));
221     }
222
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 223–230

```
223     @Transactional
224     public BookResponse update(
225         Long id,
226         BookRequest request
227     ) {
228         Book book = findBookById(id);
229         validateSupportedFormats(request.editions());
230
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 231–235

```
231         List<Author> authors = findAuthors(request.authorIds());
232         List<Category> categories = findCategories(
233             request.categoryIds()
234         );
235
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 236–239

```
236         Map<Long, Publisher> publishers = findPublishers(
237             request.editions()
238         );
239
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 240–242

```
240         validateIsbnUniqueness(request.editions(), id);
241         validateSkuUniqueness(request.editions(), id);
242
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 243–245

```
243         applyBookFields(book, request);
244         bookRepository.save(book);
245
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 246–249

```
246         replaceAuthorRelationships(book, authors);
247         replaceCategoryRelationships(book, categories);
248         replaceEditions(book, request.editions(), publishers);
249
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 250–252

```
250         return toResponse(book);
251     }
252
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 253–260

```
253     @Transactional
254     public BookResponse updateFeatured(Long id, boolean featured) {
255         Book book = findBookById(id);
256         book.setFeatured(featured);
257         bookRepository.save(book);
258         return toResponse(book);
259     }
260
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 261–274

```
261     @Transactional
262     public InventoryResponse updateInventory(
263         Long editionId,
264         InventoryRequest request
265     ) {
266         Inventory inventory = findInventoryForUpdate(editionId);
267         rejectUnsupportedDigitalSale(
268             inventory.getBookEdition().getFormat(), request.active()
269         );
270         inventory.setPrice(request.price());
271         inventory.setStockQuantity(request.stockQuantity());
272         inventory.setAvailabilityStatus(normalizeAvailability(
273             request.stockQuantity(), request.availabilityStatus()
274         ));

```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 275–278

```
275         inventory.setActive(request.active());
276         return toInventoryResponse(inventoryRepository.save(inventory));
277     }
278
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 279–288

```
279     @Transactional
280     public InventoryResponse restockInventory(
281         Long editionId,
282         int quantity
283     ) {
284         Inventory inventory = findInventoryForUpdate(editionId);
285         inventory.increaseStock(quantity);
286         return toInventoryResponse(inventoryRepository.save(inventory));
287     }
288
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 289–292

```
289     @Transactional
290     public void delete(Long id) {
291         Book book = findBookById(id);
292
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 293–301

```
293         cartItemRepository.deleteAllByBookEdition_Book_Id(id);
294         cartItemRepository.flush();
295         wishlistItemRepository.deleteAllByBook_Id(id);
296         wishlistItemRepository.flush();
297         reviewRepository.deleteAllByBook_Id(id);
298         reviewRepository.flush();
299         orderItemRepository.detachAllByBookId(id);
300         orderItemRepository.flush();
301
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 302–304

```
302         List<BookAuthor> authorLinks =
303             bookAuthorRepository.findAllByBook_Id(id);
304
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 305–307

```
305         List<BookCategory> categoryLinks =
306             bookCategoryRepository.findAllByBook_Id(id);
307
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 308–310

```
308         List<BookEdition> editions =
309             bookEditionRepository.findAllByBook_Id(id);
310
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC

in the business layer.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 311–315

```
311         deleteEditionInventory(editions);
312
313         bookEditionRepository.deleteAll(editions);
314         bookEditionRepository.flush();
315
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 316–318

```
316         bookAuthorRepository.deleteAll(authorLinks);
317         bookAuthorRepository.flush();
318
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 319–321

```
319         bookCategoryRepository.deleteAll(categoryLinks);
320         bookCategoryRepository.flush();
321
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 322–324

```
322         bookRepository.delete(book);
323     }
324
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 325–331

```
325     private Book findBookById(Long id) {
326         return bookRepository.findById(id)
327             .orElseThrow(() -> new ResourceNotFoundException(
328                 "Book not found with id: " + id
329             ));
330     }
331
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 332–343

```
332     private Inventory findInventoryForUpdate(Long editionId) {
333         if (!bookEditionRepository.existsById(editionId)) {
334             throw new ResourceNotFoundException(
335                 "Book edition not found with id: " + editionId
336             );
337         }
338         return inventoryRepository.findByBookEditionIdForUpdate(editionId)
339             .orElseThrow(() -> new ResourceNotFoundException(
340                 "Inventory not found for book edition id: " + editionId
341             ));
342     }
343 }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 344–353

```
344     private AvailabilityStatus normalizeAvailability(
345         int stock,
346         AvailabilityStatus requested
347     ) {
348         return stock == 0
349             && requested == AvailabilityStatus.IN_STOCK
350             ? AvailabilityStatus.OUT_OF_STOCK
351             : requested;
352     }
353 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 354–356

```
354     private List<Author> findAuthors(Set<Long> authorIds) {
355         List<Author> authors = authorRepository.findAllById(authorIds);
356     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 357–360

```
357         Set<Long> foundIds = authors.stream()
358             .map(Author::getId)
359             .collect(Collectors.toSet());
360     }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 361–365

```
361         List<Long> missingIds = authorIds.stream()
362             .filter(id -> !foundIds.contains(id))
363             .sorted()
364             .toList();
365     }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 366–371

```
366         if (!missingIds.isEmpty()) {
```



```

367         throw new ResourceNotFoundException(
368             "Authors not found with ids: " + missingIds
369         );
370     }
371

```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 372–374

```

372         return authors;
373     }
374

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 375–378

```

375     private List<Category> findCategories(Set<Long> categoryIds) {
376         List<Category> categories =
377             categoryRepository.findAllById(categoryIds);
378     }

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 379–382

```

379         Set<Long> foundIds = categories.stream()
380             .map(Category::getId)
381             .collect(Collectors.toSet());
382

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 383–387

```
383         List<Long> missingIds = categoryIds.stream()
384             .filter(id -> !foundIds.contains(id))
385             .sorted()
386             .toList();
387
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 388–393

```
388         if (!missingIds.isEmpty()) {
389             throw new ResourceNotFoundException(
390                 "Categories not found with ids: " + missingIds
391             );
392         }
393
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 394–396

```
394         return categories;
395     }
396
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 397–404

```
397     private Map<Long, Publisher> findPublishers(
398         List<BookEditionRequest> editionRequests
399     ) {
400         Set<Long> publisherIds = editionRequests.stream()
401             .map(BookEditionRequest::publisherId)
402             .filter(id -> id != null)
403             .collect(Collectors.toSet());
404
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 405–407

```
405         List<Publisher> publishers =
406             publisherRepository.findAllById(publisherIds);
407
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 408–411

```
408         Set<Long> foundIds = publishers.stream()
409             .map(Publisher::getId)
410             .collect(Collectors.toSet());
411
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 412–416

```
412         List<Long> missingIds = publisherIds.stream()
413             .filter(id -> !foundIds.contains(id))
414             .sorted()
415             .toList();
416
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 417–422

```
417         if (!missingIds.isEmpty()) {
418             throw new ResourceNotFoundException(
419                 "Publishers not found with ids: " + missingIds
420             );
421         }
422
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 423–429

```

423         return publishers.stream()
424             .collect(Collectors.toMap(
425                 Publisher::getId,
426                 Function.identity()
427             ));
428     }
429 
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 430–436

```

430     private void validateIsbnUniqueness(
431         List<BookEditionRequest> editions,
432         Long currentBookId
433     ) {
434         Set<String> requestIsbn10Values = new HashSet<>();
435         Set<String> requestIsbn13Values = new HashSet<>();
436 
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 437–440

```

437         for (BookEditionRequest edition : editions) {
438             String isbn10 = normalizeIsbn10(edition.isbn10());
439             String isbn13 = normalizeIsbn13(edition.isbn13());
440 
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 441–446

```

441         validateIsbn10(
442             isbn10,
443             currentBookId,
444             requestIsbn10Values

```

```
445         );  
446     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 447–454

```
447         validateIsbn13(  
448             isbn13,  
449             currentBookId,  
450             requestIsbn13Values  
451         );  
452     }  
453 }  
454
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 455–462

```
455     private void validateSupportedFormats(List<BookEditionRequest> editions) {  
456         for (BookEditionRequest edition : editions) {  
457             rejectUnsupportedDigitalSale(  
458                 edition.format(), edition.inventory().active()  
459             );  
460         }  
461     }  
462
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 463–474

```
463     private void rejectUnsupportedDigitalSale(  
464         BookFormat format,  
465         boolean active  
466     ) {  
467         if (active && format.isDigital()) {  
468             throw new com.akshara.api.common.exception.InvalidRequestException(  
469                 "Digital editions cannot be activated for sale until "  
470                 + "a licensed file and reader entitlement are configured"  
471             );  
472         }  
473     }  
474
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 475–483

```
475     private void validateIsbn10(  
476         String isbn10,  
477         Long currentBookId,  
478         Set<String> requestValues  
479     ) {  
480         if (isbn10 == null) {  
481             return;  
482         }  
483     }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 484–490

```
484         if (!isbn10.matches("[0-9]{9}[0-9X]$")  
485             || !isValidIsbn10Checksum(isbn10)) {  
486             throw new com.akshara.api.common.exception.InvalidRequestException(  
487                 "ISBN-10 '" + isbn10 + "' is invalid"  
488             );  
489         }  
490     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 491–497

```
491         if (!requestValues.add(isbn10)) {  
492             throw new DuplicateResourceException(  
493                 "ISBN-10 '" + isbn10  
494                 + "' occurs more than once in the request"  
495             );  
496         }  
497     }
```

```
496     }
497
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 498–511

```
498     bookEditionRepository.findByIsbn10(isbn10)
499         .filter(existing ->
500             currentBookId == null
501                 || !existing.getBook()
502                     .getId()
503                     .equals(currentBookId)
504         )
505         .ifPresent(existing -> {
506             throw new DuplicateResourceException(
507                 "A book edition with ISBN-10 '"
508                     + isbn10 + "' already exists"
509             );
510         });
511 }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 512–521

```
512
513     private void validateIsbn13(
514         String isbn13,
515         Long currentBookId,
516         Set<String> requestValues
517     ) {
518         if (isbn13 == null) {
519             return;
520         }
521 }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 522–528

```
522         if (!isbn13.matches("[0-9]{13}$"))
523             || !isValidIsbn13Checksum(isbn13)) {
524             throw new com.akshara.api.common.exception.InvalidRequestException(
525                 "ISBN-13 '" + isbn13 + "' is invalid"
526             );
527         }
528     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 529–535

```
529         if (!requestValues.add(isbn13)) {
530             throw new DuplicateResourceException(
531                 "ISBN-13 '" + isbn13
532                 + "' occurs more than once in the request"
533             );
534         }
535     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 536–549

```
536         bookEditionRepository.findByIsbn13(isbn13)
537             .filter(existing ->
538                 currentBookId == null
539                 || !existing.getBook()
540                     .getId()
541                     .equals(currentBookId)
542             )
543             .ifPresent(existing -> {
544                 throw new DuplicateResourceException(
545                     "A book edition with ISBN-13 '"
546                     + isbn13 + "' already exists"
547                 );
548             });
549     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 550–563

```
550
551     private void applyBookFields(
552         Book book,
553         BookRequest request
554     ) {
555         book.setTitle(request.title().trim());
556         book.setSubtitle(normalizeOptional(request.subtitle()));
557         book.setDescription(normalizeOptional(request.description()));
558         book.setCoverImageUrl(
559             normalizeOptional(request.coverImageUrl())
560         );
561         book.setLanguageCode(
562             normalizeOptional(request.languageCode())
563         );
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 564–566

```
564         book.setFeatured(request.featured());
565     }
566
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 567–580

```
567     private void validateSkuUniqueness(
568         List<BookEditionRequest> editions,
569         Long currentBookId
570     ) {
571         Set<String> requestValues = new HashSet<>();
572         for (BookEditionRequest edition : editions) {
573             String sku = edition.sku().trim();
574             String comparisonValue = sku.toLowerCase(Locale.ROOT);
575             if (!requestValues.add(comparisonValue)) {
576                 throw new DuplicateResourceException(
577                     "SKU '" + sku + "' occurs more than once in the request"
578                 );
579             }
580             bookEditionRepository.findBySkuIgnoreCase(sku)
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 581–590

```

581         .filter(existing -> currentBookId == null
582             || !existing.getBook().getId().equals(currentBookId))
583         .ifPresent(existing -> {
584             throw new DuplicateResourceException(
585                 "A book edition with SKU '" + sku + "' already exists"
586             );
587         });
588     }
589 }
590

```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 591–598

```

591     private void saveAuthorRelationships(
592         Book book,
593         List<Author> authors
594     ) {
595         List<BookAuthor> relationships = authors.stream()
596             .map(author -> new BookAuthor(book, author))
597             .toList();
598     }

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 599–601

```

599     bookAuthorRepository.saveAll(relationships);
600 }
601

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 602–609

```
602     private void saveCategoryRelationships(
603         Book book,
604         List<Category> categories
605     ) {
606         List<BookCategory> relationships = categories.stream()
607             .map(category -> new BookCategory(book, category))
608             .toList();
609     }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 610–612

```
610         bookCategoryRepository.saveAll(relationships);
611     }
612 }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 613–619

```
613     private void replaceAuthorRelationships(
614         Book book,
615         List<Author> requestedAuthors
616     ) {
617         List<BookAuthor> existingLinks =
618             bookAuthorRepository.findAllByBook_Id(book.getId());
619     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 620–623

```
620         Set<Long> requestedIds = requestedAuthors.stream()
621             .map(Author::getId)
622             .collect(Collectors.toSet());
623
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 624–627

```
624         Set<Long> existingIds = existingLinks.stream()
625             .map(link -> link.getAuthor().getId())
626             .collect(Collectors.toSet());
627
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 628–635

```
628         List<BookAuthor> linksToDelete = existingLinks.stream()
629             .filter(link ->
630                 !requestedIds.contains(
631                     link.getAuthor().getId()
632                 )
633             )
634             .toList();
635
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 636–642

```
636         List<BookAuthor> linksToAdd = requestedAuthors.stream()
637             .filter(author ->
638                 !existingIds.contains(author.getId())
639             )
640             .map(author -> new BookAuthor(book, author))
641             .toList();
642
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 643–646

```
643         bookAuthorRepository.deleteAll(linksToDelete);
644         bookAuthorRepository.saveAll(linksToAdd);
645     }
646
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 647–653

```
647     private void replaceCategoryRelationships(
648         Book book,
649         List<Category> requestedCategories
650     ) {
651         List<BookCategory> existingLinks =
652             bookCategoryRepository.findAllByBook_Id(book.getId());
653
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 654–657

```
654         Set<Long> requestedIds = requestedCategories.stream()
655             .map(Category::getId)
656             .collect(Collectors.toSet());
657
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 658–661

```
658         Set<Long> existingIds = existingLinks.stream()
659             .map(link -> link.getCategory().getId())
660             .collect(Collectors.toSet());
661
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 662–669

```
662         List<BookCategory> linksToDelete = existingLinks.stream()
663             .filter(link ->
664                 !requestedIds.contains(
665                     link.getCategory().getId()
666                 )
667             )
668             .toList();
669
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 670–679

```
670         List<BookCategory> linksToAdd =
671             requestedCategories.stream()
672                 .filter(category ->
673                     !existingIds.contains(category.getId())
674                 )
675                 .map(category ->
676                     new BookCategory(book, category)
677                 )
678                 .toList();
679
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 680–683

```
680         bookCategoryRepository.deleteAll(linksToDelete);
681         bookCategoryRepository.saveAll(linksToAdd);
682     }
683
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 684–691

```
684     private void replaceEditions(
685         Book book,
686         List<BookEditionRequest> editionRequests,
687         Map<Long, Publisher> publishers
```

```

688     ) {
689         List<BookEdition> existingEditions =
690             bookEditionRepository.findAllByBook_Id(book.getId());
691     }

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 692–696

```

692         deleteEditionInventory(existingEditions);
693
694         bookEditionRepository.deleteAll(existingEditions);
695         bookEditionRepository.flush();
696     }

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 697–699

```

697         createEditions(book, editionRequests, publishers);
698     }
699 }

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 700–711

```

700     private void deleteEditionInventory(
701         List<BookEdition> editions
702     ) {
703         List<Inventory> inventories = editions.stream()
704             .map(edition ->
705                 inventoryRepository.findByBookEdition_Id(
706                     edition.getId()
707                 )
708             )
709             .flatMap(optional -> optional.stream())
710             .toList();
711     }

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 712–715

```
712         inventoryRepository.deleteAll(inventories);
713         inventoryRepository.flush();
714     }
715
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 716–723

```
716     private void createEditions(
717         Book book,
718         List<BookEditionRequest> editionRequests,
719         Map<Long, Publisher> publishers
720     ) {
721         createEditions(book, editionRequests, publishers, null);
722     }
723
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 724–735

```
724     private void createEditions(
725         Book book,
726         List<BookEditionRequest> editionRequests,
727         Map<Long, Publisher> publishers,
728         String externalEditionId
729     ) {
730         for (BookEditionRequest request : editionRequests) {
731             BookEdition edition = new BookEdition(
732                 book,
733                 request.format()
734             );
735
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final`` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 736–741

```
736         if (request.publisherId() != null) {
737             edition.setPublisher(
738                 publishers.get(request.publisherId())
739             );
740         }
741     }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 742–751

```
742         edition.setEditionName(
743             normalizeOptional(request.editionName())
744         );
745         edition.setIsbn10(normalizeIsbn10(request.isbn10()));
746         edition.setIsbn13(normalizeIsbn13(request.isbn13()));
747         edition.setPublicationDate(request.publicationDate());
748         edition.setPageCount(request.pageCount());
749         edition.setSku(request.sku().trim());
750         edition.setExternalEditionId(externalEditionId);
751     }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 752–754

```
752         BookEdition savedEdition =
753             bookEditionRepository.save(edition);
754     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 755–757

```
755         InventoryRequest inventoryRequest =
756             request.inventory();
757     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 758–764

```
758         Inventory inventory = new Inventory(  
759             savedEdition,  
760             inventoryRequest.price(),  
761             inventoryRequest.stockQuantity(),  
762             inventoryRequest.availabilityStatus()  
763         );  
764
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 765–769

```
765         inventory.setActive(inventoryRequest.active());  
766         inventoryRepository.save(inventory);  
767     }  
768 }  
769
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 770–782

```
770     private BookResponse toResponse(Book book) {  
771         List<AuthorResponse> authors =  
772             bookAuthorRepository  
773                 .findAllByBook_Id(book.getId())  
774                 .stream()  
775                 .map(BookAuthor::getAuthor)  
776                 .sorted(Comparator.comparing(  
777                     Author::getName,  
778                     String.CASE_INSENSITIVE_ORDER  
779                 ))  
780                 .map(this::toAuthorResponse)  
781                 .toList();  
782
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 783–794

```
783         List<CategoryResponse> categories =
784             bookCategoryRepository
785                 .findAllByBook_Id(book.getId())
786                 .stream()
787                 .map(BookCategory::getCategory)
788                 .sorted(Comparator.comparing(
789                     Category::getName,
790                     String.CASE_INSENSITIVE_ORDER
791                 ))
792                 .map(this::toCategoryResponse)
793                 .toList();
794
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 795–804

```
795         List<BookEditionResponse> editions =
796             bookEditionRepository
797                 .findAllByBook_Id(book.getId())
798                 .stream()
799                 .sorted(Comparator.comparing(
800                     BookEdition::getId
801                 ))
802                 .map(this::toEditionResponse)
803                 .toList();
804
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 805–818

```
805         return new BookResponse(
806             book.getId(),
807             book.getTitle(),
808             book.getSubtitle(),
809             book.getDescription(),
810             book.getCoverImageUrl(),
811             book.getLanguageCode(),
812             book.isFeatured(),
813             book.getMetadataSource(),
814             book.getExternalWorkId(),
815             authors,
```

```
816         categories,  
817         editions,  
818         book.getCreatedAt(),
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 819–822

```
819         book.getUpdatedAt()  
820     );  
821 }  
822
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 823–829

```
823     private BookEditionResponse toEditionResponse(  
824         BookEdition edition  
825     ) {  
826         PublisherResponse publisher = edition.getPublisher() == null  
827             ? null  
828             : toPublisherResponse(edition.getPublisher());  
829
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 830–836

```
830         Inventory inventory = inventoryRepository  
831             .findByBookEdition_Id(edition.getId())  
832             .orElseThrow(() -> new ResourceNotFoundException(  
833                 "Inventory not found for book edition id: "  
834                 + edition.getId()  
835             ));  
836
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 837–850

```
837     return new BookEditionResponse(  
838         edition.getId(),  
839         publisher,  
840         edition.getFormat(),  
841         edition.getEditionName(),  
842         edition.getIsbn10(),
```

```

843         edition.getIsbn13(),
844         edition.getPublicationDate(),
845         edition.getPageCount(),
846         edition.getSku(),
847         edition.getExternalEditionId(),
848         toInventoryResponse(inventory),
849         edition.getCreatedAt(),
850         edition.getUpdatedAt()

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 851–853

```

851     );
852 }
853

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 854–863

```

854     private AuthorResponse toAuthorResponse(Author author) {
855         return new AuthorResponse(
856             author.getId(),
857             author.getName(),
858             author.getBiography(),
859             author.getCreatedAt(),
860             author.getUpdatedAt()
861         );
862     }
863

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 864–876

```

864     private CategoryResponse toCategoryResponse(
865         Category category
866     ) {
867         return new CategoryResponse(
868             category.getId(),
869             category.getName(),
870             category.getSlug(),
871             category.getDescription(),
872             category.getCreatedAt(),
873             category.getUpdatedAt()
874         );

```

```
875     }
876 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 877–888

```
877     private PublisherResponse toPublisherResponse(
878         Publisher publisher
879     ) {
880         return new PublisherResponse(
881             publisher.getId(),
882             publisher.getName(),
883             publisher.getWebsiteUrl(),
884             publisher.getCreatedAt(),
885             publisher.getUpdatedAt()
886         );
887     }
888 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 889–901

```
889     private InventoryResponse toInventoryResponse(
890         Inventory inventory
891     ) {
892         return new InventoryResponse(
893             inventory.getId(),
894             inventory.getPrice(),
895             inventory.getStockQuantity(),
896             inventory.getAvailabilityStatus(),
897             inventory.isActive(),
898             inventory.getUpdatedAt()
899         );
900     }
901 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

or persists them.

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 902–904

```
902     private String normalizeIsbn10(String value) {
903         String normalized = normalizeOptional(value);
904     }
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 905–910

```
905         return normalized == null
906             ? null
907             : normalized.replaceAll("[-\s]", "")
908               .toUpperCase(Locale.ROOT);
909     }
910 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 911–916

```
911     private String normalizeIsbn13(String value) {
912         String normalized = normalizeOptional(value);
913         return normalized == null ? null
914             : normalized.replaceAll("[-\s]", "");
915     }
916 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 917–926

```
917     private boolean isValidIsbn10Checksum(String isbn) {
918         int sum = 0;
919         for (int index = 0; index < 10; index++) {
920             char character = isbn.charAt(index);
921             int value = character == 'X' ? 10 : character - '0';
922             sum += (10 - index) * value;
923         }
924         return sum % 11 == 0;
925     }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 927–936

```

927     private boolean isValidIsbn13Checksum(String isbn) {
928         int sum = 0;
929         for (int index = 0; index < 12; index++) {
930             int value = isbn.charAt(index) - '0';
931             sum += value * (index % 2 == 0 ? 1 : 3);
932         }
933         int checkDigit = (10 - (sum % 10)) % 10;
934         return checkDigit == isbn.charAt(12) - '0';
935     }
936 
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 937–941

```

937     private String normalizeOptional(String value) {
938         if (value == null) {
939             return null;
940         }
941 
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 942–945

```
942         String trimmed = value.trim();
943         return trimmed.isEmpty() ? null : trimmed;
944     }
945
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 946–950

```
946     private Sort createSort(BookSort requestedSort) {
947         BookSort effectiveSort = requestedSort == null
948             ? BookSort.TITLE_ASC
949             : requestedSort;
950
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 951–956

```
951         return switch (effectiveSort) {
952             case TITLE_ASC -> Sort.by(
953                 Sort.Order.asc("title").ignoreCase(),
954                 Sort.Order.asc("id")
955             );
956
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 957–961

```
957             case TITLE_DESC -> Sort.by(
958                 Sort.Order.desc("title").ignoreCase(),
959                 Sort.Order.desc("id")
960             );
961
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 962–966

```
962             case NEWEST -> Sort.by(
963                 Sort.Order.desc("createdAt"),
```

```
964             Sort.Order.desc("id")
965         );
966
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 967–974

```
967         case OLDEST -> Sort.by(
968             Sort.Order.asc("createdAt"),
969             Sort.Order.asc("id")
970         );
971     };
972 }
973 }
974
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/bookimport/controller/AdminBookImportController.java

File 20 of 99 · Added or modified · Hybrid catalogue import

Purpose: Exposes secured admin endpoints for external search, permanent import and deliberate catalogue seeding.

Current file length: 70 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.controller;
2
3 import com.akshara.api.book.dto.BookResponse;
4 import com.akshara.api.bookimport.dto.BookImportRequest;
5 import com.akshara.api.bookimport.dto.BookImportSource;
6 import com.akshara.api.bookimport.dto.ExternalBookSearchPage;
7 import com.akshara.api.bookimport.dto.CatalogueSeedResponse;
8 import com.akshara.api.bookimport.service.CatalogueSeedService;
9 import com.akshara.api.bookimport.service.BookImportService;
10 import com.akshara.api.bookimport.service.ExternalBookSearchService;
11 import jakarta.validation.Valid;
12 import jakarta.validation.constraints.NotBlank;
13 import jakarta.validation.constraints.Max;
14 import jakarta.validation.constraints.Min;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–24

```
15 import jakarta.validation.constraints.Size;
16 import org.springframework.http.HttpStatus;
17 import org.springframework.http.ResponseEntity;
18 import org.springframework.web.bind.annotation.GetMapping;
19 import org.springframework.web.bind.annotation.PostMapping;
20 import org.springframework.web.bind.annotation.RequestBody;
21 import org.springframework.web.bind.annotation.RequestMapping;
22 import org.springframework.web.bind.annotation.RequestParam;
23 import org.springframework.web.bind.annotation.RestController;
24
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 25–28

```
25 @RestController
26 @RequestMapping("/api/admin/book-import")
27 public class AdminBookImportController {
28
```

How to read this block

- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

Lines 29–32

```
29     private final ExternalBookSearchService searchService;
30     private final BookImportService importService;
31     private final CatalogueSeedService seedService;
32
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 33–42

```
33     public AdminBookImportController(
34         ExternalBookSearchService searchService,
35         BookImportService importService,
36         CatalogueSeedService seedService
37     ) {
38         this.searchService = searchService;
39         this.importService = importService;
40         this.seedService = seedService;
41     }
42
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 43–54

```
43     @GetMapping("/search")
44     public ResponseEntity<ExternalBookSearchPage> search(
45         @RequestParam @NotBlank @Size(max = 200) String query,
46         @RequestParam(defaultValue = "OPEN_LIBRARY") BookImportSource source,
47         @RequestParam(defaultValue = "1") @Min(1) int page,
48         @RequestParam(defaultValue = "10") @Min(1) @Max(20) int size
49     ) {
50         return ResponseEntity.ok(
51             searchService.search(query.trim(), source, page, size)
52         );
53     }
54
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 55–62

```

55     @PostMapping
56     public ResponseEntity<BookResponse> importBook(
57         @Valid @RequestBody BookImportRequest request
58     ) {
59         return ResponseEntity.status(HttpStatus.CREATED)
60             .body(importService.importBook(request));
61     }
62

```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 63–70

```

63     @PostMapping("/seed")
64     public ResponseEntity<CatalogueSeedResponse> seed(
65         @RequestParam(defaultValue = "50") @Min(1) @Max(100) int count
66     ) {
67         return ResponseEntity.ok(seedService.seed(count));
68     }
69 }
70

```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/bookimport/dto/BookImportRequest.java

File 21 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 50 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.dto;
2
3 import com.akshara.api.book.entity.AvailabilityStatus;
4 import com.akshara.api.book.entity.BookFormat;
5 import jakarta.validation.constraints.DecimalMin;
6 import jakarta.validation.constraints.Digits;
7 import jakarta.validation.constraints.NotBlank;
8 import jakarta.validation.constraints.NotEmpty;
9 import jakarta.validation.constraints.NotNull;
10 import jakarta.validation.constraints.PastOrPresent;
11 import jakarta.validation.constraints.Pattern;
12 import jakarta.validation.constraints.Positive;
13 import jakarta.validation.constraints.PositiveOrZero;
14 import jakarta.validation.constraints.Size;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–19

```
15
16 import java.math.BigDecimal;
17 import java.time.LocalDate;
18 import java.util.List;
19
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 20–33

```
20 public record BookImportRequest(
21     @NotNull(message = "Metadata source is required")
22     BookImportSource source,
23     @Size(max = 255) String sourceId,
24     @Size(max = 255) String editionId,
25     @NotBlank(message = "Book title is required") @Size(max = 255) String title,
26     @Size(max = 255) String subtitle,
27     @Size(max = 20000) String description,
28     @NotEmpty(message = "At least one author is required")
29     List<@NotBlank @Size(max = 150) String> authors,
```

```

30     @Size(max = 150) String publisher,
31     @Pattern(regexp = "^$|^[a-z]{2,3}(:-[A-Z]{2})?$") @Size(max = 10)
32     String languageCode,
33     @Positive(message = "Page count must be greater than zero") Integer pageCount,

```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 34–47

```

34     @NotEmpty(message = "At least one category is required")
35     List<@NotBlank @Size(max = 100) String> categories,
36     @Pattern(regexp = "^$|^https?:/.+?$") @Size(max = 1000) String coverImageUrl,
37     @PastOrPresent(message = "Publication date cannot be in the future") LocalDate publicationDate,
38     @Size(max = 100) String editionName,
39     @Size(max = 20) String isbn10,
40     @Size(max = 25) String isbn13,
41     @NotNull(message = "Book format is required") BookFormat format,
42     @NotNull @DecimalMin("0.01") @Digits(integer = 8, fraction = 2) BigDecimal price,
43     @NotNull @PositiveOrZero Integer stockQuantity,
44     @NotNull AvailabilityStatus availabilityStatus,
45     @NotBlank(message = "SKU is required") @Size(max = 100) String sku,
46     @NotNull Boolean active,
47     @NotNull Boolean featured

```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 48–50

```

48 ) implements CatalogueBookCommand {
49 }
50

```

How to read this block

- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.

backend/src/main/java/com/akshara/api/bookimport/dto/BookImportSource.java

File 22 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 7 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–7

```
1 package com.akshara.api.bookimport.dto;
2
3 public enum BookImportSource {
4     OPEN_LIBRARY,
5     GOOGLE_BOOKS
6 }
7
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

backend/src/main/java/com/akshara/api/bookimport/dto/

CatalogueBookCommand.java

File 23 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 32 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.bookimport.dto;
2
3 import com.akshara.api.book.entity.AvailabilityStatus;
4 import com.akshara.api.book.entity.BookFormat;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 6–9

```
6 import java.math.BigDecimal;
7 import java.time.LocalDate;
8 import java.util.List;
9
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 10–23

```
10 public interface CatalogueBookCommand {
11     String title();
12     String subtitle();
13     String description();
14     List<String> authors();
15     String publisher();
16     String languageCode();
17     Integer pageCount();
18     List<String> categories();
19     String coverImageUrl();
20     LocalDate publicationDate();
21     String editionName();
22     String isbn10();
23     String isbn13();
24 }
```

How to read this block

- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 24–32

```
24     BookFormat format();
25     BigDecimal price();
26     Integer stockQuantity();
27     AvailabilityStatus availabilityStatus();
28     String sku();
29     Boolean active();
30     Boolean featured();
31 }
32
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/main/java/com/akshara/api/bookimport/dto/ CatalogueSeedResponse.java

File 24 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 16 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.bookimport.dto;
2
3 import java.util.List;
4 import java.util.Map;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–16

```
6 public record CatalogueSeedResponse(
7     int requested,
8     int imported,
9     int skipped,
10    long catalogueSizeBefore,
11    long catalogueSizeAfter,
12    Map<String, Integer> genreCounts,
13    List<String> importedTitles
14 ) {
15 }
16
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/bookimport/dto/

ExternalBookResult.java

File 25 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 33 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.bookimport.dto;
2
3 import java.util.List;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–18

```
5 public record ExternalBookResult(
6     BookImportSource source,
7     String sourceId,
8     String editionId,
9     String title,
10    String subtitle,
11    String description,
12    List<String> authors,
13    String publisher,
14    String languageCode,
15    Integer pageCount,
16    List<String> categories,
17    String coverImageUrl,
18    String publicationDate,
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 19–32

```
19     String isbn10,
20     String isbn13,
21     String editionName,
22     boolean alreadyImported
23 ) {
24     public ExternalBookResult withAlreadyImported(boolean imported) {
25         return new ExternalBookResult(
26             source, sourceId, editionId, title, subtitle, description,
```

```
27         authors, publisher, languageCode, pageCount, categories,  
28         coverImageUrl, publicationDate, isbn10, isbn13, editionName,  
29         imported  
30     );  
31 }  
32 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/bookimport/dto/ ExternalBookSearchPage.java

File 26 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 13 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.bookimport.dto;
2
3 import java.util.List;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–13

```
5 public record ExternalBookSearchPage(
6     BookImportSource source,
7     List<ExternalBookResult> results,
8     int page,
9     int size,
10    long totalResults
11 ) {
12 }
13
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/bookimport/dto/ ExternalBookSearchQuery.java

File 27 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 9 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–9

```
1 package com.akshara.api.bookimport.dto;
2
3 public record ExternalBookSearchQuery(
4     String query,
5     int page,
6     int size
7 ) {
8 }
9
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/bookimport/dto/ManualBookRequest.java

File 28 of 99 · Added or modified · Hybrid catalogue import

Purpose: Defines a validated API boundary type used by the hybrid catalogue import module.

Current file length: 44 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.dto;
2
3 import com.akshara.api.book.entity.AvailabilityStatus;
4 import com.akshara.api.book.entity.BookFormat;
5 import jakarta.validation.constraints.DecimalMin;
6 import jakarta.validation.constraints.Digits;
7 import jakarta.validation.constraints.NotBlank;
8 import jakarta.validation.constraints.NotEmpty;
9 import jakarta.validation.constraints.NotNull;
10 import jakarta.validation.constraints.PastOrPresent;
11 import jakarta.validation.constraints.Pattern;
12 import jakarta.validation.constraints.Positive;
13 import jakarta.validation.constraints.PositiveOrZero;
14 import jakarta.validation.constraints.Size;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–19

```
15
16 import java.math.BigDecimal;
17 import java.time.LocalDate;
18 import java.util.List;
19
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 20–33

```
20 public record ManualBookRequest(
21     @NotBlank @Size(max = 255) String title,
22     @Size(max = 255) String subtitle,
23     @Size(max = 20000) String description,
24     @NotEmpty List<@NotBlank @Size(max = 150) String> authors,
25     @Size(max = 150) String publisher,
26     @Pattern(regexp = "^$|^([a-z]{2,3}(:-[-A-Z]{2})?)" @Size(max = 10)
27     String languageCode,
28     @Positive Integer pageCount,
29     @NotEmpty List<@NotBlank @Size(max = 100) String> categories,
```



```
30      @Pattern(regexp = "^$|^https?:/.+$.") @Size(max = 1000) String coverImageUrl,
31      @PastOrPresent LocalDate publicationDate,
32      @Size(max = 100) String editionName,
33      @Size(max = 20) String isbn10,
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 34–44

```
34      @Size(max = 25) String isbn13,
35      @NotNull BookFormat format,
36      @NotNull @DecimalMin("0.01") @Digits(integer = 8, fraction = 2) BigDecimal price,
37      @NotNull @PositiveOrZero Integer stockQuantity,
38      @NotNull AvailabilityStatus availabilityStatus,
39      @NotBlank @Size(max = 100) String sku,
40      @NotNull Boolean active,
41      @NotNull Boolean featured
42 ) implements CatalogueBookCommand {
43 }
44
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/main/java/com/akshara/api/bookimport/ exception/ExternalCatalogueException.java

File 29 of 99 · Added or modified · Hybrid catalogue import

Purpose: Supports hybrid catalogue import in the current Akshara implementation.

Current file length: 12 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–7

```
1 package com.akshara.api.bookimport.exception;
2
3 public class ExternalCatalogueException extends RuntimeException {
4     public ExternalCatalogueException(String message) {
5         super(message);
6     }
7 }
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 8–12

```
8     public ExternalCatalogueException(String message, Throwable cause) {
9         super(message, cause);
10    }
11 }
12 }
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

backend/src/main/java/com/akshara/api/bookimport/provider/BookMetadataProvider.java

File 30 of 99 · Added or modified · Hybrid catalogue import

Purpose: Supports hybrid catalogue import in the current Akshara implementation.

Current file length: 12 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 package com.akshara.api.bookimport.provider;
2
3 import com.akshara.api.bookimport.dto.BookImportSource;
4 import com.akshara.api.bookimport.dto.ExternalBookSearchPage;
5 import com.akshara.api.bookimport.dto.ExternalBookSearchQuery;
6
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 7–9

```
7 public interface BookMetadataProvider {
8     BookImportSource source();
9 }
```

How to read this block

- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.

Lines 10–12

```
10     ExternalBookSearchPage search(ExternalBookSearchQuery query);
11 }
12
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/bookimport/provider/ OpenLibraryBookMetadataProvider.java

File 31 of 99 · Added or modified · Hybrid catalogue import

Purpose: Builds the server-side Open Library HTTP request, identifies Akshara with a User-Agent, normalizes work/edition JSON, and converts provider failures into a stable application exception.

Current file length: 204 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.provider;
2
3 import com.akshara.api.bookimport.dto.BookImportSource;
4 import com.akshara.api.bookimport.dto.ExternalBookResult;
5 import com.akshara.api.bookimport.dto.ExternalBookSearchPage;
6 import com.akshara.api.bookimport.dto.ExternalBookSearchQuery;
7 import com.akshara.api.bookimport.exception.ExternalCatalogueException;
8 import org.springframework.beans.factory.annotation.Value;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.http.HttpHeaders;
11 import org.springframework.http.client.SimpleClientHttpRequestFactory;
12 import org.springframework.stereotype.Component;
13 import org.springframework.web.client.RestClient;
14 import org.springframework.web.client.RestClientException;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.

Lines 15–22

```
15 import tools.jackson.databind.JsonNode;
16
17 import java.util.ArrayList;
18 import java.util.LinkedHashSet;
19 import java.util.List;
20 import java.util.Set;
21 import java.time.Duration;
22
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 23–25

```
23 @Component
24 public class OpenLibraryBookMetadataProvider implements BookMetadataProvider {
25
```

How to read this block

- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.

Lines 26–30

```
26     private static final String RESULT_FIELDS =
27         "key,title,subtitle,author_name,publisher,language,subject,"
28         + "first_publish_year,isbn,cover_i,number_of_pages_median,"
29         + "edition_key,publish_date,editions";
30
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 31–42

```
31     private final RestClient client;
32
33     @Autowired
34     public OpenLibraryBookMetadataProvider(
35         @Value("${app.book-metadata.open-library.base-url:https://openlibrary.org}") String baseUrl,
36         @Value("${app.book-import.contact:admin@akshara.local}") String contact,
37         @Value("${app.book-metadata.open-library.connect-timeout-ms:5000}") int connectTimeout,
38         @Value("${app.book-metadata.open-library.read-timeout-ms:25000}") int readTimeout
39     ) {
40         this(buildClient(baseUrl, contact, connectTimeout, readTimeout));
41     }
42
```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 43–46

```
43     OpenLibraryBookMetadataProvider(RestClient client) {
44         this.client = client;
45     }
46
```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 47–60

```
47     private static RestClient buildClient(
48         String baseUrl,
49         String contact,
50         int connectTimeout,
51         int readTimeout
```



```

77         .queryParams("offset", offset)
78         .queryParams("fields", RESULT_FIELDS)
79         .build()
80     .retrieve()
81     .body(JsonNode.class);
82

```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The URI builder safely encodes the Open Library search term, pagination and requested metadata fields instead of concatenating an unsafe URL string.
- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 83–96

```

83     List<ExternalBookResult> results = new ArrayList<>();
84     for (JsonNode work : array(root, "docs")) {
85         JsonNode editions = work.path("editions").path("docs");
86         if (editions.isArray() && !editions.isEmpty()) {
87             for (JsonNode edition : editions) {
88                 results.add(mapResult(work, edition));
89             }
90         } else {
91             results.add(mapResult(work, work));
92         }
93     }
94     long total = root == null ? 0 : root.path("numFound").asLong(0);
95     return new ExternalBookSearchPage(
96         source(), List.copyOf(results), query.page(), query.size(), total

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 97–105

```

97         );
98     } catch (RestClientException exception) {
99         throw new ExternalCatalogueException(
100             "Open Library is temporarily unavailable. Please retry or use manual entry.",
101             exception
102         );
103     }
104 }
105

```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 106–119

```

106     private ExternalBookResult mapResult(JsonNode work, JsonNode edition) {
107         List<String> isbnns = strings(edition, "isbn");
108         if (isbnns.isEmpty()) isbnns = strings(work, "isbn");
109         String isbn13 = isbnns.stream().map(this::cleanIsbn)
110             .filter(value -> value.length() == 13).findFirst().orElse(null);
111         String isbn10 = isbnns.stream().map(this::cleanIsbn)
112             .filter(value -> value.length() == 10).findFirst().orElse(null);
113         String coverId = text(edition, "cover_i", text(work, "cover_i", null));
114         String cover = coverId == null ? null
115             : "https://covers.openlibrary.org/b/id/" + coverId + "-L.jpg";
116         String publishDate = firstText(edition, "publish_date");
117         if (publishDate == null) publishDate = firstText(work, "publish_date");
118         if (publishDate == null && work.has("first_publish_year")) {
119             publishDate = work.path("first_publish_year").asText();

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 120–133

```

120     }
121     return new ExternalBookResult(
122         source(), text(work, "key", null),
123         text(edition, "key", firstText(work, "edition_key")),
124         text(edition, "title", text(work, "title", "Untitled")),
125         text(edition, "subtitle", text(work, "subtitle", null)),
126         description(edition, work), preferStrings(edition, work, "author_name"),
127         firstPreferredText(edition, work, "publisher"),
128         firstPreferredText(edition, work, "language"),
129         positiveInt(edition, "number_of_pages",
130             positiveInt(work, "number_of_pages_median", null)),
131         limited(strings(work, "subject"), 12), cover,
132         normalizeDate(publishDate), isbn10, isbn13,
133         text(edition, "edition_name", null), false

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 134–136

```
134     );  
135     }  
136
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 137–141

```
137     private Iterable<JsonNode> array(JsonNode node, String field) {  
138         JsonNode value = node == null ? null : node.path(field);  
139         return value != null && value.isArray() ? value : List.of();  
140     }  
141
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 142–153

```
142     private List<String> strings(JsonNode node, String field) {  
143         JsonNode value = node.path(field);  
144         Set<String> values = new LinkedHashSet<>();  
145         if (value.isArray()) value.forEach(item -> {  
146             if (!item.asText().isBlank()) values.add(item.asText().trim());  
147         });  
148         else if (value.isTextual() && !value.asText().isBlank()) {  
149             values.add(value.asText().trim());  
150         }  
151         return List.copyOf(values);  
152     }  
153
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 154–158

```
154     private List<String> preferStrings(JsonNode primary, JsonNode fallback, String field) {  
155         List<String> values = strings(primary, field);  
156         return values.isEmpty() ? strings(fallback, field) : values;  
157     }  
158
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 159–163

```
159     private String firstPreferredText(JsonNode primary, JsonNode fallback, String field) {  
160         String value = firstText(primary, field);  
161         return value == null ? firstText(fallback, field) : value;  
162     }  
163
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 164–168

```
164     private String firstText(JsonNode node, String field) {  
165         List<String> values = strings(node, field);  
166         return values.isEmpty() ? null : values.get(0);  
167     }  
168
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 169–174

```
169     private String text(JsonNode node, String field, String fallback) {  
170         JsonNode value = node.path(field);  
171         return value.isValueNode() && !value.asText().isBlank()  
172             ? value.asText().trim() : fallback;  
173     }  
174
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 175–179

```
175     private Integer positiveInt(JsonNode node, String field, Integer fallback) {  
176         int value = node.path(field).asInt(0);  
177         return value > 0 ? Integer.valueOf(value) : fallback;  
178     }  
179
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 180–186

```
180     private String description(JsonNode primary, JsonNode fallback) {  
181         JsonNode value = primary.path("description");  
182         if (value.isMissingNode()) value = fallback.path("description");  
183         if (value.isObjectNode()) value = value.path("value");  
184         return value.isTextual() ? value.asText() : null;  
185     }  
186
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 187–190

```
187     private List<String> limited(List<String> values, int max) {
188         return values.size() <= max ? values : values.subList(0, max);
189     }
190
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 191–194

```
191     private String cleanIsbn(String value) {
192         return value == null ? "" : value.replaceAll("[-\s]", "").toUpperCase();
193     }
194
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 195–204

```
195     private String normalizeDate(String value) {
196         if (value == null || value.isBlank()) return null;
197         String trimmed = value.trim();
198         if (trimmed.matches("\\d{4}-\\d{2}-\\d{2}")) return trimmed;
199         if (trimmed.matches("\\d{4}-\\d{2}")) return trimmed + "-01";
200         if (trimmed.matches("\\d{4}")) return trimmed + "-01-01";
201         return null;
202     }
203 }
204
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates

or persists them.

backend/src/main/java/com/akshara/api/bookimport/service/BookImportService.java

File 32 of 99 · Added or modified · Hybrid catalogue import

Purpose: Converts reviewed metadata plus Akshara commerce fields into authors, categories, publisher, book, edition and inventory inside one transaction.

Current file length: 178 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.service;
2
3 import com.akshara.api.book.dto.BookEditionRequest;
4 import com.akshara.api.book.dto.BookRequest;
5 import com.akshara.api.book.dto.BookResponse;
6 import com.akshara.api.book.dto.InventoryRequest;
7 import com.akshara.api.book.entity.Author;
8 import com.akshara.api.book.entity.AvailabilityStatus;
9 import com.akshara.api.book.entity.Category;
10 import com.akshara.api.book.entity.Publisher;
11 import com.akshara.api.book.repository.AuthorRepository;
12 import com.akshara.api.book.repository.CategoryRepository;
13 import com.akshara.api.book.repository.PublisherRepository;
14 import com.akshara.api.book.service.BookService;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–21

```
15 import com.akshara.api.bookimport.dto.BookImportRequest;
16 import com.akshara.api.bookimport.dto.CatalogueBookCommand;
17 import com.akshara.api.bookimport.dto.ManualBookRequest;
18 import com.akshara.api.common.exception.InvalidRequestException;
19 import org.springframework.stereotype.Service;
20 import org.springframework.transaction.annotation.Transactional;
21
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 22–26

```
22 import java.text.Normalizer;
23 import java.util.LinkedHashSet;
24 import java.util.Locale;
```

```
25 import java.util.Set;
26
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 27–29

```
27 @Service
28 public class BookImportService {
29
```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.

Lines 30–34

```
30 private final BookService bookService;
31 private final AuthorRepository authorRepository;
32 private final CategoryRepository categoryRepository;
33 private final PublisherRepository publisherRepository;
34
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 35–46

```
35 public BookImportService(
36     BookService bookService,
37     AuthorRepository authorRepository,
38     CategoryRepository categoryRepository,
39     PublisherRepository publisherRepository
40 ) {
41     this.bookService = bookService;
42     this.authorRepository = authorRepository;
43     this.categoryRepository = categoryRepository;
44     this.publisherRepository = publisherRepository;
45 }
46
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 47–54

```
47     @Transactional
48     public BookResponse importBook(BookImportRequest request) {
49         return save(
50             request, request.source().name(), request.sourceId(),
51             request.editionId(), true
52         );
53     }
54
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 55–59

```
55     @Transactional
56     public BookResponse createManualBook(ManualBookRequest request) {
57         return save(request, null, null, null, false);
58     }
59
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 60–72

```
60     private BookResponse save(
61         CatalogueBookCommand request,
62         String metadataSource,
63         String externalWorkId,
64         String externalEditionId,
65         boolean isbnRequired
66     ) {
67         if (isbnRequired && blank(request.isbn10()) && blank(request.isbn13())) {
68             throw new InvalidRequestException(
69                 "An ISBN-10 or ISBN-13 is required to import an edition"
70             );
71         }
72
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 73–81

```
73     Set<Long> authorIds = new LinkedHashSet<>();
74     request.authors().stream()
75         .map(String::trim)
76         .filter(name -> !name.isBlank())
77         .distinct()
78         .map(this::findOrCreateAuthor)
79         .map(Author::getId)
80         .forEach(authorIds::add);
81
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 82–90

```
82     Set<Long> categoryIds = new LinkedHashSet<>();
83     request.categories().stream()
84         .map(String::trim)
85         .filter(name -> !name.isBlank())
86         .distinct()
87         .map(this::findOrCreateCategory)
88         .map(Category::getId)
89         .forEach(categoryIds::add);
90
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 91–94

```
91     Long publisherId = blank(request.publisher())
92         ? null
93         : findOrCreatePublisher(request.publisher().trim()).getId();
94
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 95–108

```
95     InventoryRequest inventory = new InventoryRequest(
96         request.price(),
97         request.stockQuantity(),
98         normalizedAvailability(
```

```

99             request.stockQuantity(), request.availabilityStatus()
100         ),
101         request.active()
102     );
103     BookEditionRequest edition = new BookEditionRequest(
104         publisherId,
105         request.format(),
106         request.editionName(),
107         request.isbn10(),
108         request.isbn13(),

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 109–122

```

109         request.publicationDate(),
110         request.pageCount(),
111         request.sku(),
112         inventory
113     );
114     BookRequest book = new BookRequest(
115         request.title(),
116         request.subtitle(),
117         request.description(),
118         request.coverImageUrl(),
119         request.languageCode(),
120         request.featured(),
121         authorIds,
122         categoryIds,

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 123–131

```

123         java.util.List.of(edition)
124     );
125     return metadataSource == null
126         ? bookService.create(book)
127         : bookService.createImported(
128             book, metadataSource, externalWorkId, externalEditionId
129         );
130 }
131

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 132–136

```

132     private Author findOrCreateAuthor(String name) {
133         return authorRepository.findByNameIgnoreCase(name)
134             .orElseGet(() -> authorRepository.save(new Author(name)));
135     }
136

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 137–141

```
137     private Publisher findOrCreatePublisher(String name) {
138         return publisherRepository.findByNameIgnoreCase(name)
139             .orElseGet(() -> publisherRepository.save(new Publisher(name)));
140     }
141
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 142–148

```
142     private Category findOrCreateCategory(String name) {
143         return categoryRepository.findByNameIgnoreCase(name)
144             .orElseGet(() -> categoryRepository.save(
145                 new Category(name, uniqueSlug(name))
146             ));
147     }
148
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 149–162

```
149     private String uniqueSlug(String name) {
150         String base = Normalizer.normalize(name, Normalizer.Form.NFD)
151             .replaceAll("\\p{M}", "")
152             .toLowerCase(Locale.ROOT)
153             .replaceAll("[^a-z0-9]+", "-");
154     }
155
```

```

154         .replaceAll("(^|-|$)", "");
155         if (base.isBlank()) base = "category";
156         if (base.length() > 110) base = base.substring(0, 110).replaceAll("-", "");
157         String candidate = base;
158         int suffix = 2;
159         while (categoryRepository.existsBySlugIgnoreCase(candidate)) {
160             candidate = base + "-" + suffix++;
161         }
162         return candidate;

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 163–168

```

163     }
164
165     private boolean blank(String value) {
166         return value == null || value.isBlank();
167     }
168

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 169–178

```

169     private AvailabilityStatus normalizedAvailability(
170         int stock,
171         AvailabilityStatus requested
172     ) {
173         return stock == 0 && requested == AvailabilityStatus.IN_STOCK
174             ? AvailabilityStatus.OUT_OF_STOCK
175             : requested;
176     }
177 }
178

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/main/java/com/akshara/api/bookimport/service/CatalogueSeedService.java

File 33 of 99 · Added or modified · Hybrid catalogue import

Purpose: Builds a deliberately curated multi-genre demonstration catalogue by searching Open Library and importing valid, non-duplicate ISBN editions.

Current file length: 219 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–13

```
1 package com.akshara.api.bookimport.service;
2
3 import com.akshara.api.book.entity.AvailabilityStatus;
4 import com.akshara.api.book.entity.BookFormat;
5 import com.akshara.api.book.repository.BookRepository;
6 import com.akshara.api.bookimport.dto.BookImportRequest;
7 import com.akshara.api.bookimport.dto.BookImportSource;
8 import com.akshara.api.bookimport.dto.CatalogueSeedResponse;
9 import com.akshara.api.bookimport.dto.ExternalBookResult;
10 import com.akshara.api.common.exception.DuplicateResourceException;
11 import com.akshara.api.common.exception.InvalidRequestException;
12 import org.springframework.stereotype.Service;
13
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 14–25

```
14 import java.math.BigDecimal;
15 import java.time.LocalDate;
16 import java.time.format.DateTimeParseException;
17 import java.util.ArrayList;
18 import java.util.HashSet;
19 import java.util.LinkedHashMap;
20 import java.util.LinkedHashSet;
21 import java.util.List;
22 import java.util.Locale;
23 import java.util.Map;
24 import java.util.Set;
25
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 26–28

```
26 @Service
27 public class CatalogueSeedService {
28
```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.

Lines 29–39

```
29     private static final List<CuratedSearch> CURATED_SEARCHES = List.of(
30         new CuratedSearch("Classics", "subject:classic literature language:eng"),
31         new CuratedSearch("Science", "subject:science language:eng"),
32         new CuratedSearch("History", "subject:history language:eng"),
33         new CuratedSearch("Philosophy", "subject:philosophy language:eng"),
34         new CuratedSearch("Business", "subject:business language:eng"),
35         new CuratedSearch("Biography", "subject:biography language:eng"),
36         new CuratedSearch("Indian Literature", "subject:india literature language:eng"),
37         new CuratedSearch("Children's", "subject:children language:eng")
38     );
39
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 40–43

```
40     private final ExternalBookSearchService searchService;
41     private final BookImportService importService;
42     private final BookRepository bookRepository;
43
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 44–53

```
44     public CatalogueSeedService(
45         ExternalBookSearchService searchService,
46         BookImportService importService,
47         BookRepository bookRepository
48     ) {
49         this.searchService = searchService;
50         this.importService = importService;
51         this.bookRepository = bookRepository;
52     }
53
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC

in the business layer.

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 54–64

```
54     public CatalogueSeedResponse seed(int requested) {
55         long before = bookRepository.count();
56         int imported = 0;
57         int skipped = 0;
58         Set<String> attemptedWorks = new HashSet<>();
59         List<String> importedTitles = new ArrayList<>();
60         int perGenreTarget = (int) Math.ceil(
61             requested / (double) CURATED_SEARCHES.size()
62         );
63         Map<String, Integer> importedByGenre = new LinkedHashMap<>();
64     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 65–74

```
65         for (int page = 1; page <= 5 && imported < requested; page++) {
66             for (CuratedSearch search : CURATED_SEARCHES) {
67                 if (imported >= requested) break;
68                 if (importedByGenre.getOrDefault(search.genre(), 0) >= perGenreTarget) {
69                     continue;
70                 }
71                 List<ExternalBookResult> results = searchService.search(
72                     search.query(), BookImportSource.OPEN_LIBRARY, page, 20
73                 ).results();
74             }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 75–88

```
75         for (ExternalBookResult result : results) {
76             if (imported >= requested
77                 || importedByGenre.getOrDefault(search.genre(), 0)
78                 >= perGenreTarget) break;
79             String identity = identity(result);
80             if (identity == null
81                 || !attemptedWorks.add(identity))
```



```

82             || result.alreadyImported()
83             || !hasIsbn(result)) {
84                 skipped++;
85                 continue;
86             }
87         try {
88             importService.importBook(toRequest(result, imported, search.genre()));

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 89–100

```

89             importedTitles.add(result.title());
90             imported++;
91             importedByGenre.merge(search.genre(), 1, Integer::sum);
92         } catch (DuplicateResourceException | InvalidRequestException exception) {
93             skipped++;
94         } catch (RuntimeException exception) {
95             skipped++;
96         }
97     }
98 }
99 }
100

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.

Lines 101–107

```

101         return new CatalogueSeedResponse(
102             requested, imported, skipped, before,
103             bookRepository.count(), Map.copyOf(importedByGenre),
104             List.copyOf(importedTitles)
105         );
106     }
107

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 108–121

```

108     private BookImportRequest toRequest(
109         ExternalBookResult result,
110         int index,
111         String genre
112     ) {
113         String isbn = cleanIsbn(result.isbn13()) != null

```

```

114         ? result.isbn13() : result.isbn10());
115     List<String> authors = sanitizedList(result.authors(), 150, 4);
116     if (authors.isEmpty()) authors = List.of("Unknown Author");
117     List<String> categories = categories(genre, result.categories());
118     int priceBand = Math.floorMod(isbn.hashCode(), 6);
119     BigDecimal price = BigDecimal.valueOf(249L + priceBand * 50L)
120         .setScale(2);
121     int stock = 8 + Math.floorMod(isbn.hashCode(), 18);

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 122–135

```

122
123     return new BookImportRequest(
124         BookImportSource.OPEN_LIBRARY,
125         limited(result.sourceId(), 255),
126         limited(result.editionId(), 255),
127         requiredText(result.title(), "Untitled", 255),
128         limited(result.subtitle(), 255),
129         limited(result.description(), 20000),
130         authors,
131         limited(result.publisher(), 150),
132         language(result.languageCode()),
133         result.pageCount() != null && result.pageCount() > 0
134             ? result.pageCount() : null,
135         categories,

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 136–149

```

136         limited(result.coverImageUrl(), 1000),
137         date(result.publicationDate()),
138         limited(result.editionName(), 100),
139         cleanOptionalIsbn(result.isbn10()),
140         cleanOptionalIsbn(result.isbn13()),
141         BookFormat.PAPERBACK,
142         price,
143         stock,
144         AvailabilityStatus.IN_STOCK,
145         "AKS-SEED-" + isbn,
146         true,
147         index < 12
148     );
149 }

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 150–157

```
150
151     private List<String> categories(String genre, List<String> sourceCategories) {
152         Set<String> categories = new LinkedHashSet<>();
153         categories.add(genre);
154         categories.addAll(sanitizedList(sourceCategories, 100, 3));
155         return List.copyOf(categories);
156     }
157
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 158–165

```
158     private String identity(ExternalBookResult result) {
159         if (result.sourceId() != null && !result.sourceId().isBlank()) {
160             return result.sourceId().trim();
161         }
162         String isbn = result.isbn13() != null ? result.isbn13() : result.isbn10();
163         return isbn == null || isbn.isBlank() ? null : cleanIsbn(isbn);
164     }
165
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 166–170

```
166     private boolean hasIsbn(ExternalBookResult result) {
167         return (result.isbn13() != null && !result.isbn13().isBlank())
168             || (result.isbn10() != null && !result.isbn10().isBlank());
169     }
170
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 171–181

```

171     private List<String> sanitizedList(List<String> values, int maxLength, int maxItems) {
172         if (values == null) return List.of();
173         Set<String> sanitized = new LinkedHashSet<>();
174         for (String value : values) {
175             String text = limited(value, maxLength);
176             if (text != null && !text.isBlank()) sanitized.add(text);
177             if (sanitized.size() == maxItems) break;
178         }
179         return List.copyOf(sanitized);
180     }
181 
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 182–186

```

182     private String requiredText(String value, String fallback, int maxLength) {
183         String text = value == null || value.isBlank() ? fallback : value.trim();
184         return text.length() <= maxLength ? text : text.substring(0, maxLength);
185     }
186 
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 187–192

```
187     private String limited(String value, int maxLength) {
188         if (value == null || value.isBlank()) return null;
189         String text = value.trim();
190         return text.length() <= maxLength ? text : text.substring(0, maxLength);
191     }
192
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 193–198

```
193     private String language(String value) {
194         if (value == null) return "en";
195         String normalized = value.trim().toLowerCase(Locale.ROOT);
196         return normalized.matches("[a-z]{2,3}$") ? normalized : "en";
197     }
198
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 199–207

```
199     private LocalDate date(String value) {
200         if (value == null || value.isBlank()) return null;
201         try {
202             return LocalDate.parse(value);
203         } catch (DateTimeParseException exception) {
204             return null;
205         }
206     }
207
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 208–211

```
208     private String cleanOptionalIsbn(String value) {
209         return value == null || value.isBlank() ? null : cleanIsbn(value);
210     }
211
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 212–215

```
212     private String cleanIsbn(String value) {
213         return value.replaceAll("[-\s]", "").toUpperCase(Locale.ROOT);
214     }
215
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 216–219

```
216     private record CuratedSearch(String genre, String query) {
217     }
218 }
219
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

backend/src/main/java/com/akshara/api/bookimport/service/ExternalBookSearchService.java

File 34 of 99 · Added or modified · Hybrid catalogue import

Purpose: Chooses a metadata provider by source and annotates every external result with duplicate-ISBN information from MySQL.

Current file length: 68 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–11

```
1 package com.akshara.api.bookimport.service;
2
3 import com.akshara.api.book.repository.BookEditionRepository;
4 import com.akshara.api.bookimport.dto.BookImportSource;
5 import com.akshara.api.bookimport.dto.ExternalBookResult;
6 import com.akshara.api.bookimport.dto.ExternalBookSearchPage;
7 import com.akshara.api.bookimport.dto.ExternalBookSearchQuery;
8 import com.akshara.api.bookimport.provider.BookMetadataProvider;
9 import com.akshara.api.common.exception.InvalidRequestException;
10 import org.springframework.stereotype.Service;
11
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 12–17

```
12 import java.util.EnumMap;
13 import java.util.List;
14 import java.util.Map;
15 import java.util.function.Function;
16 import java.util.stream.Collectors;
17
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 18–20

```
18 @Service
19 public class ExternalBookSearchService {
20
```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.

Lines 21–23

```
21     private final Map<BookImportSource, BookMetadataProvider> providers;
22     private final BookEditionRepository editionRepository;
23
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 24–36

```
24     public ExternalBookSearchService(
25         List<BookMetadataProvider> providers,
26         BookEditionRepository editionRepository
27     ) {
28         this.providers = providers.stream().collect(Collectors.toMap(
29             BookMetadataProvider::source,
30             Function.identity(),
31             (first, ignored) -> first,
32             () -> new EnumMap<>(BookImportSource.class)
33         ));
34         this.editionRepository = editionRepository;
35     }
36
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 37–50

```
37     public ExternalBookSearchPage search(
38         String query,
39         BookImportSource source,
40         int page,
41         int size
42     ) {
43         BookMetadataProvider provider = providers.get(source);
44         if (provider == null) {
45             throw new InvalidRequestException(
46                 source + " is not configured for this Akshara installation"
47             );
48         }
49         ExternalBookSearchPage result = provider.search(
50             new ExternalBookSearchQuery(query, page, size)
51         );
52     }
53
```


How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 51–60

```
51         );
52         List<ExternalBookResult> annotated = result.results().stream()
53             .map(item -> item.withAlreadyImported(alreadyImported(item)))
54             .toList();
55         return new ExternalBookSearchPage(
56             result.source(), annotated, result.page(), result.size(),
57             result.totalResults()
58         );
59     }
60 }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 61–68

```
61     private boolean alreadyImported(ExternalBookResult result) {
62         return (result.isbn10() != null
63             && editionRepository.existsByIsbn10(result.isbn10()))
64             || (result.isbn13() != null
65             && editionRepository.existsByIsbn13(result.isbn13()));
66     }
67 }
68 }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

backend/src/main/java/com/akshara/api/cart/repository/

CartItemRepository.java

File 35 of 99 · Added or modified · Cart

Purpose: Defines Spring Data persistence queries required by the cart module.

Current file length: 35 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.cart.repository;
2
3 import com.akshara.api.cart.entity.CartItem;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 6–8

```
6 import java.util.List;
7 import java.util.Optional;
8
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 9–11

```
9 public interface CartItemRepository
10     extends JpaRepository<CartItem, Long> {
11
```

How to read this block

- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 12–15

```
12     List<CartItem> findAllByCart_IdOrderByCreatedAtAsc(
```

```
13         Long cartId
14     );
15
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 16–20

```
16     Optional<CartItem> findByCart_IdAndBookEdition_Id(
17         Long cartId,
18         Long bookEditionId
19     );
20
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 21–25

```
21     Optional<CartItem> findByIdAndCart_Id(
22         Long itemId,
23         Long cartId
24     );
25
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 26–30

```
26     boolean existsByCart_IdAndBookEdition_Id(
27         Long cartId,
28         Long bookEditionId
29     );
30
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 31–35

```
31     void deleteAllByCart_Id(Long cartId);
32
33     void deleteAllByBookEdition_Book_Id(Long bookId);
34 }
35
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/cart/service/

CartService.java

File 36 of 99 · Added or modified · Cart

Purpose: Contains business rules and orchestration for the cart module.

Current file length: 321 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.cart.service;
2
3 import com.akshara.api.auth.exception.InvalidAccessTokenException;
4 import com.akshara.api.book.entity.AvailabilityStatus;
5 import com.akshara.api.book.entity.Book;
6 import com.akshara.api.book.entity.BookEdition;
7 import com.akshara.api.book.entity.Inventory;
8 import com.akshara.api.book.entity.Publisher;
9 import com.akshara.api.book.repository.BookEditionRepository;
10 import com.akshara.api.cart.dto.AddCartItemRequest;
11 import com.akshara.api.cart.dto.CartItemResponse;
12 import com.akshara.api.cart.dto.CartResponse;
13 import com.akshara.api.cart.dto.UpdateCartItemRequest;
14 import com.akshara.api.cart.entity.Cart;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–25

```
15 import com.akshara.api.cart.entity.CartItem;
16 import com.akshara.api.cart.repository.CartItemRepository;
17 import com.akshara.api.cart.repository.CartRepository;
18 import com.akshara.api.common.exception.InvalidRequestException;
19 import com.akshara.api.common.exception.ResourceNotFoundException;
20 import com.akshara.api.user.entity.AppUser;
21 import com.akshara.api.user.repository.UserRepository;
22 import org.springframework.stereotype.Service;
23 import org.springframework.transaction.annotation.Transactional;
24 import com.akshara.api.book.repository.InventoryRepository;
25
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 26–28

```
26 import java.math.BigDecimal;
27 import java.util.List;
28
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 29–32

```
29 @Service
30 @Transactional
31 public class CartService {
32
```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.
- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.

Lines 33–38

```
33 private final CartRepository cartRepository;
34 private final CartItemRepository cartItemRepository;
35 private final UserRepository userRepository;
36 private final BookEditionRepository bookEditionRepository;
37 private final InventoryRepository inventoryRepository;
38
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 39–52

```
39 public CartService(
40     CartRepository cartRepository,
41     CartItemRepository cartItemRepository,
42     UserRepository userRepository,
43     BookEditionRepository bookEditionRepository,
44     InventoryRepository inventoryRepository
45 ) {
46     this.cartRepository = cartRepository;
47     this.cartItemRepository = cartItemRepository;
48     this.userRepository = userRepository;
49     this.bookEditionRepository = bookEditionRepository;
50     this.inventoryRepository = inventoryRepository;
51 }
52
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC

in the business layer.

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 53–56

```
53     public CartResponse getCart(String subject) {  
54         AppUser user = getAuthenticatedUser(subject);  
55         Cart cart = getOrCreateCart(user);  
56     }
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 57–59

```
57         return toResponse(cart);  
58     }  
59 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 60–66

```
60     public CartResponse addItem(  
61         String subject,  
62         AddCartItemRequest request  
63     ) {  
64         AppUser user = getAuthenticatedUser(subject);  
65         Cart cart = getOrCreateCart(user);  
66     }
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 67–74

```
67         BookEdition edition = bookEditionRepository  
68             .findById(request.bookEditionId())  
69             .orElseThrow(() -> new ResourceNotFoundException(  
70                 "Book edition with ID "  
71                     + request.bookEditionId()  
72                     + " was not found"  
73             ));  
74     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 75–80

```
75         if (edition.getFormat().isDigital()) {
76             throw new InvalidRequestException(
77                 "Digital editions are not available for purchase yet"
78             );
79         }
80     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 81–87

```
81         CartItem item = cartItemRepository
82             .findByCart_IdAndBookEdition_Id(
83                 cart.getId(),
84                 edition.getId()
85             )
86             .orElse(null);
87     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 88–91

```
88         int existingQuantity = item == null
89             ? 0
90             : item.getQuantity();
91     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 92–94

```
92         long requestedTotal =
93             (long) existingQuantity + request.quantity();
94     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 95–100

```
95         if (requestedTotal > Integer.MAX_VALUE) {
96             throw new InvalidRequestException(
97                 "Requested cart quantity is too large"
98             );
99         }
100     }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 101–107

```
101      Inventory inventory = getInventory(edition.getId());
102
103      validatePurchasableQuantity(
104          inventory,
105          (int) requestedTotal
106      );
107
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 108–117

```
108      if (item == null) {
109          item = new CartItem(
110              cart,
111              edition,
112              request.quantity()
113          );
114      } else {
115          item.setQuantity((int) requestedTotal);
116      }
117
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 118–120

```
118      cartItemRepository.save(item);
119      touchCart(cart);
120
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 121–123

```
121      return toResponse(cart);
122  }
123
```


How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 124–131

```
124     public CartResponse updateItem(  
125         String subject,  
126         Long itemId,  
127         UpdateCartItemRequest request  
128     ) {  
129         AppUser user = getAuthenticatedUser(subject);  
130         Cart cart = getOrCreateCart(user);  
131     }
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 132–137

```
132         CartItem item = findOwnedItem(cart, itemId);  
133  
134         Inventory inventory = getInventory(  
135             item.getBookEdition().getId()  
136         );  
137
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 138–142

```
138         validatePurchasableQuantity(  
139             inventory,  
140             request.quantity()  
141         );  
142
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 143–147

```
143         item.setQuantity(request.quantity());  
144  
145         cartItemRepository.save(item);  
146         touchCart(cart);  
147
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 148–150

```
148         return toResponse(cart);
149     }
150
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 151–157

```
151     public CartResponse removeItem(
152         String subject,
153         Long itemId
154     ) {
155         AppUser user = getAuthenticatedUser(subject);
156         Cart cart = getOrCreateCart(user);
157
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 158–162

```
158         CartItem item = findOwnedItem(cart, itemId);
159
160         cartItemRepository.delete(item);
161         touchCart(cart);
162
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 163–165

```
163         return toResponse(cart);
164     }
165
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 166–169

```
166     public CartResponse clearCart(String subject) {
167         AppUser user = getAuthenticatedUser(subject);
168         Cart cart = getOrCreateCart(user);
169
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 170–172

```
170         cartItemRepository.deleteAllByCart_Id(cart.getId());
171         touchCart(cart);
172
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 173–175

```
173         return toResponse(cart);
174     }
175
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 176–178

```
176     private AppUser getAuthenticatedUser(String subject) {
177         Long userId;
178
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 179–184

```
179         try {
180             userId = Long.parseLong(subject);
181         } catch (NumberFormatException exception) {
182             throw new InvalidAccessTokenException();
183         }
184
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 185–189

```
185         return userRepository.findById(userId)
186             .filter(AppUser::isEnabled)
187             .orElseThrow(InvalidAccessTokenException::new);
188     }
189
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 190–196

```
190     private Cart getOrCreateCart(AppUser user) {
191         return cartRepository.findById(user.getId())
192             .orElseGet(() ->
193                 cartRepository.save(new Cart(user))
194             );
195     }
196
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 197–209

```
197     private CartItem findOwnedItem(
198         Cart cart,
199         Long itemId
200     ) {
201         return cartItemRepository
202             .findByIdAndCart_Id(itemId, cart.getId())
203             .orElseThrow(() -> new ResourceNotFoundException(
204                 "Cart item with ID "
205                     + itemId
206                     + " was not found"
207             ));
208     }
209
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 210–219

```
210     private Inventory getInventory(Long editionId) {
211         return inventoryRepository
212             .findByBookEdition_Id(editionId)
213             .orElseThrow(() -> new ResourceNotFoundException(
214                 "Inventory for book edition with ID "
215                     + editionId
216                     + " was not found"
217             ));
218     }
219
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 220–228

```
220     private void validatePurchasableQuantity(
221         Inventory inventory,
222         int quantity
223     ) {
224         if (!inventory.isActive()
225             || inventory.getAvailabilityStatus()
226                 != AvailabilityStatus.IN_STOCK
227             || inventory.getStockQuantity() <= 0) {
228
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 229–233

```
229         throw new InvalidRequestException(
230             "This book edition is not currently available"
231         );
232     }
233
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 234–241

```
234         if (quantity > inventory.getStockQuantity()) {
235             throw new InvalidRequestException(
236                 "Requested quantity exceeds the available stock of "
237                 + inventory.getStockQuantity()
238             );
239         }
240     }
241 }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 242–246

```
242     private void touchCart(Cart cart) {
243         cart.markUpdated();
244         cartRepository.save(cart);
245     }
246 }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 247–255

```
247     private CartResponse toResponse(Cart cart) {
248         List<CartItemResponse> items = cartItemRepository
249             .findAllByCart_IdOrderByCreatedAtAsc(
250                 cart.getId()
251             )
252             .stream()
253             .map(this::toItemResponse)
254             .toList();
255     }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and

makes dependencies explicit.

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 256–259

```
256         int totalQuantity = items.stream()
257             .mapToInt(CartItemResponse::quantity)
258             .sum();
259
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 260–266

```
260         BigDecimal totalAmount = items.stream()
261             .map(CartItemResponse::subtotal)
262             .reduce(
263                 BigDecimal.ZERO.setScale(2),
264                 BigDecimal::add
265             );
266
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 267–276

```
267         return new CartResponse(
268             cart.getId(),
269             items,
270             items.size(),
271             totalQuantity,
272             totalAmount,
273             cart.getUpdatedAt()
274         );
275     }
276
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 277–282

```
277     private CartItemResponse toItemResponse(CartItem item) {
278         BookEdition edition = item.getBookEdition();
279         Book book = edition.getBook();
280         Publisher publisher = edition.getPublisher();
281         Inventory inventory = getInventory(edition.getId());
282     }
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 283–287

```
283         BigDecimal subtotal = inventory.getPrice()  
284             .multiply(  
285                 BigDecimal.valueOf(item.getQuantity())  
286             );  
287
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 288–294

```
288         boolean availableForPurchase =  
289             inventory.isActive()  
290                 && inventory.getAvailabilityStatus()  
291                 == AvailabilityStatus.IN_STOCK  
292                 && inventory.getStockQuantity()  
293                 >= item.getQuantity();  
294
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 295–297

```
295         return new CartItemResponse(  
296             item.getId(),  
297
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 298–301

```
298             book.getId(),  
299             book.getTitle(),  
300             book.getCoverImageUrl(),  
301
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 302–310

```
302             edition.getId(),  
303             edition.getFormat(),
```



```

304         edition.getEditionName(),
305         edition.getIsbn10(),
306         edition.getIsbn13(),
307         publisher == null
308             ? null
309             : publisher.getName(),
310

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 311–314

```

311         inventory.getPrice(),
312         item.getQuantity(),
313         subtotal,
314

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 315–321

```

315         inventory.getStockQuantity(),
316         inventory.getAvailabilityStatus(),
317         availableForPurchase
318     );
319 }
320 }
321

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/main/java/com/akshara/api/common/error/GlobalExceptionHandler.java

File 37 of 99 · Added or modified · Errors and shared infrastructure

Purpose: Translates domain, validation, malformed JSON, storage and provider exceptions into predictable HTTP error responses.

Current file length: 278 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.common.error;
2
3 import com.akshara.api.common.exception.InvalidRequestException;
4 import org.springframework.web.method.annotation.HandlerMethodValidationException;
5 import com.akshara.api.auth.exception.EmailAlreadyExistsException;
6 import com.akshara.api.auth.exception.InvalidAccessTokenException;
7 import com.akshara.api.auth.exception.InvalidCredentialsException;
8 import com.akshara.api.common.exception.DuplicateResourceException;
9 import com.akshara.api.common.exception.ResourceNotFoundException;
10 import com.akshara.api.bookimport.exception.ExternalCatalogueException;
11 import com.akshara.api.library.exception.PersonalLibraryStorageException;
12 import jakarta.servlet.http.HttpServletRequest;
13 import org.springframework.dao.DataIntegrityViolationException;
14 import org.springframework.http.HttpStatus;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–23

```
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.http.converter.HttpMessageNotReadableException;
17 import org.springframework.validation.FieldError;
18 import org.springframework.web.bind.MethodArgumentNotValidException;
19 import org.springframework.web.multipart.MaxUploadSizeExceededException;
20 import org.springframework.web.bind.annotation.ExceptionHandler;
21 import org.springframework.web.bind.annotation.RestControllerAdvice;
22 import org.springframework.validation.method.ParameterErrors;
23
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 24–27

```
24 import java.time.Instant;
25 import java.util.LinkedHashMap;
26 import java.util.Map;
27
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 28–31

```
28
29 @RestControllerAdvice
30 public class GlobalExceptionHandler {
31
```

How to read this block

- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.

Lines 32–44

```
32 @ExceptionHandler(MaxUploadSizeExceededException.class)
33 public ResponseEntity<ApiErrorResponse> handleMaximumUploadSize(
34     MaxUploadSizeExceededException exception,
35     HttpServletRequest request
36 ) {
37     return buildResponse(
38         HttpStatus.PAYLOAD_TOO_LARGE,
39         "Personal book files must be 25 MB or smaller",
40         request.getRequestURI(),
41         Map.of()
42     );
43 }
44
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 45–57

```
45 @ExceptionHandler(PersonalLibraryStorageException.class)
46 public ResponseEntity<ApiErrorResponse> handlePersonalLibraryStorage(
47     PersonalLibraryStorageException exception,
48     HttpServletRequest request
49 ) {
50     return buildResponse(
51         HttpStatus.INTERNAL_SERVER_ERROR,
52         exception.getMessage(),
53         request.getRequestURI(),
54         Map.of()
55     );
56 }
57
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 58–70

```

58     @ExceptionHandler(ExternalCatalogueException.class)
59     public ResponseEntity<ApiErrorResponse> handleExternalCatalogue(
60         ExternalCatalogueException exception,
61         HttpServletRequest request
62     ) {
63         return buildResponse(
64             HttpStatus.SERVICE_UNAVAILABLE,
65             exception.getMessage(),
66             request.getRequestURI(),
67             Map.of()
68         );
69     }
70

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 71–83

```

71     @ExceptionHandler(EmailAlreadyExistsException.class)
72     public ResponseEntity<ApiErrorResponse> handleDuplicateEmail(
73         EmailAlreadyExistsException exception,
74         HttpServletRequest request
75     ) {
76         return buildResponse(
77             HttpStatus.CONFLICT,
78             exception.getMessage(),
79             request.getRequestURI(),
80             Map.of()
81         );
82     }
83

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary

strings.

Lines 84–96

```
84     @ExceptionHandler(InvalidCredentialsException.class)
85     public ResponseEntity<ApiErrorResponse> handleInvalidCredentials(
86         InvalidCredentialsException exception,
87         HttpServletRequest request
88     ) {
89         return buildResponse(
90             HttpStatus.UNAUTHORIZED,
91             exception.getMessage(),
92             request.getRequestURI(),
93             Map.of()
94         );
95     }
96
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 97–109

```
97     @ExceptionHandler(InvalidAccessTokenException.class)
98     public ResponseEntity<ApiErrorResponse> handleInvalidAccessToken(
99         InvalidAccessTokenException exception,
100         HttpServletRequest request
101     ) {
102         return buildResponse(
103             HttpStatus.UNAUTHORIZED,
104             exception.getMessage(),
105             request.getRequestURI(),
106             Map.of()
107         );
108     }
109
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 110–122

```
110     @ExceptionHandler(ResourceNotFoundException.class)
111     public ResponseEntity<ApiErrorResponse> handleResourceNotFound(
```

```

112         ResourceNotFoundException exception,
113         HttpServletRequest request
114     ) {
115         return buildResponse(
116             HttpStatus.NOT_FOUND,
117             exception.getMessage(),
118             request.getRequestURI(),
119             Map.of()
120         );
121     }
122

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 123–135

```

123     @ExceptionHandler(DuplicateResourceException.class)
124     public ResponseEntity<ApiErrorResponse> handleDuplicateResource(
125         DuplicateResourceException exception,
126         HttpServletRequest request
127     ) {
128         return buildResponse(
129             HttpStatus.CONFLICT,
130             exception.getMessage(),
131             request.getRequestURI(),
132             Map.of()
133         );
134     }
135

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 136–142

```

136     @ExceptionHandler(MethodArgumentNotValidException.class)
137     public ResponseEntity<ApiErrorResponse> handleValidation(
138         MethodArgumentNotValidException exception,
139         HttpServletRequest request
140     ) {
141         Map<String, String> fieldErrors = new LinkedHashMap<>();
142

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 143–145

```
143         for (FieldError error
144             : exception.getBindingResult().getFieldErrors()) {
145
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 146–151

```
146             fieldErrors.putIfAbsent(
147                 error.getField(),
148                 error.getDefaultMessage()
149             );
150         }
151
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 152–159

```
152         return buildResponse(
153             HttpStatus.BAD_REQUEST,
154             "Request validation failed",
155             request.getRequestURI(),
156             fieldErrors
157         );
158     }
159
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 160–172

```
160     @ExceptionHandler(HttpMessageNotReadableException.class)
161     public ResponseEntity<ApiErrorResponse> handleUnreadableRequest(
162         HttpMessageNotReadableException exception,
163         HttpServletRequest request
164     ) {
165         return buildResponse(
166             HttpStatus.BAD_REQUEST,
167             "Request body is missing or contains invalid JSON",
168             request.getRequestURI(),
169             Map.of()
170         );
171     }
172
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 173–185

```
173     @ExceptionHandler(DataIntegrityViolationException.class)
174     public ResponseEntity<ApiErrorResponse> handleDataIntegrityViolation(
175         DataIntegrityViolationException exception,
176         HttpServletRequest request
177     ) {
178         return buildResponse(
179             HttpStatus.CONFLICT,
180             "The request conflicts with existing or related data",
181             request.getRequestURI(),
182             Map.of()
183         );
184     }
185
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 186–192

```
186     @ExceptionHandler(HandlerMethodValidationException.class)
187     public ResponseEntity<ApiErrorResponse> handleMethodValidation(
188         HandlerMethodValidationException exception,
189         HttpServletRequest request
190     ) {
191         Map<String, String> fieldErrors = new LinkedHashMap<>();
192
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 193–198

```
193         exception.getParameterValidationResults().forEach(result -> {
```



```

194
195         // Extract fields inside an @Valid request DTO
196         if (result instanceof ParameterErrors parameterErrors
197             && !parameterErrors.getFieldErrors().isEmpty()) {
198

```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 199–205

```

199             parameterErrors.getFieldErrors().forEach(error ->
200                 fieldErrors.putIfAbsent(
201                     error.getField(),
202                     error.getDefaultMessage()
203                 )
204             );
205

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 206–208

```

206         return;
207     }
208

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 209–213

```

209         // Handle direct constraints such as @Positive on orderId
210         String parameterName = result
211             .getMethodParameter()
212             .getParameterName();
213

```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

Lines 214–217

```

214         String fieldName = parameterName == null
215             ? "parameter"
216             : parameterName;
217

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 218–226

```
218         result.getResolvableErrors()
219             .stream()
220             .findFirst()
221             .ifPresent(error -> fieldErrors.putIfAbsent(
222                 fieldName,
223                 error.getDefaultMessage()
224             ));
225     });
226
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 227–240

```
227         return buildResponse(
228             HttpStatus.BAD_REQUEST,
229             "Request validation failed",
230             request.getRequestURI(),
231             fieldErrors
232         );
233     }
234     @ExceptionHandler(Exception.class)
235     public ResponseEntity<ApiErrorResponse> handleUnexpectedException(
236         Exception exception,
237         HttpServletRequest request
238     ) {
239         return buildResponse(
240             HttpStatus.INTERNAL_SERVER_ERROR,
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 241–246

```
241         "An unexpected server error occurred",
242         request.getRequestURI(),
243         Map.of()
244     );
245     }
246
```

How to read this block

- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 247–259

```
247     @ExceptionHandler(InvalidRequestException.class)
248     public ResponseEntity<ApiErrorResponse> handleInvalidRequest(
249         InvalidRequestException exception,
250         HttpServletRequest request
251     ) {
252         return buildResponse(
253             HttpStatus.BAD_REQUEST,
254             exception.getMessage(),
255             request.getRequestURI(),
256             Map.of()
257         );
258     }
259
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 260–273

```
260     private ResponseEntity<ApiErrorResponse> buildResponse(
261         HttpStatus status,
262         String message,
263         String path,
264         Map<String, String> fieldErrors
265     ) {
266         ApiErrorResponse response = new ApiErrorResponse(
267             Instant.now(),
268             status.value(),
269             status.getReasonPhrase(),
270             message,
271             path,
272             fieldErrors
273         );
274
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 274–278

```
274
275         return ResponseEntity.status(status).body(response);
276     }
277 }
278
```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

backend/src/main/java/com/akshara/api/config/SecurityConfig.java

File 38 of 99 · Added or modified · Security and configuration

Purpose: Defines stateless JWT security and separates public, Reader and Admin endpoint families.

Current file length: 82 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–13

```
1 package com.akshara.api.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6 import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
7 import org.springframework.security.config.http.SessionCreationPolicy;
8 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
9 import org.springframework.security.crypto.password.PasswordEncoder;
10 import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationConverter;
11 import org.springframework.security.oauth2.server.resource.authentication.JwtGrantedAuthoritiesConverter;
12 import org.springframework.security.web.SecurityFilterChain;
13
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 14–16

```
14 @Configuration
15 public class SecurityConfig {
16
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 17–21

```
17     @Bean
18     public PasswordEncoder passwordEncoder() {
19         return new BCryptPasswordEncoder();
20     }
21
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 22–26

```
22     @Bean
23     public JwtAuthenticationConverter jwtAuthenticationConverter() {
24         JwtGrantedAuthoritiesConverter authoritiesConverter =
25             new JwtGrantedAuthoritiesConverter();
26     }
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 27–29

```
27         authoritiesConverter.setAuthoritiesClaimName("role");
28         authoritiesConverter.setAuthorityPrefix("ROLE_");
29     }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 30–32

```
30         JwtAuthenticationConverter authenticationConverter =
31             new JwtAuthenticationConverter();
32     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 33–36

```
33         authenticationConverter.setJwtGrantedAuthoritiesConverter(
34             authoritiesConverter
35         );
36     }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 37–39

```
37         return authenticationConverter;
38     }
39 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 40–45

```
40     @Bean
41     public SecurityFilterChain securityFilterChain(
42         HttpSecurity http,
43         JwtAuthenticationConverter jwtAuthenticationConverter
44     ) throws Exception {
45 
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 46–59

```
46         return http
47             .csrf(AbstractHttpConfigurer::disable)
48             .sessionManagement(session -> session
49                 .sessionCreationPolicy(
50                     SessionCreationPolicy.STATELESS
51                 ))
52             .authorizeHttpRequests(authorize -> authorize
53                 .requestMatchers(
54                     "/api/public/**",
55                     "/api/auth/**",
56                     "/error"
57                 ).permitAll()
58                 .requestMatchers("/api/admin/**")
59                 .hasRole("ADMIN")
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 60–73

```
60             .requestMatchers(
61                 "/api/cart/**",
62                 "/api/wishlist/**",
63                 "/api/reviews/**",
64                 "/api/checkout",
65                 "/api/checkout/**",
66                 "/api/library",
67                 "/api/library/**",
68                 "/api/orders",
69                 "/api/orders/**"
70             )
71             .hasRole("READER")
72             .anyRequest()
73             .authenticated()
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 74–82

```
74             .oauth2ResourceServer(oauth2 -> oauth2
75                 .jwt(jwt -> jwt
76                     .jwtAuthenticationConverter(
77                         jwtAuthenticationConverter
78                     )))
```

```
79         .build();
80     }
81 }
82
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/library/controller/PersonalLibraryController.java

File 39 of 99 · Added or modified · Personal library and uploads

Purpose: Maps authenticated personal-library HTTP operations to the upload service and returns inline/download responses.

Current file length: 71 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.library.controller;
2
3 import com.akshara.api.library.dto.PersonalBookFile;
4 import com.akshara.api.library.dto.PersonalBookResponse;
5 import com.akshara.api.library.service.PersonalLibraryService;
6 import jakarta.validation.constraints.Positive;
7 import org.springframework.http.ContentDisposition;
8 import org.springframework.http.HttpHeaders;
9 import org.springframework.http.MediaType;
10 import org.springframework.http.ResponseEntity;
11 import org.springframework.security.core.annotation.AuthenticationPrincipal;
12 import org.springframework.security.oauth2.jwt.Jwt;
13 import org.springframework.validation.annotation.Validated;
14 import org.springframework.web.bind.annotation.*;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–19

```
15 import org.springframework.web.multipart.MultipartFile;
16
17 import java.nio.charset.StandardCharsets;
18 import java.util.List;
19
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 20–24

```
20 @RestController
21 @RequestMapping("/api/library/uploads")
22 @Validated
23 public class PersonalLibraryController {
24
```

How to read this block

- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.
- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

Lines 25–30

```

25     private final PersonalLibraryService personalLibraryService;
26
27     public PersonalLibraryController(PersonalLibraryService personalLibraryService) {
28         this.personalLibraryService = personalLibraryService;
29     }
30

```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 31–35

```

31     @GetMapping
32     public List<PersonalBookResponse> getBooks(@AuthenticationPrincipal Jwt jwt) {
33         return personalLibraryService.getBooks(jwt.getSubject());
34     }
35

```

How to read this block

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 36–45

```

36     @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
37     public ResponseEntity<PersonalBookResponse> upload(
38         @AuthenticationPrincipal Jwt jwt,
39         @RequestParam String title,
40         @RequestParam(required = false) String author,
41         @RequestPart("file") MultipartFile file
42     ) {
43         return ResponseEntity.status(201).body(personalLibraryService.upload(jwt.getSubject(), title, author,
44 file));
45     }
46

```

How to read this block

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 46–59

```
46     @GetMapping("/{bookId}/file")
47     public ResponseEntity<org.springframework.core.io.Resource> getFile(
48         @AuthenticationPrincipal Jwt jwt,
49         @PathVariable @Positive Long bookId,
50         @RequestParam(defaultValue = "inline") String disposition
51     ) {
52         PersonalBookFile file = personalLibraryService.getFile(jwt.getSubject(), bookId);
53         ContentDisposition contentDisposition = "attachment".equalsIgnoreCase(disposition)
54             ? ContentDisposition.attachment().filename(file.originalFilename(), StandardCharsets.UTF_8).build()
55             : ContentDisposition.inline().filename(file.originalFilename(), StandardCharsets.UTF_8).build();
56         return ResponseEntity.ok()
57             .contentType(MediaType.parseMediaType(file.mediaType()))
58             .header(HttpHeaders.CONTENT_DISPOSITION, contentDisposition.toString())
59             .body(file.resource());
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 60–71

```
60     }
61
62     @DeleteMapping("/{bookId}")
63     public ResponseEntity<Void> delete(
64         @AuthenticationPrincipal Jwt jwt,
65         @PathVariable @Positive Long bookId
66     ) {
67         personalLibraryService.delete(jwt.getSubject(), bookId);
68         return ResponseEntity.noContent().build();
69     }
70 }
71
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/library/dto/

PersonalBookFile.java

File 40 of 99 · Added or modified · Personal library and uploads

Purpose: Defines a validated API boundary type used by the personal library and uploads module.

Current file length: 11 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.library.dto;
2
3 import org.springframework.core.io.Resource;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–11

```
5 public record PersonalBookFile(
6     Resource resource,
7     String mediaType,
8     String originalFilename
9 ) {
10 }
11
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/library/dto/

PersonalBookResponse.java

File 41 of 99 · Added or modified · Personal library and uploads

Purpose: Defines a validated API boundary type used by the personal library and uploads module.

Current file length: 17 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.library.dto;
2
3 import com.akshara.api.book.entity.BookFormat;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–17

```
5 import java.time.Instant;
6
7 public record PersonalBookResponse(
8     Long id,
9     String title,
10    String author,
11    BookFormat format,
12    String originalFilename,
13    long fileSize,
14    Instant uploadedAt
15 ) {
16 }
17
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/library/entity/

PersonalBook.java

File 42 of 99 · Added or modified · Personal library and uploads

Purpose: Models persisted domain state for the personal library and uploads module using JPA mappings.

Current file length: 79 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 package com.akshara.api.library.entity;
2
3 import com.akshara.api.book.entity.BookFormat;
4 import com.akshara.api.user.entity.AppUser;
5 import jakarta.persistence.*;
6
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 7–14

```
7 import java.time.Instant;
8
9 @Entity
10 @Table(name = "personal_books", indexes = {
11     @Index(name = "idx_personal_books_user_created", columnList = "user_id, created_at")
12 })
13 public class PersonalBook {
14
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.

Lines 15–18

```
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
```

How to read this block

- This field is the database identity. Its value is generated when a new entity is inserted.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 19–22

```
19     @ManyToOne(fetch = FetchType.LAZY, optional = false)
20     @JoinColumn(name = "user_id", nullable = false)
21     private AppUser user;
22
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 23–25

```
23     @Column(nullable = false, length = 255)
24     private String title;
25
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 26–28

```
26     @Column(length = 180)
27     private String author;
28
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 29–32

```
29     @Enumerated(EnumType.STRING)
30     @Column(nullable = false, length = 20)
31     private BookFormat format;
32
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 33–35

```
33     @Column(name = "original_filename", nullable = false, length = 255)
34     private String originalFilename;
35
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 36–38

```
36     @Column(name = "stored_filename", nullable = false, unique = true, length = 100)
37     private String storedFilename;
38
```

How to read this block

- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.

Lines 39–41

```
39     @Column(name = "media_type", nullable = false, length = 100)
40     private String mediaType;
41
```

How to read this block

- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.

Lines 42–44

```
42     @Column(name = "file_size", nullable = false)
43     private Long fileSize;
44
```

How to read this block

- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.

Lines 45–47

```
45     @Column(name = "created_at", nullable = false, updatable = false)
46     private Instant createdAt;
47
```

How to read this block

- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.

Lines 48–50

```
48     protected PersonalBook() {
49     }
50
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 51–62

```
51     public PersonalBook(AppUser user, String title, String author, BookFormat format,
52                         String originalFilename, String storedFilename, String mediaType, Long fileSize) {
```

```

53         this.user = user;
54         this.title = title;
55         this.author = author;
56         this.format = format;
57         this.originalFilename = originalFilename;
58         this.storedFilename = storedFilename;
59         this.mediaType = mediaType;
60         this.fileSize = fileSize;
61     }
62

```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 63–67

```

63     @PrePersist
64     void beforeInsert() {
65         createdAt = Instant.now();
66     }
67

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 68–79

```

68     public Long getId() { return id; }
69     public AppUser getUser() { return user; }
70     public String getTitle() { return title; }
71     public String getAuthor() { return author; }
72     public BookFormat getFormat() { return format; }
73     public String getOriginalFilename() { return originalFilename; }
74     public String getStoredFilename() { return storedFilename; }
75     public String getMediaType() { return mediaType; }
76     public Long getFileSize() { return fileSize; }
77     public Instant getCreatedAt() { return createdAt; }
78 }
79

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/library/exception/PersonalLibraryStorageException.java

File 43 of 99 · Added or modified · Personal library and uploads

Purpose: Supports personal library and uploads in the current Akshara implementation.

Current file length: 8 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–8

```
1 package com.akshara.api.library.exception;
2
3 public class PersonalLibraryStorageException extends RuntimeException {
4     public PersonalLibraryStorageException(String message, Throwable cause) {
5         super(message, cause);
6     }
7 }
8
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

backend/src/main/java/com/akshara/api/library/repository/PersonalBookRepository.java

File 44 of 99 · Added or modified · Personal library and uploads

Purpose: Defines Spring Data persistence queries required by the personal library and uploads module.

Current file length: 13 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.library.repository;
2
3 import com.akshara.api.library.entity.PersonalBook;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 6–8

```
6 import java.util.List;
7 import java.util.Optional;
8
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 9–13

```
9 public interface PersonalBookRepository extends JpaRepository<PersonalBook, Long> {
10     List<PersonalBook> findAllByUser_IdOrderByCreatedAtDesc(Long userId);
11     Optional<PersonalBook> findByIdAndUser_Id(Long id, Long userId);
12 }
13
```

How to read this block

- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

backend/src/main/java/com/akshara/api/library/service/PersonalLibraryService.java

File 45 of 99 · Added or modified · Personal library and uploads

Purpose: Validates, stores, serves and deletes private PDF/EPUB uploads while enforcing file ownership and path safety.

Current file length: 186 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.library.service;
2
3 import com.akshara.api.auth.exception.InvalidAccessTokenException;
4 import com.akshara.api.book.entity.BookFormat;
5 import com.akshara.api.common.exception.InvalidRequestException;
6 import com.akshara.api.common.exception.ResourceNotFoundException;
7 import com.akshara.api.library.dto.PersonalBookFile;
8 import com.akshara.api.library.dto.PersonalBookResponse;
9 import com.akshara.api.library.entity.PersonalBook;
10 import com.akshara.api.library.exception.PersonalLibraryStorageException;
11 import com.akshara.api.library.repository.PersonalBookRepository;
12 import com.akshara.api.user.entity.AppUser;
13 import com.akshara.api.user.repository.UserRepository;
14 import org.springframework.beans.factory.annotation.Value;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–19

```
15 import org.springframework.core.io.FileSystemResource;
16 import org.springframework.stereotype.Service;
17 import org.springframework.transaction.annotation.Transactional;
18 import org.springframework.web.multipart.MultipartFile;
19
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 20–30

```
20 import java.io.IOException;
21 import java.io.InputStream;
22 import java.nio.file.Files;
23 import java.nio.file.Path;
24 import java.nio.file.StandardCopyOption;
25 import java.nio.charset.StandardCharsets;
26 import java.util.List;
```

```

27 import java.util.Locale;
28 import java.util.UUID;
29 import java.util.zip.ZipInputStream;
30

```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 31–33

```

31 @Service
32 public class PersonalLibraryService {
33

```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.

Lines 34–38

```

34     private static final long MAX_FILE_SIZE = 25L * 1024L * 1024L;
35     private final UserRepository userRepository;
36     private final PersonalBookRepository personalBookRepository;
37     private final Path storageRoot;
38

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 39–48

```

39     public PersonalLibraryService(
40         UserRepository userRepository,
41         PersonalBookRepository personalBookRepository,
42         @Value("${app.personal-library.storage-path:./data/personal-library}") String storagePath
43     ) {
44         this.userRepository = userRepository;
45         this.personalBookRepository = personalBookRepository;
46         this.storageRoot = Path.of(storagePath).toAbsolutePath().normalize();
47     }
48

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 49–55

```

49     @Transactional(readOnly = true)
50     public List<PersonalBookResponse> getBooks(String subject) {
51         AppUser user = getAuthenticatedUser(subject);
52         return personalBookRepository.findAllByUser_IdOrderByCreatedAtDesc(user.getId())
53             .stream().map(this::toResponse).toList();
54     }
55 
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 56–69

```

56     public PersonalBookResponse upload(String subject, String title, String author, MultipartFile file) {
57         AppUser user = getAuthenticatedUser(subject);
58         String cleanTitle = title == null ? "" : title.trim();
59         if (cleanTitle.isBlank() || cleanTitle.length() > 255) {
60             throw new InvalidRequestException("Title is required and must not exceed 255 characters");
61         }
62         String cleanAuthor = author == null || author.isBlank() ? null : author.trim();
63         if (cleanAuthor != null && cleanAuthor.length() > 180) {
64             throw new InvalidRequestException("Author must not exceed 180 characters");
65         }
66         if (file == null || file.isEmpty()) {
67             throw new InvalidRequestException("Choose a PDF or EPUB file to upload");
68         }
69         if (file.getSize() > MAX_FILE_SIZE) {

```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 70–72

```
70         throw new InvalidRequestException("Personal book files must be 25 MB or smaller");
71     }
72
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 73–78

```
73     String originalName = safeOriginalFilename(file.getOriginalFilename());
74     BookFormat format = detectAndValidateFormat(originalName, file);
75     String extension = format == BookFormat.PDF ? ".pdf" : ".epub";
76     String storedName = UUID.randomUUID() + extension;
77     Path target = resolveStoredFile(storedName);
78
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 79–92

```
79     try {
80         Files.createDirectories(storageRoot);
81         try (InputStream input = file.getInputStream()) {
82             Files.copy(input, target, StandardCopyOption.REPLACE_EXISTING);
83         }
84         PersonalBook saved = personalBookRepository.save(new PersonalBook(
85             user, cleanTitle, cleanAuthor, format, originalName, storedName,
86             format == BookFormat.PDF ? "application/pdf" : "application/epub+zip",
87             file.getSize()
88         ));
89         return toResponse(saved);
90     } catch (RuntimeException | IOException exception) {
91         deleteQuietly(target);
92         if (exception instanceof InvalidRequestException invalidRequestException) {
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 93–98

```
93         throw invalidRequestException;
94     }
95     throw new PersonalLibraryStorageException("The personal book could not be stored", exception);
```



```
96     }
97   }
98 }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 99–108

```
99     @Transactional(readOnly = true)
100     public PersonalBookFile getFile(String subject, Long bookId) {
101         PersonalBook book = getOwnedBook(subject, bookId);
102         Path path = resolveStoredFile(book.getStoredFilename());
103         if (!Files.isRegularFile(path)) {
104             throw new ResourceNotFoundException("The uploaded file is no longer available");
105         }
106         return new PersonalBookFile(new FileSystemResource(path), book.getMediaType(), book.getOriginalFilename());
107     }
108 }
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 109–114

```
109     public void delete(String subject, Long bookId) {
110         PersonalBook book = getOwnedBook(subject, bookId);
111         personalBookRepository.delete(book);
112         deleteQuietly(resolveStoredFile(book.getStoredFilename()));
113     }
114 }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 115–120

```
115     private PersonalBook getOwnedBook(String subject, Long bookId) {
116         AppUser user = getAuthenticatedUser(subject);
117         return personalBookRepository.findByIdAndUser_Id(bookId, user.getId())
```

```

118         .orElseThrow(() -> new ResourceNotFoundException("Personal book with ID " + bookId + " was not
found"));
119     }
120

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 121–134

```

121     private BookFormat detectAndValidateFormat(String filename, MultipartFile file) {
122         String lower = filename.toLowerCase(Locale.ROOT);
123         try (InputStream input = file.getInputStream()) {
124             byte[] signature = input.readNBytes(5);
125             if (lower.endsWith(".pdf") && signature.length >= 5
126                 && signature[0] == '%' && signature[1] == 'P' && signature[2] == 'D'
127                 && signature[3] == 'F' && signature[4] == '-') {
128                 return BookFormat.PDF;
129             }
130             if (lower.endsWith(".epub") && signature.length >= 2
131                 && signature[0] == 'P' && signature[1] == 'K'
132                 && hasEpubMimeType(file)) {
133                 return BookFormat.EPUB;
134             }

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 135–140

```

135         } catch (IOException exception) {
136             throw new PersonalLibraryStorageException("The uploaded file could not be inspected", exception);
137         }
138         throw new InvalidRequestException("Only valid PDF and EPUB files can be uploaded");
139     }
140

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-

facing error behavior.

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 141–149

```
141     private boolean hasEpubMimeType(MultipartFile file) throws IOException {
142         try (ZipInputStream zip = new ZipInputStream(file.getInputStream())) {
143             var firstEntry = zip.getNextEntry();
144             if (firstEntry == null || !"mimetype".equals(firstEntry.getName())) return false;
145             String mimeType = new String(zip.readNBytes(30), StandardCharsets.US_ASCII).trim();
146             return "application/epub+zip".equals(mimeType);
147         }
148     }
149 }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 150–158

```
150     private String safeOriginalFilename(String filename) {
151         if (filename == null || filename.isBlank()) return "personal-book";
152         String normalized = filename.replace('\\', '/');
153         String safe = normalized.substring(normalized.lastIndexOf('/') + 1)
154             .replaceAll("[\\p{Cntrl}]", "_");
155         if (safe.isBlank()) safe = "personal-book";
156         return safe.length() > 255 ? safe.substring(safe.length() - 255) : safe;
157     }
158 }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 159–166

```
159     private Path resolveStoredFile(String storedName) {
160         Path resolved = storageRoot.resolve(storedName).normalize();
161         if (!resolved.startsWith(storageRoot)) {
162             throw new InvalidRequestException("Invalid personal-library file path");
163         }
164     }
165 }
```

```
163         }
164         return resolved;
165     }
166 }
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 167–170

```
167     private void deleteQuietly(Path path) {
168         try { Files.deleteIfExists(path); } catch (IOException ignored) { }
169     }
170 }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 171–175

```
171     private PersonalBookResponse toResponse(PersonalBook book) {
172         return new PersonalBookResponse(book.getId(), book.getTitle(), book.getAuthor(), book.getFormat(),
173             book.getOriginalFilename(), book.getFileSize(), book.getCreatedAt());
174     }
175 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 176–186

```
176     private AppUser getAuthenticatedUser(String subject) {  
177         try {  
178             Long userId = Long.parseLong(subject);  
179             return userRepository.findById(userId).filter(AppUser::isEnabled)  
180                 .orElseThrow(InvalidAccessTokenException::new);  
181         } catch (NumberFormatException exception) {  
182             throw new InvalidAccessTokenException();  
183         }  
184     }  
185 }  
186
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/order/controller/

LibraryController.java

File 46 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Returns purchased non-cancelled order items as the reader's My Books shelf.

Current file length: 30 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–10

```
1 package com.akshara.api.order.controller;
2
3 import com.akshara.api.order.dto.LibraryBookResponse;
4 import com.akshara.api.order.service.OrderQueryService;
5 import org.springframework.security.core.annotation.AuthenticationPrincipal;
6 import org.springframework.security.oauth2.jwt.Jwt;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.RequestMapping;
9 import org.springframework.web.bind.annotation.RestController;
10
```

How to read this block

- The package declaration places the type in Akshara's module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 11–16

```
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/library")
15 public class LibraryController {
16
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

Lines 17–22

```
17     private final OrderQueryService orderQueryService;
18
19     public LibraryController(OrderQueryService orderQueryService) {
20         this.orderQueryService = orderQueryService;
21     }
22
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 23–30

```
23     @GetMapping
24     public List<LibraryBookResponse> getMyBooks(
25         @AuthenticationPrincipal Jwt jwt
26     ) {
27         return orderQueryService.getLibrary(jwt.getSubject());
28     }
29 }
30
```

How to read this block

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/order/dto/

LibraryBookResponse.java

File 47 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Defines a validated API boundary type used by the orders, fulfilment and purchased shelf module.

Current file length: 24 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.order.dto;
2
3 import com.akshara.api.book.entity.BookFormat;
4 import com.akshara.api.order.entity.OrderStatus;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–19

```
6 import java.time.Instant;
7
8 public record LibraryBookResponse(
9     Long orderItemId,
10    Long orderId,
11    Long bookId,
12    Long bookEditionId,
13    String title,
14    String coverImageUrl,
15    BookFormat format,
16    String editionName,
17    String isbn,
18    String publisherName,
19    int quantity,
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 20–24

```
20    OrderStatus orderStatus,
21    Instant purchasedAt
22 ) {
23 }
24
```


How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/order/dto/OrderItemResponse.java

File 48 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Defines a validated API boundary type used by the orders, fulfilment and purchased shelf module.

Current file length: 23 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.order.dto;
2
3 import com.akshara.api.book.entity.BookFormat;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–18

```
5 import java.math.BigDecimal;
6
7 public record OrderItemResponse(
8     Long orderItemId,
9     Long bookEditionId,
10    Long bookId,
11    String bookTitle,
12    String coverImageUrl,
13    BookFormat bookFormat,
14    String editionName,
15    String isbn10,
16    String isbn13,
17    String publisherName,
18    BigDecimal unitPrice,
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 19–23

```
19    int quantity,
20    BigDecimal subtotal
21 ) {
22 }
23
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/order/entity/OrderItem.java

File 49 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Models persisted domain state for the orders, fulfilment and purchased shelf module using JPA mappings.

Current file length: 187 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–8

```
1 package com.akshara.api.order.entity;
2
3 import com.akshara.api.book.entity.Book;
4 import com.akshara.api.book.entity.BookEdition;
5 import com.akshara.api.book.entity.BookFormat;
6 import com.akshara.api.book.entity.Publisher;
7 import jakarta.persistence.*;
8
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 9–22

```
9 import java.math.BigDecimal;
10
11 @Entity
12 @Table(
13     name = "order_items",
14     indexes = {
15         @Index(
16             name = "idx_order_items_order",
17             columnList = "order_id"
18         ),
19         @Index(
20             name = "idx_order_items_edition",
21             columnList = "book_edition_id"
22         )
23     }
24 )
25 public class OrderItem {
26
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.

Lines 23–26

```
23     }
24 )
25 public class OrderItem {
26
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 27–30

```
27     @Id
28     @GeneratedValue(strategy = GenerationType.IDENTITY)
29     private Long id;
30
```

How to read this block

- This field is the database identity. Its value is generated when a new entity is inserted.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 31–34

```
31     @ManyToOne(fetch = FetchType.LAZY, optional = false)
32     @JoinColumn(name = "order_id", nullable = false)
33     private Order order;
34
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 35–38

```
35     @ManyToOne(fetch = FetchType.LAZY)
36     @JoinColumn(name = "book_edition_id")
37     private BookEdition bookEdition;
38
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 39–46

```
39     @Column(
40         name = "book_title",
41         nullable = false,
42         length = 255,
43         updatable = false
44     )
45     private String bookTitle;
46
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 47–53

```
47     @Column(  
48         name = "cover_image_url",  
49         length = 1000,  
50         updatable = false  
51     )  
52     private String coverImageUrl;  
53
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 54–62

```
54     @Enumerated(EnumType.STRING)  
55     @Column(  
56         name = "book_format",  
57         nullable = false,  
58         length = 20,  
59         updatable = false  
60     )  
61     private BookFormat bookFormat;  
62
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 63–69

```
63     @Column(  
64         name = "edition_name",  
65         length = 100,  
66         updatable = false  
67     )  
68     private String editionName;  
69
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 70–72

```
70     @Column(length = 10, updatable = false)  
71     private String isbn10;  
72
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 73–75

```
73     @Column(length = 13, updatable = false)  
74     private String isbn13;
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 76–82

```
76     @Column(
77         name = "publisher_name",
78         length = 255,
79         updatable = false
80     )
81     private String publisherName;
82
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 83–91

```
83     @Column(
84         name = "unit_price",
85         nullable = false,
86         precision = 10,
87         scale = 2,
88         updatable = false
89     )
90     private BigDecimal unitPrice;
91
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 92–94

```
92     @Column(nullable = false, updatable = false)
93     private Integer quantity;
94
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 95–102

```
95     @Column(
96         nullable = false,
97         precision = 12,
98         scale = 2,
99         updatable = false
100    )
101    private BigDecimal subtotal;
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 103–106

```

103     protected OrderItem() {
104         // Required by JPA
105     }
106 
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 107–115

```

107     public OrderItem(
108         Order order,
109         BookEdition bookEdition,
110         BigDecimal unitPrice,
111         Integer quantity
112     ) {
113         Book book = bookEdition.getBook();
114         Publisher publisher = bookEdition.getPublisher();
115 
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 116–118

```

116         this.order = order;
117         this.bookEdition = bookEdition;
118 
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 119–127

```

119         this.bookTitle = book.getTitle();
120         this.coverImageUrl = book.getCoverImageUrl();
121         this.bookFormat = bookEdition.getFormat();
122         this.editionName = bookEdition.getEditionName();
123         this.isbn10 = bookEdition.getIsbn10();
124         this.isbn13 = bookEdition.getIsbn13();
125         this.publisherName = 
```



```
126         publisher == null ? null : publisher.getName();
127     }
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 128–134

```
128         this.unitPrice = unitPrice;
129         this.quantity = quantity;
130         this.subtotal = unitPrice.multiply(
131             BigDecimal.valueOf(quantity)
132         );
133     }
134 }
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 135–138

```
135     public Long getId() {
136         return id;
137     }
138 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 139–142

```
139     public Order getOrder() {
140         return order;
141     }
142 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 143–146

```
143     public BookEdition getBookEdition() {
144         return bookEdition;
145     }
146 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 147–150

```
147     public String getBookTitle() {  
148         return bookTitle;  
149     }  
150
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 151–154

```
151     public String getCoverImageUrl() {  
152         return coverImageUrl;  
153     }  
154
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 155–158

```
155     public BookFormat getBookFormat() {  
156         return bookFormat;  
157     }  
158
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 159–162

```
159     public String getEditionName() {  
160         return editionName;  
161     }  
162
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 163–166

```
163     public String getIsbn10() {  
164         return isbn10;  
165     }  
166
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 167–170

```
167     public String getIsbn13() {  
168         return isbn13;  
169     }  
170
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 171–174

```
171     public String getPublisherName() {  
172         return publisherName;  
173     }  
174
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 175–178

```
175     public BigDecimal getUnitPrice() {  
176         return unitPrice;  
177     }  
178
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 179–182

```
179     public Integer getQuantity() {  
180         return quantity;  
181     }  
182
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 183–187

```
183     public BigDecimal getSubtotal() {  
184         return subtotal;  
185     }  
186 }  
187
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/order/repository/OrderItemRepository.java

File 50 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Defines Spring Data persistence queries required by the orders, fulfilment and purchased shelf module.

Current file length: 38 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–10

```
1 package com.akshara.api.order.repository;
2
3 import com.akshara.api.order.entity.OrderItem;
4 import com.akshara.api.order.entity.OrderStatus;
5 import org.springframework.data.jpa.repository.EntityGraph;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Modifying;
8 import org.springframework.data.jpa.repository.Query;
9 import org.springframework.data.repository.query.Param;
10
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 11–15

```
11 import java.util.List;
12
13 public interface OrderItemRepository
14     extends JpaRepository<OrderItem, Long> {
15
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 16–19

```
16     List<OrderItem> findAllByOrder_IdOrderByAsc(
17         Long orderId
18     );
19
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 20–29

```
20     @EntityGraph(attributePaths = {
21         "order",
22         "bookEdition",
23         "bookEdition.book"
24     })
25     List<OrderItem> findAllByOrder_User_IdAndOrder_StatusNotOrderByOrder_PlacedAtDescIdDesc(
26         Long userId,
27         OrderStatus excludedStatus
28     );
29
```

How to read this block

- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 30–38

```
30     @Modifying(clearAutomatically = true, flushAutomatically = true)
31     @Query("""
32         update OrderItem item
33         set item.bookEdition = null
34         where item.bookEdition.book.id = :bookId
35         """)
36     int detachAllByBookId(@Param("bookId") Long bookId);
37 }
38
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/order/service/

AdminOrderService.java

File 51 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Contains business rules and orchestration for the orders, fulfilment and purchased shelf module.

Current file length: 293 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.order.service;
2
3 import com.akshara.api.book.entity.Inventory;
4 import com.akshara.api.book.repository.InventoryRepository;
5 import com.akshara.api.common.exception.InvalidRequestException;
6 import com.akshara.api.common.exception.ResourceNotFoundException;
7 import com.akshara.api.order.dto.AdminOrderDetailResponse;
8 import com.akshara.api.order.dto.AdminOrderSummaryResponse;
9 import com.akshara.api.order.entity.Order;
10 import com.akshara.api.order.entity.OrderItem;
11 import com.akshara.api.order.entity.OrderStatus;
12 import com.akshara.api.order.entity.Payment;
13 import com.akshara.api.order.repository.OrderItemRepository;
14 import com.akshara.api.order.repository.OrderRepository;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–18

```
15 import com.akshara.api.order.repository.PaymentRepository;
16 import org.springframework.stereotype.Service;
17 import org.springframework.transaction.annotation.Transactional;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 19–25

```
19 import java.util.ArrayList;
20 import java.util.EnumMap;
21 import java.util.List;
22 import java.util.Map;
23 import java.util.Set;
24 import java.util.TreeMap;
25
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 26–28

```
26 @Service
27 public class AdminOrderService {
28
```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.

Lines 29–31

```
29     private static final Map<OrderStatus, Set<OrderStatus>>
30         ALLOWED_TRANSITIONS = createAllowedTransitions();
31
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 32–37

```
32     private final OrderRepository orderRepository;
33     private final OrderItemRepository orderItemRepository;
34     private final PaymentRepository paymentRepository;
35     private final InventoryRepository inventoryRepository;
36     private final OrderResponseMapper orderResponseMapper;
37
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 38–51

```
38     public AdminOrderService(
39         OrderRepository orderRepository,
40         OrderItemRepository orderItemRepository,
41         PaymentRepository paymentRepository,
42         InventoryRepository inventoryRepository,
43         OrderResponseMapper orderResponseMapper
44     ) {
45         this.orderRepository = orderRepository;
46         this.orderItemRepository = orderItemRepository;
47         this.paymentRepository = paymentRepository;
48         this.inventoryRepository = inventoryRepository;
49         this.orderResponseMapper = orderResponseMapper;
50     }
51
```


How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 52–60

```
52     @Transactional(readOnly = true)
53     public List<AdminOrderSummaryResponse> getOrders() {
54         return orderRepository
55             .findAllByOrderByPlacedAtDesc()
56             .stream()
57             .map(this::toSummaryResponse)
58             .toList();
59     }
60
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 61–65

```
61     @Transactional(readOnly = true)
62     public AdminOrderDetailResponse getOrder(Long orderId) {
63         Order order = orderRepository.findById(orderId)
64             .orElseThrow(() -> orderNotFound(orderId));
65
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 66–68

```
66         return loadDetailResponse(order);
```

```
67     }
68
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 69–77

```
69     @Transactional
70     public AdminOrderDetailResponse updateStatus(
71         Long orderId,
72         OrderStatus requestedStatus
73     ) {
74         Order order = orderRepository
75             .findByIdForUpdate(orderId)
76             .orElseThrow(() -> orderNotFound(orderId));
77
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 78–82

```
78         validateTransition(
79             order.getStatus(),
80             requestedStatus
81         );
82
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 83–85

```
83         List<OrderItem> orderItems = orderItemRepository
84             .findAllByOrder_IdOrderByAsc(orderId);
85
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 86–93

```
86         Payment payment = paymentRepository
87             .findByOrder_Id(orderId)
```

```

88         .orElseThrow(() -> new ResourceNotFoundException(
89             "Payment for order with ID "
90             + orderId
91             + " was not found"
92         ));
93

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 94–99

```

94     if (requestedStatus == OrderStatus.CANCELLED) {
95         restoreInventory(orderItems);
96         payment.cancel();
97         paymentRepository.save(payment);
98     }
99

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 100–104

```

100     if (requestedStatus == OrderStatus.DELIVERED) {
101         payment.markPaid();
102         paymentRepository.save(payment);
103     }
104

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 105–107

```

105     order.setStatus(requestedStatus);
106     orderRepository.save(order);
107

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 108–114

```

108     return toDetailResponse(

```

```

109         order,
110         orderItems,
111         payment
112     );
113 }
114

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 115–124

```

115     private void validateTransition(
116         OrderStatus currentStatus,
117         OrderStatus requestedStatus
118     ) {
119         Set<OrderStatus> allowedStatuses =
120             ALLOWED_TRANSITIONS.getDefault(
121                 currentStatus,
122                 Set.of()
123             );
124

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 125–134

```

125         if (!allowedStatuses.contains(requestedStatus)) {
126             throw new InvalidRequestException(
127                 "Order status cannot change from "
128                 + currentStatus
129                 + " to "
130                 + requestedStatus
131             );
132         }
133     }
134

```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 135–140

```

135     private void restoreInventory(
136         List<OrderItem> orderItems

```

```

137     ) {
138         Map<Long, Integer> quantityByEditionId =
139             new TreeMap<>();
140     }

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 141–147

```

141     for (OrderItem orderItem : orderItems) {
142         if (orderItem.getBookEdition() == null) {
143             continue;
144         }
145         Long editionId =
146             orderItem.getBookEdition().getId();
147     }

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 148–154

```

148         quantityByEditionId.merge(
149             editionId,
150             orderItem.getQuantity(),
151             Integer::sum
152         );
153     }
154 }

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 155–157

```

155     List<Inventory> restoredInventories =
156         new ArrayList<>();
157 }

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 158–160

```
158         for (Map.Entry<Long, Integer> entry
159             : quantityByEditionId.entrySet()) {
160
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 161–172

```
161         Inventory inventory = inventoryRepository
162             .findByBookEditionIdForUpdate(
163                 entry.getKey()
164             )
165             .orElseThrow(() ->
166                 new ResourceNotFoundException(
167                     "Inventory for book edition with ID "
168                     + entry.getKey()
169                     + " was not found"
170                 )
171             );
172
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 173–176

```
173         inventory.increaseStock(entry.getValue());
174         restoredInventories.add(inventory);
175     }
176
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 177–179

```
177         inventoryRepository.saveAll(restoredInventories);
178     }
179
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 180–187

```
180     private AdminOrderDetailResponse loadDetailResponse(
181         Order order
182     ) {
```

```

183         List<OrderItem> orderItems = orderItemRepository
184             .findAllByOrder_IdOrderByIdAsc(
185                 order.getId()
186             );
187

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 188–195

```

188         Payment payment = paymentRepository
189             .findByOrder_Id(order.getId())
190             .orElseThrow(() -> new ResourceNotFoundException(
191                 "Payment for order with ID "
192                     + order.getId()
193                     + " was not found"
194             ));
195

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 196–202

```

196         return toDetailResponse(
197             order,
198             orderItems,
199             payment
200         );
201     }
202

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 203–216

```

203     private AdminOrderDetailResponse toDetailResponse(
204         Order order,
205         List<OrderItem> orderItems,
206         Payment payment
207     ) {
208         return new AdminOrderDetailResponse(
209             order.getUser().getId(),
210             order.getUser().getFullName(),
211             order.getUser().getEmail(),
212             orderResponseMapper.toCheckoutResponse(
213                 order,
214                 orderItems,

```

```
215         payment
216     )
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 217–219

```
217     );
218 }
219
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 220–233

```
220     private AdminOrderSummaryResponse toSummaryResponse(
221         Order order
222     ) {
223         return new AdminOrderSummaryResponse(
224             order.getId(),
225             order.getUser().getId(),
226             order.getUser().getFullName(),
227             order.getUser().getEmail(),
228             order.getStatus(),
229             order.getTotalAmount(),
230             order.getPlacedAt(),
231             order.getUpdatedAt()
232         );
233     }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 234–244

```
234
235     private ResourceNotFoundException orderNotFound(
236         Long orderId
237     ) {
238         return new ResourceNotFoundException(
239             "Order with ID "
```



```

240             + orderId
241             + " was not found"
242         );
243     }
244

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 245–247

```

245     private static Map<OrderStatus, Set<OrderStatus>>
246         createAllowedTransitions() {
247

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 248–250

```

248         Map<OrderStatus, Set<OrderStatus>> transitions =
249             new EnumMap<>(OrderStatus.class);
250

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 251–258

```

251         transitions.put(
252             OrderStatus.PLACED,
253             Set.of(
254                 OrderStatus.CONFIRMED,
255                 OrderStatus.CANCELLED
256             )
257         );
258

```

How to read this block

- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 259–266

```
259         transitions.put(  
260             OrderStatus.CONFIRMED,  
261             Set.of(  
262                 OrderStatus.PROCESSING,  
263                 OrderStatus.CANCELLED  
264             )  
265         );  
266
```

How to read this block

- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 267–274

```
267         transitions.put(  
268             OrderStatus.PROCESSING,  
269             Set.of(  
270                 OrderStatus.SHIPPED,  
271                 OrderStatus.CANCELLED  
272             )  
273         );  
274
```

How to read this block

- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 275–279

```
275         transitions.put(  
276             OrderStatus.SHIPPED,  
277             Set.of(OrderStatus.DELIVERED)  
278         );  
279
```

How to read this block

- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 280–284

```
280         transitions.put(  
281             OrderStatus.DELIVERED,  
282             Set.of()  
283         );  
284
```

How to read this block

- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 285–289

```
285         transitions.put(  
286             OrderStatus.CANCELLED,
```

```
287         Set.of()  
288     );  
289
```

How to read this block

- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 290–293

```
290         return Map.copyOf(transitions);  
291     }  
292 }  
293
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

backend/src/main/java/com/akshara/api/order/service/CheckoutService.java

File 52 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Contains business rules and orchestration for the orders, fulfilment and purchased shelf module.

Current file length: 313 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.order.service;
2
3 import com.akshara.api.auth.exception.InvalidAccessTokenException;
4 import com.akshara.api.book.entity.AvailabilityStatus;
5 import com.akshara.api.book.entity.Inventory;
6 import com.akshara.api.book.repository.InventoryRepository;
7 import com.akshara.api.cart.entity.Cart;
8 import com.akshara.api.cart.entity.CartItem;
9 import com.akshara.api.cart.repository.CartItemRepository;
10 import com.akshara.api.cart.repository.CartRepository;
11 import com.akshara.api.common.exception.InvalidRequestException;
12 import com.akshara.api.common.exception.ResourceNotFoundException;
13 import com.akshara.api.order.dto.CheckoutRequest;
14 import com.akshara.api.order.dto.CheckoutResponse;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–28

```
15 import com.akshara.api.order.dto.ShippingAddressRequest;
16 import com.akshara.api.order.entity.Address;
17 import com.akshara.api.order.entity.Order;
18 import com.akshara.api.order.entity.OrderItem;
19 import com.akshara.api.order.entity.Payment;
20 import com.akshara.api.order.entity.PaymentMethod;
21 import com.akshara.api.order.repository.OrderItemRepository;
22 import com.akshara.api.order.repository.OrderRepository;
23 import com.akshara.api.order.repository.PaymentRepository;
24 import com.akshara.api.user.entity.AppUser;
25 import com.akshara.api.user.repository.UserRepository;
26 import org.springframework.stereotype.Service;
27 import org.springframework.transaction.annotation.Transactional;
28
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC

in the business layer.

Lines 29–34

```
29 import java.math.BigDecimal;
30 import java.util.Comparator;
31 import java.util.LinkedHashMap;
32 import java.util.List;
33 import java.util.Map;
34
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 35–37

```
35 @Service
36 public class CheckoutService {
37
```

How to read this block

- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.

Lines 38–40

```
38     private static final BigDecimal SHIPPING_FEE =
39         new BigDecimal("0.00");
40
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 41–49

```
41     private final CartRepository cartRepository;
42     private final CartItemRepository cartItemRepository;
43     private final UserRepository userRepository;
44     private final InventoryRepository inventoryRepository;
45     private final OrderRepository orderRepository;
46     private final OrderItemRepository orderItemRepository;
47     private final PaymentRepository paymentRepository;
48     private final OrderResponseMapper orderResponseMapper;
49
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 50–63

```
50     public CheckoutService(
51         CartRepository cartRepository,
```

```

52         CartItemRepository cartItemRepository,
53         UserRepository userRepository,
54         InventoryRepository inventoryRepository,
55         OrderRepository orderRepository,
56         OrderItemRepository orderItemRepository,
57         PaymentRepository paymentRepository,
58         OrderResponseMapper orderResponseMapper
59     ) {
60         this.cartRepository = cartRepository;
61         this.cartItemRepository = cartItemRepository;
62         this.userRepository = userRepository;
63         this.inventoryRepository = inventoryRepository;

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 64–69

```

64         this.orderRepository = orderRepository;
65         this.orderItemRepository = orderItemRepository;
66         this.paymentRepository = paymentRepository;
67         this.orderResponseMapper = orderResponseMapper;
68     }
69

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 70–76

```

70     @Transactional
71     public CheckoutResponse checkout(
72         String subject,
73         CheckoutRequest request
74     ) {
75         AppUser user = getAuthenticatedUser(subject);
76

```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 77–82

```

77         Cart cart = cartRepository
78             .findByUserIdForUpdate(user.getId())
79             .orElseThrow(() -> new InvalidRequestException(

```

```

80         "Cart is empty"
81     });
82

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 83–87

```

83     List<CartItem> cartItems = cartItemRepository
84         .findAllByCart_IdOrderByCreatedAtAsc(
85             cart.getId()
86         );
87

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 88–93

```

88     if (cartItems.isEmpty()) {
89         throw new InvalidRequestException(
90             "Cart is empty"
91         );
92     }
93

```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 94–99

```

94     List<CartItem> sortedCartItems = cartItems.stream()
95         .sorted(Comparator.comparing(
96             item -> item.getBookEdition().getId()
97         ))
98         .toList();
99

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 100–102

```

100     Map<Long, Inventory> inventoryByEditionId =

```

```
101         lockInventories(sortedCartItems);
102
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 103–107

```
103         validateCheckoutItems(
104             sortedCartItems,
105             inventoryByEditionId
106         );
107
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 108–112

```
108         BigDecimal subtotal = calculateSubtotal(
109             sortedCartItems,
110             inventoryByEditionId
111         );
112
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 113–116

```
113         Address address = createAddress(
114             request.shippingAddress()
115         );
116
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 117–125

```
117         Order order = orderRepository.save(
118             new Order(
119                 user,
120                 address,
121                 subtotal,
122                 SHIPPING_FEE
123             )
124         );
125
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC

in the business layer.

Lines 126–130

```
126         List<OrderItem> orderItems = sortedCartItems.stream()
127             .map(cartItem -> {
128                 Long editionId =
129                     cartItem.getBookEdition().getId();
130             });
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 131–133

```
131         Inventory inventory =
132             inventoryByEditionId.get(editionId);
133     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 134–142

```
134         return new OrderItem(
135             order,
136             cartItem.getBookEdition(),
137             inventory.getPrice(),
138             cartItem.getQuantity()
139         );
140     })
141     .toList();
142 }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 143–145

```
143         List<OrderItem> savedOrderItems =
144             orderItemRepository.saveAll(orderItems);
145     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 146–153

```
146         Payment payment = paymentRepository.save(  
147             new Payment(  
148                 order,  
149                 PaymentMethod.CASH_ON_DELIVERY,  
150                 order.getTotalAmount()  
151             )  
152         );  
153
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 154–157

```
154         for (CartItem cartItem : sortedCartItems) {  
155             Long editionId =  
156                 cartItem.getBookEdition().getId();  
157
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 158–162

```
158             inventoryByEditionId  
159                 .get(editionId)  
160                 .decreaseStock(cartItem.getQuantity());  
161         }  
162
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 163–166

```
163         inventoryRepository.saveAll(  
164             inventoryByEditionId.values()  
165         );  
166
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 167–170

```
167         cartItemRepository.deleteAllByCart_Id(  
168             cart.getId()  
169         );  
170
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 171–173

```
171         cart.markUpdated();
172         cartRepository.save(cart);
173
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 174–180

```
174         return orderResponseMapper.toCheckoutResponse(
175             order,
176             savedOrderItems,
177             payment
178         );
179     }
180
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 181–183

```
181     private AppUser getAuthenticatedUser(String subject) {
182         Long userId;
183
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 184–189

```
184         try {
185             userId = Long.parseLong(subject);
186         } catch (NumberFormatException exception) {
187             throw new InvalidAccessTokenException();
188         }
189
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 190–194

```
190         return userRepository.findById(userId)
191             .filter(AppUser::isEnabled)
192             .orElseThrow(InvalidAccessTokenException::new);
193     }
194
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 195–200

```
195     private Map<Long, Inventory> lockInventories(
196         List<CartItem> sortedCartItems
197     ) {
198         Map<Long, Inventory> inventoryByEditionId =
199             new LinkedHashMap<>();
200
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 201–204

```
201         for (CartItem cartItem : sortedCartItems) {
202             Long editionId =
203                 cartItem.getBookEdition().getId();
204
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 205–214

```
205         Inventory inventory = inventoryRepository
206             .findByBookEditionIdForUpdate(editionId)
207             .orElseThrow(() ->
208                 new ResourceNotFoundException(
209                     "Inventory for book edition with ID "
210                     + editionId
211                     + " was not found"
212                 )
213             );
214
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 215–220

```
215         inventoryByEditionId.put(  
216             editionId,  
217             inventory  
218         );  
219     }  
220
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 221–223

```
221         return inventoryByEditionId;  
222     }  
223
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 224–231

```
224     private void validateCheckoutItems(  
225         List<CartItem> cartItems,  
226         Map<Long, Inventory> inventoryByEditionId  
227     ) {  
228         for (CartItem cartItem : cartItems) {  
229             Long editionId =  
230                 cartItem.getBookEdition().getId();  
231
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 232–237

```
232         if (cartItem.getBookEdition().getFormat().isDigital()) {  
233             throw new InvalidRequestException(  
234                 "Digital editions are not available for checkout yet"  
235             );  
236         }  
237
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 238–241

```
238         Integer quantity = cartItem.getQuantity();
239         Inventory inventory =
240             inventoryByEditionId.get(editionId);
241     
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 242–248

```
242         if (quantity == null || quantity <= 0) {
243             throw new InvalidRequestException(
244                 "Invalid quantity for book edition with ID "
245                     + editionId
246             );
247         }
248     
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

Lines 249–253

```
249         if (!inventory.isActive()
250             || inventory.getAvailabilityStatus()
251                 != AvailabilityStatus.IN_STOCK
252             || inventory.getStockQuantity() <= 0) {
253     
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 254–260

```
254             throw new InvalidRequestException(
255                 "Book edition with ID "
256                     + editionId
257                     + " is not available for checkout"
258             );
259         }
260     
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 261–271

```
261         if (quantity > inventory.getStockQuantity()) {
262             throw new InvalidRequestException(
263                 "Insufficient stock for book edition with ID "
264                 + editionId
265                 + ". Available quantity: "
266                 + inventory.getStockQuantity()
267             );
268         }
269     }
270 }
271
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 272–280

```
272     private BigDecimal calculateSubtotal(
273         List<CartItem> cartItems,
274         Map<Long, Inventory> inventoryByEditionId
275     ) {
276         return cartItems.stream()
277             .map(cartItem -> {
278                 Long editionId =
279                     cartItem.getBookEdition().getId();
280
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 281–285

```
281         BigDecimal price =
282             inventoryByEditionId
283                 .get(editionId)
284                 .getPrice();
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 286–297

```

286         return price.multiply(
287             BigDecimal.valueOf(
288                 cartItem.getQuantity()
289             )
290         );
291     })
292     .reduce(
293         new BigDecimal("0.00"),
294         BigDecimal::add
295     );
296 }
297
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 298–311

```

298     private Address createAddress(
299         ShippingAddressRequest request
300     ) {
301         return new Address(
302             request.recipientName(),
303             request.phone(),
304             request.addressLine1(),
305             request.addressLine2(),
306             request.city(),
307             request.state(),
308             request.postalCode(),
309             request.country()
310         );
311     }

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 312–313

```

312 }
313
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/order/service/OrderQueryService.java

File 53 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Contains business rules and orchestration for the orders, fulfilment and purchased shelf module.

Current file length: 150 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.order.service;
2
3 import com.akshara.api.auth.exception.InvalidAccessTokenException;
4 import com.akshara.api.common.exception.ResourceNotFoundException;
5 import com.akshara.api.order.dto.CheckoutResponse;
6 import com.akshara.api.order.dto.LibraryBookResponse;
7 import com.akshara.api.order.dto.OrderSummaryResponse;
8 import com.akshara.api.order.entity.Order;
9 import com.akshara.api.order.entity.OrderItem;
10 import com.akshara.api.order.entity.OrderStatus;
11 import com.akshara.api.order.entity.Payment;
12 import com.akshara.api.order.repository.OrderItemRepository;
13 import com.akshara.api.order.repository.OrderRepository;
14 import com.akshara.api.order.repository.PaymentRepository;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–19

```
15 import com.akshara.api.user.entity.AppUser;
16 import com.akshara.api.user.repository.UserRepository;
17 import org.springframework.stereotype.Service;
18 import org.springframework.transaction.annotation.Transactional;
19
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 20–24

```
20 import java.util.List;
21
22 @Service
23 public class OrderQueryService {
24
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.

Lines 25–30

```
25     private final UserRepository userRepository;
26     private final OrderRepository orderRepository;
27     private final OrderItemRepository orderItemRepository;
28     private final PaymentRepository paymentRepository;
29     private final OrderResponseMapper orderResponseMapper;
30
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 31–44

```
31     public OrderQueryService(
32         UserRepository userRepository,
33         OrderRepository orderRepository,
34         OrderItemRepository orderItemRepository,
35         PaymentRepository paymentRepository,
36         OrderResponseMapper orderResponseMapper
37     ) {
38         this.userRepository = userRepository;
39         this.orderRepository = orderRepository;
40         this.orderItemRepository = orderItemRepository;
41         this.paymentRepository = paymentRepository;
42         this.orderResponseMapper = orderResponseMapper;
43     }
44
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 45–50

```
45     @Transactional(readOnly = true)
46     public List<OrderSummaryResponse> getOrders(
47         String subject
48     ) {
49         AppUser user = getAuthenticatedUser(subject);
50
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them

back. Read-only transactions communicate query intent.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 51–57

```
51         return orderRepository
52             .findAllByUser_IdOrderByPlacedAtDesc(user.getId())
53             .stream()
54             .map(this::toSummaryResponse)
55             .toList();
56     }
57
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 58–64

```
58     @Transactional(readOnly = true)
59     public CheckoutResponse getOrder(
60         String subject,
61         Long orderId
62     ) {
63         AppUser user = getAuthenticatedUser(subject);
64
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 65–72

```
65         Order order = orderRepository
66             .findByIdAndUser_Id(orderId, user.getId())
67             .orElseThrow(() -> new ResourceNotFoundException(
68                 "Order with ID "
69                     + orderId
70                     + " was not found"
71             ));
72
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 73–75

```
73         List<OrderItem> orderItems = orderItemRepository
74             .findAllByOrder_IdOrderByIdAsc(orderId);
75     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 76–83

```
76         Payment payment = paymentRepository
77             .findByOrder_Id(orderId)
78             .orElseThrow(() -> new ResourceNotFoundException(
79                 "Payment for order with ID "
80                     + orderId
81                     + " was not found"
82             ));
83     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 84–90

```
84         return orderResponseMapper.toCheckoutResponse(
85             order,
86             orderItems,
87             payment
88         );
89     }
90 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 91–94

```
91     @Transactional(readOnly = true)
92     public List<LibraryBookResponse> getLibrary(String subject) {
93         AppUser user = getAuthenticatedUser(subject);
94     }
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 95–104

```
95         return orderItemRepository
96             .findAllByOrder_User_IdAndOrder_StatusNotOrderByOrder_PlacedAtDescIdDesc(
97                 user.getId(),
98                 OrderStatus.CANCELLED
99             )
100             .stream()
101             .map(this::toLibraryBookResponse)
102             .toList();
103     }
104
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 105–107

```
105     private LibraryBookResponse toLibraryBookResponse(OrderItem item) {
106         var edition = item.getBookEdition();
107     }
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 108–121

```
108         return new LibraryBookResponse(
109             item.getId(),
110             item.getOrder().getId(),
111             edition == null ? null : edition.getBook().getId(),
112             edition == null ? null : edition.getId(),
113             item.getBookTitle(),
114             item.getCoverImageUrl(),
115             item.getBookFormat(),
116             item.getEditionName(),
117             item.getIsbn13() == null ? item.getIsbn10() : item.getIsbn13(),
118             item.getPublisherName(),
119             item.getQuantity(),
120             item.getOrder().getStatus(),
121             item.getOrder().getPlacedAt()
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 122–124

```
122     );
123 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 125–135

```
125     private OrderSummaryResponse toSummaryResponse(  
126         Order order  
127     ) {  
128         return new OrderSummaryResponse(  
129             order.getId(),  
130             order.getStatus(),  
131             order.getTotalAmount(),  
132             order.getPlacedAt()  
133         );  
134     }  
135
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 136–138

```
136     private AppUser getAuthenticatedUser(String subject) {  
137         Long userId;  
138
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 139–144

```
139         try {  
140             userId = Long.parseLong(subject);  
141         } catch (NumberFormatException exception) {  
142             throw new InvalidAccessTokenException();  
143         }  
144
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.

Lines 145–150

```
145         return userRepository.findById(userId)
146             .filter(AppUser::isEnabled)
147             .orElseThrow(InvalidAccessTokenException::new);
148     }
149 }
150
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

backend/src/main/java/com/akshara/api/order/service/OrderResponseMapper.java

File 54 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Contains business rules and orchestration for the orders, fulfilment and purchased shelf module.

Current file length: 86 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–11

```
1 package com.akshara.api.order.service;
2
3 import com.akshara.api.order.dto.CheckoutResponse;
4 import com.akshara.api.order.dto.OrderItemResponse;
5 import com.akshara.api.order.dto.ShippingAddressResponse;
6 import com.akshara.api.order.entity.Address;
7 import com.akshara.api.order.entity.Order;
8 import com.akshara.api.order.entity.OrderItem;
9 import com.akshara.api.order.entity.Payment;
10 import org.springframework.stereotype.Component;
11
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 12–16

```
12 import java.util.List;
13
14 @Component
15 public class OrderResponseMapper {
16
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 17–25

```
17     public CheckoutResponse toCheckoutResponse(
18         Order order,
19         List<OrderItem> orderItems,
20         Payment payment
21     ) {
22         List<OrderItemResponse> itemResponses = orderItems.stream()
23             .map(this::toOrderItemResponse)
24             .toList();
25
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 26–29

```
26         int totalQuantity = itemResponses.stream()
27             .mapToInt(OrderItemResponse::quantity)
28             .sum();
29
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 30–43

```
30         return new CheckoutResponse(
31             order.getId(),
32             order.getStatus(),
33             payment.getMethod(),
34             payment.getStatus(),
35             itemResponses,
36             totalQuantity,
37             order.getSubtotal(),
38             order.getShippingFee(),
39             order.getTotalAmount(),
40             toShippingAddressResponse(
41                 order.getShippingAddress()
42             ),
43             order.getPlacedAt()
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 44–46

```
44         );
45     }
46
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 47–60

```
47     private OrderItemResponse toOrderItemResponse(
48         OrderItem orderItem
49     ) {
50         return new OrderItemResponse(
51             orderItem.getId(),
52             orderItem.getBookEdition() == null
53                 ? null
54                 : orderItem.getBookEdition().getId(),
55             orderItem.getBookEdition() == null
56                 ? null
57                 : orderItem.getBookEdition().getBook().getId(),
```

```
58         orderItem.getBookTitle(),
59         orderItem.getCoverImageUrl(),
60         orderItem.getBookFormat(),
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 61–70

```
61         orderItem.getEditionName(),
62         orderItem.getIsbn10(),
63         orderItem.getIsbn13(),
64         orderItem.getPublisherName(),
65         orderItem.getUnitPrice(),
66         orderItem.getQuantity(),
67         orderItem.getSubtotal()
68     );
69 }
70
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 71–84

```
71     private ShippingAddressResponse toShippingAddressResponse(
72         Address address
73     ) {
74         return new ShippingAddressResponse(
75             address.getRecipientName(),
76             address.getPhone(),
77             address.getAddressLine1(),
78             address.getAddressLine2(),
79             address.getCity(),
80             address.getState(),
81             address.getPostalCode(),
82             address.getCountry()
83         );
84     }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 85–86

```
85 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/main/java/com/akshara/api/review/controller/ PublicReviewController.java

File 55 of 99 · Added or modified · Reviews and discussions

Purpose: Defines HTTP routes for the reviews and discussions module and delegates business decisions to services.

Current file length: 47 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.review.controller;
2
3 import com.akshara.api.review.dto.BookReviewsResponse;
4 import com.akshara.api.review.dto.ReviewResponse;
5 import com.akshara.api.review.dto.ReviewReplyResponse;
6 import com.akshara.api.review.service.ReviewService;
7 import jakarta.validation.constraints.Max;
8 import jakarta.validation.constraints.Min;
9 import jakarta.validation.constraints.Positive;
10 import org.springframework.validation.annotation.Validated;
11 import org.springframework.web.bind.annotation.GetMapping;
12 import org.springframework.web.bind.annotation.PathVariable;
13 import org.springframework.web.bind.annotation.RequestMapping;
14 import org.springframework.web.bind.annotation.RequestParam;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–18

```
15 import org.springframework.web.bind.annotation.RestController;
16
17 import java.util.List;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 19–23

```
19 @RestController
20 @RequestMapping("/api/public")
21 @Validated
22 public class PublicReviewController {
23
```

How to read this block

- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.
- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

Lines 24–29

```
24     private final ReviewService reviewService;
25
26     public PublicReviewController(ReviewService reviewService) {
27         this.reviewService = reviewService;
28     }
29
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 30–34

```
30     @GetMapping("/books/{bookId}/reviews")
31     public BookReviewsResponse getBookReviews(@PathVariable @Positive Long bookId) {
32         return reviewService.getBookReviews(bookId);
33     }
34
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 35–41

```
35     @GetMapping("/reviews/recent")
36     public List<ReviewResponse> getRecent(
37         @RequestParam(defaultValue = "6") @Min(1) @Max(20) int limit
38     ) {
39         return reviewService.getRecent(limit);
40     }
41
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns

become DTOs or domain values for the caller.

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 42–47

```
42     @GetMapping("/reviews/{reviewId}/replies")
43     public List<ReviewReplyResponse> getReplies(@PathVariable @Positive Long reviewId) {
44         return reviewService.getReplies(reviewId);
45     }
46 }
47
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/review/controller/

ReviewController.java

File 56 of 99 · Added or modified · Reviews and discussions

Purpose: Defines HTTP routes for the reviews and discussions module and delegates business decisions to services.

Current file length: 98 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.review.controller;
2
3 import com.akshara.api.review.dto.ReviewRequest;
4 import com.akshara.api.review.dto.ReviewResponse;
5 import com.akshara.api.review.dto.UpdateReviewRequest;
6 import com.akshara.api.review.service.ReviewService;
7 import com.akshara.api.review.dto.ReviewReplyRequest;
8 import com.akshara.api.review.dto.ReviewReplyResponse;
9 import jakarta.validation.Valid;
10 import jakarta.validation.constraints.Positive;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.http.ResponseEntity;
13 import org.springframework.security.core.annotation.AuthenticationPrincipal;
14 import org.springframework.security.oauth2.jwt.Jwt;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–24

```
15 import org.springframework.validation.annotation.Validated;
16 import org.springframework.web.bind.annotation.DeleteMapping;
17 import org.springframework.web.bind.annotation.PathVariable;
18 import org.springframework.web.bind.annotation.PostMapping;
19 import org.springframework.web.bind.annotation.PutMapping;
20 import org.springframework.web.bind.annotation.RequestBody;
21 import org.springframework.web.bind.annotation.RequestMapping;
22 import org.springframework.web.bind.annotation.RestController;
23 import org.springframework.web.bind.annotation.GetMapping;
24
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 25–31

```
25 import java.util.List;
26
27 @RestController
28 @RequestMapping("/api/reviews")
29 @Validated
30 public class ReviewController {
31
```


How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Spring registers this class as an HTTP controller. Mapping annotations connect URLs and HTTP verbs to Java methods.
- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

Lines 32–37

```
32     private final ReviewService reviewService;
33
34     public ReviewController(ReviewService reviewService) {
35         this.reviewService = reviewService;
36     }
37
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 38–42

```
38     @GetMapping("/me")
39     public List<ReviewResponse> getMine(@AuthenticationPrincipal Jwt jwt) {
40         return reviewService.getMine(jwt.getSubject());
41     }
42
```

How to read this block

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 43–51

```
43     @PostMapping
44     public ResponseEntity<ReviewResponse> create(
45         @AuthenticationPrincipal Jwt jwt,
46         @Valid @RequestBody ReviewRequest request
47     ) {
48         return ResponseEntity.status(HttpStatus.CREATED)
49             .body(reviewService.create(jwt.getSubject(), request));
50     }
51
```

```
50     }  
51 }
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 52–60

```
52     @PutMapping("/{reviewId}")  
53     public ReviewResponse update(  
54         @AuthenticationPrincipal Jwt jwt,  
55         @PathVariable @Positive Long reviewId,  
56         @Valid @RequestBody UpdateReviewRequest request  
57     ) {  
58         return reviewService.update(jwt.getSubject(), reviewId, request);  
59     }  
60 }
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 61–69

```
61     @DeleteMapping("/{reviewId}")  
62     public ResponseEntity<Void> delete(  
63         @AuthenticationPrincipal Jwt jwt,  
64         @PathVariable @Positive Long reviewId  
65     ) {  
66         reviewService.delete(jwt.getSubject(), reviewId);  
67         return ResponseEntity.noContent().build();  
68     }  
69 }
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles

rules that depend on database state.

- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 70–79

```
70     @PostMapping("/{reviewId}/replies")
71     public ResponseEntity<ReviewReplyResponse> createReply(
72         @AuthenticationPrincipal Jwt jwt,
73         @PathVariable @Positive Long reviewId,
74         @Valid @RequestBody ReviewReplyRequest request
75     ) {
76         return ResponseEntity.status(HttpStatus.CREATED)
77             .body(reviewService.createReply(jwt.getSubject(), reviewId, request));
78     }
79 }
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 80–88

```
80     @PutMapping("/replies/{replyId}")
81     public ReviewReplyResponse updateReply(
82         @AuthenticationPrincipal Jwt jwt,
83         @PathVariable @Positive Long replyId,
84         @Valid @RequestBody ReviewReplyRequest request
85     ) {
86         return reviewService.updateReply(jwt.getSubject(), replyId, request);
87     }
88 }
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 89–98

```
89     @DeleteMapping("/replies/{replyId}")
90     public ResponseEntity<Void> deleteReply(
91         @AuthenticationPrincipal Jwt jwt,
92         @PathVariable @Positive Long replyId
93     ) {
94         reviewService.deleteReply(jwt.getSubject(), replyId);
95         return ResponseEntity.noContent().build();
96     }
97 }
98
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- The mapping declares the public HTTP contract. Path variables identify resources, request parameters control searches, and request bodies carry JSON or multipart data.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/review/dto/

ReviewReplyRequest.java

File 57 of 99 · Added or modified · Reviews and discussions

Purpose: Defines a validated API boundary type used by the reviews and discussions module.

Current file length: 12 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.review.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.Size;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–12

```
6 public record ReviewReplyRequest(
7     @NotBlank(message = "Reply text is required")
8     @Size(min = 2, max = 1000, message = "Reply must contain between 2 and 1000 characters")
9     String content
10 ) {
11 }
12
```

How to read this block

- Jakarta Validation rejects malformed input before unsafe business work proceeds. Service validation still handles rules that depend on database state.
- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/review/dto/

ReviewReplyResponse.java

File 58 of 99 · Added or modified · Reviews and discussions

Purpose: Defines a validated API boundary type used by the reviews and discussions module.

Current file length: 15 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–4

```
1 package com.akshara.api.review.dto;
2
3 import java.time.Instant;
4
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 5–15

```
5 public record ReviewReplyResponse(
6     Long id,
7     Long reviewId,
8     Long userId,
9     String readerName,
10    String content,
11    Instant createdAt,
12    Instant updatedAt
13 ) {
14 }
15
```

How to read this block

- A Java record is an immutable data carrier. The compiler supplies a constructor, accessors, equality, hash code and text representation.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

backend/src/main/java/com/akshara/api/review/entity/

Review.java

File 59 of 99 · Added or modified · Reviews and discussions

Purpose: Models persisted domain state for the reviews and discussions module using JPA mappings.

Current file length: 107 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.review.entity;
2
3 import com.akshara.api.book.entity.Book;
4 import com.akshara.api.user.entity.AppUser;
5 import jakarta.persistence.Column;
6 import jakarta.persistence.Entity;
7 import jakarta.persistence.FetchType;
8 import jakarta.persistence.GeneratedValue;
9 import jakarta.persistence.GenerationType;
10 import jakarta.persistence.Id;
11 import jakarta.persistence.Index;
12 import jakarta.persistence.JoinColumn;
13 import jakarta.persistence.ManyToOne;
14 import jakarta.persistence.OneToMany;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–20

```
15 import jakarta.persistence.CascadeType;
16 import jakarta.persistence.PrePersist;
17 import jakarta.persistence.PreUpdate;
18 import jakarta.persistence.Table;
19 import jakarta.persistence.UniqueConstraint;
20
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 21–24

```
21 import java.time.Instant;
22 import java.util.ArrayList;
23 import java.util.List;
24
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 25–38

```
25 @Entity
26 @Table(
27     name = "reviews",
28     uniqueConstraints = @UniqueConstraint(
29         name = "uk_reviews_user_book",
30         columnNames = {"user_id", "book_id"}
31     ),
32     indexes = {
33         @Index(name = "idx_reviews_book", columnList = "book_id"),
34         @Index(name = "idx_reviews_created", columnList = "created_at")
35     }
36 )
37 public class Review {
38
```

How to read this block

- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.

Lines 39–42

```
39 @Id
40 @GeneratedValue(strategy = GenerationType.IDENTITY)
41 private Long id;
42
```

How to read this block

- This field is the database identity. Its value is generated when a new entity is inserted.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 43–46

```
43 @ManyToOne(fetch = FetchType.LAZY, optional = false)
44 @JoinColumn(name = "user_id", nullable = false)
45 private AppUser user;
46
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 47–50

```
47 @ManyToOne(fetch = FetchType.LAZY, optional = false)
48 @JoinColumn(name = "book_id", nullable = false)
49 private Book book;
50
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and

makes dependencies explicit.

Lines 51–53

```
51     @Column(nullable = false)
52     private Integer rating;
53
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 54–56

```
54     @Column(length = 120)
55     private String headline;
56
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 57–59

```
57     @Column(nullable = false, columnDefinition = "TEXT")
58     private String content;
59
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 60–62

```
60     @Column(name = "created_at", nullable = false, updatable = false)
61     private Instant createdAt;
62
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 63–65

```
63     @Column(name = "updated_at", nullable = false)
64     private Instant updatedAt;
65
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 66–68

```
66     @OneToMany(mappedBy = "review", cascade = CascadeType.ALL, orphanRemoval = true)
67     private List<ReviewReply> replies = new ArrayList<>();
68
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 69–71

```
69     protected Review() {
70     }
71
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 72–79

```
72     public Review(AppUser user, Book book, Integer rating, String headline, String content) {
73         this.user = user;
74         this.book = book;
75         this.rating = rating;
76         this.headline = headline;
77         this.content = content;
78     }
79
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 80–86

```
80     @PrePersist
81     void beforeInsert() {
82         Instant now = Instant.now();
83         createdAt = now;
84         updatedAt = now;
85     }
86
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 87–91

```
87     @PreUpdate
88     void beforeUpdate() {
89         updatedAt = Instant.now();
90     }
91
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 92–100

```
92     public Long getId() { return id; }
93     public AppUser getUser() { return user; }
94     public Book getBook() { return book; }
95     public Integer getRating() { return rating; }
96     public String getHeadline() { return headline; }
97     public String getContent() { return content; }
98     public Instant getCreatedAt() { return createdAt; }
99     public Instant getUpdatedAt() { return updatedAt; }
100
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 101–107

```
101     public void update(Integer rating, String headline, String content) {
102         this.rating = rating;
103         this.headline = headline;
104         this.content = content;
105     }
106 }
107
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

backend/src/main/java/com/akshara/api/review/entity/

ReviewReply.java

File 60 of 99 · Added or modified · Reviews and discussions

Purpose: Persists a reader reply belonging to one review and one author.

Current file length: 62 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.review.entity;
2
3 import com.akshara.api.user.entity.AppUser;
4 import jakarta.persistence.*;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–14

```
6 import java.time.Instant;
7
8 @Entity
9 @Table(name = "review_replies", indexes = {
10     @Index(name = "idx_review_replies_review_created", columnList = "review_id, created_at"),
11     @Index(name = "idx_review_replies_user", columnList = "user_id")
12 })
13 public class ReviewReply {
14
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.

Lines 15–18

```
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
```

How to read this block

- This field is the database identity. Its value is generated when a new entity is inserted.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 19–22

```
19     @ManyToOne(fetch = FetchType.LAZY, optional = false)
20     @JoinColumn(name = "review_id", nullable = false)
21     private Review review;
22
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 23–26

```
23     @ManyToOne(fetch = FetchType.LAZY, optional = false)
24     @JoinColumn(name = "user_id", nullable = false)
25     private AppUser user;
26
```

How to read this block

- These annotations describe a database relationship and its foreign-key/ownership behavior; fetch and cascade settings affect loading and lifecycle operations.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 27–29

```
27     @Column(nullable = false, columnDefinition = "TEXT")
28     private String content;
29
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 30–32

```
30     @Column(name = "created_at", nullable = false, updatable = false)
31     private Instant createdAt;
32
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 33–35

```
33     @Column(name = "updated_at", nullable = false)
34     private Instant updatedAt;
35
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and

makes dependencies explicit.

Lines 36–43

```
36     protected ReviewReply() { }
37
38     public ReviewReply(Review review, AppUser user, String content) {
39         this.review = review;
40         this.user = user;
41         this.content = content;
42     }
43
```

How to read this block

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 44–50

```
44     @PrePersist
45     void beforeInsert() {
46         Instant now = Instant.now();
47         createdAt = now;
48         updatedAt = now;
49     }
50
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 51–53

```
51     @PreUpdate
52     void beforeUpdate() { updatedAt = Instant.now(); }
53
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 54–62

```
54     public Long getId() { return id; }
55     public Review getReview() { return review; }
56     public AppUser getUser() { return user; }
57     public String getContent() { return content; }
58     public Instant getCreatedAt() { return createdAt; }
59     public Instant getUpdatedAt() { return updatedAt; }
60     public void update(String content) { this.content = content; }
61 }
62
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- Fields hold injected collaborators, constants or entity state. ``final`` prevents reassignment after construction and makes dependencies explicit.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

backend/src/main/java/com/akshara/api/review/repository/

ReviewReplyRepository.java

File 61 of 99 · Added or modified · Reviews and discussions

Purpose: Defines Spring Data persistence queries required by the reviews and discussions module.

Current file length: 13 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 package com.akshara.api.review.repository;
2
3 import com.akshara.api.review.entity.ReviewReply;
4 import org.springframework.data.jpa.repository.EntityGraph;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 7–13

```
7 import java.util.List;
8
9 public interface ReviewReplyRepository extends JpaRepository<ReviewReply, Long> {
10     @EntityGraph(attributePaths = "user")
11     List<ReviewReply> findAllByReview_IdOrderByCreatedAtAsc(Long reviewId);
12 }
13
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- JPA maps this class to relational data. The table annotation controls the database table or indexes/constraints when present.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

backend/src/main/java/com/akshara/api/review/repository/

ReviewRepository.java

File 62 of 99 · Added or modified · Reviews and discussions

Purpose: Defines Spring Data persistence queries required by the reviews and discussions module.

Current file length: 23 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–7

```
1 package com.akshara.api.review.repository;
2
3 import com.akshara.api.review.entity.Review;
4 import org.springframework.data.domain.Page;
5 import org.springframework.data.domain.Pageable;
6 import org.springframework.data.jpa.repository.JpaRepository;
7
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 8–10

```
8 import java.util.List;
9 import java.util.Optional;
10
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 11–14

```
11 public interface ReviewRepository extends JpaRepository<Review, Long> {
12
13     List<Review> findAllByBook_IdOrderByCreatedAtDesc(Long bookId);
14
```

How to read this block

- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 15–18

```
15    Optional<Review> findByUser_IdAndBook_Id(Long userId, Long bookId);
16
17    Page<Review> findAllByOrderByCreatedAtDesc(Pageable pageable);
18
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 19–23

```
19    List<Review> findAllByUser_IdOrderByUpdatedAtDesc(Long userId);
20
21    void deleteAllByBook_Id(Long bookId);
22 }
23
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

backend/src/main/java/com/akshara/api/review/service/

ReviewService.java

File 63 of 99 · Added or modified · Reviews and discussions

Purpose: Owns review CRUD, recent-community queries and ownership-protected discussion reply operations.

Current file length: 196 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.review.service;
2
3 import com.akshara.api.book.entity.Book;
4 import com.akshara.api.book.repository.BookRepository;
5 import com.akshara.api.common.exception.DuplicateResourceException;
6 import com.akshara.api.common.exception.ResourceNotFoundException;
7 import com.akshara.api.review.dto.BookReviewsResponse;
8 import com.akshara.api.review.dto.ReviewRequest;
9 import com.akshara.api.review.dto.ReviewResponse;
10 import com.akshara.api.review.dto.UpdateReviewRequest;
11 import com.akshara.api.review.dto.ReviewReplyRequest;
12 import com.akshara.api.review.dto.ReviewReplyResponse;
13 import com.akshara.api.review.entity.Review;
14 import com.akshara.api.review.entity.ReviewReply;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–22

```
15 import com.akshara.api.review.repository.ReviewRepository;
16 import com.akshara.api.review.repository.ReviewReplyRepository;
17 import com.akshara.api.user.entity.AppUser;
18 import com.akshara.api.user.service.UserService;
19 import org.springframework.data.domain.PageRequest;
20 import org.springframework.stereotype.Service;
21 import org.springframework.transaction.annotation.Transactional;
22
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 23–28

```
23 import java.util.List;
24
25 @Service
26 @Transactional
```

```
27 public class ReviewService {
28
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Spring creates one service bean. Controllers depend on it through constructor injection, keeping business rules out of HTTP code.
- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.

Lines 29–33

```
29 private final ReviewRepository reviewRepository;
30 private final BookRepository bookRepository;
31 private final UserService userService;
32 private final ReviewReplyRepository reviewReplyRepository;
33
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 34–45

```
34 public ReviewService(
35     ReviewRepository reviewRepository,
36     BookRepository bookRepository,
37     UserService userService,
38     ReviewReplyRepository reviewReplyRepository
39 ) {
40     this.reviewRepository = reviewRepository;
41     this.bookRepository = bookRepository;
42     this.userService = userService;
43     this.reviewReplyRepository = reviewReplyRepository;
44 }
45
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 46–51

```
46 @Transactional(readOnly = true)
47 public BookReviewsResponse getBookReviews(Long bookId) {
48     if (!bookRepository.existsById(bookId)) {
49         throw new ResourceNotFoundException("Book not found with id: " + bookId);
50     }
51
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 52–57

```
52         List<ReviewResponse> reviews = reviewRepository
53             .findAllByBook_IdOrderByCreatedAtDesc(bookId)
54             .stream()
55             .map(this::toResponse)
56             .toList();
57
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 58–62

```
58         double average = reviews.stream()
59             .mapToInt(ReviewResponse::rating)
60             .average()
61             .orElse(0.0);
62
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 63–65

```
63         return new BookReviewsResponse(bookId, reviews.size(), average, reviews);
64     }
65
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 66–74

```
66     @Transactional(readonly = true)
67     public List<ReviewResponse> getRecent(int limit) {
68         return reviewRepository
69             .findAllByOrderByCreatedAtDesc(PageRequest.of(0, limit))
70             .stream()
71             .map(this::toResponse)
72             .toList();
73     }
74
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 75–81

```
75     @Transactional(readonly = true)
76     public List<ReviewResponse> getMine(String subject) {
77         AppUser user = userService.getCurrentUserEntity(subject);
78         return reviewRepository.findAllByUser_IdOrderByUpdatedAtDesc(user.getId())
79             .stream().map(this::toResponse).toList();
80     }
81
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 82–90

```
82     @Transactional(readonly = true)
```

```

83     public List<ReviewReplyResponse> getReplies(Long reviewId) {
84         if (!reviewRepository.existsById(reviewId)) {
85             throw new ResourceNotFoundException("Review with ID " + reviewId + " was not found");
86         }
87         return reviewReplyRepository.findAllByReview_IdOrderByCreatedAtAsc(reviewId)
88             .stream().map(this::toReplyResponse).toList();
89     }
90

```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.
- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 91–98

```

91     public ReviewReplyResponse createReply(String subject, Long reviewId, ReviewReplyRequest request) {
92         AppUser user = userService.getCurrentUserEntity(subject);
93         Review review = reviewRepository.findById(reviewId)
94             .orElseThrow(() -> new ResourceNotFoundException("Review with ID " + reviewId + " was not found"));
95         ReviewReply reply = new ReviewReply(review, user, request.content().trim());
96         return toReplyResponse(reviewReplyRepository.save(reply));
97     }
98

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 99–105

```

99     public ReviewReplyResponse updateReply(String subject, Long replyId, ReviewReplyRequest request) {
100         AppUser user = userService.getCurrentUserEntity(subject);
101         ReviewReply reply = findOwnedReply(replyId, user.getId());
102         reply.update(request.content().trim());
103         return toReplyResponse(reviewReplyRepository.save(reply));
104     }
105

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 106–110

```
106     public void deleteReply(String subject, Long replyId) {
107         AppUser user = userService.getCurrentUserEntity(subject);
108         reviewReplyRepository.delete(findOwnedReply(replyId, user.getId()));
109     }
110
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 111–113

```
111     public ReviewResponse create(String subject, ReviewRequest request) {
112         AppUser user = userService.getCurrentUserEntity(subject);
113
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 114–117

```
114         if (reviewRepository.findByUser_IdAndBook_Id(user.getId(), request.bookId()).isPresent()) {
115             throw new DuplicateResourceException("You have already reviewed this book");
116         }
117
```

How to read this block

- Fail-fast domain exceptions stop invalid state changes. The global exception handler later translates them into a stable HTTP status and message.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 118–122

```
118         Book book = bookRepository.findById(request.bookId())
119             .orElseThrow(() -> new ResourceNotFoundException(
120                 "Book not found with id: " + request.bookId()
121             ));
122
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC

in the business layer.

Lines 123–130

```
123         Review review = new Review(  
124             user,  
125             book,  
126             request.rating(),  
127             normalizeHeadline(request.headline()),  
128             request.content().trim()  
129         );  
130
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 131–133

```
131         return toResponse(reviewRepository.save(review));  
132     }  
133
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 134–137

```
134     public ReviewResponse update(String subject, Long reviewId, UpdateReviewRequest request) {  
135         AppUser user = userService.getCurrentUserEntity(subject);  
136         Review review = findOwned(reviewId, user.getId());  
137
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 138–143

```
138         review.update(  
139             request.rating(),  
140             normalizeHeadline(request.headline()),  
141             request.content().trim()  
142         );  
143
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 144–146

```
144         return toResponse(reviewRepository.save(review));  
145     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 147–151

```

147     public void delete(String subject, Long reviewId) {
148         AppUser user = userService.getCurrentUserEntity(subject);
149         reviewRepository.delete(findOwned(reviewId, user.getId()));
150     }
151 
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 152–159

```

152     private Review findOwned(Long reviewId, Long userId) {
153         return reviewRepository.findById(reviewId)
154             .filter(review -> review.getUser().getId().equals(userId))
155             .orElseThrow(() -> new ResourceNotFoundException(
156                 "Review with ID " + reviewId + " was not found"
157             ));
158     }
159 
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 160–165

```

160     private ReviewReply findOwnedReply(Long replyId, Long userId) {
161         return reviewReplyRepository.findById(replyId)
162             .filter(reply -> reply.getUser().getId().equals(userId))
163             .orElseThrow(() -> new ResourceNotFoundException("Discussion reply with ID " + replyId + " was not
found"));
164     }
165 
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 166–172

```
166     private String normalizeHeadline(String headline) {  
167         if (headline == null || headline.isBlank()) {  
168             return null;  
169         }  
170         return headline.trim();  
171     }  
172
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 173–186

```
173     private ReviewResponse toResponse(Review review) {  
174         return new ReviewResponse(  
175             review.getId(),  
176             review.getBook().getId(),  
177             review.getBook().getTitle(),  
178             review.getBook().getCoverImageUrl(),  
179             review.getUser().getId(),  
180             review.getUser().getFullName(),  
181             review.getRating(),  
182             review.getHeadline(),  
183             review.getContent(),  
184             review.getCreatedAt(),  
185             review.getUpdatedAt()  
186         );  
187
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 187–196

```
187     }
188
189     private ReviewReplyResponse toReplyResponse(ReviewReply reply) {
190         return new ReviewReplyResponse(
191             reply.getId(), reply.getReview().getId(), reply.getUser().getId(),
192             reply.getUser().getFullName(), reply.getContent(), reply.getCreatedAt(), reply.getUpdatedAt()
193         );
194     }
195 }
196
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

backend/src/main/java/com/akshara/api/wishlist/repository/

WishlistItemRepository.java

File 64 of 99 · Added or modified · Wishlist

Purpose: Defines Spring Data persistence queries required by the wishlist module.

Current file length: 17 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.wishlist.repository;
2
3 import com.akshara.api.wishlist.entity.WishlistItem;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 6–8

```
6 import java.util.List;
7 import java.util.Optional;
8
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 9–12

```
9 public interface WishlistItemRepository extends JpaRepository<WishlistItem, Long> {
10
11     List<WishlistItem> findAllByUser_IdOrderByCreatedAtDesc(Long userId);
12
```

How to read this block

- Spring Data generates the repository implementation at runtime; method names describe the SQL-style lookup or delete operation.
- The interface/implementation boundary keeps provider or persistence details replaceable and makes isolated testing easier.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 13–17

```
13     Optional<WishlistItem> findByUser_IdAndBook_Id(Long userId, Long bookId);  
14  
15     void deleteAllByBook_Id(Long bookId);  
16 }  
17
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

backend/src/main/resources/application.properties

File 65 of 99 · Added or modified · Runtime configuration

Purpose: Configures MySQL, JPA, JWT, Open Library timeouts/contact identity and private upload limits/storage.

Current file length: 29 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–3

```
1 spring.application.name=akshara-api
2 server.port=8080
3
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 4–8

```
4 # MySQL connection
5 spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/akshara}
6 spring.datasource.username=${DB_USERNAME:akshara_app}
7 spring.datasource.password=${DB_PASSWORD}
8
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 9–14

```
9 # JPA and Hibernate
10 spring.jpa.hibernate.ddl-auto=update
11 spring.jpa.show-sql=true
12 spring.jpa.properties.hibernate.format_sql=true
13 spring.jpa.open-in-view=false
14
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 15–18

```
15 app.jwt.secret=${JWT_SECRET}
16 app.jwt.issuer=${JWT_ISSUER:akshara-api}
17 app.jwt.expiration-seconds=${JWT_EXPIRATION_SECONDS:3600}
18
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 19–24

```
19 # External catalogue metadata (commerce data is never sent to these APIs)
20 app.book-import.contact=${BOOK_IMPORT_CONTACT:admin@akshara.local}
21 app.book-metadata.open-library.base-url=https://openlibrary.org
22 app.book-metadata.open-library.connect-timeout-ms=5000
23 app.book-metadata.open-library.read-timeout-ms=25000
24
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 25–29

```
25 # Private reader uploads. Files stay outside the public catalogue and require authentication.
26 app.personal-library.storage-path=${PERSONAL_LIBRARY_STORAGE:./data/personal-library}
27 spring.servlet.multipart.max-file-size=25MB
28 spring.servlet.multipart.max-request-size=26MB
29
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/test/java/com/akshara/api/

AksharaApiApplicationTests.java

File 66 of 99 · Added or modified · Automated verification

Purpose: Supports automated verification in the current Akshara implementation.

Current file length: 21 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–15

```
6 @SpringBootTest(properties = {
7     "spring.datasource.url=jdbc:h2:mem:context_test;MODE=MySQL;DB_CLOSE_DELAY=-1",
8     "spring.datasource.driver-class-name=org.h2.Driver",
9     "spring.datasource.username=sa",
10    "spring.datasource.password=",
11    "spring.jpa.hibernate.ddl-auto=create-drop",
12    "app.jwt.secret=01234567890123456789012345678901"
13 })
14 class AksharaApiApplicationTests {
15
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 16–19

```
16 @Test
17 void contextLoads() {
18 }
19
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 20–21

```
20 }
21
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

backend/src/test/java/com/akshara/api/book/controller/ AdminCatalogueControllerTest.java

File 67 of 99 · Added or modified · Catalogue and inventory

Purpose: Verifies AdminCatalogueController behavior, including success paths and important validation, authorization or error cases.

Current file length: 146 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.book.controller;
2
3 import com.akshara.api.book.dto.InventoryRequest;
4 import com.akshara.api.book.dto.InventoryResponse;
5 import com.akshara.api.book.entity.AvailabilityStatus;
6 import com.akshara.api.book.service.BookService;
7 import com.akshara.api.bookimport.service.BookImportService;
8 import com.akshara.api.config.SecurityConfig;
9 import org.junit.jupiter.api.Test;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
12 import org.springframework.context.annotation.Import;
13 import org.springframework.http.MediaType;
14 import org.springframework.security.core.authority.SimpleGrantedAuthority;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–18

```
15 import org.springframework.security.oauth2.jwt.JwtDecoder;
16 import org.springframework.test.context.bean.override.mockito.MockitoBean;
17 import org.springframework.test.web.servlet.MockMvc;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 19–21

```
19 import java.math.BigDecimal;
20 import java.time.Instant;
21
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 22–33

```
22 import static org.mockito.ArgumentMatchers.any;
23 import static org.mockito.Mockito.verify;
24 import static org.mockito.Mockito.verifyNoInteractions;
25 import static org.mockito.Mockito.when;
26 import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.jwt;
27 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.patch;
28 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
29 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
30 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
31 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
32 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
33
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 34–37

```
34 @WebMvcTest({AdminBookController.class, AdminEditionController.class})
35 @Import({SecurityConfig.class})
36 class AdminCatalogueControllerTest {
37
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 38–42

```
38 @Autowired private MockMvc mockMvc;
39 @MockitoBean private BookService bookService;
40 @MockitoBean private BookImportService bookImportService;
41 @MockitoBean private JwtDecoder jwtDecoder;
42
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 43–48

```
43 @Test
44 void adminCanUpdateEditionInventory() throws Exception {
45     InventoryResponse response = inventoryResponse(9, AvailabilityStatus.IN_STOCK);
46     when(bookService.updateInventory(any(Long.class), any(InventoryRequest.class)))
47         .thenReturn(response);
48
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 49–62

```
49      mockMvc.perform(put("/api/admin/editions/7/inventory")
50                      .with(adminJwt())
51                      .contentType(MediaType.APPLICATION_JSON)
52                      .content("""
53                          {
54                              "price": 549.00,
55                              "stockQuantity": 9,
56                              "availabilityStatus": "IN_STOCK",
57                              "active": true
58                          }
59                      """))
60      .andExpect(status().isOk())
61      .andExpect(jsonPath("$.stockQuantity").value(9))
62      .andExpect(jsonPath("$.price").value(549.00));
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 63–72

```
63
64      verify(bookService).updateInventory(
65          7L,
66          new InventoryRequest(
67              new BigDecimal("549.00"), 9,
68              AvailabilityStatus.IN_STOCK, true
69          )
70      );
71  }
72
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 73–77

```
73      @Test
74      void adminCanRestockAnEdition() throws Exception {
75          when(bookService.restockInventory(7L, 12))
76              .thenReturn(inventoryResponse(12, AvailabilityStatus.IN_STOCK));
77      }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 78–84

```
78         mockMvc.perform(post("/api/admin/editions/7/inventory/restock")
79                         .with(adminJwt())
80                         .contentType(MediaType.APPLICATION_JSON)
81                         .content("{\"quantity\":12}"))
82         .andExpect(status().isOk())
83         .andExpect(jsonPath("$.stockQuantity").value(12));
84
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 85–87

```
85         verify(bookService).restockInventory(7L, 12);
86     }
87
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 88–95

```
88     @Test
89     void invalidRestockQuantityIsRejectedBeforeServiceCall() throws Exception {
90         mockMvc.perform(post("/api/admin/editions/7/inventory/restock")
91                         .with(adminJwt())
92                         .contentType(MediaType.APPLICATION_JSON)
93                         .content("{\"quantity\":0}"))
94         .andExpect(status().isBadRequest());
95
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 96–98

```
96         verifyNoInteractions(bookService, bookImportService);
97     }
98
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 99–108

```
99     @Test
100     void readerCannotChangeInventory() throws Exception {
101         mockMvc.perform(post("/api/admin/editions/7/inventory/restock")
102             .with(jwt().authorities(
103                 new SimpleGrantedAuthority("ROLE_READER")
104             ))
105             .contentType(MediaType.APPLICATION_JSON)
106             .content("{\"quantity\":3}"))
107         .andExpect(status().isForbidden());
108     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 109–111

```
109         verifyNoInteractions(bookService, bookImportService);
110     }
111 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 112–119

```
112     @Test
113     void adminCanChangeFeaturedStatus() throws Exception {
114         mockMvc.perform(patch("/api/admin/books/4/featured")
115             .with(adminJwt())
116             .contentType(MediaType.APPLICATION_JSON)
117             .content("{\"featured\":true}"))
118         .andExpect(status().isOk());
119     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 120–122

```
120         verify(bookService).updateFeatured(4L, true);
121     }
122 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 123–128

```
123     @Test
124     void adminCanDeleteAnUnreferencedBook() throws Exception {
125         mockMvc.perform(delete("/api/admin/books/4")
126             .with(adminJwt()))
127             .andExpect(status().isNoContent());
128     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 129–131

```
129         verify(bookService).delete(4L);
130     }
131 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 132–141

```
132     private InventoryResponse inventoryResponse(
133         int stock,
134         AvailabilityStatus status
135     ) {
136         return new InventoryResponse(
137             3L, new BigDecimal("549.00"), stock,
138             status, true, Instant.parse("2026-08-03T00:00:00Z")
139         );
140     }
141 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 142–146

```
142     private static org.springframework.test.web.servlet.request.RequestPostProcessor adminJwt() {
143         return jwt().authorities(new SimpleGrantedAuthority("ROLE_ADMIN"));
144     }
145 }
146 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

backend/src/test/java/com/akshara/api/book/entity/

InventoryTest.java

File 68 of 99 · Added or modified · Catalogue and inventory

Purpose: Verifies Inventory behavior, including success paths and important validation, authorization or error cases.

Current file length: 218 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 package com.akshara.api.book.entity;
2
3 import com.akshara.api.common.exception.InvalidRequestException;
4 import org.junit.jupiter.api.Test;
5
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–9

```
6 import java.math.BigDecimal;
7
8 import static org.junit.jupiter.api.Assertions.*;
9
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 10–18

```
10 class InventoryTest {
11
12     @Test
13     void decreaseStockShouldReduceAvailableQuantity() {
14         Inventory inventory = createInventory(
15             10,
16             AvailabilityStatus.IN_STOCK
17         );
18
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 19–27

```
19         inventory.decreaseStock(3);
20
```

```

21      assertEquals(7, inventory.getStockQuantity());
22      assertEquals(
23          AvailabilityStatus.IN_STOCK,
24          inventory.getAvailabilityStatus()
25      );
26  }
27

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 28–34

```

28  @Test
29  void decreaseStockShouldMarkInventoryOutOfStockWhenQuantityReachesZero() {
30      Inventory inventory = createInventory(
31          5,
32          AvailabilityStatus.IN_STOCK
33      );
34

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 35–43

```

35      inventory.decreaseStock(5);
36
37      assertEquals(0, inventory.getStockQuantity());
38      assertEquals(
39          AvailabilityStatus.OUT_OF_STOCK,
40          inventory.getAvailabilityStatus()
41      );
42  }
43

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 44–50

```

44  @Test
45  void decreaseStockShouldRejectZeroQuantity() {
46      Inventory inventory = createInventory(
47          10,
48          AvailabilityStatus.IN_STOCK
49      );
50

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 51–55

```
51      assertThrows(
52          InvalidRequestException.class,
53          () -> inventory.decreaseStock(0)
54      );
55
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 56–58

```
56      assertEquals(10, inventory.getStockQuantity());
57  }
58
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 59–65

```
59  @Test
60  void decreaseStockShouldRejectNegativeQuantity() {
61      Inventory inventory = createInventory(
62          10,
63          AvailabilityStatus.IN_STOCK
64      );
65
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 66–70

```
66      assertThrows(
67          InvalidRequestException.class,
68          () -> inventory.decreaseStock(-2)
69      );
70
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important

collaboration where appropriate.

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 71–73

```
71      assertEquals(10, inventory.getStockQuantity());
72  }
73
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 74–81

```
74  @Test
75  void decreaseStockShouldRejectInactiveInventory() {
76      Inventory inventory = createInventory(
77          10,
78          AvailabilityStatus.IN_STOCK
79      );
80      inventory.setActive(false);
81
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 82–86

```
82      assertThrows(
83          InvalidRequestException.class,
84          () -> inventory.decreaseStock(1)
85      );
86
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 87–89

```
87      assertEquals(10, inventory.getStockQuantity());
88  }
89
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 90–96

```
90     @Test
91     void decreaseStockShouldRejectUnavailableInventory() {
92         Inventory inventory = createInventory(
93             10,
94             AvailabilityStatus.UNAVAILABLE
95         );
96     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 97–101

```
97         assertThrows(
98             InvalidRequestException.class,
99             () -> inventory.decreaseStock(1)
100         );
101     }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 102–104

```
102         assertEquals(10, inventory.getStockQuantity());
103     }
104 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 105–111

```
105     @Test
106     void decreaseStockShouldRejectQuantityGreaterThanAvailableStock() {
107         Inventory inventory = createInventory(
108             3,
109             AvailabilityStatus.IN_STOCK
110         );
111     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 112–116

```
112         assertThrows(  
113             InvalidRequestException.class,  
114             () -> inventory.decreaseStock(4)  
115         );  
116
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 117–123

```
117         assertEquals(3, inventory.getStockQuantity());  
118         assertEquals(  
119             AvailabilityStatus.IN_STOCK,  
120             inventory.getAvailabilityStatus()  
121         );  
122     }  
123
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 124–130

```
124     @Test  
125     void increaseStockShouldRestoreQuantity() {  
126         Inventory inventory = createInventory(  
127             4,  
128             AvailabilityStatus.IN_STOCK  
129         );  
130
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 131–139

```
131         inventory.increaseStock(3);  
132  
133         assertEquals(7, inventory.getStockQuantity());  
134         assertEquals(  
135             AvailabilityStatus.IN_STOCK,  
136             inventory.getAvailabilityStatus()  
137         );  
138     }  
139
```

```
135         AvailabilityStatus.IN_STOCK,  
136         inventory.getAvailabilityStatus()  
137     );  
138 }  
139
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 140–146

```
140     @Test  
141     void increaseStockShouldChangeOutOfStockToInStock() {  
142         Inventory inventory = createInventory(  
143             0,  
144             AvailabilityStatus.OUT_OF_STOCK  
145         );  
146
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 147–155

```
147         inventory.increaseStock(2);  
148  
149         assertEquals(2, inventory.getStockQuantity());  
150         assertEquals(  
151             AvailabilityStatus.IN_STOCK,  
152             inventory.getAvailabilityStatus()  
153         );  
154     }  
155
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 156–162

```
156     @Test  
157     void increaseStockShouldPreserveUnavailableStatus() {  
158         Inventory inventory = createInventory(  
159             0,  
160             AvailabilityStatus.UNAVAILABLE  
161         );  
162
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 163–171

```
163         inventory.increaseStock(2);
164
165         assertEquals(2, inventory.getStockQuantity());
166         assertEquals(
167             AvailabilityStatus.UNAVAILABLE,
168             inventory.getAvailabilityStatus()
169         );
170     }
171
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 172–178

```
172     @Test
173     void increaseStockShouldRejectNonPositiveQuantity() {
174         Inventory inventory = createInventory(
175             5,
176             AvailabilityStatus.IN_STOCK
177         );
178
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 179–183

```
179         InvalidRequestException exception = assertThrows(
180             InvalidRequestException.class,
181             () -> inventory.increaseStock(0)
182         );
183
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 184–188

```
184         assertEquals(
185             "Returned quantity must be greater than zero",
186             exception.getMessage()
187         );
188
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 189–191

```
189         assertEquals(5, inventory.getStockQuantity());
190     }
191
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 192–198

```
192     @Test
193     void increaseStockShouldRejectIntegerOverflow() {
194         Inventory inventory = createInventory(
195             Integer.MAX_VALUE,
196             AvailabilityStatus.IN_STOCK
197         );
198
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 199–205

```
199         assertThrows(
200             InvalidRequestException.class,
201             () -> inventory.increaseStock(1)
202         );
203         assertEquals(Integer.MAX_VALUE, inventory.getStockQuantity());
204     }
205
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 206–218

```
206     private Inventory createInventory(
207         int stockQuantity,
208         AvailabilityStatus availabilityStatus
209     ) {
210         return new Inventory(
211             null,
212             new BigDecimal("499.00"),
213             stockQuantity,
```

```
214         availabilityStatus
215     );
216 }
217 }
218
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/test/java/com/akshara/api/bookimport/controller/ AdminBookImportControllerTest.java

File 69 of 99 · Added or modified · Hybrid catalogue import

Purpose: Verifies AdminBookImportController behavior, including success paths and important validation, authorization or error cases.

Current file length: 113 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.controller;
2
3 import com.akshara.api.bookimport.dto.BookImportSource;
4 import com.akshara.api.bookimport.dto.CatalogueSeedResponse;
5 import com.akshara.api.bookimport.dto.ExternalBookSearchPage;
6 import com.akshara.api.bookimport.service.BookImportService;
7 import com.akshara.api.bookimport.service.CatalogueSeedService;
8 import com.akshara.api.bookimport.service.ExternalBookSearchService;
9 import com.akshara.api.config.SecurityConfig;
10 import org.junit.jupiter.api.Test;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
13 import org.springframework.context.annotation.Import;
14 import org.springframework.security.core.authority.SimpleGrantedAuthority;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–18

```
15 import org.springframework.security.oauth2.jwt.JwtDecoder;
16 import org.springframework.test.context.bean.override.mockito.MockitoBean;
17 import org.springframework.test.web.servlet.MockMvc;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 19–21

```
19 import java.util.List;
20 import java.util.Map;
21
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 22–30

```
22 import static org.mockito.Mockito.verify;
23 import static org.mockito.Mockito.verifyNoInteractions;
24 import static org.mockito.Mockito.when;
25 import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.jwt;
26 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
27 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
28 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
29 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
30
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 31–34

```
31 @WebMvcTest(AdminBookImportController.class)
32 @Import(SecurityConfig.class)
33 class AdminBookImportControllerTest {
34
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 35–40

```
35     @Autowired private MockMvc mockMvc;
36     @MockitoBean private ExternalBookSearchService searchService;
37     @MockitoBean private BookImportService importService;
38     @MockitoBean private CatalogueSeedService seedService;
39     @MockitoBean private JwtDecoder jwtDecoder;
40
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 41–48

```
41     @Test
42     void adminCanPopulateCuratedCatalogue() throws Exception {
43         when(seedService.seed(50)).thenReturn(new CatalogueSeedResponse(
44             50, 50, 3, 0, 50,
45             Map.of("Classics", 7, "Science", 7),
46             List.of("Pride and Prejudice", "A Brief History of Time")
47         ));
48
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- The fixed value set models a closed domain choice such as format, availability or status, preventing arbitrary strings.

Lines 49–56

```
49         mockMvc.perform(post("/api/admin/book-import/seed")
50                         .param("count", "50")
51                         .with(adminJwt()))
52         .andExpect(status().isOk())
53         .andExpect(jsonPath("$.imported").value(50))
54         .andExpect(jsonPath("$.catalogueSizeAfter").value(50))
55         .andExpect(jsonPath("$.genreCounts.Classics").value(7));
56
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 57–59

```
57         verify(seedService).seed(50);
58     }
59
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 60–67

```
60     @Test
61     void adminCanSearchWithBoundedPagination() throws Exception {
62         when(searchService.search(
63             "Atomic Habits", BookImportSource.OPEN_LIBRARY, 1, 10
64         )).thenReturn(new ExternalBookSearchPage(
65             BookImportSource.OPEN_LIBRARY, List.of(), 1, 10, 0
66         ));
67
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 68–74

```
68         mockMvc.perform(get("/api/admin/book-import/search")
69                         .param("query", "Atomic Habits")
70                         .with(adminJwt()))
71         .andExpect(status().isOk())
72         .andExpect(jsonPath("$.source").value("OPEN_LIBRARY"))
73         .andExpect(jsonPath("$.results").isArray());
74
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 75–79

```
75         verify(searchService).search(
```

```

76         "Atomic Habits", BookImportSource.OPEN_LIBRARY, 1, 10
77     );
78 }
79

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 80–90

```

80     @Test
81     void readerCannotSearchExternalCatalogue() throws Exception {
82         mockMvc.perform(get("/api/admin/book-import/search")
83             .param("query", "Atomic Habits")
84             .with(jwt().authorities(
85                 new SimpleGrantedAuthority("ROLE_READER")
86             )))
87         .andExpect(status().isForbidden());
88         verifyNoInteractions(searchService, importService, seedService);
89     }
90

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 91–98

```

91     @Test
92     void unauthenticatedClientCannotSearchExternalCatalogue() throws Exception {
93         mockMvc.perform(get("/api/admin/book-import/search")
94             .param("query", "Atomic Habits"))
95         .andExpect(status().isUnauthorized());
96         verifyNoInteractions(searchService, importService, seedService);
97     }
98

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 99–108

```

99     @Test
100    void pageSizeAboveProviderLimitIsRejected() throws Exception {
101        mockMvc.perform(get("/api/admin/book-import/search")
102            .param("query", "Atomic Habits")
103            .param("size", "21")
104            .with(adminJwt()))
105        .andExpect(status().isBadRequest());
106        verifyNoInteractions(searchService, importService, seedService);
107    }
108

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 109–113

```
109     private static org.springframework.test.web.servlet.request.RequestPostProcessor adminJwt() {  
110         return jwt().authorities(new SimpleGrantedAuthority("ROLE_ADMIN"));  
111     }  
112 }  
113
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

backend/src/test/java/com/akshara/api/bookimport/provider/ OpenLibraryBookMetadataProviderTest.java

File 70 of 99 · Added or modified · Hybrid catalogue import

Purpose: Verifies OpenLibraryBookMetadataProvider behavior, including success paths and important validation, authorization or error cases.

Current file length: 95 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–9

```
1 package com.akshara.api.bookimport.provider;
2
3 import com.akshara.api.bookimport.dto.ExternalBookSearchPage;
4 import com.akshara.api.bookimport.dto.ExternalBookSearchQuery;
5 import org.junit.jupiter.api.Test;
6 import org.springframework.http.MediaType;
7 import org.springframework.test.web.client.MockRestServiceServer;
8 import org.springframework.web.client.RestClient;
9
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.

Lines 10–14

```
10 import static org.junit.jupiter.api.Assertions.assertEquals;
11 import static org.junit.jupiter.api.Assertions.assertNull;
12 import static org.springframework.test.web.client.match.MockRestRequestMatchers.queryParam;
13 import static org.springframework.test.web.client.response.MockRestResponseCreators.withSuccess;
14
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The URI builder safely encodes the Open Library search term, pagination and requested metadata fields instead of concatenating an unsafe URL string.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 15–28

```
15 class OpenLibraryBookMetadataProviderTest {
16
17     @Test
18     void normalizesEditionMetadataAndEncodedQuery() {
19         RestClient.Builder builder = RestClient.builder()
20             .baseUrl("https://openlibrary.test");
21         MockRestServiceServer server = MockRestServiceServer.bindTo(builder).build();
```

```

22     server.expect(queryParam("q", "Atomic%20Habits%20%26%20change"))
23         .andExpect(queryParam("limit", "10"))
24         .andExpect(queryParam("offset", "10"))
25         .andRespond(withSuccess("""
26             {"numFound":21,"docs":[{"
27                 "key":"/works/OL1W","title":"Atomic Habits",
28                 "author_name":["James Clear"],"subject":["Habits"],

```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- The URI builder safely encodes the Open Library search term, pagination and requested metadata fields instead of concatenating an unsafe URL string.
- Each `@Test` method is an independently executable behavior specification.

Lines 29–37

```

29         "editions":{"docs":[{"
30             "key":"/books/OL1M","title":"Atomic Habits",
31             "publisher":["Avery"],"publish_date":["2018"],
32             "language":["eng"],"number_of_pages":320,
33             "cover_i":123,"isbn":["0735211299","9780735211292"]
34         }]}
35     }]}
36     """, MediaType.APPLICATION_JSON));
37

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 38–44

```

38     ExternalBookSearchPage result =
39         new OpenLibraryBookMetadataProvider(builder.build()).search(
40             new ExternalBookSearchQuery(
41                 "Atomic Habits & change", 2, 10
42             )
43         );
44

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 45–54

```

45     assertEquals(21, result.totalResults());
46     assertEquals(2, result.page());
47     assertEquals(1, result.results().size());
48     assertEquals("/works/OL1W", result.results().get(0).sourceId());
49     assertEquals("/books/OL1M", result.results().get(0).editionId());
50     assertEquals("9780735211292", result.results().get(0).isbn13());
51     assertEquals("2018-01-01", result.results().get(0).publicationDate());
52     server.verify();
53 }
54

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 55–65

```
55     @Test
56     void missingOptionalFieldsProduceSafeNullsAndLists() {
57         RestClient.Builder builder = RestClient.builder()
58             .baseUrl("https://openlibrary.test");
59         MockRestServiceServer server = MockRestServiceServer.bindTo(builder).build();
60         server.expect(request -> { })
61             .andRespond(withSuccess(
62                 "{\"numFound\":1,\"docs\":[{\"key\":\"/works/X\",\"title\":\"Rare Book\"}]}",
63                 MediaType.APPLICATION_JSON
64             ));
65     }
```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- Each `@Test` method is an independently executable behavior specification.

Lines 66–70

```
66         ExternalBookSearchPage result =
67             new OpenLibraryBookMetadataProvider(builder.build()).search(
68                 new ExternalBookSearchQuery("Rare Book", 1, 10)
69             );
70     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 71–76

```
71         assertEquals("Rare Book", result.results().get(0).title());
72         assertEquals(0, result.results().get(0).authors().size());
73         assertNull(result.results().get(0).coverImageUrl());
74         assertNull(result.results().get(0).isbn13());
75     }
76 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 77–85

```
77     @Test
78     void zeroMatchesReturnSuccessfulEmptyPage() {
79         RestClient.Builder builder = RestClient.builder()
80             .baseUrl("https://openlibrary.test");
81         MockRestServiceServer server = MockRestServiceServer.bindTo(builder).build();
82         server.expect(request -> { }).andRespond(withSuccess(
83             "{\"numFound\":0,\"docs\":[]}", MediaType.APPLICATION_JSON
84         ));
85     }
```

How to read this block

- This code performs a backend HTTP call and deserializes the JSON response. The frontend never contacts the provider directly.
- Each `@Test`` method is an independently executable behavior specification.

Lines 86–90

```
86         ExternalBookSearchPage result =  
87             new OpenLibraryBookMetadataProvider(builder.build()).search(  
88                 new ExternalBookSearchQuery("nothing", 1, 10)  
89             );  
90
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 91–95

```
91         assertEquals(0, result.totalResults());  
92         assertEquals(0, result.results().size());  
93     }  
94 }  
95
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

backend/src/test/java/com/akshara/api/bookimport/service/BookImportServiceIntegrationTest.java

File 71 of 99 · Added or modified · Hybrid catalogue import

Purpose: Verifies BookImportServiceIntegration behavior, including success paths and important validation, authorization or error cases.

Current file length: 280 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.service;
2
3 import com.akshara.api.book.dto.BookResponse;
4 import com.akshara.api.book.dto.InventoryRequest;
5 import com.akshara.api.book.entity.AvailabilityStatus;
6 import com.akshara.api.book.entity.BookFormat;
7 import com.akshara.api.book.repository.AuthorRepository;
8 import com.akshara.api.book.repository.BookEditionRepository;
9 import com.akshara.api.book.repository.BookRepository;
10 import com.akshara.api.book.repository.CategoryRepository;
11 import com.akshara.api.book.repository.InventoryRepository;
12 import com.akshara.api.book.repository.PublisherRepository;
13 import com.akshara.api.book.service.BookService;
14 import com.akshara.api.cart.entity.Cart;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–28

```
15 import com.akshara.api.cart.entity.CartItem;
16 import com.akshara.api.cart.repository.CartItemRepository;
17 import com.akshara.api.cart.repository.CartRepository;
18 import com.akshara.api.bookimport.dto.BookImportRequest;
19 import com.akshara.api.bookimport.dto.BookImportSource;
20 import com.akshara.api.bookimport.dto.ManualBookRequest;
21 import com.akshara.api.common.exception.DuplicateResourceException;
22 import com.akshara.api.common.exception.InvalidRequestException;
23 import com.akshara.api.review.entity.Review;
24 import com.akshara.api.review.repository.ReviewRepository;
25 import com.akshara.api.order.entity.Address;
26 import com.akshara.api.order.entity.Order;
27 import com.akshara.api.order.entity.OrderItem;
28 import com.akshara.api.order.repository.OrderItemRepository;
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 29–39

```
29 import com.akshara.api.order.repository.OrderRepository;
30 import com.akshara.api.user.entity.AppUser;
31 import com.akshara.api.user.entity.Role;
32 import com.akshara.api.user.repository.UserRepository;
33 import com.akshara.api.wishlist.entity.WishlistItem;
34 import com.akshara.api.wishlist.repository.WishlistItemRepository;
35 import org.junit.jupiter.api.Test;
36 import org.springframework.beans.factory.annotation.Autowired;
37 import org.springframework.boot.test.context.SpringBootTest;
38 import org.springframework.transaction.annotation.Transactional;
39
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 40–43

```
40 import java.math.BigDecimal;
41 import java.time.LocalDate;
42 import java.util.List;
43
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 44–48

```
44 import static org.junit.jupiter.api.Assertions.assertEquals;
45 import static org.junit.jupiter.api.Assertions.assertNotNull;
46 import static org.junit.jupiter.api.Assertions.assertFalse;
47 import static org.junit.jupiter.api.Assertions.assertThrows;
48
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 49–59

```
49 @SpringBootTest(properties = {
50     "spring.datasource.url=jdbc:h2:mem:import_test;DB_CLOSE_DELAY=-1",
51     "spring.datasource.driver-class-name=org.h2.Driver",
52     "spring.datasource.username=sa",
53     "spring.datasource.password=",
54     "spring.jpa.hibernate.ddl-auto=create-drop",
55     "app.jwt.secret=01234567890123456789012345678901"
56 })
57 @Transactional
```

```
58 class BookImportServiceIntegrationTest {
59
```

How to read this block

- The method runs as one database transaction: either all required writes commit together or an exception rolls them back. Read-only transactions communicate query intent.

Lines 60–73

```
60     @Autowired private BookImportService service;
61     @Autowired private BookService bookService;
62     @Autowired private BookRepository bookRepository;
63     @Autowired private BookEditionRepository editionRepository;
64     @Autowired private InventoryRepository inventoryRepository;
65     @Autowired private AuthorRepository authorRepository;
66     @Autowired private CategoryRepository categoryRepository;
67     @Autowired private PublisherRepository publisherRepository;
68     @Autowired private UserRepository userRepository;
69     @Autowired private CartRepository cartRepository;
70     @Autowired private CartItemRepository cartItemRepository;
71     @Autowired private WishListItemRepository wishListItemRepository;
72     @Autowired private ReviewRepository reviewRepository;
73     @Autowired private OrderRepository orderRepository;
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 74–81

```
74     @Autowired private OrderItemRepository orderItemRepository;
75
76     @Test
77     void importPersistsCompleteGraphAndNormalizesZeroStock() {
78         BookResponse response = service.importBook(importRequest(
79             "978-0-7352-1129-2", "AKS-ATOMIC-PB", "/books/OL1M"
80         ));
81
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 82–93

```
82         assertNotNull(response.id());
83         assertEquals(1, bookRepository.count());
84         assertEquals(1, editionRepository.count());
85         assertEquals(1, inventoryRepository.count());
86         assertEquals(1, authorRepository.count());
87         assertEquals(1, categoryRepository.count());
88         assertEquals(1, publisherRepository.count());
89         assertEquals("9780735211292", response.editions().get(0).isbn13());
90         assertEquals(AvailabilityStatus.OUT_OF_STOCK,
91             response.editions().get(0).inventory().availabilityStatus());
92     }
93
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 94–102

```
94     @Test
95     void secondEditionFromSameWorkReusesConceptualBookAndNames() {
96         BookResponse first = service.importBook(importRequest(
97             "9780735211292", "AKS-ATOMIC-PB", "/books/OL1M"
98         ));
99         BookResponse second = service.importBook(importRequest(
100             "9780735211308", "AKS-ATOMIC-HB", "/books/OL2M"
101         ));
102     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 103–111

```
103         assertEquals(first.id(), second.id());
104         assertEquals(1, bookRepository.count());
105         assertEquals(2, editionRepository.count());
106         assertEquals(1, authorRepository.count());
107         assertEquals(1, categoryRepository.count());
108         assertEquals(1, publisherRepository.count());
109         assertEquals(2, second.editions().size());
110     }
111 }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 112–117

```
112     @Test
113     void duplicateNormalizedIsbnAndSkuAreRejected() {
114         service.importBook(importRequest(
115             "9780735211292", "AKS-ATOMIC-PB", "/books/OL1M"
116         ));
117     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 118–129

```
118         assertThrows(DuplicateResourceException.class, () ->
119             service.importBook(importRequest(
120                 "978-0-7352-1129-2", "AKS-OTHER", "/books/OL3M"
121             ))
122         );
123         assertThrows(DuplicateResourceException.class, () ->
124             service.importBook(importRequest(
125                 "9780735211308", "aks-atomic-pb", "/books/OL4M"
126             ))
127         );
128     }
129
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 130–140

```
130     @Test
131     void manualEntryCanStoreRareBookWithoutIsbn() {
132         BookResponse response = service.createManualBook(new ManualBookRequest(
133             "Local Oral Histories", null, "A local publication",
134             List.of("A. Archivist"), "Akshara Local Press", "en", 80,
135             List.of("Local History"), null, LocalDate.of(2020, 1, 1),
136             "First edition", null, null, BookFormat.PAPERBACK,
137             new BigDecimal("199.00"), 3, AvailabilityStatus.IN_STOCK,
138             "AKS-LOCAL-001", true, false
139         ));
140
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 141–144

```
141         assertNotNull(response.id());
142         assertEquals("AKS-LOCAL-001", response.editions().get(0).sku());
143     }
144
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 145–158

```
145     @Test
146     void activeDigitalEditionIsRejectedUntilDeliveryIsConfigured() {
147         InvalidRequestException exception = assertThrows(
148             InvalidRequestException.class,
149             () -> service.createManualBook(new ManualBookRequest(
150                 "Licensed eBook", null, "Not yet deliverable",
151                 List.of("Digital Author"), "Digital Press", "en", 120,
```

```

152             List.of("Technology"), null, LocalDate.of(2024, 1, 1),
153             "EPUB edition", null, null, BookFormat.EPUB,
154             new BigDecimal("199.00"), 1, AvailabilityStatus.IN_STOCK,
155             "AKS-DIGITAL-001", true, false
156         ))
157     );
158

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 159–166

```

159         assertEquals(
160             "Digital editions cannot be activated for sale until "
161             + "a licensed file and reader entitlement are configured",
162             exception.getMessage()
163         );
164         assertEquals(0, bookRepository.count());
165     }
166

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 167–173

```

167     @Test
168     void restockAddsUnitsAndMakesAnOutOfStockEditionAvailable() {
169         BookResponse imported = service.importBook(importRequest(
170             "9780735211292", "AKS-ATOMIC-PB", "/books/OL1M"
171         ));
172         Long editionId = imported.editions().get(0).id();
173     }

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 174–179

```

174         var inventory = bookService.restockInventory(editionId, 15);
175
176         assertEquals(15, inventory.stockQuantity());
177         assertEquals(AvailabilityStatus.IN_STOCK, inventory.availabilityStatus());
178     }
179

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 180–186

```
180     @Test
181     void inventoryUpdateNormalizesImpossibleZeroInStockCombination() {
182         BookResponse imported = service.importBook(importRequest(
183             "9780735211292", "AKS-ATOMIC-PB", "/books/OL1M"
184         ));
185         Long editionId = imported.editions().get(0).id();
186     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 187–194

```
187         var inventory = bookService.updateInventory(
188             editionId,
189             new InventoryRequest(
190                 new BigDecimal("525.00"), 0,
191                 AvailabilityStatus.IN_STOCK, false
192             )
193         );
194     }
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 195–200

```
195         assertEquals(new BigDecimal("525.00"), inventory.price());
196         assertEquals(0, inventory.stockQuantity());
197         assertEquals(AvailabilityStatus.OUT_OF_STOCK, inventory.availabilityStatus());
198         assertEquals(false, inventory.active());
199     }
200 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 201–214

```
201     @Test
202     void deletionCleansDisposableReaderReferences() {
203         BookResponse imported = service.importBook(importRequest(
204             "9780735211292", "AKS-ATOMIC-PB", "/books/OL1M"
205         ));
206         var book = bookRepository.findById(imported.id()).orElseThrow();
```

```

207         var edition = editionRepository.findById(
208             imported.editions().get(0).id()
209         ).orElseThrow();
210         AppUser reader = userRepository.save(new AppUser(
211             "Reader One", "reader-one@example.com",
212             "encoded-password", Role.READER
213         ));
214         Cart cart = cartRepository.save(new Cart(reader));

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.

Lines 215–228

```

215         cartItemRepository.save(new CartItem(cart, edition, 1));
216         wishlistItemRepository.save(new WishlistItem(reader, book));
217         reviewRepository.save(new Review(
218             reader, book, 5, "Excellent", "A useful book."
219         ));
220         Order order = orderRepository.save(new Order(
221             reader,
222             new Address(
223                 "Reader One", "9999999999", "1 Book Lane", null,
224                 "Pune", "Maharashtra", "411001", "India"
225             ),
226             new BigDecimal("299.00"), BigDecimal.ZERO
227         ));
228         orderItemRepository.save(new OrderItem(

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 229–231

```

229             order, edition, new BigDecimal("299.00"), 1
230         ));
231

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 232–245

```

232         bookService.delete(imported.id());
233
234         assertFalse(bookRepository.existsById(imported.id()));
235         assertEquals(0, cartItemRepository.count());
236         assertEquals(0, wishlistItemRepository.count());
237         assertEquals(0, reviewRepository.count());
238         assertEquals(1, cartRepository.count());
239         assertEquals(1, userRepository.count());
240         OrderItem historicalItem = orderItemRepository
241             .findAllByOrder_IdOrderByIdAsc(order.getId())
242             .get(0);
243         assertEquals(null, historicalItem.getBookEdition());
244         assertEquals("Atomic Habits", historicalItem.getBookTitle());
245         assertEquals(1, orderRepository.count());

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 246–259

```
246     }
247
248     private BookImportRequest importRequest(
249         String isbn13,
250         String sku,
251         String editionId
252     ) {
253         return new BookImportRequest(
254             BookImportSource.OPEN_LIBRARY,
255             "/works/OL17930368W",
256             editionId,
257             "Atomic Habits",
258             null,
259             "Tiny changes, remarkable results.",
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 260–273

```
260         List.of("James Clear"),
261         "Avery",
262         "en",
263         320,
264         List.of("Self-help"),
265         "https://covers.openlibrary.org/b/id/1-L.jpg",
266         LocalDate.of(2018, 10, 16),
267         "First edition",
268         null,
269         isbn13,
270         sku.endsWith("HB") ? BookFormat.HARDCOVER : BookFormat.PAPERBACK,
271         new BigDecimal("499.00"),
272         0,
273         AvailabilityStatus.IN_STOCK,
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 274–280

```
274         sku,
275         true,
276         true
```

```
277         );  
278     }  
279 }  
280
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

backend/src/test/java/com/akshara/api/bookimport/service/CatalogueSeedServiceTest.java

File 72 of 99 · Added or modified · Hybrid catalogue import

Purpose: Verifies CatalogueSeedService behavior, including success paths and important validation, authorization or error cases.

Current file length: 102 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.bookimport.service;
2
3 import com.akshara.api.book.repository.BookRepository;
4 import com.akshara.api.bookimport.dto.BookImportRequest;
5 import com.akshara.api.bookimport.dto.BookImportSource;
6 import com.akshara.api.bookimport.dto.CatalogueSeedResponse;
7 import com.akshara.api.bookimport.dto.ExternalBookResult;
8 import com.akshara.api.bookimport.dto.ExternalBookSearchPage;
9 import org.junit.jupiter.api.Test;
10 import org.junit.jupiter.api.extension.ExtendWith;
11 import org.mockito.ArgumentCaptor;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–18

```
15 import java.util.ArrayList;
16 import java.util.List;
17 import java.util.concurrent.atomic.AtomicInteger;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 19–26

```
19 import static org.assertj.core.api.Assertions.assertThat;
20 import static org.mockito.ArgumentMatchers.anyInt;
21 import static org.mockito.ArgumentMatchers.anyString;
22 import static org.mockito.ArgumentMatchers.eq;
23 import static org.mockito.Mockito.times;
24 import static org.mockito.Mockito.verify;
25 import static org.mockito.Mockito.when;
26
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 27–29

```
27 @ExtendWith(MockitoExtension.class)
28 class CatalogueSeedServiceTest {
29
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 30–33

```
30 @Mock private ExternalBookSearchService searchService;
31 @Mock private BookImportService importService;
32 @Mock private BookRepository bookRepository;
33
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 34–43

```
34 @Test
35 void importsFiftyBooksAcrossEveryCuratedGenre() {
36     AtomicInteger sequence = new AtomicInteger();
37     when(bookRepository.count()).thenReturn(1L, 51L);
38     when(searchService.search(
39         anyString(), eq(BookImportSource.OPEN_LIBRARY), anyInt(), eq(20)
40     )).thenAnswer(invocation -> results(
41         invocation.getArgument(0), sequence.getAndAdd(20)
42     ));
43
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 44–47

```
44 CatalogueSeedResponse response = new CatalogueSeedService(
45     searchService, importService, bookRepository
46 ).seed(50);
47
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 48–53

```
48      assertThat(response.imported()).isEqualTo(50);
49      assertThat(response.catalogueSizeBefore()).isEqualTo(1);
50      assertThat(response.catalogueSizeAfter()).isEqualTo(51);
51      assertThat(response.genreCounts()).hasSize(8);
52      assertThat(response.genreCounts().values()).allMatch(count -> count > 0);
53
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 54–62

```
54      ArgumentCaptor<BookImportRequest> requests =
55          ArgumentCaptor.forClass(BookImportRequest.class);
56      verify(importService, times(50)).importBook(requests.capture());
57      assertThat(requests.getAllValues())
58          .allMatch(request -> request.categories() != null
59              && !request.categories().isEmpty())
60          .allMatch(request -> request.sku().startsWith("AKS-SEED-"));
61  }
62
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 63–76

```
63      private ExternalBookSearchPage results(String query, int start) {
64          List<ExternalBookResult> results = new ArrayList<>();
65          for (int index = 0; index < 20; index++) {
66              String isbn = isbn13(start + index);
67              results.add(new ExternalBookResult(
68                  BookImportSource.OPEN_LIBRARY,
69                  "/works/OL" + (start + index) + "W",
70                  "/books/OL" + (start + index) + "M",
71                  "Curated book " + (start + index),
72                  null,
73                  "A catalogue seed test book.",
74                  List.of("Test Author"),
75                  "Test Publisher",
76                  "eng",
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 77–90

```

77         240,
78         List.of(),
79         "https://covers.openlibrary.org/test.jpg",
80         "2020-01-01",
81         null,
82         isbn,
83         query,
84         false
85     ));
86 }
87 return new ExternalBookSearchPage(
88     BookImportSource.OPEN_LIBRARY, results, 1, 20, 200
89 );
90 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 91–102

```

91
92 private String isbn13(int value) {
93     String firstTwelve = "978" + String.format("%09d", value);
94     int sum = 0;
95     for (int index = 0; index < firstTwelve.length(); index++) {
96         int digit = firstTwelve.charAt(index) - '0';
97         sum += digit * (index % 2 == 0 ? 1 : 3);
98     }
99     return firstTwelve + ((10 - sum % 10) % 10);
100 }
101 }
102 }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

backend/src/test/java/com/akshara/api/cart/service/

CartServiceTest.java

File 73 of 99 · Added or modified · Cart

Purpose: Verifies CartService behavior, including success paths and important validation, authorization or error cases.

Current file length: 358 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.cart.service;
2
3 import com.akshara.api.book.repository.BookEditionRepository;
4 import com.akshara.api.cart.dto.AddCartItemRequest;
5 import com.akshara.api.cart.dto.CartResponse;
6 import com.akshara.api.cart.dto.UpdateCartItemRequest;
7 import com.akshara.api.cart.entity.Cart;
8 import com.akshara.api.cart.entity.CartItem;
9 import com.akshara.api.cart.repository.CartItemRepository;
10 import com.akshara.api.cart.repository.CartRepository;
11 import com.akshara.api.common.exception.InvalidRequestException;
12 import com.akshara.api.common.exception.ResourceNotFoundException;
13 import com.akshara.api.user.entity.AppUser;
14 import com.akshara.api.user.entity.Role;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–22

```
15 import com.akshara.api.user.repository.UserRepository;
16 import org.junit.jupiter.api.BeforeEach;
17 import org.junit.jupiter.api.Test;
18 import org.junit.jupiter.api.extension.ExtendWith;
19 import org.mockito.Mock;
20 import org.mockito.junit.jupiter.MockitoExtension;
21 import org.springframework.test.util.ReflectionTestUtils;
22
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 23–27

```
23 import java.math.BigDecimal;
24 import java.util.List;
25 import java.util.Optional;
26 import java.util.concurrent.atomic.AtomicReference;
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 28–40

```
28 import static org.assertj.core.api.Assertions.assertThat;
29 import static org.assertj.core.api.Assertions.assertThatThrownBy;
30 import static org.mockito.ArgumentMatchers.any;
31 import static org.mockito.Mockito.never;
32 import static org.mockito.Mockito.verify;
33 import static org.mockito.Mockito.when;
34 import com.akshara.api.book.repository.InventoryRepository;
35 import com.akshara.api.book.entity.AvailabilityStatus;
36 import com.akshara.api.book.entity.Inventory;
37 import com.akshara.api.book.entity.Book;
38 import com.akshara.api.book.entity.BookEdition;
39 import com.akshara.api.book.entity.BookFormat;
40
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 41–44

```
41
42 @ExtendWith(MockitoExtension.class)
43 class CartServiceTest {
44
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 45–50

```
45 private static final Long USER_ID = 1L;
46 private static final Long CART_ID = 10L;
47 private static final Long BOOK_ID = 20L;
48 private static final Long EDITION_ID = 30L;
49 private static final Long ITEM_ID = 40L;
50
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 51–53

```
51     @Mock
52     private CartRepository cartRepository;
53
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 54–56

```
54     @Mock
55     private CartItemRepository cartItemRepository;
56
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 57–59

```
57     @Mock
58     private UserRepository userRepository;
59
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 60–62

```
60     @Mock
61     private BookEditionRepository bookEditionRepository;
62
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 63–65

```
63     @Mock
64     private InventoryRepository inventoryRepository;
65
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 66–72

```
66     private CartService cartService;
67
68     private AppUser user;
69     private Cart cart;
70     private BookEdition edition;
71     private Inventory inventory;
72
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 73–82

```
73     @BeforeEach
74     void setUp() {
75         cartService = new CartService(
76             cartRepository,
77             cartItemRepository,
78             userRepository,
79             bookEditionRepository,
80             inventoryRepository
81         );
82
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 83–90

```
83         user = new AppUser(
84             "Cart Test Reader",
85             "cart.reader@example.com",
86             "password-hash",
87             Role.READER
88         );
89         ReflectionTestUtils.setField(user, "id", USER_ID);
90
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 91–93

```
91         cart = new Cart(user);
92         ReflectionTestUtils.setField(cart, "id", CART_ID);
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 94–96

```

94      Book book = new Book("Clean Code");
95      ReflectionTestUtils.setField(book, "id", BOOK_ID);
96

```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 97–108

```

97      edition = new BookEdition(
98          book,
99          BookFormat.PAPERBACK
100     );
101     edition.setEditionName("First Edition");
102     edition.setIsbn13("9780132350884");
103     ReflectionTestUtils.setField(
104         edition,
105         "id",
106         EDITION_ID
107     );
108

```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 109–116

```

109     inventory = new Inventory(
110         edition,
111         new BigDecimal("299.00"),
112         5,
113         AvailabilityStatus.IN_STOCK
114     );
115 }
116

```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 117–120

```

117     @Test
118     void addItemCreatesNewItemAndCalculatesTotals() {
119         stubAuthenticatedCart();
120

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 121–123

```
121         when(bookEditionRepository.findById(EDITION_ID))
122             .thenReturn(Optional.of(edition));
123
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 124–130

```
124         when(cartItemRepository
125             .findByCart_IdAndBookEdition_Id(
126                 CART_ID,
127                 EDITION_ID
128             ))
129             .thenReturn(Optional.empty());
130
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 131–134

```
131         when(inventoryRepository
132             .findByBookEdition_Id(EDITION_ID))
133             .thenReturn(Optional.of(inventory));
134
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 135–137

```
135         AtomicReference<CartItem> savedItem =
136             new AtomicReference<>();
137
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 138–149

```
138         when(cartItemRepository.save(any(CartItem.class)))
139             .thenAnswer(invocation -> {
140                 CartItem item = invocation.getArgument(0);
141                 ReflectionTestUtils.setField(
142                     item,
143                     "id",
144                     ITEM_ID
145                 );
146                 savedItem.set(item);
147                 return item;
148             });
149
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 150–155

```
150         when(cartItemRepository
151             .findAllByCart_IdOrderByCreatedAtAsc(CART_ID))
152             .thenAnswer(invocation ->
153                 List.of(savedItem.get())
154             );
155
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 156–160

```
156         CartResponse response = cartService.addItem(
157             USER_ID.toString(),
158             new AddCartItemRequest(EDITION_ID, 2)
159         );
160
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 161–168

```
161         assertThat(savedItem.get().getQuantity()).isEqualTo(2);
162         assertThat(response.distinctItems()).isEqualTo(1);
163         assertThat(response.totalQuantity()).isEqualTo(2);
```

```

164      assertThat(response.totalAmount())
165          .isEqualToByComparingTo("598.00");
166      assertThat(response.items().get(0).subtotal())
167          .isEqualToByComparingTo("598.00");
168

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 169–172

```

169      verify(cartItemRepository).save(savedItem.get());
170      verify(cartRepository).save(cart);
171  }
172

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 173–176

```

173      @Test
174      void addItemMergesQuantityForExistingEdition() {
175          stubAuthenticatedCart();
176

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 177–179

```

177          CartItem existingItem =
178              new CartItem(cart, edition, 2);
179

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 180–185

```

180          ReflectionTestUtils.setField(
181              existingItem,
182              "id",
183              ITEM_ID
184          );
185

```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 186–188

```
186         when(bookEditionRepository.findById(EDITION_ID))
187             .thenReturn(Optional.of(edition));
188
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 189–195

```
189         when(cartItemRepository
190             .findByCart_IdAndBookEdition_Id(
191                 CART_ID,
192                 EDITION_ID
193             ))
194             .thenReturn(Optional.of(existingItem));
195
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 196–199

```
196         when(inventoryRepository
197             .findByBookEdition_Id(EDITION_ID))
198             .thenReturn(Optional.of(inventory));
199
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 200–202

```
200         when(cartItemRepository.save(existingItem))
201             .thenReturn(existingItem);
202
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 203–206

```
203         when(cartItemRepository
204             .findAllByCart_IdOrderByCreatedAtAsc(CART_ID))
205             .thenReturn(List.of(existingItem));
206
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 207–211

```
207         CartResponse response = cartService.addItem(
208             USER_ID.toString(),
209             new AddCartItemRequest(EDITION_ID, 2)
210         );
211
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 212–217

```
212         assertThat(existingItem.getQuantity()).isEqualTo(4);
213         assertThat(response.distinctItems()).isEqualTo(1);
214         assertThat(response.totalQuantity()).isEqualTo(4);
215         assertThat(response.totalAmount())
216             .isEqualByComparingTo("1196.00");
217
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 218–220

```
218         verify(cartItemRepository).save(existingItem);
219     }
220
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 221–224

```
221     @Test
222     void addItemRejectsQuantityGreaterThanStock() {
223         stubAuthenticatedCart();
224
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 225–227

```
225         CartItem existingItem =  
226             new CartItem(cart, edition, 3);  
227
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 228–233

```
228         ReflectionTestUtils.setField(  
229             existingItem,  
230             "id",  
231             ITEM_ID  
232         );  
233
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 234–236

```
234         when(bookEditionRepository.findById(EDITION_ID))  
235             .thenReturn(Optional.of(edition));  
236
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 237–243

```
237         when(cartItemRepository  
238             .findByCart_IdAndBookEdition_Id(  
239                 CART_ID,  
240                 EDITION_ID  
241             ))  
242             .thenReturn(Optional.of(existingItem));  
243
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important

collaboration where appropriate.

Lines 244–247

```
244         when(inventoryRepository
245                 .findByBookEdition_Id(EDITION_ID))
246                 .thenReturn(Optional.of(inventory));
247
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 248–258

```
248         assertThatThrownBy(() ->
249             cartService.addItem(
250                 USER_ID.toString(),
251                 new AddCartItemRequest(EDITION_ID, 3)
252             )
253         )
254         .isInstanceOf(InvalidRequestException.class)
255         .hasMessageContaining(
256             "exceeds the available stock of 5"
257         );
258
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 259–265

```
259         assertThat(existingItem.getQuantity()).isEqualTo(3);
260
261         verify(
262             cartItemRepository,
263             never()
264         ).save(any(CartItem.class));
265
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 266–271

```
266         verify(
267             cartRepository,
268             never()
269         ).save(any(Cart.class));
270     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 272–278

```

272     @Test
273     void addItemRejectsDigitalEditionWithoutDeliveryEntitlement() {
274         stubAuthenticatedCart();
275         edition.setFormat(BookFormat.EPUB);
276         when(bookEditionRepository.findById(EDITION_ID))
277             .thenReturn(Optional.of(edition));
278     }

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 279–287

```

279         assertThatThrownBy(() ->
280             cartService.addItem(
281                 USER_ID.toString(),
282                 new AddCartItemRequest(EDITION_ID, 1)
283             )
284         )
285             .assertInstanceOf(InvalidRequestException.class)
286             .hasMessage("Digital editions are not available for purchase yet");
287     }

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 288–293

```

288         verify(inventoryRepository, never())
289             .findByBookEdition_Id(any());
290         verify(cartItemRepository, never())
291             .save(any(CartItem.class));
292     }
293

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 294–297

```
294     @Test
295     void updateItemRejectsItemNotOwnedByReader() {
296         stubAuthenticatedCart();
297     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 298–302

```
298         when(cartItemRepository.findByIdAndCart_Id(
299             ITEM_ID,
300             CART_ID
301         )).thenReturn(Optional.empty());
302     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 303–314

```
303         assertThatThrownBy(() ->
304             cartService.updateItem(
305                 USER_ID.toString(),
306                 ITEM_ID,
307                 new UpdateCartItemRequest(1)
308             )
309         )
310             .isInstanceOf(ResourceNotFoundException.class)
311             .hasMessageContaining(
312                 "Cart item with ID 40 was not found"
313             );
314     }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 315–319

```
315         verify(
316             inventoryRepository,
317             never()
318         ).findByBookEdition_Id(any());
319     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 320–325

```
320         verify(  
321             cartItemRepository,  
322             never()  
323         ).save(any(CartItem.class));  
324     }  
325
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 326–329

```
326     @Test  
327     void clearCartDeletesEveryItemAndReturnsEmptyCart() {  
328         stubAuthenticatedCart();  
329
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 330–333

```
330         when(cartItemRepository  
331             .findAllByCart_IdOrderByCreatedAtAsc(CART_ID))  
332             .thenReturn(List.of());  
333
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 334–337

```
334         CartResponse response = cartService.clearCart(  
335             USER_ID.toString()  
336         );  
337
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 338–340

```
338         verify(cartItemRepository)  
339             .deleteAllByCart_Id(CART_ID);
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 341–349

```

341         verify(cartRepository).save(cart);
342
343         assertThat(response.distinctItems()).isZero();
344         assertThat(response.totalQuantity()).isZero();
345         assertThat(response.totalAmount())
346             .isEqualByComparingTo("0.00");
347         assertThat(response.items()).isEmpty();
348     }
349 
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 350–353

```

350     private void stubAuthenticatedCart() {
351         when(userRepository.findById(USER_ID))
352             .thenReturn(Optional.of(user));
353     }

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 354–358

```

354         when(cartRepository.findByUser_Id(USER_ID))
355             .thenReturn(Optional.of(cart));
356     }
357 }
358 
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC

in the business layer.

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

backend/src/test/java/com/akshara/api/library/service/PersonalLibraryServiceTest.java

File 74 of 99 · Added or modified · Personal library and uploads

Purpose: Verifies PersonalLibraryService behavior, including success paths and important validation, authorization or error cases.

Current file length: 108 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.library.service;
2
3 import com.akshara.api.book.entity.BookFormat;
4 import com.akshara.api.common.exception.InvalidRequestException;
5 import com.akshara.api.common.exception.ResourceNotFoundException;
6 import com.akshara.api.library.entity.PersonalBook;
7 import com.akshara.api.library.repository.PersonalBookRepository;
8 import com.akshara.api.user.entity.AppUser;
9 import com.akshara.api.user.repository.UserRepository;
10 import org.junit.jupiter.api.BeforeEach;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.junit.jupiter.api.io.TempDir;
14 import org.mockito.ArgumentCaptor;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–18

```
15 import org.mockito.Mock;
16 import org.mockito.junit.jupiter.MockitoExtension;
17 import org.springframework.mock.web.MockMultipartFile;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 19–22

```
19 import java.nio.file.Files;
20 import java.nio.file.Path;
21 import java.util.Optional;
22
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 23–27

```
23 import static org.assertj.core.api.Assertions.assertThat;
24 import static org.assertj.core.api.Assertions.assertThatThrownBy;
25 import static org.mockito.ArgumentMatchers.any;
26 import static org.mockito.Mockito.*;
27
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 28–30

```
28 @ExtendWith(MockitoExtension.class)
29 class PersonalLibraryServiceTest {
30
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 31–33

```
31     @TempDir
32     Path storageDirectory;
33
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 34–36

```
34     @Mock
35     UserRepository userRepository;
36
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 37–39

```
37     @Mock
38     PersonalBookRepository personalBookRepository;
39
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 40–42

```
40     @Mock
41     AppUser user;
42
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 43–49

```
43     private PersonalLibraryService service;
44
45     @BeforeEach
46     void setUp() {
47         service = new PersonalLibraryService(userRepository, personalBookRepository, storageDirectory.toString());
48     }
49
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 50–58

```
50     @Test
51     void uploadShouldStoreAValidPdfWithGeneratedFilename() throws Exception {
52         authenticate();
53         when(personalBookRepository.save(any(PersonalBook.class)))
54             .thenReturn(invocation -> invocation.getArgument(0));
55         MockMultipartFile file = new MockMultipartFile(
56             "file", "../my-book.pdf", "application/pdf", "%PDF-1.7\ncontent".getBytes()
57         );
58
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 59–65

```
59         var response = service.upload("1", " My Book ", " An Author ", file);
60
```

```

61     assertThat(response.title()).isEqualTo("My Book");
62     assertThat(response.author()).isEqualTo("An Author");
63     assertThat(response.format()).isEqualTo(BookFormat.PDF);
64     assertThat(response.originalFilename()).isEqualTo("my-book.pdf");
65

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 66–74

```

66     ArgumentCaptor<PersonalBook> captor = ArgumentCaptor.forClass(PersonalBook.class);
67     verify(personalBookRepository).save(captor.capture());
68     assertThat(captor.getValue().getStoredFilename())
69         .matches("[0-9a-f-]{36}\\..pdf");
70     try (var files = Files.list(storageDirectory)) {
71         assertThat(files).hasSize(1);
72     }
73 }
74

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 75–81

```

75     @Test
76     void uploadShouldRejectAFileWhoseContentsDoNotMatchItsExtension() throws Exception {
77         authenticate();
78         MockMultipartFile file = new MockMultipartFile(
79             "file", "unsafe.pdf", "application/pdf", "not a pdf".getBytes()
80         );
81

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 82–85

```

82     assertThatThrownBy(() -> service.upload("1", "Unsafe", null, file))
83         .isInstanceOf(InvalidRequestException.class)
84         .hasMessage("Only valid PDF and EPUB files can be uploaded");
85

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 86–91

```
86         verifyNoInteractions(personalBookRepository);
87         try (var files = Files.list(storageDirectory)) {
88             assertThat(files).isEmpty();
89         }
90     }
91
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- The file code normalizes paths, validates content signatures and performs storage operations behind an authenticated ownership check.

Lines 92–97

```
92     @Test
93     void getFileShouldNotExposeAnotherUsersUpload() {
94         authenticate();
95         when(user.getId()).thenReturn(1L);
96         when(personalBookRepository.findByIdAndUser_Id(9L, 1L)).thenReturn(Optional.empty());
97
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 98–102

```
98         assertThatThrownBy(() -> service.getFile("1", 9L))
99             .isInstanceOf(ResourceNotFoundException.class)
100             .hasMessage("Personal book with ID 9 was not found");
101     }
102
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 103–108

```
103     private void authenticate() {
104         when(userRepository.findById(1L)).thenReturn(Optional.of(user));
105         when(user.isEnabled()).thenReturn(true);
106     }
107 }
108
```


How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

backend/src/test/java/com/akshara/api/order/service/CheckoutServiceTest.java

File 75 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Verifies CheckoutService behavior, including success paths and important validation, authorization or error cases.

Current file length: 400 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.order.service;
2
3 import com.akshara.api.book.entity.AvailabilityStatus;
4 import com.akshara.api.book.entity.Book;
5 import com.akshara.api.book.entity.BookEdition;
6 import com.akshara.api.book.entity.BookFormat;
7 import com.akshara.api.book.entity.Inventory;
8 import com.akshara.api.book.repository.InventoryRepository;
9 import com.akshara.api.cart.entity.Cart;
10 import com.akshara.api.cart.entity.CartItem;
11 import com.akshara.api.cart.repository.CartItemRepository;
12 import com.akshara.api.cart.repository.CartRepository;
13 import com.akshara.api.common.exception.InvalidRequestException;
14 import com.akshara.api.order.dto.CheckoutRequest;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–28

```
15 import com.akshara.api.order.dto.CheckoutResponse;
16 import com.akshara.api.order.dto.ShippingAddressRequest;
17 import com.akshara.api.order.entity.Order;
18 import com.akshara.api.order.entity.OrderItem;
19 import com.akshara.api.order.entity.OrderStatus;
20 import com.akshara.api.order.entity.Payment;
21 import com.akshara.api.order.entity.PaymentMethod;
22 import com.akshara.api.order.entity.PaymentStatus;
23 import com.akshara.api.order.repository.OrderItemRepository;
24 import com.akshara.api.order.repository.OrderRepository;
25 import com.akshara.api.order.repository.PaymentRepository;
26 import com.akshara.api.user.entity.AppUser;
27 import com.akshara.api.user.repository.UserRepository;
28 import org.junit.jupiter.api.BeforeEach;
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 29–34

```
29 import org.junit.jupiter.api.Test;
30 import org.junit.jupiter.api.extension.ExtendWith;
31 import org.mockito.Mock;
32 import org.mockito.junit.jupiter.MockitoExtension;
33 import org.springframework.test.util.ReflectionTestUtils;
34
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 35–40

```
35 import java.math.BigDecimal;
36 import java.time.Instant;
37 import java.util.List;
38 import java.util.Optional;
39 import java.util.concurrent.atomic.AtomicReference;
40
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 41–50

```
41 import static org.assertj.core.api.Assertions.assertThat;
42 import static org.assertj.core.api.Assertions.assertThatThrownBy;
43 import static org.mockito.ArgumentMatchers.any;
44 import static org.mockito.ArgumentMatchers.anyList;
45 import static org.mockito.ArgumentMatchers.anyLong;
46 import static org.mockito.Mockito.never;
47 import static org.mockito.Mockito.verify;
48 import static org.mockito.Mockito.verifyNoInteractions;
49 import static org.mockito.Mockito.when;
50
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 51–53

```
51 @ExtendWith(MockitoExtension.class)
52 class CheckoutServiceTest {
53
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 54–57

```
54     private static final Long USER_ID = 1L;
55     private static final Long CART_ID = 10L;
56     private static final Long EDITION_ID = 30L;
57
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 58–60

```
58     @Mock
59     private CartRepository cartRepository;
60
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 61–63

```
61     @Mock
62     private CartItemRepository cartItemRepository;
63
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 64–66

```
64     @Mock
65     private UserRepository userRepository;
66
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 67–69

```
67     @Mock
68     private InventoryRepository inventoryRepository;
69
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 70–72

```
70     @Mock
71     private OrderRepository orderRepository;
72
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 73–75

```
73     @Mock
74     private OrderItemRepository orderItemRepository;
75
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 76–78

```
76     @Mock
77     private PaymentRepository paymentRepository;
78
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 79–81

```
79     @Mock
80     private OrderResponseMapper orderResponseMapper;
81
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 82–84

```
82     @Mock
83     private AppUser user;
84
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 85–89

```
85     private CheckoutService checkoutService;
86     private Cart cart;
87     private BookEdition edition;
88     private CheckoutRequest checkoutRequest;
89
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 90–102

```
90     @BeforeEach
91     void setUp() {
92         checkoutService = new CheckoutService(
93             cartRepository,
94             cartItemRepository,
95             userRepository,
96             inventoryRepository,
97             orderRepository,
98             orderItemRepository,
99             paymentRepository,
100             orderResponseMapper
101         );
102
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 103–105

```
103         when(user.getId()).thenReturn(USER_ID);
104         when(user.isEnabled()).thenReturn(true);
105
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 106–108

```
106         cart = new Cart(user);
```

```
107         ReflectionTestUtils.setField(cart, "id", CART_ID);
108     }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 109–117

```
109         Book book = new Book("Clean Code");
110
111         edition = new BookEdition(
112             book,
113             BookFormat.PAPERBACK
114         );
115         edition.setEditionName("First Edition");
116         edition.setIsbn13("9780132350884");
117     }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 118–123

```
118         ReflectionTestUtils.setField(
119             edition,
120             "id",
121             EDITION_ID
122         );
123     }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 124–137

```
124         checkoutRequest = new CheckoutRequest(
125             new ShippingAddressRequest(
126                 "Ayush Thakur",
127                 "9876543210",
128                 "123 Main Road",
129                 null,
130                 "New Delhi",
131                 "Delhi",
132                 "110001",
133                 "India"
134             )
135         );
136     }
137 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 138–145

```
138     @Test
139     void checkoutShouldCreateOrderReduceStockAndClearCart() {
140         CartItem cartItem = new CartItem(
141             cart,
142             edition,
143             2
144         );
145     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 146–152

```
146         Inventory inventory = new Inventory(
147             edition,
148             new BigDecimal("299.00"),
149             5,
150             AvailabilityStatus.IN_STOCK
151         );
152     }
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 153–166

```
153         CheckoutResponse expectedResponse =
154             new CheckoutResponse(
155                 101L,
156                 OrderStatus.PLACED,
157                 PaymentMethod.CASH_ON_DELIVERY,
158                 PaymentStatus.PENDING,
159                 List.of(),
160                 2,
161                 new BigDecimal("598.00"),
162                 new BigDecimal("0.00"),
163                 new BigDecimal("598.00"),
164                 null,
165                 Instant.now()
166             );
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 167–169

```
167
168         stubAuthenticatedCart();
169     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because

formatting has split one logical construct across line ranges.

Lines 170–173

```
170         when(cartItemRepository
171             .findAllByCart_IdOrderByCreatedAtAsc(CART_ID))
172             .thenReturn(List.of(cartItem));
173
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 174–177

```
174         when(inventoryRepository
175             .findByBookEditionIdForUpdate(EDITION_ID))
176             .thenReturn(Optional.of(inventory));
177
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 178–180

```
178         AtomicReference<Order> savedOrder =
179             new AtomicReference<>();
180
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 181–187

```
181         when(orderRepository.save(any(Order.class)))
182             .thenAnswer(invocation -> {
183             Order order = invocation.getArgument(0);
184             savedOrder.set(order);
185             return order;
186         });
187
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important

collaboration where appropriate.

Lines 188–190

```
188         AtomicReference<List<OrderItem>> savedOrderItems =  
189             new AtomicReference<>();  
190
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 191–195

```
191         when(orderItemRepository.saveAll(anyList()))  
192             .thenAnswer(invocation -> {  
193                 List<OrderItem> items =  
194                     invocation.getArgument(0);  
195
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 196–199

```
196                 savedOrderItems.set(items);  
197                 return items;  
198             });  
199
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 200–202

```
200         AtomicReference<Payment> savedPayment =  
201             new AtomicReference<>();  
202
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 203–209

```
203         when(paymentRepository.save(any(Payment.class)))  
204             .thenAnswer(invocation -> {  
205                 Payment payment = invocation.getArgument(0);  
206                 savedPayment.set(payment);  
207                 return payment;  
208             });  
209
```

```
208         });  
209     }  
210 }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 210–215

```
210         when(orderResponseMapper.toCheckoutResponse(  
211             any(Order.class),  
212             anyList(),  
213             any(Payment.class)  
214         )).thenReturn(expectedResponse);  
215     }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 216–221

```
216         CheckoutResponse actualResponse =  
217             checkoutService.checkout(  
218                 USER_ID.toString(),  
219                 checkoutRequest  
220             );  
221     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 222–224

```
222         assertThat(actualResponse)  
223             .isEqualTo(expectedResponse);  
224     }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 225–227

```
225         assertThat(inventory.getStockQuantity())  
226             .isEqualTo(3);  
227     }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 228–230

```
228         assertThat(inventory.getAvailabilityStatus())
229             .isEqualTo(AvailabilityStatus.IN_STOCK);
230
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 231–235

```
231         Order order = savedOrder.get();
232
233         assertThat(order.getSubtotal())
234             .isEqualToComparingTo("598.00");
235
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 236–238

```
236         assertThat(order.getShippingFee())
237             .isEqualToComparingTo("0.00");
238
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 239–241

```
239         assertThat(order.getTotalAmount())
240             .isEqualToComparingTo("598.00");
241
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 242–244

```
242         assertThat(order.getShippingAddress().getRecipientName())
243             .isEqualTo("Ayush Thakur");
244
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 245–247

```
245         assertThat(order.getShippingAddress().getPostalCode())
246             .isEqualTo("110001");
247
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 248–250

```
248         assertThat(savedOrderItems.get())
249             .hasSize(1);
250
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 251–253

```
251         OrderItem orderItem =
252             savedOrderItems.get().get(0);
253
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 254–256

```
254         assertThat(orderItem.getBookTitle())
255             .isEqualTo("Clean Code");
256
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 257–259

```
257         assertThat(orderItem.getBookFormat())
258             .isEqualTo(BookFormat.PAPERBACK);
259
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 260–262

```
260         assertThat(orderItem.getUnitPrice())
261             .isEqualToByComparingTo("299.00");
262
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 263–265

```
263         assertThat(orderItem.getQuantity())
264             .isEqualTo(2);
265
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 266–268

```
266         assertThat(orderItem.getSubtotal())
267             .isEqualToByComparingTo("598.00");
268
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 269–273

```
269         Payment payment = savedPayment.get();
270
271         assertThat(payment.getMethod())
272             .isEqualTo(PaymentMethod.CASH_ON_DELIVERY);
273
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 274–276

```
274         assertThat(payment.getStatus())
275             .isEqualTo(PaymentStatus.PENDING);
276
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 277–279

```
277         assertThat(payment.getAmount())
278             .isEqualToComparingTo("598.00");
279
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 280–282

```
280         verify(inventoryRepository)
281             .findByBookEditionIdForUpdate(EDITION_ID);
282
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 283–285

```
283         verify(cartItemRepository)
284             .deleteAllByCart_Id(CART_ID);
285
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 286–288

```
286         verify(cartRepository).save(cart);
287     }
288
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 289–293

```
289     @Test
290     void checkoutShouldRejectMissingOrEmptyCart() {
291         when(userRepository.findById(USER_ID))
292             .thenReturn(Optional.of(user));
293
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 294–296

```
294         when(cartRepository.findByUserIdForUpdate(USER_ID))
295             .thenReturn(Optional.empty());
296
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 297–304

```
297         assertThatThrownBy(() ->
298             checkoutService.checkout(
299                 USER_ID.toString(),
300                 checkoutRequest
301             ))
302             .isInstanceOf(InvalidRequestException.class)
303             .hasMessage("Cart is empty");
304
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 305–313

```
305         verifyNoInteractions(
306             inventoryRepository,
307             orderRepository,
308             orderItemRepository,
309             paymentRepository,
310             orderResponseMapper
311         );
312     }
313
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 314–321

```
314     @Test
315     void checkoutShouldRejectQuantityGreaterThanStock() {
316         CartItem cartItem = new CartItem(
317             cart,
318             edition,
```



```
319         2
320     );
321
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 322–328

```
322     Inventory inventory = new Inventory(
323         edition,
324         new BigDecimal("299.00"),
325         1,
326         AvailabilityStatus.IN_STOCK
327     );
328
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 329–334

```
329     stubAuthenticatedCart();
330
331     when(cartItemRepository
332         .findAllByCart_IdOrderByCreatedAtAsc(CART_ID))
333         .thenReturn(List.of(cartItem));
334
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 335–338

```
335     when(inventoryRepository
336         .findByBookEditionIdForUpdate(EDITION_ID))
337         .thenReturn(Optional.of(inventory));
338
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 339–349

```
339     assertThatThrownBy(() ->
340         checkoutService.checkout(
```

```

341                USER_ID.toString(),
342                checkoutRequest
343            ))
344            .isInstanceOf(InvalidRequestException.class)
345            .hasMessageContaining("Insufficient stock")
346            .hasMessageContaining(
347                "Available quantity: 1"
348            );
349

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 350–352

```

350        assertThat(inventory.getStockQuantity())
351            .isEqualTo(1);
352

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 353–355

```

353        verify(orderRepository, never())
354            .save(any(Order.class));
355

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 356–358

```

356        verify(paymentRepository, never())
357            .save(any(Payment.class));
358

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 359–362

```
359         verify(cartItemRepository, never())
360             .deleteAllByCart_Id(anyLong());
361     }
362
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 363–373

```
363     @Test
364     void checkoutRejectsDigitalEditionWithoutDeliveryEntitlement() {
365         edition.setFormat(BookFormat.PDF);
366         CartItem cartItem = new CartItem(cart, edition, 1);
367         Inventory inventory = new Inventory(
368             edition,
369             new BigDecimal("199.00"),
370             1,
371             AvailabilityStatus.IN_STOCK
372         );
373
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Each `@Test` method is an independently executable behavior specification.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 374–381

```
374         stubAuthenticatedCart();
375         when(cartItemRepository
376             .findAllByCart_IdOrderByCreatedAtAsc(CART_ID))
377             .thenReturn(List.of(cartItem));
378         when(inventoryRepository
379             .findByBookEditionIdForUpdate(EDITION_ID))
380             .thenReturn(Optional.of(inventory));
381
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 382–387

```
382         assertThatThrownBy(() ->
383             checkoutService.checkout(USER_ID.toString(), checkoutRequest)
384         )
385             .isInstanceOf(InvalidRequestException.class)
386             .hasMessage("Digital editions are not available for checkout yet");
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 388–391

```
388         verify(orderRepository, never()).save(any(Order.class));
389         verify(paymentRepository, never()).save(any(Payment.class));
390     }
391
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 392–395

```
392     private void stubAuthenticatedCart() {
393         when(userRepository.findById(USER_ID))
394             .thenReturn(Optional.of(user));
395     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 396–400

```
396         when(cartRepository.findByIdForUpdate(USER_ID))
397             .thenReturn(Optional.of(cart));
398     }
399 }
400
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

backend/src/test/java/com/akshara/api/order/service/OrderQueryServiceTest.java

File 76 of 99 · Added or modified · Orders, fulfilment and purchased shelf

Purpose: Verifies OrderQueryService behavior, including success paths and important validation, authorization or error cases.

Current file length: 262 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.order.service;
2
3 import com.akshara.api.auth.exception.InvalidAccessTokenException;
4 import com.akshara.api.common.exception.ResourceNotFoundException;
5 import com.akshara.api.book.entity.Book;
6 import com.akshara.api.book.entity.BookEdition;
7 import com.akshara.api.book.entity.BookFormat;
8 import com.akshara.api.order.dto.CheckoutResponse;
9 import com.akshara.api.order.dto.LibraryBookResponse;
10 import com.akshara.api.order.dto.OrderSummaryResponse;
11 import com.akshara.api.order.entity.Order;
12 import com.akshara.api.order.entity.OrderItem;
13 import com.akshara.api.order.entity.OrderStatus;
14 import com.akshara.api.order.entity.Payment;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–25

```
15 import com.akshara.api.order.repository.OrderItemRepository;
16 import com.akshara.api.order.repository.OrderRepository;
17 import com.akshara.api.order.repository.PaymentRepository;
18 import com.akshara.api.user.entity.AppUser;
19 import com.akshara.api.user.repository.UserRepository;
20 import org.junit.jupiter.api.BeforeEach;
21 import org.junit.jupiter.api.Test;
22 import org.junit.jupiter.api.extension.ExtendWith;
23 import org.mockito.Mock;
24 import org.mockito.junit.jupiter.MockitoExtension;
25
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 26–30

```
26 import java.math.BigDecimal;
27 import java.time.Instant;
28 import java.util.List;
```

```
29 import java.util.Optional;
30
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 31–36

```
31 import static org.assertj.core.api.Assertions.assertThat;
32 import static org.assertj.core.api.Assertions.assertThatThrownBy;
33 import static org.mockito.Mockito.verify;
34 import static org.mockito.Mockito.verifyNoInteractions;
35 import static org.mockito.Mockito.when;
36
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 37–39

```
37 @ExtendWith(MockitoExtension.class)
38 class OrderQueryServiceTest {
39
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 40–42

```
40     private static final Long USER_ID = 1L;
41     private static final Long ORDER_ID = 101L;
42
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 43–45

```
43     @Mock
44     private UserRepository userRepository;
45
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 46–48

```
46     @Mock
47     private OrderRepository orderRepository;
48
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 49–51

```
49     @Mock
50     private OrderItemRepository orderItemRepository;
51
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 52–54

```
52     @Mock
53     private PaymentRepository paymentRepository;
54
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 55–57

```
55     @Mock
56     private OrderResponseMapper orderResponseMapper;
57
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 58–60

```
58     @Mock
59     private AppUser user;
60
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 61–63

```
61    @Mock
62    private Order order;
63
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 64–66

```
64    @Mock
65    private OrderItem orderItem;
66
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 67–69

```
67    @Mock
68    private Payment payment;
69
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 70–72

```
70    @Mock
71    private BookEdition bookEdition;
72
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 73–75

```
73    @Mock
74    private Book book;
75
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 76–78

```
76     @Mock
77     private CheckoutResponse checkoutResponse;
78
```

How to read this block

- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 79–91

```
79     private OrderQueryService orderQueryService;
80
81     @BeforeEach
82     void setUp() {
83         orderQueryService = new OrderQueryService(
84             userRepository,
85             orderRepository,
86             orderItemRepository,
87             paymentRepository,
88             orderResponseMapper
89         );
90     }
91
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 92–96

```
92     @Test
93     void getOrdersShouldReturnAuthenticatedUsersOrders() {
94         Instant placedAt =
95             Instant.parse("2026-08-02T16:42:29Z");
96
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 97–102

```
97         stubAuthenticatedUser();
98
99         when(orderRepository
100             .findAllByUser_IdOrderByPlacedAtDesc(USER_ID))
101             .thenReturn(List.of(order));
102
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 103–108

```
103         when(order.getId()).thenReturn(ORDER_ID);
104         when(order.getStatus()).thenReturn(OrderStatus.PLACED);
105         when(order.getTotalAmount())
106             .thenReturn(new BigDecimal("598.00"));
107         when(order.getPlacedAt()).thenReturn(placedAt);
108     }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 109–113

```
109         List<OrderSummaryResponse> response =
110             orderQueryService.getOrders(
111                 USER_ID.toString()
112             );
113     }
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 114–123

```
114         assertThat(response).containsExactly(
115             new OrderSummaryResponse(
116                 ORDER_ID,
117                 OrderStatus.PLACED,
118                 new BigDecimal("598.00"),
119                 placedAt
120             )
121         );
122     }
123 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 124–127

```
124     @Test
125     void getOrderShouldReturnOwnedOrderDetails() {
126         stubAuthenticatedUser();
127     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 128–132

```
128         when(orderRepository.findByIdAndUser_Id(
```

```
129         ORDER_ID,  
130         USER_ID  
131     })).thenReturn(Optional.of(order));  
132
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 133–136

```
133     when(orderItemRepository  
134         .findAllByOrder_IdOrderByIdAsc(ORDER_ID))  
135         .thenReturn(List.of(orderItem));  
136
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 137–139

```
137     when(paymentRepository.findByOrder_Id(ORDER_ID))  
138         .thenReturn(Optional.of(payment));  
139
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 140–145

```
140     when(orderResponseMapper.toCheckoutResponse(  
141         order,  
142         List.of(orderItem),  
143         payment  
144     })).thenReturn(checkoutResponse);  
145
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 146–151

```
146     CheckoutResponse response =  
147         orderQueryService.getOrder(  
148             USER_ID.toString(),  
149             ORDER_ID
```

```
150         );  
151     }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 152–156

```
152         assertThat(response).isSameAs(checkoutResponse);  
153     }  
154     verify(orderRepository)  
155         .findByIdAndUser_Id(ORDER_ID, USER_ID);  
156 }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 157–164

```
157     verify(orderResponseMapper)  
158         .toCheckoutResponse(  
159         order,  
160         List.of(orderItem),  
161         payment  
162     );  
163 }  
164 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 165–168

```
165     @Test  
166     void getOrderShouldHideAnotherUsersOrder() {  
167         stubAuthenticatedUser();  
168     }
```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 169–173

```
169         when(orderRepository.findByIdAndUser_Id(  
170             ORDER_ID,  
171             USER_ID  
172         )).thenReturn(Optional.empty());  
173     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 174–183

```

174         assertThatThrownBy(() ->
175             orderQueryService.getOrder(
176                 USER_ID.toString(),
177                 ORDER_ID
178             ))
179             .assertInstanceOf(ResourceNotFoundException.class)
180             .hasMessage(
181                 "Order with ID 101 was not found"
182             );
183

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 184–190

```

184         verifyNoInteractions(
185             orderItemRepository,
186             paymentRepository,
187             orderResponseMapper
188         );
189     }
190

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 191–200

```

191     @Test
192     void getOrdersShouldRejectInvalidJwtSubject() {
193         assertThatThrownBy(() ->
194             orderQueryService.getOrders(
195                 "not-a-user-id"
196             ))
197             .assertInstanceOf(
198                 InvalidAccessTokenException.class
199             );
200

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 201–209

```

201         verifyNoInteractions(
202             userRepository,
203             orderRepository,

```

```

204         orderItemRepository,
205         paymentRepository,
206         orderResponseMapper
207     );
208 }
209

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 210–214

```

210     @Test
211     void getLibraryShouldReturnBooksFromNonCancelledOrders() {
212         Instant placedAt = Instant.parse("2026-08-03T08:18:00Z");
213         stubAuthenticatedUser();
214

```

How to read this block

- Each `@Test` method is an independently executable behavior specification.

Lines 215–228

```

215         when(orderItemRepository
216             .findAllByOrder_User_IdAndOrder_StatusNotOrderByOrder_PlacedAtDescIdDesc(
217                 USER_ID,
218                 OrderStatus.CANCELLED
219             )).thenReturn(List.of(orderItem));
220         when(orderItem.getId()).thenReturn(501L);
221         when(orderItem.getOrder()).thenReturn(order);
222         when(orderItem.getBookEdition()).thenReturn(bookEdition);
223         when(orderItem.getBookTitle()).thenReturn("The Discovery of India");
224         when(orderItem.getCoverImageUrl()).thenReturn("https://example.test/cover.jpg");
225         when(orderItem.getBookFormat()).thenReturn(BookFormat.PAPERBACK);
226         when(orderItem.getIsbn13()).thenReturn("9780143031031");
227         when(orderItem.getPublisherName()).thenReturn("Penguin");
228         when(orderItem.getQuantity()).thenReturn(1);

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 229–235

```

229         when(order.getId()).thenReturn(ORDER_ID);
230         when(order.getStatus()).thenReturn(OrderStatus.SHIPPED);
231         when(order.getPlacedAt()).thenReturn(placedAt);
232         when(bookEdition.getId()).thenReturn(201L);
233         when(bookEdition.getBook()).thenReturn(book);
234         when(book.getId()).thenReturn(301L);
235

```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 236–249

```
236         assertThat(orderQueryService.getLibrary(USER_ID.toString()))
237             .containsExactly(new LibraryBookResponse(
238                 501L,
239                 ORDER_ID,
240                 301L,
241                 201L,
242                 "The Discovery of India",
243                 "https://example.test/cover.jpg",
244                 BookFormat.PAPERBACK,
245                 null,
246                 "9780143031031",
247                 "Penguin",
248                 1,
249                 OrderStatus.SHIPPED,
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 250–253

```
250             placedAt
251         ));
252     }
253 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 254–257

```
254     private void stubAuthenticatedUser() {
255         when(userRepository.findById(USER_ID))
256             .thenReturn(Optional.of(user));
257     }
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.
- This method groups one named responsibility. Parameters are its inputs; its body validates, transforms, delegates or persists them.

Lines 258–262

```
258         when(user.isEnabled()).thenReturn(true);
259         when(user.getId()).thenReturn(USER_ID);
260     }
261 }
262 }
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

backend/src/test/java/com/akshara/api/review/service/

ReviewDiscussionServiceTest.java

File 77 of 99 · Added or modified · Reviews and discussions

Purpose: Verifies ReviewDiscussionService behavior, including success paths and important validation, authorization or error cases.

Current file length: 79 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 package com.akshara.api.review.service;
2
3 import com.akshara.api.book.repository.BookRepository;
4 import com.akshara.api.common.exception.ResourceNotFoundException;
5 import com.akshara.api.review.dto.ReviewReplyRequest;
6 import com.akshara.api.review.entity.Review;
7 import com.akshara.api.review.entity.ReviewReply;
8 import com.akshara.api.review.repository.ReviewReplyRepository;
9 import com.akshara.api.review.repository.ReviewRepository;
10 import com.akshara.api.user.entity.AppUser;
11 import com.akshara.api.user.service.UserService;
12 import org.junit.jupiter.api.BeforeEach;
13 import org.junit.jupiter.api.Test;
14 import org.junit.jupiter.api.extension.ExtendWith;
```

How to read this block

- The package declaration places the type in Akshara’s module namespace; imports below it make external and neighbouring types available without fully qualified names.
- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 15–18

```
15 import org.mockito.ArgumentCaptor;
16 import org.mockito.Mock;
17 import org.mockito.junit.jupiter.MockitoExtension;
18
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 19–26

```
19 import java.util.Optional;
20
21 import static org.assertj.core.api.Assertions.assertThat;
22 import static org.assertj.core.api.Assertions.assertThatThrownBy;
23 import static org.mockito.ArgumentMatchers.any;
24 import static org.mockito.Mockito.verify;
25 import static org.mockito.Mockito.when;
26
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 27–29

```
27 @ExtendWith(MockitoExtension.class)
28 class ReviewDiscussionServiceTest {
29
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 30–38

```
30     @Mock ReviewRepository reviewRepository;
31     @Mock BookRepository bookRepository;
32     @Mock UserService userService;
33     @Mock ReviewReplyRepository reviewReplyRepository;
34     @Mock AppUser user;
35     @Mock AppUser anotherUser;
36     @Mock Review review;
37     @Mock ReviewReply reply;
38
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.

Lines 39–45

```
39     private ReviewService service;
40
41     @BeforeEach
42     void setUp() {
43         service = new ReviewService(reviewRepository, bookRepository, userService, reviewReplyRepository);
44     }
45
```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Fields hold injected collaborators, constants or entity state. `final` prevents reassignment after construction and makes dependencies explicit.

Lines 46–54

```
46     @Test
47     void createReplyShouldAttachAuthenticatedReaderToReview() {
48         when(userService.getCurrentUserEntity("1")).thenReturn(user);
49         when(reviewRepository.findById(7L)).thenReturn(Optional.of(review));
```

```

50         when(reviewReplyRepository.save(any(ReviewReply.class))).thenAnswer(invocation ->
invocation.getArgument(0));
51         when(user.getId()).thenReturn(1L);
52         when(user.getFullName()).thenReturn("A Reader");
53         when(review.getId()).thenReturn(7L);
54

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 55–65

```

55         var response = service.createReply("1", 7L, new ReviewReplyRequest(" A thoughtful reply "));
56
57         assertThat(response.reviewId()).isEqualTo(7L);
58         assertThat(response.userId()).isEqualTo(1L);
59         assertThat(response.content()).isEqualTo("A thoughtful reply");
60         ArgumentCaptor<ReviewReply> captor = ArgumentCaptor.forClass(ReviewReply.class);
61         verify(reviewReplyRepository).save(captor.capture());
62         assertThat(captor.getValue().getReview()).isSameAs(review);
63         assertThat(captor.getValue().getUser()).isSameAs(user);
64     }
65

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 66–73

```

66     @Test
67     void updateReplyShouldHideAnotherReadersReply() {
68         when(userService.getCurrentUserEntity("1")).thenReturn(user);
69         when(user.getId()).thenReturn(1L);
70         when(reviewReplyRepository.findById(9L)).thenReturn(Optional.of(reply));
71         when(reply.getUser()).thenReturn(anotherUser);
72         when(anotherUser.getId()).thenReturn(2L);
73

```

How to read this block

- Repository calls are the persistence boundary: services ask for domain data or save/delete it without writing JDBC in the business layer.
- Each `@Test` method is an independently executable behavior specification.
- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

Lines 74–79

```
74         assertThatThrownBy(() -> service.updateReply("1", 9L, new ReviewReplyRequest("Changed")))
75             .isInstanceOf(ResourceNotFoundException.class)
76             .hasMessage("Discussion reply with ID 9 was not found");
77     }
78 }
79
```

How to read this block

- The test arranges dependencies/input, performs one operation, asserts the observable result, and verifies important collaboration where appropriate.

frontend/src/App.jsx

File 78 of 99 · Added or modified · Project support

Purpose: Renders and coordinates the App React interface.

Current file length: 56 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 import { Route, Routes, useLocation } from 'react-router-dom'
2 import Footer from './components/Footer'
3 import Navbar from './components/Navbar'
4 import CataloguePage from './pages/CataloguePage'
5 import BookDetailsPage from './pages/BookDetailsPage'
6 import AuthPage from './pages/AuthPage'
7 import WishlistPage from './pages/WishlistPage'
8 import CartPage from './pages/CartPage'
9 import CheckoutPage from './pages/CheckoutPage'
10 import OrdersPage from './pages/OrdersPage'
11 import CommunityPage from './pages/CommunityPage'
12 import ProtectedRoute from './components/ProtectedRoute'
13 import AdminRoute from './components/AdminRoute'
14 import BookImportPage from './pages/BookImportPage'
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 15–20

```
15 import CatalogueAdminPage from './pages/CatalogueAdminPage'
16 import RouteErrorBoundary from './components/RouteErrorBoundary'
17 import MyBooksPage from './pages/MyBooksPage'
18 import AdminOrdersPage from './pages/AdminOrdersPage'
19 import MyReviewsPage from './pages/MyReviewsPage'
20
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 21–23

```
21 function App() {
22   const location = useLocation()
23
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 24–27

```
24   return (
25     <>
26       <Navbar />
27
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 28–41

```
28     <main className="site-main">
29       <RouteErrorBoundary resetKey={location.pathname}>
30         <Routes>
31           <Route path="/" element={<CataloguePage />} />
32           <Route path="/books/:bookId" element={<BookDetailsPage />} />
33           <Route path="/community" element={<CommunityPage />} />
34           <Route path="/login" element={<AuthPage mode="login" />} />
35           <Route path="/register" element={<AuthPage mode="register" />} />
36           <Route path="/wishlist" element={<ProtectedRoute><WishlistPage /></ProtectedRoute>} />
37           <Route path="/cart" element={<ProtectedRoute><CartPage /></ProtectedRoute>} />
38           <Route path="/checkout" element={<ProtectedRoute><CheckoutPage /></ProtectedRoute>} />
39           <Route path="/orders" element={<ProtectedRoute><OrdersPage /></ProtectedRoute>} />
40           <Route path="/orders/:orderId" element={<ProtectedRoute><OrdersPage /></ProtectedRoute>} />
41           <Route path="/my-books" element={<ProtectedRoute><MyBooksPage /></ProtectedRoute>} />
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 42–49

```
42           <Route path="/my-reviews" element={<ProtectedRoute><MyReviewsPage /></ProtectedRoute>} />
43           <Route path="/admin/books/import" element={<AdminRoute><BookImportPage /></AdminRoute>} />
44           <Route path="/admin/books" element={<AdminRoute><CatalogueAdminPage /></AdminRoute>} />
45           <Route path="/admin/orders" element={<AdminRoute><AdminOrdersPage /></AdminRoute>} />
46         </Routes>
47       </RouteErrorBoundary>
48     </main>
49
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 50–54

```
50     <Footer />
51   </>
52 )
53 }
54
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 55–56

```
55 export default App
56
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/components/AdminRoute.jsx

File 79 of 99 · Added or modified · Frontend component

Purpose: Prevents non-admin navigation into administrator pages while backend security remains authoritative.

Current file length: 24 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–3

```
1 import { Navigate, useLocation } from 'react-router-dom'
2 import useAuth from '../hooks/useAuth'
3
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 4–7

```
4 function AdminRoute({ children }) {
5   const { ready, isAuthenticated, user } = useAuth()
6   const location = useLocation()
7
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 8–11

```
8   if (!ready) {
9     return <div className="page-loading"><span className="spinner-border" /> Checking admin access...</div>
10  }
11
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 12–15

```
12   if (!isAuthenticated) {
13     return <Navigate to="/login" state={{ from: location }} replace />
14   }
15
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 16–19

```
16   if (user?.role !== 'ADMIN') {  
17     return <Navigate to="/" replace />  
18   }  
19
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 20–22

```
20   return children  
21 }  
22
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 23–24

```
23 export default AdminRoute  
24
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/components/DiscussionThread.jsx

File 80 of 99 · Added or modified · Frontend component

Purpose: Loads, creates and deletes replies under a review while respecting reader ownership.

Current file length: 119 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–11

```
1 import { useState } from 'react'
2 import { Link } from 'react-router-dom'
3 import useAuth from '../hooks/useAuth'
4 import { getApiErrorMessage } from '../services/api'
5 import {
6   createReviewReply,
7   deleteReviewReply,
8   getReviewReplies,
9   updateReviewReply,
10 } from '../services/readerService'
11
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 12–23

```
12 function DiscussionThread({ reviewId }) {
13   const { user } = useAuth()
14   const [open, setOpen] = useState(false)
15   const [loaded, setLoaded] = useState(false)
16   const [loading, setLoading] = useState(false)
17   const [replies, setReplies] = useState([])
18   const [draft, setDraft] = useState('')
19   const [editingId, setEditingId] = useState(null)
20   const [editingText, setEditingText] = useState('')
21   const [busy, setBusy] = useState(false)
22   const [error, setError] = useState('')
23
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 24–37

```
24 async function toggleThread() {
25   const nextOpen = !open
26   setOpen(nextOpen)
27   if (!nextOpen || loaded) return
28   setLoading(true)
29   setError('')
30   try {
31     setReplies(await getReviewReplies(reviewId))
32     setLoaded(true)
33   } catch (requestError) {
34     setError(getApiErrorMessage(requestError, 'The discussion could not be loaded.'))
35   } finally {
```

```
36     setLoading(false)
37   }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 38–51

```
38   }
39
40   async function submitReply(event) {
41     event.preventDefault()
42     if (draft.trim().length < 2) return
43     setBusy(true)
44     setError('')
45     try {
46       const created = await createReviewReply(reviewId, draft.trim())
47       setReplies((items) => [...items, created])
48       setDraft('')
49     } catch (requestError) {
50       setError(getApiErrorMessage(requestError, 'Your reply could not be posted.'))
51     } finally {
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 52–55

```
52     setBusy(false)
53   }
54 }
55
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 56–69

```
56   async function saveEdit(replyId) {
57     if (editingText.trim().length < 2) return
```

```

58     setBusy(true)
59     setError('')
60     try {
61         const updated = await updateReviewReply(replyId, editingText.trim())
62         setReplies((items) => items.map((reply) => reply.id === replyId ? updated : reply))
63         setEditingId(null)
64         setEditingText('')
65     } catch (requestError) {
66         setError(getApiErrorMessage(requestError, 'Your reply could not be updated.'))
67     } finally {
68         setBusy(false)
69     }

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 70–83

```

70     }
71
72     async function removeReply(reply) {
73         if (!window.confirm('Delete this discussion reply?')) return
74         setBusy(true)
75         setError('')
76         try {
77             await deleteReviewReply(reply.id)
78             setReplies((items) => items.filter((item) => item.id !== reply.id))
79         } catch (requestError) {
80             setError(getApiErrorMessage(requestError, 'Your reply could not be deleted.'))
81         } finally {
82             setBusy(false)
83         }

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 84–97

```
84   }
85
86   return (
87     <section className={`discussion-thread${open ? ' is-open' : ''}`}>
88       <button className="discussion-toggle" type="button" onClick={toggleThread} aria-expanded={open}>
89         <span><i className="bi bi-chat-left-text" /> {open ? 'Close discussion' : 'Open discussion'}</span>
90         {loaded && <small>{replies.length} {replies.length === 1 ? 'reply' : 'replies'}</small>}
91       </button>
92       {open && (
93         <div className="discussion-body">
94           <p className="discussion-explainer"><strong>Discussion is different from a review.</strong> Reply to the
reader's perspective, ask a question, or add another interpretation.</p>
95           {loading && <p className="muted-message">Loading discussion...</p>}
96           {!loading && replies.map((reply) => {
97             const mine = Number(reply.userId) === Number(user?.id)
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 98–111

```
98     return (
99       <article className="discussion-reply" key={reply.id}>
100         <span className="reader-avatar">{reply.readerName?.split(/\s+/).map((part) => part[0]).slice(0,
2).join('')}</span>
101         <div>
102           <header><strong>{reply.readerName}</strong><small>{new
Date(reply.createdAt).toLocaleDateString('en-IN', { dateStyle: 'medium' })}</small></header>
103           {editingId === reply.id ? <textarea maxLength="1000" minLength="2" value={editingText}
onChange={(event) => setEditingText(event.target.value)} /> : <p>{reply.content}</p>}
104           {mine && <div className="discussion-owner-actions">{editingId === reply.id ? <><button
disabled={busy} type="button" onClick={() => saveEdit(reply.id)}>Save</button><button type="button" onClick={() =>
setEditingId(null)}>Cancel</button></> : <><button type="button" onClick={() => { setEditingId(reply.id);
setEditingText(reply.content) }}>Edit</button><button className="danger" disabled={busy} type="button" onClick={() =>
removeReply(reply)}>Delete</button></></div>}
105         </div>
106       </article>
107     )
108   )}
109   {!loading && loaded && !replies.length && <p className="muted-message">No replies yet. Start a focused
conversation about this review.</p>}
110   {user?.role === 'READER' ? <form className="discussion-reply-form" onSubmit={submitReply}><textarea
value={draft} minLength="2" maxLength="1000" required placeholder="Reply thoughtfully to this reader..." onChange={(event)
=> setDraft(event.target.value)} /><button disabled={busy} type="submit">{busy ? 'Posting...' : 'Post
reply'}</button></form> : <p className="discussion-signin"><Link to="/login">Sign in as a reader</Link> to join this
discussion.</p>}
111   {error && <p className="form-alert" role="alert">{error}</p>}
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 112–117

```
112         </div>
113     )}
114 </section>
115 )
116 }
117
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 118–119

```
118 export default DiscussionThread
119
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/components/Footer.jsx

File 81 of 99 · Added or modified · Frontend component

Purpose: Renders and coordinates the Footer React interface.

Current file length: 48 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 import { Link } from 'react-router-dom'
2
3 function Footer() {
4   return (
5     <footer className="site-footer">
6       <div className="container-xl">
7         <div className="footer-grid">
8           <div>
9             <Link className="footer-wordmark" to="/">AKSHARA</Link>
10            <p className="footer-description">
11              A reader-first bookstore for discovering ideas, sharing perspectives,
12              and building a life around books.
13            </p>
14          </div>
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 15–22

```
15
16     <div>
17       <p className="footer-heading">Explore</p>
18       <a href="/#catalogue">Book catalogue</a>
19       <a href="/#journey">Reading journey</a>
20       <Link to="/community">Community</Link>
21     </div>
22
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 23–31

```
23     <div>
24       <p className="footer-heading">For readers</p>
25       <Link to="/my-books">My Books</Link>
```

```

26         <Link to="/wishlist">Wishlist</Link>
27         <Link to="/orders">Order history</Link>
28         <Link to="/my-reviews">My reviews</Link>
29         <Link to="/community">Reader reviews</Link>
30     </div>
31

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 32–37

```

32         <div className="footer-note">
33             <span className="footer-letter">अ</span>
34             <p>Every story, idea, and discovery begins with a letter.</p>
35         </div>
36     </div>
37

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 38–46

```

38         <div className="footer-bottom">
39             <span>© 2026 Akshara</span>
40             <span>Discover · Read · Grow</span>
41         </div>
42     </div>
43 </footer>
44 )
45 }
46

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 47–48

```

47 export default Footer
48

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/components/Navbar.jsx

File 82 of 99 · Added or modified · Frontend component

Purpose: Renders and coordinates the Navbar React interface.

Current file length: 99 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 import { Link, useNavigate } from 'react-router-dom'
2 import logo from '../assets/akshara-logo.png'
3 import useAuth from '../hooks/useAuth'
4 import useReaderData from '../hooks/useReaderData'
5
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 6–10

```
6 function Navbar() {
7   const { user, logout } = useAuth()
8   const { wishlist, cart } = useReaderData()
9   const navigate = useNavigate()
10
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 11–15

```
11   function signOut() {
12     logout()
13     navigate('/')
14   }
15
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 16–29

```
16   return (
17     <nav className="navbar navbar-expand-lg akshara-navbar sticky-top">
18       <div className="container-xl">
19         <Link className="navbar-brand akshara-brand" to="/" aria-label="Akshara home">
20           <span
21             className="akshara-brand-mark"
22             style={{ backgroundImage: `url(${logo})` }}
23             aria-hidden="true"
24           />
25           <span className="akshara-brand-copy">
26             <strong>AKSHARA</strong>
27             <small>Discover · Read · Grow</small>
28           </span>
29         </div>
30       </nav>
31     )
32
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 30–42

```

30
31     <button
32         className="navbar-toggler"
33         type="button"
34         data-bs-toggle="collapse"
35         data-bs-target="#aksharaNavbar"
36         aria-controls="aksharaNavbar"
37         aria-expanded="false"
38         aria-label="Toggle navigation"
39     >
40         <span className="navbar-toggler-icon" />
41     </button>
42

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 43–51

```

43     <div
44         className="collapse navbar-collapse"
45         id="aksharaNavbar"
46     >
47         <ul className="navbar-nav ms-auto align-items-lg-center akshara-nav-links">
48             <li className="nav-item">
49                 <a className="nav-link" href="/#catalogue">Explore</a>
50             </li>
51

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 52–55

```

52         <li className="nav-item">
53             {user?.role === 'READER' ? <Link className="nav-link" to="/my-books">My Books</Link> : <a
54             className="nav-link" href="/#journey">Reading journey</a>}
55         </li>
56

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 56–59

```
56         <li className="nav-item">
57             <Link className="nav-link" to="/community">Community</Link>
58         </li>
59     </>
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 60–73

```
60         {user?.role === 'ADMIN' ? (
61             <>
62                 <li className="nav-item"><Link className="nav-link admin-nav-link" to="/admin/orders"><i
className="bi bi-box-seam" /> Orders</Link></li>
63                 <li className="nav-item"><Link className="nav-link admin-nav-link" to="/admin/books"><i
className="bi bi-sliders" /> Catalogue</Link></li>
64             </>
65         ) : (
66             <>
67                 <li className="nav-item">
68                     <Link className="nav-icon-button nav-count-link" to="/wishlist" aria-label="Open wishlist">
69                         <i className="bi bi-heart" />
70                         {wishlist.length > 0 && <span>{wishlist.length}</span>}
71                     </Link>
72                 </li>
73             </>
64         )
65     </>
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 74–82

```
74         <li className="nav-item">
75             <Link className="nav-icon-button nav-count-link" to="/cart" aria-label="Open cart">
76                 <i className="bi bi-bag" />
77                 {cart?.totalQuantity > 0 && <span>{cart.totalQuantity}</span>}
78             </Link>
79         </li>
80     </>
81 )}
82
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props

carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 83–96

```
83         <li className="nav-item ms-lg-1">
84             {user ? (
85                 <div className="nav-reader-menu">
86                     <Link to={user.role === 'ADMIN' ? '/admin/books' : '/my-books'}><i className="bi bi-person-circle"
/> {user.fullName.split(' ')[0]}</Link>
87                     <button type="button" onClick={signOut}>Sign out</button>
88                 </div>
89             ) : <Link className="btn btn-ink nav-join-button" to="/register">Join Akshara</Link>}
90         </li>
91     </ul>
92 </div>
93 </div>
94 </nav>
95 )
96 }
```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 97–99

```
97
98 export default Navbar
99
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/components/ReviewSection.jsx

File 83 of 99 · Added or modified · Frontend component

Purpose: Renders and coordinates the ReviewSection React interface.

Current file length: 177 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–11

```
1 import { useEffect, useMemo, useState } from 'react'
2 import { Link } from 'react-router-dom'
3 import useAuth from '../hooks/useAuth'
4 import { getApiErrorMessage } from '../services/api'
5 import {
6   createReview,
7   deleteReview,
8   getBookReviews,
9   updateReview,
10 } from '../services/readerService'
11
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 12–23

```
12 function ReviewSection({ bookId }) {
13   const { user } = useAuth()
14   const [data, setData] = useState({ reviews: [], reviewCount: 0, averageRating: 0 })
15   const [form, setForm] = useState({ rating: 5, headline: '', content: '' })
16   const [loading, setLoading] = useState(true)
17   const [saving, setSaving] = useState(false)
18   const [error, setError] = useState('')
19   const ownReview = useMemo(
20     () => data.reviews.find((review) => Number(review.userId) === Number(user?.id)),
21     [data.reviews, user],
22   )
23 }
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 24–37

```
24 async function loadReviews() {
25   try {
26     const nextData = await getBookReviews(bookId)
27     setData(nextData)
28     const currentReview = nextData.reviews.find((review) => Number(review.userId) === Number(user?.id))
29     if (currentReview) {
30       setForm({
31         rating: currentReview.rating,
32         headline: currentReview.headline ?? '',
33         content: currentReview.content,
34       })
35     }
36   }
37 }
```

```
36     } catch (requestError) {
37       setError(getApiErrorMessage(requestError, 'Reviews could not be loaded.'))
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 38–42

```
38     } finally {
39       setLoading(false)
40     }
41   }
42 }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 43–56

```
43   useEffect(() => {
44     let active = true
45     getBookReviews(bookId)
46       .then((nextData) => {
47         if (!active) return
48         setData(nextData)
49         const currentReview = nextData.reviews.find((review) => Number(review.userId) === Number(user?.id))
50         if (currentReview) {
51           setForm({ rating: currentReview.rating, headline: currentReview.headline ?? '', content:
currentReview.content })
52         }
53       })
54     .catch((requestError) => {
55       if (active) setError(getApiErrorMessage(requestError, 'Reviews could not be loaded.'))
56     })
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 57–60

```
57     .finally(() => {
58         if (active) setLoading(false)
59     })
60
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 61–63

```
61     return () => { active = false }
62 }, [bookId, user])
63
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 64–77

```
64 async function handleSubmit(event) {
65     event.preventDefault()
66     setSaving(true)
67     setError('')
68     try {
69         const payload = { rating: Number(form.rating), headline: form.headline, content: form.content }
70         if (ownReview) {
71             await updateReview(ownReview.id, payload)
72         } else {
73             await createReview({ ...payload, bookId: Number(bookId) })
74         }
75         await loadReviews()
76     } catch (requestError) {
77         setError(getApiErrorMessage(requestError, 'Your review could not be saved.'))
78     } finally {
79         setSaving(false)
80     }
81 }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 78–82

```
78     } finally {
79         setSaving(false)
80     }
81 }
```

```
80     }
81   }
82 }
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 83–96

```
83  async function handleDelete() {
84    if (!ownReview || !window.confirm('Delete your review?')) return
85    setSaving(true)
86    try {
87      await deleteReview(ownReview.id)
88      setForm({ rating: 5, headline: '', content: '' })
89      await loadReviews()
90    } catch (requestError) {
91      setError(getApiErrorMessage(requestError, 'Your review could not be deleted.'))
92    } finally {
93      setSaving(false)
94    }
95  }
96 }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 97–109

```
97  return (
98    <section className="reviews-section" id="reviews">
99      <div className="section-heading review-heading">
100        <div>
101          <p className="eyebrow">Reader perspectives</p>
102          <h2>What the community thinks</h2>
103        </div>
104        <div className="review-summary">
105          <strong>{data.reviewCount ? data.averageRating.toFixed(1) : 'New'}</strong>
106          <span>{data.reviewCount} {data.reviewCount === 1 ? 'review' : 'reviews'}</span>
107        </div>
108      </div>
109    </section>
110  )
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props

carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 110–123

```
110      {user ? (  
111        <form className="review-form" onSubmit={handleSubmit}>  
112          {ownReview && <div className="review-editing-notice"><i className="bi bi-pencil-square" /> You are editing  
your published review. You can update or delete it here, or from <Link to="/my-reviews">My Reviews</Link>.</div>  
113          <div className="rating-picker" aria-label="Rating">  
114            {[1, 2, 3, 4, 5].map((rating) => (  
115              <button  
116                type="button"  
117                key={rating}  
118                className={rating <= form.rating ? 'is-active' : ''}  
119                onClick={() => setForm((current) => ({ ...current, rating })))}  
120                aria-label={` ${rating} stars`}  
121              >★</button>  
122            )]}  
123          </div>
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 124–137

```
124      <input  
125        value={form.headline}  
126        onChange={(event) => setForm((current) => ({ ...current, headline: event.target.value })))  
127        placeholder="A short headline (optional)"  
128        maxLength="120"  
129      />  
130      <textarea  
131        value={form.content}  
132        onChange={(event) => setForm((current) => ({ ...current, content: event.target.value })))  
133        placeholder="Share what stayed with you about this book..."  
134        minLength="10"  
135        maxLength="2000"  
136        required  
137      />
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.

Lines 138–148

```
138         <div>
139             <button className="btn btn-ink" disabled={saving} type="submit">
140                 {saving ? 'Saving...' : ownReview ? 'Update my review' : 'Publish review'}
141             </button>
142             {ownReview && <button className="text-button danger" disabled={saving} type="button"
onClick={handleDelete}>Delete</button>}
143         </div>
144     </form>
145 ) : (
146     <div className="review-signin">Already read it? <Link to="/login">Sign in to write a review.</Link></div>
147 )}
148
```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 149–162

```
149     {error && <p className="form-alert" role="alert">{error}</p>}
150     {loading ? (
151         <p className="muted-message">Loading reader reviews...</p>
152     ) : data.reviews.length ? (
153         <div className="review-list">
154             {data.reviews.map((review) => (
155                 <article className="review-card" key={review.id}>
156                     <div className="reader-avatar">{review.readerName.split(/\s+/).map((part) => part[0]).slice(0,
2).join('')}</div>
157                     <div>
158                         <div className="review-card-meta">
159                             <strong>{review.readerName}</strong>
160                             <span>{'★'.repeat(review.rating)}{'★'.repeat(5 - review.rating)}</span>
161                         </div>
162                     {review.headline && <h3>{review.headline}</h3>}
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 163–175

```
163         <p>{review.content}</p>
164         <small>{new Date(review.createdAt).toLocaleDateString('en-IN', { dateStyle: 'medium' })}</small>
165     </div>
166 </article>
167 )}
168 </div>
169 ) : (
170     <p className="muted-message">No reviews yet. Begin the conversation for this book.</p>
171 )}
172 </section>
```

```
173    )  
174  }  
175
```

How to read this block

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 176–177

```
176 export default ReviewSection  
177
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/components/RouteErrorBoundary.jsx

File 84 of 99 · Added or modified · Frontend component

Purpose: Catches React rendering failures and replaces blank screens with a recovery interface.

Current file length: 46 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–3

```
1 import { Component } from 'react'
2 import { Link } from 'react-router-dom'
3
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 4–9

```
4 class RouteErrorBoundary extends Component {
5   constructor(props) {
6     super(props)
7     this.state = { error: null }
8   }
9
```

How to read this block

- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 10–13

```
10 static getDerivedStateFromError(error) {
11   return { error }
12 }
13
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 14–17

```
14 componentDidCatch(error, details) {
15   console.error('Page rendering failed', error, details)
16 }
17
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 18–23

```
18   componentDidUpdate(previousProps) {
19     if (previousProps.resetKey !== this.props.resetKey && this.state.error) {
20       this.setState({ error: null })
21     }
22   }
23 }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 24–26

```
24   render() {
25     if (!this.state.error) return this.props.children
26   }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- Constructor injection/assignment makes required collaborators and initial state explicit, avoiding hidden global dependencies.

Lines 27–40

```
27   return (
28     <section className="inner-page">
29       <div className="container-xl">
30         <div className="catalogue-message catalogue-error">
31           <span><i className="bi bi-exclamation-triangle" /></span>
32           <div>
33             <h1>This page could not be displayed</h1>
34             <p>Akshara encountered an unexpected display error. Your account and stored data were not changed.</p>
35             <button type="button" onClick={() => window.location.reload()}>Reload this page</button>
36             <Link to="/">Return to the catalogue</Link>
37           </div>
38         </div>
39       </div>
40     </section>
  )
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 41–44

```
41     )  
42   }  
43 }  
44
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 45–46

```
45 export default RouteErrorBoundary  
46
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/index.css

File 85 of 99 · Added or modified · Visual system and responsive layout

Purpose: Defines the Akshara editorial visual language and responsive layouts for catalogue, admin, orders, library, reviews and community.

Current file length: 4944 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–18

```
1  :root {
2    --ink: #102a26;
3    --ink-soft: #25443e;
4    --forest: #174f42;
5    --forest-light: #2f6f5e;
6    --paper: #f7f2e8;
7    --paper-deep: #eee5d6;
8    --white: #ffffdfa;
9    --saffron: #df8d2d;
10   --saffron-light: #f2b862;
11   --navy: #163c66;
12   --clay: #a64d35;
13   --plum: #6c3d5e;
14   --teal: #196b69;
15   --muted: #697873;
16   --line: #dcd4c7;
17   --serif: "Iowan Old Style", "Palatino Linotype", "Book Antiqua", Georgia, serif;
18   --sans: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 19–24

```
19  font-family: var(--sans);
20  color: var(--ink);
21  background: var(--paper);
22  font-synthesis: none;
23  text-rendering: optimizeLegibility;
24  }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 25–27

```
25
26  /* Catalogue administration */
27  .catalogue-admin-page { min-height: 100vh; background: #f4efe5; color: #15332e; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 28–28

```
28 .catalogue-admin-hero { padding: 4.5rem 0 5.8rem; background: linear-gradient(120deg, #0d382f, #174f40); color: #fff; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 29–29

```
29 .catalogue-admin-hero .container-xl { display: flex; align-items: center; justify-content: space-between; gap: 2rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 30–30

```
30 .catalogue-admin-hero h1 { max-width: 720px; margin: .35rem 0 .75rem; font-family: Georgia, serif; font-size: clamp(2.6rem, 5vw, 5rem); line-height: .98; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 31–31

```
31 .catalogue-admin-hero p:last-child { max-width: 720px; margin: 0; color: #c8d9d3; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 32–32

```
32 .catalogue-admin-hero .btn { flex: 0 0 auto; padding: .85rem 1.15rem; border-radius: 999px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 33–33

```
33 .catalogue-admin-hero-actions { display: flex; align-items: center; gap: .65rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 34–34

```
34 .catalogue-admin-hero-actions button { border: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 35–35

```
35 .catalogue-admin-workspace { position: relative; margin-top: -2.4rem; padding-bottom: 5rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 36–36

```
36 .catalogue-admin-summary { display: grid; grid-template-columns: repeat(4, 1fr); overflow: hidden; background: #fffd8; border: 1px solid #ded4c5; border-radius: 18px; box-shadow: 0 14px 40px rgba(25, 47, 39, .1); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 37–37

```
37 .catalogue-admin-summary div { display: flex; flex-direction: column; padding: 1.35rem 1.5rem; border-right: 1px solid #e8dfd2; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 38–38

```
38 .catalogue-admin-summary div:last-child { border-right: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 39–39

```
39 .catalogue-admin-summary strong { font-family: Georgia, serif; font-size: 2rem; line-height: 1; color: #123b32; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 40–40

```
40 .catalogue-admin-summary span { margin-top: .4rem; color: #697a75; font-size: .83rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 41–41

```
41 .catalogue-admin-toolbar { display: flex; justify-content: space-between; gap: 1rem; margin: 1.5rem 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 42–42

```
42 .catalogue-admin-toolbar label { display: flex; align-items: center; flex: 1; max-width: 620px; padding: 0 1rem; background: #ffffdf8; border: 1px solid #d9cfbf; border-radius: 12px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 43–43

```
43 .catalogue-admin-toolbar input { width: 100%; padding: .85rem .7rem; border: 0; outline: 0; background: transparent; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 44–44

```
44 .catalogue-admin-toolbar button { padding: .75rem 1rem; border: 1px solid #cfc4b3; border-radius: 10px; background: #ffffdf8; color: #17483d; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 45–45

```
45 .admin-feedback { margin: 1rem 0; padding: .9rem 1rem; border-radius: 10px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 46–46

```
46 .admin-feedback-error { background: #f8dfd8; border: 1px solid #efb9af; color: #8b3025; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 47–47

```
47 .admin-feedback-success { background: #e0eee7; border: 1px solid #b2d2c1; color: #17533f; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 48–48

```
48 .catalogue-admin-list { display: grid; gap: 1.25rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 49–49

```
49 .catalogue-admin-book { overflow: hidden; background: #ffffd8; border: 1px solid #dcd2c2; border-radius: 18px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 50–50

```
50 .catalogue-admin-book > header { display: grid; grid-template-columns: 64px minmax(0, 1fr) auto; align-items: center; gap: 1rem; padding: 1.25rem 1.4rem; border-bottom: 1px solid #e7ded1; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 51–51

```
51 .catalogue-admin-cover { display: grid; place-items: center; width: 58px; height: 76px; overflow: hidden; border-radius: 6px; background: #e8dfd0; color: #b37c1d; font-size: 1.5rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 52–52

```
52 .catalogue-admin-cover img { width: 100%; height: 100%; object-fit: cover; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 53–53

```
53 .catalogue-admin-book header p { margin: 0 0 .2rem; color: #a26717; font-size: .76rem; text-transform: uppercase; letter-spacing: .08em; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 54–54

```
54 .catalogue-admin-book header h2 { margin: 0; font-family: Georgia, serif; font-size: 1.45rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 55–55

```
55 .catalogue-admin-book header span { color: #75827e; font-size: .82rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 56–56

```
56 .catalogue-admin-book-actions { display: flex; align-items: center; gap: .6rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 57–57

```
57 .catalogue-admin-book-actions button, .catalogue-admin-book-actions a { padding: .58rem .72rem; border: 1px solid #d7ccbc; border-radius: 8px; background: #fff; color: #184a3f; font-size: .82rem; text-decoration: none; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 58–58

```
58 .catalogue-admin-book-actions button.is-featured { background: #fff1cf; border-color: #e4bd60; color: #86580b; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 59–59

```
59 .catalogue-admin-book-actions button.catalogue-delete-button { border-color: #e0b6ad; color: #9a3327; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 60–60

```
60 .catalogue-admin-book-actions button.catalogue-delete-button:hover { background: #fae5e0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 61–61

```
61 .catalogue-admin-editions { padding: .25rem 1.4rem 1.4rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 62–62

```
62 .catalogue-admin-edition { padding: 1.15rem 0; border-bottom: 1px solid #ebe3d7; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 63–63

```
63 .catalogue-admin-edition:last-child { border-bottom: 0; padding-bottom: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 64–64

```
64 .edition-identity { display: flex; align-items: center; flex-wrap: wrap; gap: .65rem; margin-bottom: .8rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 65–65

```
65 .edition-identity strong { padding: .3rem .5rem; border-radius: 5px; background: #17493d; color: #fff; font-size: .7rem; letter-spacing: .06em; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 66–66

```
66 .edition-identity span { color: #687975; font-size: .8rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 67–67

```
67 .edition-inventory-fields { display: grid; grid-template-columns: 150px 140px minmax(170px, 1fr) 145px auto; align-items: end; gap: .75rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 68–68

```
68 .edition-inventory-fields label, .edition-restock label { display: grid; gap: .35rem; color: #687975; font-size: .75rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 69–69

```
69 .edition-inventory-fields input, .edition-inventory-fields select, .edition-restock input { width: 100%; min-height: 42px; padding: .55rem .65rem; border: 1px solid #d8cebe; border-radius: 8px; background: #fff; color: #163a32; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 70–70

```
70 .edition-active-toggle { display: flex !important; flex-direction: row; align-items: center; min-height: 42px; padding: 0 .7rem; border: 1px solid #d8cebe; border-radius: 8px; background: #fff; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 71–71

```
71 .edition-active-toggle input { width: auto; min-height: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 72–72

```
72 .edition-save-button, .edition-restock button { min-height: 42px; padding: .55rem .85rem; border: 0; border-radius: 8px; background: #17493d; color: #fff; white-space: nowrap; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 73–73

```
73 .edition-restock { display: flex; align-items: end; gap: .6rem; max-width: 360px; margin-top: .8rem; padding: .8rem; border-radius: 10px; background: #f4efe5; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 74–74

```
74 .edition-restock label { flex: 1; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 75–75

```
75 .catalogue-admin-empty { padding: 4rem 1rem; text-align: center; color: #667873; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 76–76

```
76 .catalogue-admin-empty i { color: #c08425; font-size: 2rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 77–77

```
77 .catalogue-admin-empty h2 { margin: .7rem 0 .2rem; font-family: Georgia, serif; color: #163a32; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 78–84

```
78
79 @media (max-width: 992px) {
80   .catalogue-admin-summary { grid-template-columns: repeat(2, 1fr); }
81   .catalogue-admin-summary div:nth-child(2) { border-right: 0; }
82   .catalogue-admin-summary div:nth-child(-n+2) { border-bottom: 1px solid #e8dfd2; }
83   .edition-inventory-fields { grid-template-columns: repeat(2, minmax(0, 1fr)); }
84 }
```


How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 85–93

```
85
86 @media (max-width: 640px) {
87   .catalogue-admin-hero .container-xl, .catalogue-admin-hero-actions, .catalogue-admin-toolbar, .catalogue-admin-
book-actions { align-items: stretch; flex-direction: column; }
88   .catalogue-admin-summary { grid-template-columns: 1fr 1fr; }
89   .catalogue-admin-book > header { grid-template-columns: 52px minmax(0, 1fr); }
90   .catalogue-admin-book-actions { grid-column: 1 / -1; }
91   .edition-inventory-fields { grid-template-columns: 1fr; }
92   .edition-restock { max-width: none; }
93 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 94–97

```
94
95 * {
96   box-sizing: border-box;
97 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 98–102

```
98
99 html {
100   scroll-behavior: smooth;
101   scroll-padding-top: 92px;
102 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 103–110

```
103
104 body {
105   margin: 0;
```

```
106 min-width: 320px;
107 min-height: 100vh;
108 color: var(--ink);
109 background: var(--paper);
110 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 111–117

```
111
112 button,
113 input,
114 select,
115 textarea {
116   font: inherit;
117 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 118–122

```
118
119 button,
120 a {
121   -webkit-tap-highlight-color: transparent;
122 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 123–126

```
123
124 a {
125   color: inherit;
126 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 127–135

```
127
128 button:focus-visible,
129 a:focus-visible,
130 input:focus-visible,
131 select:focus-visible,
132 textarea:focus-visible {
133   outline: 3px solid rgb(223 141 45 / 45%);
134   outline-offset: 3px;
135 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 136–145

```
136
137 /* Catalogue administration */
138 .admin-nav-link {
139     display: inline-flex !important;
140     align-items: center;
141     gap: 0.45rem;
142     color: var(--forest) !important;
143     background: var(--paper-deep);
144     border-radius: 999px;
145 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 146–150

```
146
147 .admin-import-page {
148     min-height: 100vh;
149     background: #f3eee5;
150 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 151–158

```
151
152 .admin-import-hero {
153     padding: 5rem 0 6.8rem;
154     color: var(--white);
155     background:
156         radial-gradient(circle at 83% 12%, rgb(223 141 45 / 20%), transparent 27%),
157         linear-gradient(120deg, #0d2926, #174f42);
158 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 159–165

```
159
160 .admin-import-hero .container-xl {
161     display: flex;
162     align-items: end;
163     justify-content: space-between;
164     gap: 3rem;
165 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 166–174

```
166
167 .admin-import-hero h1 {
168   margin: 0;
169   font-family: var(--serif);
170   font-size: clamp(2.8rem, 5vw, 5rem);
171   font-weight: 500;
172   letter-spacing: -0.045em;
173   line-height: 1;
174 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 175–181

```
175
176 .admin-import-hero div > p:last-child {
177   max-width: 660px;
178   margin: 1.4rem 0 0;
179   color: #bfd1cb;
180   line-height: 1.7;
181 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 182–195

```
182
183 .admin-import-principle {
184   display: flex;
185   max-width: 330px;
186   padding: 1.2rem 1.35rem;
187   align-items: center;
188   gap: 0.9rem;
189   color: #dbe7e3;
190   font-size: 0.82rem;
191   line-height: 1.5;
192   background: rgb(255 255 255 / 8%);
193   border: 1px solid rgb(255 255 255 / 13%);
194   border-radius: 12px;
195 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 196–200

```
196
197 .admin-import-principle i {
198   color: var(--saffron-light);
199   font-size: 1.5rem;
200 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 201–204

```
201
202 .admin-import-principle strong {
203   color: var(--white);
204 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 205–210

```
205
206 .admin-import-workspace {
207   position: relative;
208   margin-top: -2.3rem;
209   padding-bottom: 7rem;
210 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 211–222

```
211
212 .admin-mode-tabs {
213   display: grid;
214   max-width: 660px;
215   margin: 0 auto 2rem;
216   padding: 0.38rem;
217   grid-template-columns: 1fr 1fr;
218   background: var(--white);
219   border: 1px solid var(--line);
220   border-radius: 14px;
221   box-shadow: 0 16px 40px rgb(16 42 38 / 12%);
222 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 223–235

```
223
224 .admin-mode-tabs button {
225     display: flex;
226     padding: 0.85rem 1rem;
227     align-items: center;
228     justify-content: center;
229     gap: 0.8rem;
230     color: var(--muted);
231     text-align: left;
232     background: transparent;
233     border: 0;
234     border-radius: 10px;
235 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 236–240

```
236
237 .admin-mode-tabs button.is-active {
238     color: var(--white);
239     background: var(--ink);
240 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 241–244

```
241
242 .admin-mode-tabs i {
243     font-size: 1.25rem;
244 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 245–251

```
245
246 .admin-mode-tabs span {
247     display: flex;
248     flex-direction: column;
249     font-size: 0.88rem;
250     font-weight: 800;
251 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 252–258

```
252
253 .admin-mode-tabs small {
254   margin-top: 0.18rem;
255   font-size: 0.66rem;
256   font-weight: 500;
257   opacity: 0.72;
258 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 259–268

```
259
260 .admin-notice {
261   display: flex;
262   max-width: 900px;
263   margin: 0 auto 1.3rem;
264   padding: 1rem 1.2rem;
265   align-items: center;
266   gap: 0.75rem;
267   border-radius: 10px;
268 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 269–270

```
269
270 .admin-notice.success { color: #185c47; background: #dcefe7; border: 1px solid #b9ddcf; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 271–271

```
271 .admin-notice.error { color: #842f28; background: #f6e2dd; border: 1px solid #e9c0b7; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 272–272

```
272 .admin-notice span { flex: 1; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 273–273

```
273 .admin-notice a { font-size: 0.8rem; font-weight: 800; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 274–274

```
274 .admin-notice button { color: inherit; font-size: 1.4rem; background: transparent; border: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 275–284

```
275  
276 .admin-search-panel,  
277 .external-results,  
278 .catalogue-book-form {  
279     max-width: 1120px;  
280     margin: 0 auto 2rem;  
281     background: var(--white);  
282     border: 1px solid var(--line);  
283     border-radius: 18px;  
284 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 285–292

```
285  
286 .admin-search-panel {  
287     display: grid;  
288     padding: 2.3rem;  
289     grid-template-columns: 0.78fr 1.22fr;  
290     align-items: center;  
291     gap: 3rem;  
292 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 293–302

```
293
```



```

294 .admin-search-panel h2,
295 .external-results h2,
296 .catalogue-form-heading h1 {
297     margin: 0;
298     font-family: var(--serif);
299     font-size: 2rem;
300     font-weight: 550;
301     letter-spacing: -0.03em;
302 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 303–309

```

303
304 .admin-search-panel > div > p:last-child {
305     margin: 0.8rem 0 0;
306     color: var(--muted);
307     font-size: 0.83rem;
308     line-height: 1.65;
309 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 310–320

```

310
311 .admin-search-input {
312     display: flex;
313     min-height: 58px;
314     padding: 0.3rem 0.3rem 0.3rem 1rem;
315     align-items: center;
316     gap: 0.65rem;
317     background: #f5f1e9;
318     border: 1px solid var(--line);
319     border-radius: 12px;
320 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 321–322

```

321
322 .admin-search-input > i { color: var(--forest-light); }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 323–323

```
323 .admin-search-input input { min-width: 80px; flex: 1; background: transparent; border: 0; outline: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 324–324

```
324 .admin-search-input select { max-width: 145px; color: var(--ink); background: transparent; border: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 325–325

```
325 .admin-source-label { color: var(--ink); font-weight: 700; white-space: nowrap; padding: 0 .25rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 326–334

```
326 .admin-search-input button {  
327   align-self: stretch;  
328   min-width: 92px;  
329   color: var(--white);  
330   font-weight: 800;  
331   background: var(--forest);  
332   border: 0;  
333   border-radius: 9px;  
334 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 335–341

```
335  
336 .admin-search-panel form > small {  
337   display: block;  
338   margin-top: 0.7rem;  
339   color: var(--muted);  
340   font-size: 0.7rem;  
341 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 342–343

```
342
343 .external-results { padding: 2.3rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 344–344

```
344 .external-results-heading { display: flex; margin-bottom: 1.6rem; align-items: end; justify-content: space-between;
gap: 2rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 345–345

```
345 .external-results-heading > span { color: var(--muted); font-size: 0.78rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 346–346

```
346 .external-result-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 347–358

```
347
348 .external-book-card {
349   display: grid;
350   min-width: 0;
351   padding: 1.1rem;
352   grid-template-columns: 118px minmax(0, 1fr);
353   gap: 1.2rem;
354   background: #faf7f1;
355   border: 1px solid var(--line);
356   border-radius: 13px;
357   transition: border-color 160ms ease, box-shadow 160ms ease;
358 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 359–360

```
359
360 .external-book-card.is-selected { border-color: var(--forest); box-shadow: 0 0 0 2px rgb(23 79 66 / 10%); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 361–361

```
361 .external-book-card.is-imported { opacity: 0.66; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 362–362

```
362 .external-cover { height: 174px; background: var(--paper-deep); border-radius: 5px; overflow: hidden; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 363–363

```
363 .external-cover img { width: 100%; height: 100%; object-fit: cover; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 364–364

```
364 .external-cover > span { display: flex; height: 100%; align-items: center; flex-direction: column; justify-content: center; gap: 0.5rem; color: var(--forest); font-family: var(--serif); font-size: 2.5rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 365–365

```
365 .external-cover small { color: var(--muted); font-family: var(--sans); font-size: 0.6rem; text-transform: uppercase; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 366–366

```
366 .external-book-copy { min-width: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 367–367

```
367 .external-source { display: flex; justify-content: space-between; color: var(--forest-light); font-size: 0.58rem; font-weight: 850; letter-spacing: 0.1em; text-transform: uppercase; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 368–368

```
368 .external-source strong { color: var(--clay); font-size: inherit; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 369–369

```
369 .external-book-copy h3 { margin: 0.4rem 0 0; overflow: hidden; font-family: var(--serif); font-size: 1.25rem; text-overflow: ellipsis; white-space: nowrap; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 370–370

```
370 .external-book-copy > p { margin: 0.2rem 0 0.65rem; color: var(--muted); font-size: 0.74rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 371–371

```
371 .external-book-copy dl { display: grid; margin: 0; grid-template-columns: 1fr 1fr; gap: 0.45rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 372–372

```
372 .external-book-copy dl div { min-width: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 373–373

```
373 .external-book-copy dt { color: #8a948f; font-size: 0.55rem; font-weight: 800; letter-spacing: 0.07em; text-transform: uppercase; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 374–374

```
374 .external-book-copy dd { margin: 0.1rem 0 0; overflow: hidden; font-size: 0.67rem; font-weight: 700; text-overflow: ellipsis; white-space: nowrap; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 375–375

```
375 .external-book-copy button { display: flex; width: 100%; margin-top: 0.75rem; padding: 0.55rem 0; align-items: center; justify-content: space-between; color: var(--forest); font-size: 0.7rem; font-weight: 850; background: transparent; border: 0; border-top: 1px solid var(--line); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 376–376

```
376 .external-book-copy button:disabled { color: var(--clay); cursor: not-allowed; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 377–378

```
377
378 .admin-empty-results { padding: 3rem; color: var(--muted); text-align: center; background: #faf7f1; border-radius:
12px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 379–379

```
379 .admin-empty-results i { color: var(--saffron); font-size: 2rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 380–380

```
380 .admin-empty-results h3 { margin: 0.65rem 0 0.3rem; color: var(--ink); font-family: var(--serif); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 381–381

```
381 .admin-empty-results p { margin: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 382–383

```
382
383 .catalogue-book-form { overflow: hidden; scroll-margin-top: 96px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 384–384

```
384 .catalogue-form-heading { display: flex; min-height: 150px; padding: 2.3rem; align-items: center; justify-content:
space-between; color: var(--white); background: var(--ink); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 385–385

```
385 .catalogue-form-heading img { width: 78px; height: 110px; object-fit: cover; border-radius: 3px; box-shadow: -5px 8px 18px rgb(0 0 0 / 32%); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 386–386

```
386 .admin-form-section { padding: 2.4rem; border-bottom: 1px solid var(--line); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 387–387

```
387 .admin-owned-section { background: #fbf5e9; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 388–388

```
388 .admin-section-title { display: grid; margin-bottom: 1.7rem; grid-template-columns: 38px 1fr; gap: 0.8rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 389–389

```
389 .admin-section-title > span { color: var(--saffron); font-family: var(--serif); font-size: 0.85rem; font-style: italic; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 390–390

```
390 .admin-section-title h2 { margin: 0; font-family: var(--serif); font-size: 1.5rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

Lines 391–391

```
391 .admin-section-title p { margin: 0.3rem 0 0; color: var(--muted); font-size: 0.75rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 392–392

```
392 .admin-field-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 1.1rem 1.3rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 393–393

```
393 .admin-field-grid-three { grid-template-columns: repeat(3, 1fr); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 394–394

```
394 .admin-field-grid .span-two { grid-column: span 2; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 395–395

```
395 .admin-field-grid label { display: flex; min-width: 0; flex-direction: column; gap: 0.42rem; color: var(--ink-soft); font-size: 0.7rem; font-weight: 800; letter-spacing: 0.02em; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 396–398

```
396 .admin-field-grid input,  
397 .admin-field-grid select,
```

```
398 .admin-field-grid textarea { width: 100%; padding: 0.72rem 0.8rem; color: var(--ink); font-size: 0.84rem; font-weight: 500; background: var(--white); border: 1px solid var(--line); border-radius: 8px; outline: 0; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 399–400

```
399 .admin-field-grid input,  
400 .admin-field-grid select { min-height: 45px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 401–401

```
401 .admin-field-grid textarea { resize: vertical; line-height: 1.55; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 402–404

```
402 .admin-field-grid input:focus,  
403 .admin-field-grid select:focus,  
404 .admin-field-grid textarea:focus { border-color: var(--forest-light); box-shadow: 0 0 0 3px rgb(47 111 94 / 10%); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 405–405

```
405 .admin-field-grid label > small { color: var(--muted); font-size: 0.61rem; font-weight: 500; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 406–406

```
406 .admin-checks { display: flex; align-items: center; flex-direction: column; justify-content: center; gap: 0.55rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 407–407

```
407 .admin-checks label { width: 100%; align-items: center; flex-direction: row; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 408–408

```
408 .admin-checks input { width: 16px; min-height: 16px; accent-color: var(--forest); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 409–409

```
409 .admin-submit-bar { display: flex; padding: 1.4rem 2.3rem; align-items: center; justify-content: space-between; gap: 2rem; background: var(--white); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 410–410

```
410 .admin-submit-bar > div { display: flex; align-items: center; gap: 0.8rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 411–411

```
411 .admin-submit-bar > div > i { color: var(--forest); font-size: 1.5rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 412–412

```
412 .admin-submit-bar span { display: flex; flex-direction: column; font-size: 0.75rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 413–413

```
413 .admin-submit-bar small { margin-top: 0.1rem; color: var(--muted); font-size: 0.65rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 414–414

```
414 .admin-submit-bar button { min-width: 210px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 415–420

```
415  
416 @media (max-width: 991.98px) {  
417   .my-books-grid {  
418     grid-template-columns: 1fr;  
419   }  
420 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 421–425

```
421 .admin-import-hero .container-xl { align-items: flex-start; flex-direction: column; }  
422 .admin-import-principle { max-width: 520px; }  
423 .admin-search-panel { grid-template-columns: 1fr; gap: 1.5rem; }  
424 .external-result-grid { grid-template-columns: 1fr; }  
425 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 426–433

```
426
427 @media (max-width: 767.98px) {
428   .my-books-hero {
429     padding: 2rem;
430     grid-template-columns: 1fr;
431     gap: 1.5rem;
432   }
433
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 434–437

```
434   .my-books-toolbar {
435     overflow-x: auto;
436   }
437
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 438–442

```
438   .my-books-toolbar button,
439   .my-books-toolbar a {
440     flex: 0 0 auto;
441   }
442
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 443–457

```
443   .admin-import-hero { padding: 3.8rem 0 5.8rem; }
444   .admin-mode-tabs { margin-right: 0.5rem; margin-left: 0.5rem; }
445   .admin-search-panel,
446   .external-results,
447   .admin-form-section { padding: 1.4rem; }
448   .admin-search-input { align-items: stretch; flex-wrap: wrap; }
449   .admin-search-input > i { align-self: center; }
450   .admin-search-input input { min-width: calc(100% - 36px); }
451   .admin-search-input select { max-width: none; flex: 1; }
452   .admin-field-grid,
453   .admin-field-grid-three { grid-template-columns: 1fr; }
454   .admin-field-grid .span-two { grid-column: auto; }
```

```
455 .admin-submit-bar { align-items: stretch; flex-direction: column; }
456 .admin-submit-bar button { width: 100%; }
457 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 458–464

```
458
459 @media (max-width: 575.98px) {
460   .my-book-card {
461     grid-template-columns: 90px minmax(0, 1fr);
462     gap: 1rem;
463   }
464 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 465–470

```
465 .my-book-cover > img,
466 .my-book-fallback {
467   width: 90px;
468   height: 134px;
469 }
470
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 471–475

```
471 .order-detail-navigation {
472   align-items: flex-start;
473   flex-direction: column;
474 }
475
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 476–484

```
476 .admin-mode-tabs button { padding: 0.75rem 0.45rem; }
477 .admin-mode-tabs i { display: none; }
```

```
478 .external-book-card { grid-template-columns: 82px minmax(0, 1fr); }
479 .external-cover { height: 125px; }
480 .external-book-copy dl { grid-template-columns: 1fr; }
481 .external-book-copy dl div:nth-child(n + 3) { display: none; }
482 .catalogue-form-heading { padding: 1.5rem; }
483 .catalogue-form-heading img { width: 62px; height: 88px; }
484 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 485–488

```
485
486 .site-main {
487     overflow: hidden;
488 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 489–497

```
489
490 .eyebrow {
491     margin: 0 0 0.85rem;
492     color: var(--forest);
493     font-size: 0.72rem;
494     font-weight: 800;
495     letter-spacing: 0.18em;
496     text-transform: uppercase;
497 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 498–501

```
498
499 .eyebrow-light {
500     color: var(--saffron-light);
501 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 502–516

```
502
503 .btn-ink {
504     display: inline-flex;
505     min-height: 44px;
506     padding: 0.7rem 1.15rem;
```

```

507 align-items: center;
508 justify-content: center;
509 gap: 0.55rem;
510 color: var(--white);
511 font-weight: 700;
512 background: var(--ink);
513 border: 1px solid var(--ink);
514 border-radius: 999px;
515 transition: background 160ms ease, transform 160ms ease;
516 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 517–524

```

517
518 .btn-ink:hover,
519 .btn-ink:focus {
520   color: var(--white);
521   background: var(--forest-light);
522   border-color: var(--forest-light);
523   transform: translateY(-1px);
524 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 525–532

```

525
526 /* Navigation */
527 .akshara-navbar {
528   min-height: 76px;
529   background: rgb(255 253 250 / 92%);
530   border-bottom: 1px solid rgb(16 42 38 / 9%);
531   backdrop-filter: blur(16px);
532 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 533–539

```

533
534 .akshara-brand {
535   display: inline-flex;
536   padding: 0;
537   align-items: center;
538   gap: 0.7rem;
539 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 540–550

```
540
541 .akshara-brand-mark {
542     display: block;
543     width: 43px;
544     height: 43px;
545     flex: 0 0 43px;
546     background-repeat: no-repeat;
547     background-position: -57px -14px;
548     background-size: 168px 91.6px;
549     border-radius: 50%;
550 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 551–556

```
551
552 .akshara-brand-copy {
553     display: flex;
554     flex-direction: column;
555     line-height: 1;
556 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 557–562

```
557
558 .akshara-brand-copy strong {
559     font-family: var(--serif);
560     font-size: 1.2rem;
561     letter-spacing: 0.18em;
562 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 563–571

```
563
564 .akshara-brand-copy small {
565     margin-top: 0.38rem;
566     color: #64716d;
567     font-size: 0.52rem;
568     font-weight: 700;
569     letter-spacing: 0.14em;
```

```
570 text-transform: uppercase;
571 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 572–575

```
572
573 .akshara-nav-links {
574     gap: 0.2rem;
575 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 576–582

```
576
577 .akshara-nav-links .nav-link {
578     padding: 0.55rem 0.78rem !important;
579     color: #3f534e;
580     font-size: 0.9rem;
581     font-weight: 650;
582 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 583–586

```
583
584 .akshara-nav-links .nav-link:hover {
585     color: var(--forest);
586 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 587–599

```
587
588 .nav-icon-button {
589     display: grid;
590     width: 39px;
591     height: 39px;
592     padding: 0;
593     place-items: center;
594     color: var(--ink);
595     background: transparent;
596     border: 0;
```

```
597   border-radius: 50%;
598   transition: background 160ms ease;
599 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 600–603

```
600
601 .nav-icon-button:hover {
602   background: var(--paper-deep);
603 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 604–609

```
604
605 .nav-join-button {
606   min-height: 39px;
607   padding: 0.55rem 1.05rem;
608   font-size: 0.84rem;
609 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 610–614

```
610
611 .navbar-toggler {
612   border: 0;
613   box-shadow: none !important;
614 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 615–623

```
615
616 /* Hero */
617 .home-hero {
618   position: relative;
619   color: var(--white);
620   background:
621     linear-gradient(112deg, rgb(7 34 31 / 98%), rgb(20 76 63 / 96%)),
622     var(--ink);
623 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 624–634

```
624
625 .home-hero::before {
626   position: absolute;
627   inset: 0;
628   background-image:
629     linear-gradient(rgb(255 255 255 / 2%) 1px, transparent 1px),
630     linear-gradient(90deg, rgb(255 255 255 / 2%) 1px, transparent 1px);
631   background-size: 52px 52px;
632   content: "";
633   mask-image: linear-gradient(to right, black, transparent 82%);
634 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 635–641

```
635
636 .hero-orb {
637   position: absolute;
638   border-radius: 50%;
639   filter: blur(2px);
640   pointer-events: none;
641 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 642–649

```
642
643 .hero-orb-one {
644   width: 330px;
645   height: 330px;
646   top: -170px;
647   left: 46%;
648   background: rgb(223 141 45 / 13%);
649 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 650–657

```
650
651 .hero-orb-two {
652   width: 360px;
653   height: 360px;
654   right: -160px;
655   bottom: -180px;
656   background: rgb(62 132 110 / 24%);
```

```
657 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 658–669

```
658
659 .hero-layout {
660   position: relative;
661   z-index: 1;
662   display: grid;
663   min-height: 660px;
664   padding-top: 5.8rem;
665   padding-bottom: 5.8rem;
666   align-items: center;
667   grid-template-columns: minmax(0, 1.02fr) minmax(440px, 0.98fr);
668   gap: 4rem;
669 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 670–673

```
670
671 .hero-copy {
672   max-width: 690px;
673 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 674–681

```
674
675 .hero-eyebrow {
676   display: flex;
677   margin-bottom: 1.25rem;
678   align-items: center;
679   gap: 0.8rem;
680   color: #c9d8d3;
681 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 682–687

```
682
683 .hero-eyebrow span {
```

```
684 width: 34px;
685 height: 1px;
686 background: var(--saffron);
687 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 688–696

```
688
689 .hero-copy h1 {
690   margin: 0;
691   font-family: var(--serif);
692   font-size: clamp(3.35rem, 5.4vw, 5.65rem);
693   font-weight: 500;
694   letter-spacing: -0.045em;
695   line-height: 0.98;
696 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 697–702

```
697
698 .hero-copy h1 em {
699   position: relative;
700   color: var(--saffron-light);
701   font-style: italic;
702 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 703–712

```
703
704 .hero-copy h1 em::after {
705   position: absolute;
706   right: 0;
707   bottom: -0.18rem;
708   left: 0.1rem;
709   height: 7px;
710   background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 200 12'%3E%3Cpath d='M2
9c58-9 123-7 196-3' fill='none' stroke='%23df8d2d' stroke-width='3' stroke-linecap='round'/%3E%3C/svg%3E") center / 100%
100% no-repeat;
711   content: "";
712 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 713–720

```
713
714 .hero-lead {
715     max-width: 620px;
716     margin: 1.7rem 0 0;
717     color: #c8d7d2;
718     font-size: 1.1rem;
719     line-height: 1.75;
720 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 721–734

```
721
722 .hero-search {
723     display: flex;
724     max-width: 660px;
725     min-height: 64px;
726     margin-top: 2rem;
727     padding: 0.38rem 0.42rem 0.38rem 1.15rem;
728     align-items: center;
729     gap: 0.8rem;
730     color: var(--ink);
731     background: var(--white);
732     border-radius: 999px;
733     box-shadow: 0 22px 55px rgb(0 0 0 / 22%);
734 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 735–739

```
735
736 .hero-search > i {
737     color: var(--forest-light);
738     font-size: 1.05rem;
739 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 740–749

```
740
741 .hero-search input {
742     min-width: 0;
743     height: 48px;
744     flex: 1;
745     color: var(--ink);
746     background: transparent;
747     border: 0;
748     outline: 0;
749 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 750–753

```
750
751 .hero-search input::placeholder {
752   color: #82908c;
753 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 754–765

```
754
755 .hero-search button {
756   height: 52px;
757   padding: 0 1.55rem;
758   color: #fff;
759   font-weight: 750;
760   white-space: nowrap;
761   background: var(--saffron);
762   border: 0;
763   border-radius: 999px;
764   transition: background 160ms ease, transform 160ms ease;
765 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 766–770

```
766
767 .hero-search button:hover {
768   background: #c9771e;
769   transform: translateX(1px);
770 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 771–779

```
771
772 .hero-proof {
773   display: flex;
774   margin-top: 1.5rem;
775   flex-wrap: wrap;
776   gap: 1.25rem;
777   color: #aebfba;
778   font-size: 0.78rem;
779 }
```


How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 780–785

```
780
781 .hero-proof span {
782     display: inline-flex;
783     align-items: center;
784     gap: 0.45rem;
785 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 786–789

```
786
787 .hero-proof i {
788     color: var(--saffron-light);
789 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 790–795

```
790
791 .hero-visual {
792     position: relative;
793     min-height: 500px;
794     perspective: 1000px;
795 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 796–806

```
796
797 .hero-sun {
798     position: absolute;
799     width: 320px;
800     height: 320px;
801     top: 46px;
802     right: 60px;
803     background: linear-gradient(145deg, var(--saffron-light), #bb641d);
804     border-radius: 50%;
805     box-shadow: 0 0 0 34px rgb(255 255 255 / 3%);
806 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 807–820

```
807
808 .hero-book {
809     position: absolute;
810     display: flex;
811     width: 205px;
812     height: 300px;
813     padding: 1.55rem;
814     flex-direction: column;
815     justify-content: space-between;
816     border: 1px solid rgb(255 255 255 / 14%);
817     border-radius: 5px 10px 10px 5px;
818     box-shadow: -14px 18px 42px rgb(0 0 0 / 28%);
819     transform-origin: bottom center;
820 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 821–830

```
821
822 .hero-book::before {
823     position: absolute;
824     top: 0;
825     bottom: 0;
826     left: 10px;
827     width: 1px;
828     background: rgb(255 255 255 / 24%);
829     content: "";
830 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 831–838

```
831
832 .hero-book span,
833 .hero-book small {
834     font-size: 0.58rem;
835     font-weight: 750;
836     letter-spacing: 0.17em;
837     text-transform: uppercase;
838 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 839–845

```
839
840 .hero-book strong {
```

```
841 font-family: var(--serif);
842 font-size: 1.75rem;
843 font-weight: 500;
844 line-height: 1.05;
845 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 846–856

```
846
847 .hero-book-one {
848   z-index: 2;
849   top: 90px;
850   left: 24px;
851   color: #f7e4c0;
852   background:
853     radial-gradient(circle at 72% 30%, transparent 0 35px, rgb(255 255 255 / 18%) 36px 37px, transparent 38px),
854     linear-gradient(150deg, #153c64, #102947);
855   transform: rotate(-9deg) translateZ(10px);
856 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 857–867

```
857
858 .hero-book-two {
859   z-index: 3;
860   top: 50px;
861   left: 180px;
862   color: #fff4df;
863   background:
864     linear-gradient(140deg, transparent 62%, rgb(255 255 255 / 10%) 62%),
865     var(--clay);
866   transform: rotate(4deg) translateZ(32px);
867 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 868–878

```
868
869 .hero-book-three {
870   z-index: 1;
871   top: 126px;
872   right: 0;
873   color: #d8eee7;
874   background:
875     repeating-linear-gradient(45deg, transparent 0 17px, rgb(255 255 255 / 5%) 18px 19px),
876     #1a6658;
877   transform: rotate(10deg);
878 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 879–892

```
879
880 .hero-quote-card {
881   position: absolute;
882   z-index: 5;
883   width: 300px;
884   right: 20px;
885   bottom: 3px;
886   padding: 1.1rem 1.25rem;
887   color: var(--ink);
888   background: rgb(255 253 250 / 96%);
889   border-radius: 12px;
890   box-shadow: 0 18px 45px rgb(0 0 0 / 24%);
891   transform: rotate(-2deg);
892 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 893–897

```
893
894 .hero-quote-card i {
895   color: var(--saffron);
896   font-size: 1.25rem;
897 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 898–904

```
898
899 .hero-quote-card p {
900   margin: 0.35rem 0;
901   font-family: var(--serif);
902   font-size: 1rem;
903   line-height: 1.42;
904 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 905–912

```
905
906 .hero-quote-card span {
907   color: var(--muted);
908   font-size: 0.66rem;
909   font-weight: 700;
910   letter-spacing: 0.08em;
911   text-transform: uppercase;
912 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 913–929

```
913
914 .hero-seal {
915     position: absolute;
916     z-index: 6;
917     display: grid;
918     width: 88px;
919     height: 88px;
920     right: 42px;
921     top: 12px;
922     place-content: center;
923     color: var(--ink);
924     text-align: center;
925     background: var(--paper);
926     border: 1px solid rgb(16 42 38 / 18%);
927     border-radius: 50%;
928     box-shadow: 0 10px 30px rgb(0 0 0 / 18%);
929 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 930–937

```
930
931 .hero-seal::before {
932     position: absolute;
933     inset: 6px;
934     border: 1px dashed var(--saffron);
935     border-radius: 50%;
936     content: "";
937 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 938–943

```
938
939 .hero-seal span {
940     font-family: var(--serif);
941     font-size: 1.8rem;
942     line-height: 1;
943 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 944–950

```
944
945 .hero-seal small {
946   font-size: 0.52rem;
947   font-weight: 800;
948   letter-spacing: 0.12em;
949   text-transform: uppercase;
950 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 951–957

```
951
952 /* Category shelf */
953 .category-strip {
954   padding: 4.7rem 0 5.1rem;
955   background: var(--white);
956   border-bottom: 1px solid var(--line);
957 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 958–965

```
958
959 .category-strip-heading,
960 .section-heading {
961   display: flex;
962   align-items: end;
963   justify-content: space-between;
964   gap: 2rem;
965 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 966–976

```
966
967 .category-strip-heading h2,
968 .section-heading h2,
969 .edition-section h2 {
970   margin: 0;
971   font-family: var(--serif);
972   font-size: clamp(2rem, 3vw, 3.15rem);
973   font-weight: 500;
974   letter-spacing: -0.035em;
975   line-height: 1.08;
976 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 977–982

```
977
978 .category-strip-heading > span {
979     padding-bottom: 0.45rem;
980     color: var(--muted);
981     font-size: 0.85rem;
982 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 983–991

```
983
984 .category-list {
985     display: grid;
986     margin-top: 2.3rem;
987     grid-template-columns: repeat(6, minmax(0, 1fr));
988     border: 1px solid var(--line);
989     border-radius: 14px;
990     overflow: hidden;
991 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 992–1008

```
992
993 .category-list button {
994     display: flex;
995     min-height: 132px;
996     padding: 1.25rem 1rem;
997     align-items: center;
998     flex-direction: column;
999     justify-content: center;
1000     gap: 0.7rem;
1001     color: var(--ink);
1002     font-size: 0.83rem;
1003     font-weight: 750;
1004     background: var(--white);
1005     border: 0;
1006     border-right: 1px solid var(--line);
1007     transition: color 180ms ease, background 180ms ease;
1008 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1009–1012

```
1009
1010 .category-list button:last-child {
1011     border-right: 0;
1012 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1013–1023

```
1013
1014 .category-list button > span {
1015     display: grid;
1016     width: 40px;
1017     height: 40px;
1018     place-items: center;
1019     color: var(--forest);
1020     background: var(--paper);
1021     border-radius: 50%;
1022     transition: color 180ms ease, background 180ms ease, transform 180ms ease;
1023 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1024–1030

```
1024
1025 .category-list button > .bi-arrow-right {
1026     color: var(--saffron);
1027     opacity: 0;
1028     transform: translateX(-6px);
1029     transition: opacity 180ms ease, transform 180ms ease;
1030 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1031–1035

```
1031
1032 .category-list button:hover {
1033     color: var(--white);
1034     background: var(--forest);
1035 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1036–1041

```
1036
1037 .category-list button:hover > span {
1038     color: var(--white);
1039     background: rgb(255 255 255 / 12%);
1040     transform: translateY(-3px);
1041 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1042–1046

```
1042
1043 .category-list button:hover > .bi-arrow-right {
1044     opacity: 1;
1045     transform: translateX(0);
1046 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1047–1052

```
1047
1048 /* Catalogue */
1049 .catalogue-section {
1050     padding: 6.5rem 0 7rem;
1051     background: var(--paper);
1052 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1053–1056

```
1053
1054 .catalogue-heading {
1055     margin-bottom: 2.7rem;
1056 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1057–1063

```
1057
1058 .section-heading > div > p:last-child {
1059     max-width: 610px;
1060     margin: 0.8rem 0 0;
```

```
1061 color: var(--muted);
1062 line-height: 1.65;
1063 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1064–1069

```
1064
1065 .catalogue-tools {
1066   display: flex;
1067   align-items: center;
1068   gap: 0.85rem;
1069 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1070–1075

```
1070
1071 .result-count {
1072   color: var(--muted);
1073   font-size: 0.8rem;
1074   white-space: nowrap;
1075 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1076–1086

```
1076
1077 .clear-search-button {
1078   padding: 0.45rem 0.75rem;
1079   color: var(--forest);
1080   font-size: 0.76rem;
1081   font-weight: 750;
1082   background: transparent;
1083   border: 0;
1084   text-decoration: underline;
1085   text-underline-offset: 4px;
1086 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1087–1092

```
1087
1088 .sort-control {
1089   display: flex;
```

```
1090     align-items: center;
1091     gap: 0.6rem;
1092 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1093–1098

```
1093
1094 .sort-control span {
1095     color: var(--muted);
1096     font-size: 0.78rem;
1097     font-weight: 700;
1098 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1099–1109

```
1099
1100 .sort-control select {
1101     min-height: 42px;
1102     padding: 0 2.3rem 0 0.95rem;
1103     color: var(--ink);
1104     font-size: 0.82rem;
1105     font-weight: 700;
1106     background-color: var(--white);
1107     border: 1px solid var(--line);
1108     border-radius: 999px;
1109 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1110–1115

```
1110
1111 .book-grid {
1112     display: grid;
1113     grid-template-columns: repeat(4, minmax(0, 1fr));
1114     gap: 2.1rem 1.4rem;
1115 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1116–1119

```
1116
```

```
1117 .book-card {
1118   min-width: 0;
1119 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1120–1129

```
1120
1121 .book-card-cover-wrap {
1122   position: relative;
1123   height: 350px;
1124   padding: 1.7rem;
1125   background: #e9e2d6;
1126   border-radius: 12px;
1127   overflow: hidden;
1128   transition: transform 220ms ease, box-shadow 220ms ease;
1129 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1130–1134

```
1130
1131 .book-card:hover .book-card-cover-wrap {
1132   box-shadow: 0 20px 46px rgb(37 47 42 / 14%);
1133   transform: translateY(-5px);
1134 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1135–1140

```
1135
1136 .book-card-cover-link {
1137   display: block;
1138   width: 100%;
1139   height: 100%;
1140 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1141–1150

```
1141
1142 .book-card-cover,
1143 .fallback-book-cover {
1144   display: block;
1145   width: 100%;
```

```
1146 height: 100%;
1147 object-fit: cover;
1148 border-radius: 4px 8px 8px 4px;
1149 box-shadow: -7px 10px 22px rgb(21 31 28 / 24%);
1150 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1151–1160

```
1151
1152 .fallback-book-cover {
1153   position: relative;
1154   display: flex;
1155   padding: 1.35rem;
1156   flex-direction: column;
1157   color: #fff;
1158   background: var(--navy);
1159   overflow: hidden;
1160 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1161–1168

```
1161
1162 .fallback-book-cover::before,
1163 .fallback-book-cover::after {
1164   position: absolute;
1165   content: "";
1166   border: 1px solid rgb(255 255 255 / 25%);
1167   border-radius: 50%;
1168 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1169–1175

```
1169
1170 .fallback-book-cover::before {
1171   width: 140px;
1172   height: 140px;
1173   right: -62px;
1174   top: -35px;
1175 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1176–1182

```
1176
1177 .fallback-book-cover::after {
1178     width: 210px;
1179     height: 210px;
1180     right: -100px;
1181     top: -71px;
1182 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1183–1192

```
1183
1184 .fallback-book-cover strong {
1185     position: relative;
1186     z-index: 1;
1187     margin: auto 0 0.8rem;
1188     font-family: var(--serif);
1189     font-size: 3.2rem;
1190     font-weight: 500;
1191     letter-spacing: -0.08em;
1192 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1193–1200

```
1193
1194 .fallback-book-cover small {
1195     position: relative;
1196     z-index: 1;
1197     font-family: var(--serif);
1198     font-size: 0.95rem;
1199     line-height: 1.25;
1200 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1201–1209

```
1201
1202 .fallback-book-mark {
1203     position: relative;
1204     z-index: 1;
1205     align-self: flex-start;
1206     padding-bottom: 0.45rem;
1207     font-size: 1.25rem;
1208     border-bottom: 1px solid rgb(255 255 255 / 45%);
1209 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1210–1211

```
1210
1211 .cover-tone-navy { background-color: var(--navy); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1212–1212

```
1212 .cover-tone-forest { background-color: var(--forest); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1213–1213

```
1213 .cover-tone-saffron { color: var(--ink); background-color: var(--saffron); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1214–1214

```
1214 .cover-tone-clay { background-color: var(--clay); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1215–1215

```
1215 .cover-tone-plum { background-color: var(--plum); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1216–1216

```
1216 .cover-tone-teal { background-color: var(--teal); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1217–1233

```
1217
1218 .save-book-button {
1219     position: absolute;
1220     z-index: 2;
1221     display: grid;
1222     width: 38px;
1223     height: 38px;
1224     top: 0.8rem;
1225     right: 0.8rem;
1226     padding: 0;
1227     place-items: center;
1228     color: var(--ink);
1229     background: rgb(255 253 250 / 94%);
1230     border: 0;
1231     border-radius: 50%;
1232     box-shadow: 0 4px 14px rgb(16 42 38 / 12%);
1233 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1234–1237

```
1234
1235 .save-book-button.is-saved {
1236     color: #b13b34;
1237 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1238–1250

```
1238
1239 .availability-badge {
1240     display: inline-flex;
1241     align-items: center;
1242     gap: 0.35rem;
1243     color: var(--ink);
1244     font-size: 0.63rem;
1245     font-weight: 800;
1246     letter-spacing: 0.08em;
1247     text-transform: uppercase;
1248     background: var(--white);
1249     border-radius: 999px;
1250 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1251–1259

```
1251
1252 .book-card-cover-wrap > .availability-badge {
1253   position: absolute;
1254   z-index: 2;
1255   bottom: 0.8rem;
1256   left: 0.8rem;
1257   padding: 0.42rem 0.62rem;
1258   box-shadow: 0 5px 15px rgb(16 42 38 / 12%);
1259 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1260–1267

```
1260
1261 .availability-badge::before {
1262   width: 6px;
1263   height: 6px;
1264   background: #7b8a85;
1265   border-radius: 50%;
1266   content: "";
1267 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1268–1269

```
1268
1269 .availability-available::before { background: #2e8a62; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1270–1270

```
1270 .availability-unavailable::before { background: #b34b40; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1271–1271

```
1271 .availability-muted::before { background: #bd842b; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1272–1275

```
1272
1273 .book-card-content {
1274     padding: 1.05rem 0.15rem 0;
1275 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1276–1287

```
1276
1277 .book-card-meta {
1278     display: flex;
1279     min-height: 18px;
1280     justify-content: space-between;
1281     gap: 0.8rem;
1282     color: var(--forest-light);
1283     font-size: 0.63rem;
1284     font-weight: 800;
1285     letter-spacing: 0.09em;
1286     text-transform: uppercase;
1287 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1288–1301

```
1288
1289 .book-card-title {
1290     display: -webkit-box;
1291     margin-top: 0.45rem;
1292     overflow: hidden;
1293     color: var(--ink);
1294     font-family: var(--serif);
1295     font-size: 1.35rem;
1296     font-weight: 600;
1297     line-height: 1.18;
1298     text-decoration: none;
1299     -webkit-box-orient: vertical;
1300     -webkit-line-clamp: 2;
1301 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1302–1305

```
1302
1303 .book-card-title:hover {
1304     color: var(--forest-light);
1305 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1306–1311

```
1306
1307 .book-card-author {
1308     margin: 0.4rem 0 0;
1309     color: var(--muted);
1310     font-size: 0.78rem;
1311 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1312–1320

```
1312
1313 .book-card-rating {
1314     display: flex;
1315     margin-top: 0.8rem;
1316     align-items: center;
1317     gap: 0.35rem;
1318     color: var(--muted);
1319     font-size: 0.65rem;
1320 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1321–1324

```
1321
1322 .book-card-rating > span:first-child {
1323     color: var(--saffron);
1324 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1325–1329

```
1325
1326 .book-card-rating strong {
1327   color: var(--ink-soft);
1328   font-size: 0.66rem;
1329 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1330–1339

```
1330
1331 .book-card-footer {
1332   display: flex;
1333   min-height: 47px;
1334   margin-top: 0.8rem;
1335   padding-top: 0.75rem;
1336   align-items: center;
1337   justify-content: space-between;
1338   border-top: 1px solid var(--line);
1339 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1340–1344

```
1340
1341 .book-card-price {
1342   color: var(--ink);
1343   font-size: 0.92rem;
1344 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1345–1356

```
1345
1346 .book-card-arrow {
1347   display: grid;
1348   width: 34px;
1349   height: 34px;
1350   place-items: center;
1351   color: var(--forest);
1352   border: 1px solid var(--line);
1353   border-radius: 50%;
1354   text-decoration: none;
1355   transition: color 160ms ease, background 160ms ease, transform 160ms ease;
1356 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1357–1363

```
1357
1358 .book-card-arrow:hover {
1359     color: #fff;
1360     background: var(--forest);
1361     border-color: var(--forest);
1362     transform: rotate(7deg);
1363 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1364–1375

```
1364
1365 .catalogue-message {
1366     display: flex;
1367     max-width: 780px;
1368     margin: 1rem auto;
1369     padding: 2.5rem;
1370     align-items: flex-start;
1371     gap: 1.3rem;
1372     background: var(--white);
1373     border: 1px solid var(--line);
1374     border-radius: 14px;
1375 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1376–1387

```
1376
1377 .catalogue-message > span {
1378     display: grid;
1379     width: 52px;
1380     height: 52px;
1381     flex: 0 0 52px;
1382     place-items: center;
1383     color: var(--forest);
1384     font-size: 1.2rem;
1385     background: var(--paper-deep);
1386     border-radius: 50%;
1387 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1388–1394

```
1388
1389 .catalogue-message h1,
1390 .catalogue-message h3 {
1391   margin: 0;
1392   font-family: var(--serif);
1393   font-size: 1.55rem;
1394 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1395–1400

```
1395
1396 .catalogue-message p {
1397   margin: 0.55rem 0 1rem;
1398   color: var(--muted);
1399   line-height: 1.6;
1400 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1401–1411

```
1401
1402 .catalogue-message button,
1403 .catalogue-message a {
1404   padding: 0;
1405   color: var(--forest);
1406   font-weight: 750;
1407   background: transparent;
1408   border: 0;
1409   text-decoration: underline;
1410   text-underline-offset: 4px;
1411 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1412–1416

```
1412
1413 .catalogue-error > span {
1414   color: var(--clay);
1415   background: #f5e6df;
1416 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1417–1422

```
1417
1418 .book-card-skeleton {
1419     display: flex;
1420     flex-direction: column;
1421     gap: 0.7rem;
1422 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1423–1430

```
1423
1424 .book-card-skeleton span {
1425     display: block;
1426     background: linear-gradient(90deg, #e6ded1 25%, #f1eadd 50%, #e6ded1 75%);
1427     background-size: 200% 100%;
1428     border-radius: 7px;
1429     animation: shimmer 1.45s infinite linear;
1430 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1431–1432

```
1431
1432 .skeleton-cover { height: 350px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1433–1433

```
1433 .skeleton-line { width: 100%; height: 14px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1434–1434

```
1434 .skeleton-line-short { width: 35%; height: 9px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1435–1435

```
1435 .skeleton-line-medium { width: 62%; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1436–1439

```
1436
1437 @keyframes shimmer {
1438   to { background-position: -200% 0; }
1439 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1440–1448

```
1440
1441 /* Reader journey */
1442 .journey-section {
1443   padding: 7rem 0;
1444   color: var(--white);
1445   background:
1446     radial-gradient(circle at 92% 0, rgb(223 141 45 / 16%), transparent 28%),
1447     var(--ink);
1448 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1449–1455

```
1449
1450 .journey-intro {
1451   display: grid;
1452   grid-template-columns: minmax(200px, 0.7fr) minmax(400px, 1.25fr) minmax(260px, 0.75fr);
1453   gap: 3rem;
1454   align-items: start;
1455 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1456–1464

```
1456
1457 .journey-intro h2 {
1458   margin: -0.15rem 0 0;
1459   font-family: var(--serif);
1460   font-size: clamp(2.3rem, 4vw, 4.2rem);
1461   font-weight: 500;
```



```
1462 letter-spacing: -0.045em;
1463 line-height: 1.02;
1464 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1465–1471

```
1465
1466 .journey-intro > p:last-child {
1467   margin: 0;
1468   color: #afc2bc;
1469   font-size: 0.92rem;
1470   line-height: 1.75;
1471 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1472–1478

```
1472
1473 .journey-steps {
1474   display: grid;
1475   margin-top: 4.5rem;
1476   grid-template-columns: repeat(5, minmax(0, 1fr));
1477   border-top: 1px solid rgb(255 255 255 / 18%);
1478 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1479–1486

```
1479
1480 .journey-steps article {
1481   position: relative;
1482   min-height: 230px;
1483   padding: 1.7rem 1.35rem 1rem 0;
1484   border-right: 1px solid rgb(255 255 255 / 14%);
1485   transition: background 180ms ease, transform 180ms ease;
1486 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1487–1488

```
1487
1488 .journey-steps article:hover { background: rgb(255 255 255 / 4%); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1489–1492

```
1489
1490 .journey-steps article:not(:first-child) {
1491     padding-left: 1.35rem;
1492 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1493–1496

```
1493
1494 .journey-steps article:last-child {
1495     border-right: 0;
1496 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1497–1508

```
1497
1498 .journey-steps article::before {
1499     position: absolute;
1500     width: 9px;
1501     height: 9px;
1502     top: -5px;
1503     left: -1px;
1504     background: var(--saffron);
1505     border-radius: 50%;
1506     content: "";
1507     box-shadow: 0 0 6px rgb(223 141 45 / 12%);
1508 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1509–1512

```
1509
1510 .journey-steps article:not(:first-child)::before {
1511     left: 1.35rem;
1512 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1513–1519

```
1513
1514 .journey-steps span {
1515     color: var(--saffron-light);
1516     font-family: var(--serif);
1517     font-size: 0.78rem;
1518     font-style: italic;
1519 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1520–1526

```
1520
1521 .journey-steps h3 {
1522     margin: 2.3rem 0 0.65rem;
1523     font-family: var(--serif);
1524     font-size: 1.65rem;
1525     font-weight: 500;
1526 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1527–1533

```
1527
1528 .journey-steps p {
1529     margin: 0;
1530     color: #aebfba;
1531     font-size: 0.78rem;
1532     line-height: 1.6;
1533 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1534–1544

```
1534
1535 .journey-step-link {
1536     display: inline-flex;
1537     align-items: center;
1538     gap: .45rem;
1539     margin-top: 1.15rem;
1540     color: var(--saffron-light);
1541     font-size: .78rem;
1542     font-weight: 700;
1543     text-decoration: none;
1544 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1545–1546

```
1545
1546 .journey-step-link i { transition: transform 180ms ease; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1547–1547

```
1547 .journey-step-link:hover { color: #ffd18b; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1548–1548

```
1548 .journey-step-link:hover i { transform: translateX(4px); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1549–1554

```
1549
1550 /* Community */
1551 .community-section {
1552     padding: 7rem 0 7.5rem;
1553     background: var(--white);
1554 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1555–1558

```
1555
1556 .community-heading {
1557     align-items: center;
1558 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1559–1562

```
1559
1560 .community-heading .btn {
1561     white-space: nowrap;
1562 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1563–1569

```
1563
1564 .community-grid {
1565     display: grid;
1566     margin-top: 3rem;
1567     grid-template-columns: repeat(3, minmax(0, 1fr));
1568     gap: 1.25rem;
1569 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1570–1579

```
1570
1571 .community-card {
1572     position: relative;
1573     min-height: 260px;
1574     padding: 1.6rem;
1575     background: var(--paper);
1576     border: 1px solid var(--line);
1577     border-radius: 14px;
1578     transition: transform 180ms ease, box-shadow 180ms ease;
1579 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1580–1584

```
1580
1581 .community-card:hover {
1582     box-shadow: 0 18px 40px rgb(16 42 38 / 9%);
1583     transform: translateY(-4px);
1584 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1585–1590

```
1585
1586 .community-card-top {
1587     display: flex;
1588     align-items: center;
1589     gap: 0.75rem;
1590 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1591–1603

```
1591
1592 .reader-avatar {
1593     display: grid;
1594     width: 42px;
1595     height: 42px;
1596     flex: 0 0 42px;
1597     place-items: center;
1598     color: var(--white);
1599     font-size: 0.68rem;
1600     font-weight: 800;
1601     background: var(--forest);
1602     border-radius: 50%;
1603 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1604–1605

```
1604
1605 .community-card:nth-child(2) .reader-avatar { background: var(--clay); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1606–1606

```
1606 .community-card:nth-child(3) .reader-avatar { background: var(--navy); }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1607–1613

```
1607
1608 .community-card-top > div {
1609     display: flex;
```

```
1610 min-width: 0;
1611 flex: 1;
1612 flex-direction: column;
1613 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1614–1617

```
1614
1615 .community-card-top strong {
1616   font-size: 0.82rem;
1617 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1618–1622

```
1618
1619 .community-card-top small {
1620   color: var(--muted);
1621   font-size: 0.68rem;
1622 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1623–1626

```
1623
1624 .community-card-top > i {
1625   color: var(--muted);
1626 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1627–1634

```
1627
1628 .community-card h3 {
1629   margin: 2rem 0 0.75rem;
1630   font-family: var(--serif);
1631   font-size: 1.55rem;
1632   font-weight: 500;
1633   line-height: 1.2;
1634 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1635–1640

```
1635
1636 .community-card > p {
1637   margin: 0;
1638   color: var(--muted);
1639   font-size: 0.72rem;
1640 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1641–1655

```
1641
1642 .community-card > button {
1643   position: absolute;
1644   display: grid;
1645   width: 38px;
1646   height: 38px;
1647   right: 1.3rem;
1648   bottom: 1.3rem;
1649   padding: 0;
1650   place-items: center;
1651   color: var(--forest);
1652   background: transparent;
1653   border: 1px solid var(--line);
1654   border-radius: 50%;
1655 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1656–1662

```
1656
1657 /* Book details */
1658 .detail-page {
1659   min-height: 70vh;
1660   padding: 2.3rem 0 7rem;
1661   background: var(--paper);
1662 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1663–1671

```
1663
1664 .detail-breadcrumb {
1665   display: flex;
1666   margin-bottom: 2.5rem;
1667   align-items: center;
1668   gap: 0.55rem;
```



```
1669 color: var(--muted);
1670 font-size: 0.75rem;
1671 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1672–1676

```
1672
1673 .detail-breadcrumb a {
1674   color: var(--forest);
1675   text-decoration: none;
1676 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1677–1683

```
1677
1678 .detail-breadcrumb span {
1679   max-width: 380px;
1680   overflow: hidden;
1681   text-overflow: ellipsis;
1682   white-space: nowrap;
1683 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1684–1689

```
1684
1685 .detail-layout {
1686   display: grid;
1687   grid-template-columns: minmax(280px, 390px) minmax(0, 1fr);
1688   gap: clamp(3rem, 7vw, 7rem);
1689 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1690–1693

```
1690
1691 .detail-cover-column {
1692   min-width: 0;
1693 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1694–1700

```
1694
1695 .detail-cover-frame {
1696     height: 520px;
1697     padding: 2.6rem;
1698     background: #e9e2d6;
1699     border-radius: 18px;
1700 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1701–1709

```
1701
1702 .detail-cover-frame > img,
1703 .detail-cover-frame > .fallback-book-cover {
1704     width: 100%;
1705     height: 100%;
1706     object-fit: cover;
1707     border-radius: 5px 10px 10px 5px;
1708     box-shadow: -10px 16px 34px rgb(16 42 38 / 24%);
1709 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1710–1713

```
1710
1711 .detail-fallback-cover {
1712     padding: 2rem;
1713 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1714–1717

```
1714
1715 .detail-fallback-cover strong {
1716     font-size: 4.5rem;
1717 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1718–1721

```
1718
1719 .detail-fallback-cover small {
1720     font-size: 1.2rem;
1721 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1722–1728

```
1722
1723 .detail-cover-note {
1724     margin: 1rem 0 0;
1725     color: var(--muted);
1726     font-size: 0.72rem;
1727     text-align: center;
1728 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1729–1732

```
1729
1730 .detail-cover-note i {
1731     color: var(--forest-light);
1732 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1733–1736

```
1733
1734 .detail-content {
1735     padding-top: 1.3rem;
1736 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1737–1743

```
1737
1738 .detail-kicker-row {
1739     display: flex;
1740     align-items: center;
1741     flex-wrap: wrap;
1742     gap: 0.7rem;
1743 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1744–1748

```
1744
1745 .detail-kicker-row .availability-badge {
1746     padding: 0.45rem 0.7rem;
1747     border: 1px solid var(--line);
1748 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1749–1756

```
1749
1750 .detail-kicker-row > span:not(.availability-badge) {
1751     color: var(--muted);
1752     font-size: 0.66rem;
1753     font-weight: 800;
1754     letter-spacing: 0.1em;
1755     text-transform: uppercase;
1756 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1757–1766

```
1757
1758 .detail-content h1 {
1759     max-width: 760px;
1760     margin: 1.3rem 0 0;
1761     font-family: var(--serif);
1762     font-size: clamp(2.8rem, 5vw, 5rem);
1763     font-weight: 500;
1764     letter-spacing: -0.048em;
1765     line-height: 0.99;
1766 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1767–1775

```
1767
1768 .detail-subtitle {
1769     max-width: 650px;
1770     margin: 0.8rem 0 0;
```

```
1771 color: var(--muted);
1772 font-family: var(--serif);
1773 font-size: 1.35rem;
1774 font-style: italic;
1775 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1776–1781

```
1776
1777 .detail-author {
1778   margin: 1rem 0 0;
1779   color: var(--muted);
1780   font-size: 0.95rem;
1781 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1782–1785

```
1782
1783 .detail-author strong {
1784   color: var(--forest);
1785 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1786–1794

```
1786
1787 .detail-rating-row {
1788   display: flex;
1789   margin-top: 1.2rem;
1790   align-items: center;
1791   gap: 0.65rem;
1792   color: var(--muted);
1793   font-size: 0.72rem;
1794 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1795–1800

```
1795
1796 .detail-rating-row .stars {
1797   color: var(--saffron);
1798   font-size: 0.8rem;
1799   letter-spacing: 0.08em;
```

```
1800 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1801–1804

```
1801
1802 .detail-rating-row strong {
1803   color: var(--ink);
1804 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1805–1811

```
1805
1806 .detail-price-row {
1807   display: flex;
1808   margin-top: 2rem;
1809   align-items: baseline;
1810   gap: 0.9rem;
1811 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1812–1817

```
1812
1813 .detail-price-row strong {
1814   font-family: var(--serif);
1815   font-size: 2rem;
1816   font-weight: 600;
1817 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1818–1822

```
1818
1819 .detail-price-row span {
1820   color: var(--muted);
1821   font-size: 0.68rem;
```

```
1822 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 1823–1828

```
1823
1824 .detail-actions {
1825     display: flex;
1826     margin-top: 1.4rem;
1827     gap: 0.8rem;
1828 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1829–1833

```
1829
1830 .detail-cart-button {
1831     min-width: 190px;
1832     border-radius: 7px;
1833 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1834–1838

```
1834
1835 .detail-cart-button.is-added {
1836     background: var(--forest-light);
1837     border-color: var(--forest-light);
1838 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1839–1845

```
1839
1840 .detail-cart-button:disabled {
1841     color: #73817d;
1842     background: #d5d8d3;
1843     border-color: #d5d8d3;
1844     transform: none;
```

```
1845 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1846–1856

```
1846
1847 .detail-save-button {
1848     min-height: 44px;
1849     padding: 0.7rem 1.1rem;
1850     color: var(--ink);
1851     font-size: 0.82rem;
1852     font-weight: 750;
1853     background: transparent;
1854     border: 1px solid #bcb4a8;
1855     border-radius: 7px;
1856 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1857–1860

```
1857
1858 .detail-save-button i {
1859     margin-right: 0.45rem;
1860 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1861–1866

```
1861
1862 .detail-save-button.is-saved {
1863     color: #a33b34;
1864     background: #f7e8e4;
1865     border-color: #dcb7af;
1866 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 1867–1873

```
1867
1868 .detail-description {
1869     max-width: 730px;
1870     margin-top: 2.3rem;
1871     padding-top: 2rem;
```



```
1872 border-top: 1px solid var(--line);
1873 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1874–1880

```
1874
1875 .detail-description > p:last-child {
1876     margin: 0;
1877     color: #4f615c;
1878     line-height: 1.8;
1879     white-space: pre-line;
1880 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1881–1887

```
1881
1882 .detail-categories {
1883     display: flex;
1884     margin-top: 1.4rem;
1885     flex-wrap: wrap;
1886     gap: 0.55rem;
1887 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1888–1896

```
1888
1889 .detail-categories span {
1890     padding: 0.48rem 0.8rem;
1891     color: var(--forest);
1892     font-size: 0.7rem;
1893     font-weight: 750;
1894     background: #e4ece6;
1895     border-radius: 999px;
1896 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1897–1902

```
1897
1898 .edition-section {
1899     margin-top: 6.5rem;
1900     padding-top: 4rem;
```

```
1901   border-top: 1px solid var(--line);
1902 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1903–1909

```
1903
1904 .edition-grid {
1905     display: grid;
1906     margin-top: 2rem;
1907     grid-template-columns: repeat(2, minmax(0, 1fr));
1908     gap: 1rem;
1909 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1910–1917

```
1910
1911 .edition-card {
1912     display: block;
1913     padding: 1.4rem;
1914     background: var(--white);
1915     border: 1px solid var(--line);
1916     border-radius: 12px;
1917 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1918–1925

```
1918
1919 .edition-radio {
1920     position: absolute;
1921     width: 1px;
1922     height: 1px;
1923     overflow: hidden;
1924     opacity: 0;
1925 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1926–1931

```
1926
1927 .edition-card > div:first-child {
1928     display: flex;
1929     align-items: center;
```

```
1930     gap: 0.8rem;
1931 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1932–1942

```
1932
1933 .edition-icon {
1934     display: grid;
1935     width: 42px;
1936     height: 42px;
1937     flex: 0 0 42px;
1938     place-items: center;
1939     color: var(--forest);
1940     background: var(--paper-deep);
1941     border-radius: 50%;
1942 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1943–1948

```
1943
1944 .edition-card h3 {
1945     margin: 0;
1946     font-family: var(--serif);
1947     font-size: 1.2rem;
1948 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1949–1954

```
1949
1950 .edition-card > div p {
1951     margin: 0.2rem 0 0;
1952     color: var(--muted);
1953     font-size: 0.72rem;
1954 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1955–1961

```
1955
1956 .edition-card dl {
```

```
1957   display: grid;
1958   margin: 1.3rem 0 0;
1959   grid-template-columns: repeat(2, minmax(0, 1fr));
1960   gap: 0.8rem;
1961 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1962–1966

```
1962
1963 .edition-card dl div {
1964   padding-top: 0.7rem;
1965   border-top: 1px solid var(--line);
1966 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1967–1974

```
1967
1968 .edition-card dt {
1969   color: var(--muted);
1970   font-size: 0.6rem;
1971   font-weight: 800;
1972   letter-spacing: 0.1em;
1973   text-transform: uppercase;
1974 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1975–1981

```
1975
1976 .edition-card dd {
1977   margin: 0.25rem 0 0;
1978   overflow-wrap: anywhere;
1979   font-size: 0.78rem;
1980   font-weight: 700;
1981 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1982–1986

```
1982
1983 .edition-empty {
1984   margin-top: 1.5rem;
1985   color: var(--muted);
```

```
1986 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 1987–1992

```
1987
1988 .detail-loading {
1989     display: grid;
1990     grid-template-columns: minmax(280px, 390px) minmax(0, 1fr);
1991     gap: 6rem;
1992 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 1993–1999

```
1993
1994 .detail-cover-skeleton,
1995 .detail-copy-skeleton span {
1996     background: linear-gradient(90deg, #e6ded1 25%, #f1eadd 50%, #e6ded1 75%);
1997     background-size: 200% 100%;
1998     animation: shimmer 1.45s infinite linear;
1999 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2000–2004

```
2000
2001 .detail-cover-skeleton {
2002     height: 520px;
2003     border-radius: 18px;
2004 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2005–2011

```
2005
2006 .detail-copy-skeleton {
2007     display: flex;
2008     padding-top: 4rem;
2009     flex-direction: column;
2010     gap: 1rem;
2011 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2012–2017

```
2012
2013 .detail-copy-skeleton span {
2014     display: block;
2015     height: 18px;
2016     border-radius: 8px;
2017 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2018–2019

```
2018
2019 .detail-copy-skeleton span:nth-child(1) { width: 22%; height: 10px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2020–2020

```
2020 .detail-copy-skeleton span:nth-child(2) { width: 82%; height: 72px; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2021–2021

```
2021 .detail-copy-skeleton span:nth-child(3) { width: 42%; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2022–2022

```
2022 .detail-copy-skeleton span:nth-child(4) { width: 66%; height: 120px; margin-top: 2rem; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2023–2030

```
2023
```

```
2024 /* Footer */
2025 .site-footer {
2026     padding: 5rem 0 1.6rem;
2027     color: #c6d4cf;
2028     background: #0a211e;
2029     border-top: 1px solid rgb(255 255 255 / 8%);
2030 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2031–2036

```
2031
2032 .footer-grid {
2033     display: grid;
2034     grid-template-columns: 2fr 0.8fr 0.95fr 1.05fr;
2035     gap: 3.5rem;
2036 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2037–2045

```
2037
2038 .footer-wordmark {
2039     color: var(--white);
2040     font-family: var(--serif);
2041     font-size: 1.5rem;
2042     font-weight: 600;
2043     letter-spacing: 0.2em;
2044     text-decoration: none;
2045 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2046–2053

```
2046
2047 .footer-description {
2048     max-width: 390px;
2049     margin: 1.15rem 0 0;
2050     color: #91a7a0;
2051     font-size: 0.85rem;
2052     line-height: 1.75;
2053 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2054–2062

```
2054
2055 .footer-heading {
2056     margin: 0 0 1rem;
2057     color: var(--saffron-light);
2058     font-size: 0.67rem;
2059     font-weight: 800;
2060     letter-spacing: 0.14em;
2061     text-transform: uppercase;
2062 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2063–2068

```
2063
2064 .footer-grid > div:not(:first-child) {
2065     display: flex;
2066     flex-direction: column;
2067     align-items: flex-start;
2068 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2069–2076

```
2069
2070 .footer-grid a:not(.footer-wordmark),
2071 .footer-grid span:not(.footer-letter) {
2072     margin-bottom: 0.58rem;
2073     color: #a9bab5;
2074     font-size: 0.78rem;
2075     text-decoration: none;
2076 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2077–2080

```
2077
2078 .footer-grid a:hover {
2079     color: var(--white);
2080 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2081–2085

```
2081
2082 .footer-note {
2083   padding-left: 2rem;
2084   border-left: 1px solid rgb(255 255 255 / 12%);
2085 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2086–2092

```
2086
2087 .footer-letter {
2088   color: var(--saffron-light);
2089   font-family: var(--serif);
2090   font-size: 2.7rem;
2091   line-height: 1;
2092 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2093–2101

```
2093
2094 .footer-note p {
2095   margin: 0.7rem 0 0;
2096   color: #91a7a0;
2097   font-family: var(--serif);
2098   font-size: 1rem;
2099   font-style: italic;
2100   line-height: 1.5;
2101 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2102–2113

```
2102
2103 .footer-bottom {
2104   display: flex;
2105   margin-top: 4rem;
2106   padding-top: 1.3rem;
2107   justify-content: space-between;
2108   color: #718a82;
2109   font-size: 0.65rem;
2110   letter-spacing: 0.09em;
2111   text-transform: uppercase;
2112   border-top: 1px solid rgb(255 255 255 / 9%);
2113 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2114–2121

```
2114
2115 /* Responsive */
2116 @media (max-width: 1199.98px) {
2117     .hero-layout {
2118         grid-template-columns: minmax(0, 1fr) minmax(380px, 0.85fr);
2119         gap: 2rem;
2120     }
2121 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2122–2126

```
2122 .hero-book {
2123     width: 180px;
2124     height: 275px;
2125 }
2126
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2127–2130

```
2127 .hero-book-two { left: 145px; }
2128 .hero-book-three { right: -18px; }
2129 .hero-quote-card { right: 0; }
2130
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2131–2134

```
2131 .category-list {
2132     grid-template-columns: repeat(3, minmax(0, 1fr));
2133 }
2134
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2135–2137

```
2135 .category-list button:nth-child(3) { border-right: 0; }
2136 .category-list button:nth-child(-n + 3) { border-bottom: 1px solid var(--line); }
2137
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2138–2141

```
2138 .book-grid {
2139     grid-template-columns: repeat(3, minmax(0, 1fr));
2140 }
2141
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2142–2146

```
2142 .journey-intro {
2143     grid-template-columns: 0.45fr 1.15fr 0.75fr;
2144     gap: 2rem;
2145 }
2146 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2147–2152

```
2147
2148 @media (max-width: 991.98px) {
2149     .akshara-navbar {
2150         min-height: 68px;
2151     }
2152 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.

Lines 2153–2158

```
2153 .navbar-collapse {
2154     margin-top: 0.8rem;
2155     padding: 0.8rem 0 1rem;
2156     border-top: 1px solid var(--line);
2157 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2159–2162

```
2159 .akshara-nav-links {  
2160     align-items: stretch !important;  
2161 }  
2162
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2163–2166

```
2163 .nav-icon-button {  
2164     display: none;  
2165 }  
2166
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2167–2171

```
2167 .nav-join-button {  
2168     width: 100%;  
2169     margin-top: 0.5rem;  
2170 }  
2171
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2172–2178

```
2172 .hero-layout {  
2173     min-height: auto;  
2174     padding-top: 5.5rem;  
2175     padding-bottom: 3rem;  
2176     grid-template-columns: 1fr;  
2177 }  
2178
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2179–2182

```
2179  .hero-copy {
2180      max-width: 760px;
2181  }
2182
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2183–2188

```
2183  .hero-visual {
2184      width: min(100%, 610px);
2185      min-height: 460px;
2186      margin: 0 auto;
2187  }
2188
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2189–2193

```
2189  .hero-book {
2190      width: 205px;
2191      height: 300px;
2192  }
2193
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2194–2197

```
2194  .hero-book-one { left: 30px; }
2195  .hero-book-two { left: 195px; }
2196  .hero-book-three { right: 0; }
2197
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2198–2202

```
2198  .journey-intro {
2199      grid-template-columns: 1fr;
2200      gap: 1.2rem;
2201  }
2202
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2203–2206

```
2203 .journey-intro .eyebrow {  
2204     margin-bottom: 0;  
2205 }  
2206
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2207–2210

```
2207 .journey-intro > p:last-child {  
2208     max-width: 650px;  
2209 }  
2210
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2211–2217

```
2211 .journey-steps {  
2212     grid-template-columns: repeat(3, minmax(0, 1fr));  
2213     border-top: 0;  
2214     gap: 1px;  
2215     background: rgb(255 255 255 / 14%);  
2216 }  
2217
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2218–2225

```
2218 .journey-steps article,  
2219 .journey-steps article:not(:first-child) {  
2220     min-height: 205px;  
2221     padding: 1.5rem;  
2222     background: var(--ink);  
2223     border: 0;  
2224 }  
2225
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2226–2230

```
2226 .journey-steps article::before,  
2227 .journey-steps article:not(:first-child)::before {  
2228     display: none;  
2229 }  
2230
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2231–2234

```
2231 .journey-steps h3 {  
2232     margin-top: 1.8rem;  
2233 }  
2234
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2235–2238

```
2235 .community-grid {  
2236     grid-template-columns: 1fr 1fr;  
2237 }  
2238
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2239–2242

```
2239 .community-card:last-child {  
2240     grid-column: 1 / -1;  
2241 }  
2242
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2243–2247

```
2243 .detail-layout {  
2244     grid-template-columns: minmax(250px, 330px) minmax(0, 1fr);  
2245     gap: 3rem;  
2246 }  
2247
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2248–2252

```
2248 .detail-cover-frame {
2249     height: 470px;
2250     padding: 2rem;
2251 }
2252
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2253–2257

```
2253 .detail-loading {
2254     grid-template-columns: 330px minmax(0, 1fr);
2255     gap: 3rem;
2256 }
2257
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2258–2261

```
2258 .footer-grid {
2259     grid-template-columns: 1.6fr 0.8fr 1fr;
2260 }
2261
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2262–2265

```
2262 .footer-note {
2263     display: none !important;
2264 }
2265 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2266–2275

```
2266
```



```
2267 @media (max-width: 767.98px) {
2268   .akshara-brand-mark {
2269     width: 39px;
2270     height: 39px;
2271     flex-basis: 39px;
2272     background-position: -52px -13px;
2273     background-size: 153px 83.5px;
2274   }
2275 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2276–2279

```
2276 .hero-layout {
2277   padding-top: 4rem;
2278 }
2279
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2280–2283

```
2280 .hero-copy h1 {
2281   font-size: clamp(3rem, 14vw, 4.4rem);
2282 }
2283
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2284–2287

```
2284 .hero-lead {
2285   font-size: 1rem;
2286 }
2287
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2288–2291

```
2288 .hero-search {
2289   min-height: 58px;
2290 }
2291
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2292–2295

```
2292 .hero-search input {  
2293   height: 44px;  
2294 }  
2295
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2296–2301

```
2296 .hero-search button {  
2297   height: 46px;  
2298   padding: 0 1rem;  
2299   font-size: 0.8rem;  
2300 }  
2301
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2302–2305

```
2302 .hero-proof {  
2303   gap: 0.8rem 1.1rem;  
2304 }  
2305
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2306–2312

```
2306 .hero-visual {  
2307   min-height: 410px;  
2308   transform: scale(0.88);  
2309   transform-origin: center top;  
2310   margin-bottom: -3.5rem;  
2311 }  
2312
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2313–2319

```
2313 .category-strip-heading,  
2314 .section-heading {  
2315     align-items: flex-start;  
2316     flex-direction: column;  
2317     gap: 1rem;  
2318 }  
2319
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2320–2323

```
2320 .category-strip-heading > span {  
2321     padding: 0;  
2322 }  
2323
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2324–2327

```
2324 .category-list {  
2325     grid-template-columns: repeat(2, minmax(0, 1fr));  
2326 }  
2327
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2328–2334

```
2328 .category-list button,  
2329 .category-list button:nth-child(3) {  
2330     min-height: 120px;  
2331     border-right: 1px solid var(--line);  
2332     border-bottom: 1px solid var(--line);  
2333 }  
2334
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2335–2337

```
2335 .category-list button:nth-child(even) { border-right: 0; }  
2336 .category-list button:nth-last-child(-n + 2) { border-bottom: 0; }  
2337
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2338–2344

```
2338 .catalogue-section,  
2339 .journey-section,  
2340 .community-section {  
2341     padding-top: 5rem;  
2342     padding-bottom: 5rem;  
2343 }  
2344
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2345–2349

```
2345 .catalogue-tools {  
2346     width: 100%;  
2347     flex-wrap: wrap;  
2348 }  
2349
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2350–2353

```
2350 .sort-control {  
2351     margin-left: auto;  
2352 }  
2353
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2354–2357

```
2354 .book-grid {  
2355     grid-template-columns: repeat(2, minmax(0, 1fr));  
2356 }  
2357
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2358–2361

```
2358 .book-card-cover-wrap {  
2359     height: 330px;  
2360 }  
2361
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2362–2365

```
2362 .journey-steps {  
2363     grid-template-columns: 1fr 1fr;  
2364 }  
2365
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2366–2369

```
2366 .community-heading .btn {  
2367     align-self: flex-start;  
2368 }  
2369
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2370–2374

```
2370 .detail-layout,  
2371 .detail-loading {  
2372     grid-template-columns: 1fr;  
2373 }  
2374
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2375–2380

```
2375 .detail-cover-column,  
2376 .detail-cover-skeleton {  
2377     width: min(100%, 390px);  
2378     margin: 0 auto;  
2379 }  
2380
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2381–2385

```
2381 .detail-cover-frame,  
2382 .detail-cover-skeleton {  
2383     height: 520px;  
2384 }  
2385
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2386–2389

```
2386 .detail-content h1 {  
2387     font-size: clamp(2.8rem, 12vw, 4.2rem);  
2388 }  
2389
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2390–2393

```
2390 .edition-grid {  
2391     grid-template-columns: 1fr;  
2392 }  
2393
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2394–2398

```
2394 .footer-grid {  
2395     grid-template-columns: 1.5fr 1fr;  
2396     gap: 2.5rem;  
2397 }  
2398
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2399–2402

```
2399 .footer-grid > div:nth-child(3) {
```

```
2400     display: none;
2401   }
2402 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2403–2408

```
2403
2404 @media (max-width: 575.98px) {
2405   .container-xl {
2406     --bs-gutter-x: 2rem;
2407   }
2408 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.

Lines 2409–2412

```
2409   .akshara-brand-copy small {
2410     display: none;
2411   }
2412 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2413–2416

```
2413   .hero-layout {
2414     padding-top: 3.5rem;
2415   }
2416 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2417–2420

```
2417   .hero-copy h1 {
2418     font-size: clamp(2.75rem, 14vw, 3.7rem);
2419   }
2420 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

Lines 2421–2428

```
2421  .hero-search {
2422      min-height: auto;
2423      padding: 0.7rem;
2424      align-items: stretch;
2425      flex-wrap: wrap;
2426      border-radius: 18px;
2427  }
2428
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2429–2433

```
2429  .hero-search > i {
2430      align-self: center;
2431      margin-left: 0.3rem;
2432  }
2433
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2434–2438

```
2434  .hero-search input {
2435      width: calc(100% - 2.5rem);
2436      flex: none;
2437  }
2438
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2439–2443

```
2439  .hero-search button {
2440      width: 100%;
2441      border-radius: 12px;
2442  }
2443
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2444–2448

```
2444 .hero-proof {  
2445     align-items: flex-start;  
2446     flex-direction: column;  
2447 }  
2448
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2449–2455

```
2449 .hero-visual {  
2450     width: 580px;  
2451     left: 50%;  
2452     transform: translateX(-50%) scale(0.62);  
2453     margin-bottom: -9rem;  
2454 }  
2455
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2456–2459

```
2456 .category-strip {  
2457     padding: 4rem 0;  
2458 }  
2459
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2460–2464

```
2460 .category-list button {  
2461     padding: 1rem 0.5rem;  
2462     font-size: 0.75rem;  
2463 }  
2464
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2465–2469

```
2465 .book-grid {  
2466     grid-template-columns: 1fr;  
2467     gap: 2.7rem;  
2468 }  
2469
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2470–2474

```
2470 .book-card {  
2471     max-width: 330px;  
2472     margin: 0 auto;  
2473 }  
2474
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2475–2478

```
2475 .book-card-cover-wrap {  
2476     height: 420px;  
2477 }  
2478
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2479–2483

```
2479 .catalogue-message {  
2480     padding: 1.5rem;  
2481     flex-direction: column;  
2482 }  
2483
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2484–2487

```
2484 .journey-steps {  
2485     grid-template-columns: 1fr;  
2486 }  
2487
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2488–2493

```
2488 .journey-steps article,  
2489 .journey-steps article:not(:first-child) {  
2490   min-height: auto;  
2491   padding: 1.6rem;  
2492 }  
2493
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2494–2497

```
2494 .journey-steps h3 {  
2495   margin-top: 1rem;  
2496 }  
2497
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2498–2501

```
2498 .community-grid {  
2499   grid-template-columns: 1fr;  
2500 }  
2501
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2502–2505

```
2502 .community-card:last-child {  
2503   grid-column: auto;  
2504 }  
2505
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2506–2509

```
2506 .detail-breadcrumb span {  
2507   display: none;  
2508 }  
2509
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2510–2515

```
2510 .detail-cover-frame,  
2511 .detail-cover-skeleton {  
2512     height: 450px;  
2513     padding: 1.8rem;  
2514 }  
2515
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2516–2519

```
2516 .detail-actions {  
2517     flex-direction: column;  
2518 }  
2519
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2520–2524

```
2520 .detail-cart-button,  
2521 .detail-save-button {  
2522     width: 100%;  
2523 }  
2524
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2525–2530

```
2525 .detail-price-row {  
2526     align-items: flex-start;  
2527     flex-direction: column;  
2528     gap: 0.2rem;  
2529 }  
2530
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 2531–2534

```
2531 .edition-card dl {
2532   grid-template-columns: 1fr;
2533 }
2534
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2535–2538

```
2535 .site-footer {
2536   padding-top: 4rem;
2537 }
2538
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2539–2542

```
2539 .footer-grid {
2540   grid-template-columns: 1fr;
2541 }
2542
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2543–2546

```
2543 .footer-grid > div:nth-child(2) {
2544   display: none;
2545 }
2546
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2547–2552

```
2547 .footer-bottom {
2548   align-items: flex-start;
2549   flex-direction: column;
2550   gap: 0.5rem;
2551 }
2552 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2553–2563

```
2553
2554 @media (prefers-reduced-motion: reduce) {
2555     *,
2556     *::before,
2557     *::after {
2558         scroll-behavior: auto !important;
2559         animation-duration: 0.01ms !important;
2560         animation-iteration-count: 1 !important;
2561         transition-duration: 0.01ms !important;
2562     }
2563 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.

Lines 2564–2569

```
2564
2565 /* Connected reader experience */
2566 .nav-count-link {
2567     position: relative;
2568     text-decoration: none;
2569 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2570–2586

```
2570
2571 .nav-count-link > span {
2572     position: absolute;
2573     top: -2px;
2574     right: -3px;
2575     display: grid;
2576     min-width: 17px;
2577     height: 17px;
2578     padding: 0 4px;
2579     place-items: center;
2580     color: white;
2581     font-size: 0.62rem;
2582     font-weight: 800;
2583     background: var(--clay);
2584     border: 2px solid var(--white);
2585     border-radius: 999px;
2586 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2587–2592

```
2587
2588 .nav-reader-menu {
2589     display: flex;
2590     align-items: center;
2591     gap: 0.7rem;
2592 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2593–2600

```
2593
2594 .nav-reader-menu a {
2595     display: inline-flex;
2596     align-items: center;
2597     gap: 0.4rem;
2598     font-weight: 750;
2599     text-decoration: none;
2600 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2601–2612

```
2601
2602 .nav-reader-menu button,
2603 .text-button {
2604     padding: 0;
2605     color: var(--muted);
2606     font-size: 0.78rem;
2607     font-weight: 700;
2608     background: transparent;
2609     border: 0;
2610     text-decoration: underline;
2611     text-underline-offset: 3px;
2612 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2613–2617

```
2613
2614 .nav-reader-menu button:hover,
2615 .text-button:hover {
2616     color: var(--clay);
2617 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 2618–2622

```
2618
2619 .book-card-actions {
2620     display: flex;
2621     gap: 0.45rem;
2622 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2623–2626

```
2623
2624 button.book-card-arrow {
2625     border: 0;
2626 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2627–2631

```
2627
2628 button.book-card-arrow:disabled {
2629     cursor: not-allowed;
2630     opacity: 0.4;
2631 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 2632–2640

```
2632
2633 .book-card-message {
2634     display: block;
2635     min-height: 1.1rem;
2636     margin-top: 0.55rem;
2637     color: var(--forest);
2638     font-size: 0.72rem;
2639     font-weight: 700;
2640 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

Lines 2641–2647

```
2641
2642 .detail-action-message {
2643     margin: -0.4rem 0 1rem;
2644     color: var(--forest);
2645     font-size: 0.84rem;
2646     font-weight: 750;
2647 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2648–2655

```
2648
2649 .edition-choice {
2650     width: 100%;
2651     color: inherit;
2652     text-align: left;
2653     cursor: pointer;
2654     transition: border-color 160ms ease, box-shadow 160ms ease, transform 160ms ease;
2655 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2656–2662

```
2656
2657 .edition-choice:hover,
2658 .edition-choice.is-selected {
2659     border-color: var(--forest-light);
2660     box-shadow: 0 12px 32px rgb(16 42 38 / 10%);
2661     transform: translateY(-2px);
2662 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 2663–2676

```
2663
2664 .edition-choice.is-selected::after {
2665     position: absolute;
2666     top: 1rem;
2667     right: 1rem;
2668     display: grid;
2669     width: 24px;
2670     height: 24px;
2671     place-items: center;
2672     color: white;
2673     background: var(--forest);
2674     border-radius: 50%;
2675     content: "✓";
2676 }
```

```
2676 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2677–2680

```
2677  
2678 .edition-card {  
2679     position: relative;  
2680 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2681–2690

```
2681  
2682 .page-loading {  
2683     display: flex;  
2684     min-height: 55vh;  
2685     align-items: center;  
2686     justify-content: center;  
2687     gap: 0.8rem;  
2688     color: var(--muted);  
2689     font-weight: 700;  
2690 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2691–2696

```
2691  
2692 .inner-page {  
2693     min-height: 68vh;  
2694     padding: 5rem 0 7rem;  
2695     background: var(--paper);  
2696 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2697–2701

```
2697  
2698 .inner-page-heading {  
2699     max-width: 760px;  
2700     margin-bottom: 3rem;
```

```
2701 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2702–2710

```
2702
2703 .inner-page-heading h1,
2704 .auth-form h2,
2705 .order-detail-heading h1 {
2706     margin: 0 0 0.7rem;
2707     font-family: var(--serif);
2708     font-size: clamp(2.7rem, 6vw, 5rem);
2709     line-height: 0.98;
2710 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2711–2715

```
2711
2712 .inner-page-heading > p:last-child {
2713     color: var(--muted);
2714     font-size: 1.05rem;
2715 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2716–2728

```
2716
2717 .empty-reader-page {
2718     display: flex;
2719     min-height: 340px;
2720     padding: 3rem;
2721     align-items: center;
2722     justify-content: center;
2723     flex-direction: column;
2724     text-align: center;
2725     background: rgb(255 253 250 / 72%);
2726     border: 1px dashed #c9c0b2;
2727     border-radius: 24px;
2728 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2729–2734

```
2729
```

```
2730 .empty-reader-page > i {
2731   margin-bottom: 1rem;
2732   color: var(--saffron);
2733   font-size: 2.5rem;
2734 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2735–2739

```
2735
2736 .empty-reader-page h2 {
2737   font-family: var(--serif);
2738   font-size: 2rem;
2739 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2740–2744

```
2740
2741 .empty-reader-page p,
2742 .muted-message {
2743   color: var(--muted);
2744 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2745–2750

```
2745
2746 .auth-page {
2747   display: grid;
2748   min-height: calc(100vh - 76px);
2749   grid-template-columns: minmax(0, 0.9fr) minmax(480px, 1.1fr);
2750 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2751–2757

```
2751
2752 .auth-panel {
2753   display: flex;
2754   padding: clamp(3rem, 7vw, 8rem);
2755   justify-content: center;
2756   flex-direction: column;
2757 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2758–2764

```
2758
2759 .auth-story {
2760     position: relative;
2761     overflow: hidden;
2762     color: var(--white);
2763     background: linear-gradient(145deg, var(--ink), var(--forest-light));
2764 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2765–2776

```
2765
2766 .auth-story::after {
2767     position: absolute;
2768     right: -12rem;
2769     bottom: -18rem;
2770     width: 34rem;
2771     height: 34rem;
2772     border: 1px solid rgb(255 255 255 / 17%);
2773     border-radius: 50%;
2774     box-shadow: 0 0 0 5rem rgb(255 255 255 / 3%), 0 0 0 10rem rgb(255 255 255 / 2%);
2775     content: "";
2776 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2777–2791

```
2777
2778 .auth-akshar {
2779     position: relative;
2780     z-index: 1;
2781     display: grid;
2782     width: 80px;
2783     height: 80px;
2784     margin-bottom: 3rem;
2785     place-items: center;
2786     color: var(--ink);
2787     font-family: var(--serif);
2788     font-size: 3rem;
2789     background: var(--saffron-light);
2790     border-radius: 50%;
2791 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2792–2800

```
2792
2793 .auth-story h1 {
2794     position: relative;
2795     z-index: 1;
2796     max-width: 560px;
2797     font-family: var(--serif);
2798     font-size: clamp(3rem, 6vw, 5.7rem);
2799     line-height: 0.96;
2800 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2801–2810

```
2801
2802 .auth-story > p:last-child {
2803     position: relative;
2804     z-index: 1;
2805     max-width: 520px;
2806     margin-top: 1.5rem;
2807     color: rgb(255 253 250 / 72%);
2808     font-size: 1.06rem;
2809     line-height: 1.8;
2810 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2811–2814

```
2811
2812 .auth-form-panel {
2813     background: var(--white);
2814 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2815–2819

```
2815
2816 .auth-form {
2817     width: min(100%, 460px);
2818     margin: 0 auto;
2819 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2820–2824

```
2820
2821 .auth-form h2 {
2822     margin-bottom: 2.5rem;
2823     font-size: 3.6rem;
2824 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2825–2835

```
2825
2826 .auth-form label,
2827 .checkout-form label {
2828     display: flex;
2829     margin-bottom: 1rem;
2830     color: var(--ink-soft);
2831     font-size: 0.8rem;
2832     font-weight: 800;
2833     flex-direction: column;
2834     gap: 0.45rem;
2835 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2836–2848

```
2836
2837 .auth-form input,
2838 .checkout-form input,
2839 .review-form input,
2840 .review-form textarea {
2841     width: 100%;
2842     min-height: 50px;
2843     padding: 0.8rem 0.95rem;
2844     color: var(--ink);
2845     background: var(--paper);
2846     border: 1px solid var(--line);
2847     border-radius: 10px;
2848 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2849–2853

```
2849
2850 .auth-form .btn {
2851     width: 100%;
2852     margin-top: 0.75rem;
2853 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2854–2859

```
2854
2855 .auth-switch {
2856     margin-top: 1.5rem;
2857     color: var(--muted);
2858     text-align: center;
2859 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2860–2865

```
2860
2861 .auth-switch a,
2862 .review-signin a {
2863     color: var(--forest);
2864     font-weight: 800;
2865 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2866–2873

```
2866
2867 .form-alert {
2868     padding: 0.8rem 1rem;
2869     color: #7c2c25;
2870     background: #fbe8e3;
2871     border: 1px solid #efc4ba;
2872     border-radius: 10px;
2873 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2874–2881

```
2874
2875 .cart-layout,
2876 .checkout-layout {
2877     display: grid;
2878     align-items: start;
2879     grid-template-columns: minmax(0, 1fr) 360px;
2880     gap: 3rem;
2881 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2882–2890

```
2882
2883 .cart-items,
2884 .checkout-form,
2885 .order-detail-card,
2886 .order-detail-grid > div {
2887   background: var(--white);
2888   border: 1px solid var(--line);
2889   border-radius: 18px;
2890 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2891–2899

```
2891
2892 .cart-item {
2893   display: grid;
2894   padding: 1.5rem;
2895   align-items: center;
2896   grid-template-columns: 78px minmax(0, 1fr) auto auto;
2897   gap: 1.2rem;
2898   border-bottom: 1px solid var(--line);
2899 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2900–2903

```
2900
2901 .cart-item:last-child {
2902   border-bottom: 0;
2903 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2904–2917

```
2904
2905 .cart-thumb,
2906 .order-book-cover {
2907   display: grid;
2908   overflow: hidden;
2909   width: 72px;
2910   height: 100px;
2911   place-items: center;
2912   color: var(--white);
2913   font-family: var(--serif);
2914   font-size: 2rem;
```

```
2915     background: var(--navy);
2916     border-radius: 6px;
2917 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2918–2924

```
2918
2919 .cart-thumb img,
2920 .order-book-cover img {
2921     width: 100%;
2922     height: 100%;
2923     object-fit: cover;
2924 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2925–2928

```
2925
2926 .cart-item-copy a {
2927     text-decoration: none;
2928 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2929–2934

```
2929
2930 .cart-item-copy h2 {
2931     margin: 0 0 0.35rem;
2932     font-family: var(--serif);
2933     font-size: 1.3rem;
2934 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2935–2940

```
2935
2936 .cart-item-copy p {
2937     margin: 0 0 0.6rem;
2938     color: var(--muted);
2939     font-size: 0.8rem;
2940 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2941–2947

```
2941
2942 .quantity-control {
2943     display: flex;
2944     align-items: center;
2945     border: 1px solid var(--line);
2946     border-radius: 999px;
2947 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2948–2954

```
2948
2949 .quantity-control button {
2950     width: 34px;
2951     height: 34px;
2952     background: transparent;
2953     border: 0;
2954 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2955–2959

```
2955
2956 .quantity-control span {
2957     min-width: 28px;
2958     text-align: center;
2959 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2960–2966

```
2960
2961 .cart-item-total {
2962     display: flex;
2963     align-items: flex-end;
2964     flex-direction: column;
2965     gap: 0.7rem;
2966 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2967–2975

```
2967
2968 .cart-item-total button {
2969     padding: 0;
2970     color: var(--clay);
2971     font-size: 0.72rem;
2972     font-weight: 750;
2973     background: transparent;
2974     border: 0;
2975 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2976–2983

```
2976
2977 .order-summary {
2978     position: sticky;
2979     top: 100px;
2980     padding: 1.8rem;
2981     background: var(--paper-deep);
2982     border-radius: 18px;
2983 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2984–2989

```
2984
2985 .order-summary h2 {
2986     margin-bottom: 1.5rem;
2987     font-family: var(--serif);
2988     font-size: 2rem;
2989 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 2990–2997

```
2990
2991 .order-summary > div {
2992     display: flex;
2993     margin: 0.9rem 0;
2994     justify-content: space-between;
2995     gap: 1rem;
2996     font-size: 0.85rem;
2997 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 2998–3001

```
2998
2999 .order-summary > div strong {
3000     text-align: right;
3001 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3002–3006

```
3002
3003 .order-summary .btn {
3004     width: 100%;
3005     margin-top: 1.4rem;
3006 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3007–3013

```
3007
3008 .order-summary small {
3009     display: block;
3010     margin-top: 1rem;
3011     color: var(--muted);
3012     line-height: 1.6;
3013 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3014–3017

```
3014
3015 .checkout-form {
3016     padding: 2rem;
3017 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3018–3024

```
3018
3019 .checkout-form h2,
3020 .order-detail-card h2,
3021 .order-detail-grid h2 {
3022     margin-bottom: 1.5rem;
3023     font-family: var(--serif);
3024 }
```

```
3024 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3025–3030

```
3025
3026 .form-grid {
3027     display: grid;
3028     grid-template-columns: 1fr 1fr;
3029     gap: 0 1rem;
3030 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3031–3034

```
3031
3032 .form-grid .span-two {
3033     grid-column: span 2;
3034 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3035–3039

```
3035
3036 .form-grid label small {
3037     color: var(--muted);
3038     font-weight: 500;
3039 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3040–3043

```
3040
3041 .checkout-form > .btn {
3042     margin-top: 1rem;
3043 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3044–3050

```
3044
3045 .orders-list {
3046     overflow: hidden;
3047     background: var(--white);
3048     border: 1px solid var(--line);
3049     border-radius: 18px;
3050 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3051–3060

```
3051
3052 .order-row {
3053     display: grid;
3054     padding: 1.4rem 1.6rem;
3055     align-items: center;
3056     grid-template-columns: minmax(0, 1fr) auto auto auto;
3057     gap: 1.5rem;
3058     text-decoration: none;
3059     border-bottom: 1px solid var(--line);
3060 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3061–3064

```
3061
3062 .order-row:last-child {
3063     border-bottom: 0;
3064 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3065–3068

```
3065
3066 .order-row:hover {
3067     background: var(--paper);
3068 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3069–3073

```
3069
3070 .order-row > div {
3071     display: flex;
3072     flex-direction: column;
3073 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3074–3078

```
3074
3075 .order-row span {
3076     color: var(--muted);
3077     font-size: 0.8rem;
3078 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3079–3090

```
3079
3080 .status-pill {
3081     display: inline-flex;
3082     width: fit-content;
3083     padding: 0.35rem 0.65rem;
3084     color: var(--forest) !important;
3085     font-size: 0.68rem !important;
3086     font-weight: 850;
3087     background: #e1eee8;
3088     border-radius: 999px;
3089     text-transform: capitalize;
3090 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3091–3094

```
3091
3092 .narrow-page {
3093     max-width: 1000px;
3094 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3095–3105

```
3095
3096 .success-banner {
3097     display: flex;
3098     margin-bottom: 2rem;
3099     padding: 1rem 1.2rem;
3100     align-items: center;
3101     gap: 0.9rem;
3102     color: var(--forest);
3103     background: #e1eee8;
3104     border-radius: 12px;
3105 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3106–3109

```
3106
3107 .success-banner i {
3108     font-size: 1.5rem;
3109 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3110–3114

```
3110
3111 .success-banner div {
3112     display: flex;
3113     flex-direction: column;
3114 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3115–3123

```
3115
3116 .back-link {
3117     display: inline-flex;
3118     margin-bottom: 2rem;
3119     align-items: center;
3120     gap: 0.45rem;
3121     font-weight: 800;
3122     text-decoration: none;
3123 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3124–3130

```
3124
3125 .order-detail-navigation {
3126     display: flex;
3127     align-items: center;
3128     justify-content: space-between;
3129     gap: 1rem;
3130 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3131–3135

```
3131
3132 .order-access-note a {
3133     color: var(--white);
3134     font-weight: 800;
3135 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3136–3139

```
3136
3137 .my-books-page {
3138     min-height: 70vh;
3139 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3140–3151

```
3140
3141 .my-books-hero {
3142     display: grid;
3143     margin-bottom: 2rem;
3144     padding: 3rem;
3145     grid-template-columns: minmax(0, 1fr) auto;
3146     align-items: end;
3147     gap: 3rem;
3148     color: var(--white);
3149     background: var(--forest);
3150     border-radius: 22px;
3151 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3152–3155

```
3152
3153 .my-books-hero .eyebrow {
3154   color: var(--saffron);
3155 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3156–3162

```
3156
3157 .my-books-hero h1 {
3158   margin-bottom: 0.65rem;
3159   color: var(--white);
3160   font-family: var(--serif);
3161   font-size: clamp(3rem, 7vw, 5.5rem);
3162 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3163–3168

```
3163
3164 .my-books-hero p:last-child {
3165   max-width: 620px;
3166   margin: 0;
3167   color: rgba(255, 255, 255, 0.72);
3168 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3169–3176

```
3169
3170 .my-books-stats {
3171   display: grid;
3172   min-width: 220px;
3173   grid-template-columns: auto 1fr;
3174   align-items: baseline;
3175   gap: 0.2rem 0.8rem;
3176 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3177–3182

```
3177
3178 .my-books-stats strong {
3179     color: var(--saffron);
3180     font-family: var(--serif);
3181     font-size: 2.2rem;
3182 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3183–3187

```
3183
3184 .my-books-stats span {
3185     color: rgba(255, 255, 255, 0.75);
3186     font-size: 0.78rem;
3187 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3188–3196

```
3188
3189 .my-books-toolbar {
3190     display: flex;
3191     margin-bottom: 2rem;
3192     padding-bottom: 1rem;
3193     align-items: center;
3194     gap: 0.5rem;
3195     border-bottom: 1px solid var(--line);
3196 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3197–3204

```
3197
3198 .personal-upload-panel {
3199     margin-bottom: 1.4rem;
3200     overflow: hidden;
3201     background: var(--white);
3202     border: 1px solid var(--line);
3203     border-radius: 16px;
3204 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3205–3209

```
3205
3206 .personal-upload-panel.is-open {
3207   border-color: rgba(20, 78, 65, 0.35);
3208   box-shadow: 0 14px 30px rgba(20, 47, 41, 0.07);
3209 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3210–3221

```
3210
3211 .personal-upload-toggle {
3212   display: flex;
3213   width: 100%;
3214   padding: 1.15rem 1.3rem;
3215   align-items: center;
3216   justify-content: space-between;
3217   color: var(--forest);
3218   text-align: left;
3219   background: transparent;
3220   border: 0;
3221 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3222–3227

```
3222
3223 .personal-upload-toggle > span {
3224   display: flex;
3225   align-items: center;
3226   gap: 0.9rem;
3227 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3228–3238

```
3228
3229 .personal-upload-toggle > span > i {
3230   display: grid;
3231   width: 42px;
3232   height: 42px;
3233   place-items: center;
3234   color: var(--white);
3235   font-size: 1.15rem;
3236   background: var(--forest);
3237   border-radius: 50%;
3238 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3239–3243

```
3239
3240 .personal-upload-toggle span span {
3241     display: flex;
3242     flex-direction: column;
3243 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3244–3249

```
3244
3245 .personal-upload-toggle small {
3246     color: var(--muted);
3247     font-size: 0.72rem;
3248     font-weight: 500;
3249 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3250–3259

```
3250
3251 .personal-upload-form {
3252     display: grid;
3253     padding: 1.3rem;
3254     grid-template-columns: 1fr 1fr 1.35fr auto;
3255     align-items: end;
3256     gap: 1rem;
3257     background: var(--paper);
3258     border-top: 1px solid var(--line);
3259 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3260–3270

```
3260
3261 .personal-upload-form label {
3262     display: flex;
3263     margin: 0;
3264     min-width: 0;
3265     flex-direction: column;
3266     gap: 0.4rem;
```

```
3267 color: var(--forest);
3268 font-size: 0.72rem;
3269 font-weight: 800;
3270 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3271–3279

```
3271
3272 .personal-upload-form input {
3273   width: 100%;
3274   min-height: 44px;
3275   padding: 0.65rem 0.75rem;
3276   background: var(--white);
3277   border: 1px solid var(--line);
3278   border-radius: 9px;
3279 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3280–3284

```
3280
3281 .personal-upload-form input[type='file'] {
3282   padding: 0.48rem;
3283   font-size: 0.72rem;
3284 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3285–3290

```
3285
3286 .personal-upload-form label > span small,
3287 .personal-file-field > small {
3288   color: var(--muted);
3289   font-weight: 500;
3290 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3291–3295

```
3291
3292 .personal-file-field > small {
3293   min-height: 1em;
3294   font-size: 0.61rem;
3295 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3296–3300

```
3296
3297 .personal-upload-form .btn {
3298   min-height: 44px;
3299   white-space: nowrap;
3300 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3301–3309

```
3301
3302 .personal-upload-message {
3303   margin: 0;
3304   padding: 0.8rem 1.3rem;
3305   color: var(--forest);
3306   font-size: 0.75rem;
3307   font-weight: 750;
3308   border-top: 1px solid var(--line);
3309 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3310–3321

```
3310
3311 .my-books-toolbar button,
3312 .my-books-toolbar a {
3313   padding: 0.65rem 1rem;
3314   color: var(--muted);
3315   font-size: 0.78rem;
3316   font-weight: 800;
3317   background: transparent;
3318   border: 1px solid transparent;
3319   border-radius: 999px;
3320   text-decoration: none;
3321 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3322–3326

```
3322
3323 .my-books-toolbar button.is-active {
3324   color: var(--white);
3325   background: var(--forest);
3326 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3327–3333

```
3327
3328 .my-books-toolbar > a,
3329 .my-books-toolbar-links a {
3330     margin-left: auto;
3331     color: var(--forest);
3332     border-color: var(--line);
3333 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3334–3339

```
3334
3335 .my-books-toolbar-links {
3336     display: flex;
3337     margin-left: auto;
3338     gap: 0.45rem;
3339 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3340–3343

```
3340
3341 .my-books-toolbar-links a {
3342     margin-left: 0;
3343 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3344–3349

```
3344
3345 .my-books-grid {
3346     display: grid;
3347     grid-template-columns: repeat(2, minmax(0, 1fr));
3348     gap: 1.25rem;
3349 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3350–3360

```
3350
3351 .my-book-card {
3352   display: grid;
3353   min-width: 0;
3354   padding: 1.25rem;
3355   grid-template-columns: 125px minmax(0, 1fr);
3356   gap: 1.4rem;
3357   background: var(--white);
3358   border: 1px solid var(--line);
3359   border-radius: 18px;
3360 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3361–3364

```
3361
3362 .my-book-cover {
3363   position: relative;
3364 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3365–3373

```
3365
3366 .my-book-cover > img,
3367 .my-book-fallback {
3368   width: 125px;
3369   height: 184px;
3370   object-fit: cover;
3371   border-radius: 7px;
3372   box-shadow: 0 12px 26px rgba(20, 47, 41, 0.16);
3373 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3374–3382

```
3374
3375 .my-book-fallback {
3376   display: flex;
3377   padding: 1rem;
3378   align-items: center;
3379   justify-content: space-between;
3380   flex-direction: column;
3381   color: var(--white);
3382 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3383–3387

```
3383
3384 .my-book-fallback span {
3385     color: var(--saffron);
3386     font-size: 1.5rem;
3387 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3388–3392

```
3388
3389 .my-book-fallback strong {
3390     font-family: var(--serif);
3391     font-size: 2rem;
3392 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3393–3405

```
3393
3394 .my-book-status {
3395     position: absolute;
3396     right: -8px;
3397     bottom: 8px;
3398     padding: 0.35rem 0.55rem;
3399     color: var(--white);
3400     font-size: 0.62rem;
3401     font-weight: 850;
3402     background: var(--forest);
3403     border-radius: 999px;
3404     box-shadow: 0 4px 10px rgba(20, 47, 41, 0.18);
3405 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3406–3410

```
3406
3407 .my-book-status.status-transit {
3408     color: var(--ink);
3409     background: var(--saffron);
3410 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

Lines 3411–3414

```
3411
3412 .my-book-status.status-uploaded {
3413     background: #60446f;
3414 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3415–3420

```
3415
3416 .my-book-copy {
3417     display: flex;
3418     min-width: 0;
3419     flex-direction: column;
3420 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3421–3429

```
3421
3422 .my-book-copy > p {
3423     margin: 0 0 0.45rem;
3424     color: var(--ochre);
3425     font-size: 0.67rem;
3426     font-weight: 850;
3427     letter-spacing: 0.08em;
3428     text-transform: uppercase;
3429 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3430–3435

```
3430
3431 .my-book-copy h2 {
3432     margin-bottom: 0.35rem;
3433     font-family: var(--serif);
3434     font-size: 1.55rem;
3435 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3436–3443

```
3436
3437 .my-book-copy > span {
3438     overflow: hidden;
3439     color: var(--muted);
3440     font-size: 0.76rem;
3441     text-overflow: ellipsis;
3442     white-space: nowrap;
3443 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3444–3452

```
3444
3445 .my-book-progress {
3446     display: flex;
3447     margin-top: auto;
3448     padding: 0.8rem 0;
3449     align-items: center;
3450     gap: 0.65rem;
3451     border-bottom: 1px solid var(--line);
3452 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3453–3456

```
3453
3454 .my-book-progress i {
3455     color: var(--ochre);
3456 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3457–3462

```
3457
3458 .my-book-progress div {
3459     display: flex;
3460     flex-direction: column;
3461     font-size: 0.73rem;
3462 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3463–3466

```
3463
3464 .my-book-progress small {
3465     color: var(--muted);
3466 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3467–3474

```
3467
3468 .my-book-actions {
3469     display: flex;
3470     padding-top: 0.8rem;
3471     align-items: center;
3472     justify-content: space-between;
3473     gap: 0.7rem;
3474 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3475–3480

```
3475
3476 .my-book-actions a {
3477     font-size: 0.73rem;
3478     font-weight: 850;
3479     text-decoration: none;
3480 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3481–3484

```
3481
3482 .personal-book-actions {
3483     justify-content: flex-start;
3484 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3485–3493

```
3485
3486 .personal-book-actions button {
3487     padding: 0;
```

```
3488 color: var(--forest);
3489 font-size: 0.7rem;
3490 font-weight: 850;
3491 background: transparent;
3492 border: 0;
3493 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3494–3498

```
3494
3495 .personal-book-actions button.danger {
3496   margin-left: auto;
3497   color: #a43d30;
3498 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3499–3502

```
3499
3500 .personal-book-actions button:disabled {
3501   opacity: 0.45;
3502 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3503–3509

```
3503
3504 /* Admin order operations */
3505 .admin-orders-page {
3506   min-height: 100vh;
3507   color: #15332e;
3508   background: #f4efe5;
3509 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3510–3513

```
3510
3511 .admin-orders-hero {
3512   padding-bottom: 6rem;
3513 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3514–3519

```
3514
3515 .admin-orders-workspace {
3516     position: relative;
3517     margin-top: -2.4rem;
3518     padding-bottom: 5rem;
3519 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3520–3526

```
3520
3521 .admin-orders-toolbar {
3522     display: flex;
3523     margin: 1.5rem 0 0.8rem;
3524     justify-content: space-between;
3525     gap: 1rem;
3526 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3527–3537

```
3527
3528 .admin-orders-toolbar label {
3529     display: flex;
3530     max-width: 620px;
3531     flex: 1;
3532     padding: 0 1rem;
3533     align-items: center;
3534     background: #ffffdf8;
3535     border: 1px solid #d9cfbf;
3536     border-radius: 12px;
3537 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3538–3545

```
3538
3539 .admin-orders-toolbar input {
3540     width: 100%;
3541     padding: 0.85rem 0.7rem;
3542     background: transparent;
3543     border: 0;
3544     outline: 0;
3545 }
```


How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3546–3553

```
3546
3547 .admin-orders-toolbar > button {
3548     padding: 0.75rem 1rem;
3549     color: #17483d;
3550     background: #ffffdf8;
3551     border: 1px solid #cfc4b3;
3552     border-radius: 10px;
3553 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3554–3560

```
3554
3555 .admin-order-filters {
3556     display: flex;
3557     margin-bottom: 1.3rem;
3558     overflow-x: auto;
3559     gap: 0.4rem;
3560 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3561–3571

```
3561
3562 .admin-order-filters button {
3563     flex: 0 0 auto;
3564     padding: 0.48rem 0.8rem;
3565     color: #657670;
3566     font-size: 0.7rem;
3567     font-weight: 800;
3568     background: transparent;
3569     border: 1px solid #d4cabb;
3570     border-radius: 999px;
3571 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3572–3577

```
3572
3573 .admin-order-filters button.is-active {
3574     color: #fff;
3575     background: #17483d;
```

```
3576     border-color: #17483d;
3577 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3578–3584

```
3578
3579 .admin-order-layout {
3580     display: grid;
3581     grid-template-columns: minmax(330px, 0.8fr) minmax(500px, 1.2fr);
3582     align-items: start;
3583     gap: 1.25rem;
3584 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3585–3592

```
3585
3586 .admin-order-list,
3587 .admin-order-detail {
3588     overflow: hidden;
3589     background: #ffffdf8;
3590     border: 1px solid #dcd2c2;
3591     border-radius: 18px;
3592 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3593–3606

```
3593
3594 .admin-order-row {
3595     display: grid;
3596     width: 100%;
3597     padding: 1.15rem 1.2rem;
3598     grid-template-columns: minmax(0, 1fr) auto 16px;
3599     align-items: center;
3600     gap: 1rem;
3601     color: #15332e;
3602     text-align: left;
3603     background: transparent;
3604     border: 0;
3605     border-bottom: 1px solid #e8dfd2;
3606 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3607–3610

```
3607
3608 .admin-order-row:last-child {
3609     border-bottom: 0;
3610 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3611–3615

```
3611
3612 .admin-order-row:hover,
3613 .admin-order-row.is-selected {
3614     background: #eef3ed;
3615 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3616–3619

```
3616
3617 .admin-order-row.is-selected {
3618     box-shadow: inset 4px 0 #d99024;
3619 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3620–3626

```
3620
3621 .admin-order-row > div {
3622     display: flex;
3623     min-width: 0;
3624     flex-direction: column;
3625     gap: 0.2rem;
3626 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3627–3630

```
3627
3628 .admin-order-row > div:nth-child(2) {
3629     align-items: flex-end;
3630 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3631–3639

```
3631
3632 .admin-order-row span,
3633 .admin-order-row small {
3634     overflow: hidden;
3635     color: #71807b;
3636     font-size: 0.7rem;
3637     text-overflow: ellipsis;
3638     white-space: nowrap;
3639 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3640–3650

```
3640
3641 .admin-order-status {
3642     display: inline-flex;
3643     width: fit-content;
3644     padding: 0.3rem 0.55rem;
3645     color: #17483d !important;
3646     font-size: 0.62rem !important;
3647     font-weight: 850;
3648     background: #e2eee8;
3649     border-radius: 999px;
3650 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3651–3652

```
3651
3652 .admin-order-status.status-placed { color: #89580c !important; background: #fff0ce; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3653–3654

```
3653 .admin-order-status.status-processing,  
3654 .admin-order-status.status-shipped { color: #275c80 !important; background: #e0edf5; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3655–3655

```
3655 .admin-order-status.status-delivered { color: #24633e !important; background: #dff0e5; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3656–3656

```
3656 .admin-order-status.status-cancelled { color: #96372d !important; background: #f8e2de; }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3657–3661

```
3657  
3658 .admin-order-detail {  
3659     position: sticky;  
3660     top: 100px;  
3661 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3662–3667

```
3662  
3663 .admin-order-placeholder {  
3664     padding: 5rem 2rem;  
3665     color: #6f7d78;  
3666     text-align: center;  
3667 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3668–3672

```
3668  
3669 .admin-order-placeholder i {
```

```
3670 color: #bf8124;
3671 font-size: 2rem;
3672 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3673–3678

```
3673
3674 .admin-order-placeholder h2 {
3675   margin: 0.8rem 0 0.3rem;
3676   color: #15332e;
3677   font-family: Georgia, serif;
3678 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3679–3687

```
3679
3680 .admin-order-detail > header {
3681   display: flex;
3682   padding: 1.5rem;
3683   align-items: flex-start;
3684   justify-content: space-between;
3685   gap: 1rem;
3686   border-bottom: 1px solid #e7ded1;
3687 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3688–3696

```
3688
3689 .admin-order-detail header p {
3690   margin: 0 0 0.25rem;
3691   color: #a26717;
3692   font-size: 0.7rem;
3693   font-weight: 800;
3694   letter-spacing: 0.08em;
3695   text-transform: uppercase;
3696 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3697–3702

```
3697
3698 .admin-order-detail header h2 {
```

```
3699 margin: 0;
3700 font-family: Georgia, serif;
3701 font-size: 1.7rem;
3702 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3703–3707

```
3703
3704 .admin-order-detail header div > span {
3705     color: #75827e;
3706     font-size: 0.75rem;
3707 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3708–3717

```
3708
3709 .admin-order-primary-action {
3710     display: flex;
3711     padding: 1rem 1.5rem;
3712     align-items: center;
3713     justify-content: space-between;
3714     gap: 1rem;
3715     background: #eef3ed;
3716     border-bottom: 1px solid #d8e1d9;
3717 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3718–3727

```
3718
3719 .admin-order-primary-action > button:first-child {
3720     padding: 0.7rem 1rem;
3721     color: #fff;
3722     font-size: 0.78rem;
3723     font-weight: 850;
3724     background: #17483d;
3725     border: 0;
3726     border-radius: 9px;
3727 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3728–3735

```
3728
3729 .admin-order-primary-action .cancel-order-button {
3730     color: #9a3327;
3731     font-size: 0.7rem;
3732     font-weight: 800;
3733     background: transparent;
3734     border: 0;
3735 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3736–3742

```
3736
3737 .admin-order-primary-action p {
3738     margin: 0;
3739     color: #48645c;
3740     font-size: 0.8rem;
3741     font-weight: 800;
3742 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3743–3747

```
3743
3744 .admin-order-items {
3745     padding: 1.4rem 1.5rem;
3746     border-bottom: 1px solid #e7ded1;
3747 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3748–3757

```
3748
3749 .admin-order-items h3,
3750 .admin-order-facts h3 {
3751     margin: 0 0 0.8rem;
3752     color: #6d7b76;
3753     font-size: 0.68rem;
3754     font-weight: 850;
3755     letter-spacing: 0.08em;
3756     text-transform: uppercase;
3757 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3758–3766

```
3758
3759 .admin-order-items > div {
3760   display: grid;
3761   padding: 0.7rem 0;
3762   grid-template-columns: 42px minmax(0, 1fr) auto;
3763   align-items: center;
3764   gap: 0.8rem;
3765   border-top: 1px solid #eee6db;
3766 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3767–3777

```
3767
3768 .admin-order-items > div > span {
3769   display: grid;
3770   width: 38px;
3771   height: 52px;
3772   overflow: hidden;
3773   place-items: center;
3774   color: #b2781d;
3775   background: #e8dfd0;
3776   border-radius: 4px;
3777 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3778–3783

```
3778
3779 .admin-order-items img {
3780   width: 100%;
3781   height: 100%;
3782   object-fit: cover;
3783 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3784–3789

```
3784
3785 .admin-order-items div div {
3786   display: flex;
3787   min-width: 0;
3788   flex-direction: column;
3789 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3790–3794

```
3790
3791 .admin-order-items small {
3792     color: #75827e;
3793     font-size: 0.68rem;
3794 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3795–3799

```
3795
3796 .admin-order-facts {
3797     display: grid;
3798     grid-template-columns: 1fr 1fr;
3799 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3800–3804

```
3800
3801 .admin-order-facts section {
3802     padding: 1.4rem 1.5rem;
3803     border-right: 1px solid #e7ded1;
3804 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3805–3810

```
3805
3806 .admin-order-facts section:last-child {
3807     display: flex;
3808     border-right: 0;
3809     flex-direction: column;
3810 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3811–3818

```
3811
3812 .admin-order-facts p,
3813 .admin-order-facts small {
3814   margin: 0;
3815   color: #64736e;
3816   font-size: 0.75rem;
3817   line-height: 1.65;
3818 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3819–3823

```
3819
3820 .admin-order-facts section > strong {
3821   margin-top: auto;
3822   padding-top: 0.8rem;
3823 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3824–3829

```
3824
3825 @media (max-width: 1100px) {
3826   .admin-order-layout {
3827     grid-template-columns: 1fr;
3828   }
3829 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3830–3833

```
3830 .admin-order-detail {
3831   position: static;
3832 }
3833 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3834–3840

```
3834
3835 @media (max-width: 767.98px) {
```

```
3836 .admin-orders-toolbar {
3837     align-items: stretch;
3838     flex-direction: column;
3839 }
3840
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3841–3844

```
3841 .admin-orders-toolbar label {
3842     max-width: none;
3843 }
3844
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3845–3848

```
3845 .admin-order-facts {
3846     grid-template-columns: 1fr;
3847 }
3848
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3849–3853

```
3849 .admin-order-facts section {
3850     border-right: 0;
3851     border-bottom: 1px solid #e7ded1;
3852 }
3853 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3854–3859

```
3854
3855 @media (max-width: 575.98px) {
3856     .admin-order-row {
3857         grid-template-columns: 1fr auto;
3858     }
3859 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3860–3863

```
3860 .admin-order-row > i {
3861     display: none;
3862 }
3863
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3864–3868

```
3864 .admin-order-primary-action {
3865     align-items: flex-start;
3866     flex-direction: column;
3867 }
3868 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3869–3876

```
3869
3870 .order-detail-heading {
3871     display: flex;
3872     margin-bottom: 2rem;
3873     align-items: flex-end;
3874     justify-content: space-between;
3875     gap: 2rem;
3876 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3877–3881

```
3877
3878 .order-detail-heading h1 {
3879     font-size: 3.5rem;
3880     text-transform: capitalize;
3881 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3882–3886

```
3882
3883 .order-detail-heading > strong {
3884     font-family: var(--serif);
3885     font-size: 2rem;
3886 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3887–3890

```
3887
3888 .order-detail-card {
3889     padding: 2rem;
3890 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3891–3897

```
3891
3892 .order-journey-card {
3893     margin-bottom: 1.5rem;
3894     padding: 2rem;
3895     background: var(--forest);
3896     border-radius: 18px;
3897 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3898–3905

```
3898
3899 .order-journey-heading {
3900     display: flex;
3901     margin-bottom: 2rem;
3902     align-items: flex-start;
3903     justify-content: space-between;
3904     gap: 1rem;
3905 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3906–3909

```
3906
3907 .order-journey-heading .eyebrow {
3908     color: var(--saffron);
3909 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3910–3915

```
3910
3911 .order-journey-heading h2 {
3912     margin: 0;
3913     color: var(--white);
3914     font-family: var(--serif);
3915 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3916–3923

```
3916
3917 .delivery-timeline {
3918     display: grid;
3919     margin: 0;
3920     padding: 0;
3921     grid-template-columns: repeat(5, 1fr);
3922     list-style: none;
3923 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3924–3931

```
3924
3925 .delivery-timeline li {
3926     position: relative;
3927     display: flex;
3928     min-width: 0;
3929     flex-direction: column;
3930     gap: 0.7rem;
3931 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3932–3942

```
3932
3933 .delivery-timeline li::after {
3934     position: absolute;
3935     z-index: 0;
3936     top: 17px;
3937     right: 0;
3938     left: 34px;
3939     height: 1px;
3940     content: '';
3941     background: rgba(255, 255, 255, 0.2);
3942 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3943–3946

```
3943
3944 .delivery-timeline li:last-child::after {
3945     display: none;
3946 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3947–3960

```
3947
3948 .delivery-timeline li > span {
3949     z-index: 1;
3950     display: grid;
3951     width: 34px;
3952     height: 34px;
3953     place-items: center;
3954     color: rgba(255, 255, 255, 0.65);
3955     font-size: 0.75rem;
3956     font-weight: 800;
3957     background: #315b51;
3958     border: 1px solid rgba(255, 255, 255, 0.18);
3959     border-radius: 50%;
3960 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3961–3967

```
3961
3962 .delivery-timeline .is-complete > span,
3963 .delivery-timeline .is-current > span {
3964     color: var(--forest);
3965     background: var(--saffron);
3966     border-color: var(--saffron);
3967 }
```


How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3968–3971

```
3968
3969 .delivery-timeline .is-complete::after {
3970     background: var(--saffron);
3971 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 3972–3977

```
3972
3973 .delivery-timeline li > div {
3974     display: flex;
3975     padding-right: 0.5rem;
3976     flex-direction: column;
3977 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 3978–3982

```
3978
3979 .delivery-timeline strong {
3980     color: var(--white);
3981     font-size: 0.75rem;
3982 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3983–3987

```
3983
3984 .delivery-timeline small {
3985     color: rgba(255, 255, 255, 0.52);
3986     font-size: 0.68rem;
3987 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3988–3996

```
3988
3989 .order-access-note {
3990     margin: 1.8rem 0 0;
3991     padding-top: 1.2rem;
3992     color: rgba(255, 255, 255, 0.7);
3993     font-size: 0.8rem;
3994     line-height: 1.6;
3995     border-top: 1px solid rgba(255, 255, 255, 0.14);
3996 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 3997–4001

```
3997
3998 .order-access-note i {
3999     margin-right: 0.4rem;
4000     color: var(--saffron);
4001 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4002–4008

```
4002
4003 .order-cancelled-message {
4004     display: flex;
4005     align-items: center;
4006     gap: 0.8rem;
4007     color: var(--white);
4008 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4009–4013

```
4009
4010 .order-cancelled-message > i {
4011     color: #ffb3a9;
4012     font-size: 1.6rem;
4013 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4014–4018

```
4014
```

```
4015 .order-cancelled-message > div {
4016     display: flex;
4017     flex-direction: column;
4018 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4019–4027

```
4019
4020 .order-book {
4021     display: grid;
4022     padding: 1rem 0;
4023     align-items: center;
4024     grid-template-columns: 52px minmax(0, 1fr) auto;
4025     gap: 1rem;
4026     border-top: 1px solid var(--line);
4027 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4028–4033

```
4028
4029 .order-book-cover {
4030     width: 48px;
4031     height: 66px;
4032     font-size: 1.25rem;
4033 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4034–4039

```
4034
4035 .order-book > div {
4036     display: flex;
4037     flex-direction: column;
4038     align-items: flex-start;
4039 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4040–4045

```
4040
4041 .order-book-title {
4042   color: var(--ink);
4043   font-weight: 800;
4044   text-decoration: none;
4045 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4046–4056

```
4046
4047 .order-reflection-link {
4048   display: inline-flex;
4049   margin-top: 0.45rem;
4050   align-items: center;
4051   gap: 0.35rem;
4052   color: var(--forest);
4053   font-size: 0.73rem;
4054   font-weight: 800;
4055   text-decoration: none;
4056 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4057–4061

```
4057
4058 .order-access-warning {
4059   margin-top: 0.45rem;
4060   color: var(--clay);
4061 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4062–4066

```
4062
4063 .order-book span {
4064   color: var(--muted);
4065   font-size: 0.8rem;
4066 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4067–4073

```
4067
4068 .order-detail-grid {
4069     display: grid;
4070     margin-top: 1.5rem;
4071     grid-template-columns: 1fr 1fr;
4072     gap: 1.5rem;
4073 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4074–4077

```
4074
4075 .order-detail-grid > div {
4076     padding: 2rem;
4077 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4078–4082

```
4078
4079 .order-detail-grid p {
4080     color: var(--muted);
4081     line-height: 1.8;
4082 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4083–4087

```
4083
4084 .reviews-section {
4085     padding: 5rem 0 1rem;
4086     border-top: 1px solid var(--line);
4087 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4088–4093

```
4088
4089 .review-heading {
```

```
4090 display: flex;
4091 align-items: flex-end;
4092 justify-content: space-between;
4093 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4094–4099

```
4094
4095 .review-summary {
4096   display: flex;
4097   align-items: flex-end;
4098   flex-direction: column;
4099 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4100–4104

```
4100
4101 .review-summary strong {
4102   font-family: var(--serif);
4103   font-size: 2.4rem;
4104 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4105–4109

```
4105
4106 .review-summary span {
4107   color: var(--muted);
4108   font-size: 0.78rem;
4109 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4110–4120

```
4110
4111 .review-form {
4112   display: grid;
4113   max-width: 760px;
4114   margin: 2rem 0 3rem;
4115   padding: 1.5rem;
4116   gap: 0.9rem;
```

```
4117 background: var(--white);
4118 border: 1px solid var(--line);
4119 border-radius: 16px;
4120 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4121–4125

```
4121
4122 .review-form textarea {
4123   min-height: 120px;
4124   resize: vertical;
4125 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4126–4131

```
4126
4127 .review-form > div:last-child {
4128   display: flex;
4129   align-items: center;
4130   gap: 1rem;
4131 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4132–4135

```
4132
4133 .rating-picker {
4134   display: flex;
4135 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4136–4143

```
4136
4137 .rating-picker button {
4138   padding: 0 0.15rem;
4139   color: #c9c2b6;
4140   font-size: 1.6rem;
4141   background: transparent;
```

```
4142 border: 0;
4143 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4144–4147

```
4144
4145 .rating-picker button.is-active {
4146   color: var(--saffron);
4147 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 4148–4151

```
4148
4149 .text-button.danger {
4150   color: var(--clay);
4151 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4152–4158

```
4152
4153 .review-signin {
4154   margin: 2rem 0;
4155   padding: 1rem;
4156   background: var(--paper-deep);
4157   border-radius: 10px;
4158 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4159–4164

```
4159
4160 .review-list {
4161   display: grid;
4162   grid-template-columns: 1fr 1fr;
4163   gap: 1.2rem;
4164 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4165–4174

```
4165
4166 .review-card {
4167     display: grid;
4168     padding: 1.4rem;
4169     grid-template-columns: 44px 1fr;
4170     gap: 1rem;
4171     background: var(--white);
4172     border: 1px solid var(--line);
4173     border-radius: 14px;
4174 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4175–4181

```
4175
4176 .review-card-meta {
4177     display: flex;
4178     align-items: center;
4179     justify-content: space-between;
4180     gap: 1rem;
4181 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4182–4186

```
4182
4183 .review-card-meta span {
4184     color: var(--saffron);
4185     font-size: 0.8rem;
4186 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4187–4192

```
4187
4188 .review-card h3 {
4189     margin: 0.7rem 0 0.4rem;
4190     font-family: var(--serif);
4191     font-size: 1.2rem;
4192 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4193–4197

```
4193
4194 .review-card p {
4195     color: var(--ink-soft);
4196     line-height: 1.65;
4197 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4198–4201

```
4198
4199 .review-card small {
4200     color: var(--muted);
4201 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4202–4207

```
4202
4203 .community-review-grid {
4204     display: grid;
4205     grid-template-columns: repeat(2, minmax(0, 1fr));
4206     gap: 1.3rem;
4207 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4208–4215

```
4208
4209 .community-page-hero {
4210     display: flex;
4211     margin-bottom: 2.5rem;
4212     align-items: flex-end;
4213     justify-content: space-between;
4214     gap: 2rem;
4215 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4216–4222

```
4216
4217 .community-hero-actions {
4218     display: flex;
4219     flex: 0 0 auto;
4220     align-items: center;
4221     gap: 0.65rem;
4222 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4223–4234

```
4223
4224 .review-editing-notice {
4225     display: flex;
4226     margin-bottom: 1rem;
4227     padding: 0.8rem 0.9rem;
4228     align-items: center;
4229     gap: 0.5rem;
4230     color: var(--forest);
4231     font-size: 0.73rem;
4232     background: #e5eee8;
4233     border-radius: 8px;
4234 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4235–4238

```
4235
4236 .review-editing-notice a {
4237     font-weight: 850;
4238 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4239–4243

```
4239
4240 .discussion-thread {
4241     margin: 1.1rem -1.4rem -1.4rem;
4242     border-top: 1px solid var(--line);
4243 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4244–4247

```
4244
4245 .discussion-thread.is-open {
4246     background: #f8f4ec;
4247 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- This state selector gives visible feedback for interaction, selection, validation or disabled behavior.

Lines 4248–4260

```
4248
4249 .discussion-toggle {
4250     display: flex;
4251     width: 100%;
4252     padding: 0.95rem 1.4rem;
4253     align-items: center;
4254     justify-content: space-between;
4255     color: var(--forest);
4256     font-size: 0.74rem;
4257     font-weight: 850;
4258     background: transparent;
4259     border: 0;
4260 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4261–4266

```
4261
4262 .discussion-toggle span {
4263     display: flex;
4264     align-items: center;
4265     gap: 0.45rem;
4266 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4267–4271

```
4267
4268 .discussion-toggle small {
4269     color: var(--muted);
4270     font-weight: 650;
4271 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

Lines 4272–4275

```
4272
4273 .discussion-body {
4274     padding: 0 1.4rem 1.3rem;
4275 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4276–4285

```
4276
4277 .discussion-explainer {
4278     margin: 0 0 1rem !important;
4279     padding: 0.8rem;
4280     color: #52645e !important;
4281     font-size: 0.69rem !important;
4282     line-height: 1.55;
4283     background: #edf1eb;
4284     border-radius: 8px;
4285 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4286–4293

```
4286
4287 .discussion-reply {
4288     display: grid;
4289     padding: 0.9rem 0;
4290     grid-template-columns: 34px minmax(0, 1fr);
4291     gap: 0.7rem;
4292     border-bottom: 1px solid var(--line);
4293 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4294–4300

```
4294
4295 .discussion-reply .reader-avatar {
4296     width: 34px;
4297     height: 34px;
4298     flex-basis: 34px;
4299     font-size: 0.58rem;
4300 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4301–4307

```
4301
4302 .discussion-reply header {
4303     display: flex;
4304     align-items: center;
4305     justify-content: space-between;
4306     gap: 1rem;
4307 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4308–4312

```
4308
4309 .discussion-reply header strong,
4310 .discussion-reply header small {
4311     font-size: 0.68rem;
4312 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4313–4316

```
4313
4314 .discussion-reply header small {
4315     color: var(--muted);
4316 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4317–4323

```
4317
4318 .discussion-reply p {
4319     margin: 0.35rem 0 0 !important;
4320     color: #435750 !important;
4321     font-size: 0.73rem !important;
4322     line-height: 1.6;
4323 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4324–4334

```
4324
```

```
4325 .discussion-reply textarea,  
4326 .discussion-reply-form textarea {  
4327     width: 100%;  
4328     min-height: 76px;  
4329     padding: 0.7rem;  
4330     resize: vertical;  
4331     background: var(--white);  
4332     border: 1px solid var(--line);  
4333     border-radius: 8px;  
4334 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4335–4340

```
4335  
4336 .discussion-owner-actions {  
4337     display: flex;  
4338     margin-top: 0.45rem;  
4339     gap: 0.7rem;  
4340 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4341–4349

```
4341  
4342 .discussion-owner-actions button {  
4343     padding: 0;  
4344     color: var(--forest);  
4345     font-size: 0.65rem;  
4346     font-weight: 800;  
4347     background: transparent;  
4348     border: 0;  
4349 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4350–4353

```
4350  
4351 .discussion-owner-actions button.danger {  
4352     color: #a13d31;  
4353 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4354–4360

```
4354
4355 .discussion-reply-form {
4356     display: flex;
4357     margin-top: 1rem;
4358     flex-direction: column;
4359     gap: 0.55rem;
4360 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4361–4371

```
4361
4362 .discussion-reply-form button {
4363     align-self: flex-end;
4364     padding: 0.55rem 0.8rem;
4365     color: var(--white);
4366     font-size: 0.68rem;
4367     font-weight: 850;
4368     background: var(--forest);
4369     border: 0;
4370     border-radius: 7px;
4371 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4372–4376

```
4372
4373 .discussion-signin {
4374     margin: 1rem 0 0 !important;
4375     font-size: 0.72rem !important;
4376 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4377–4380

```
4377
4378 .my-reviews-page {
4379     min-height: 70vh;
4380 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4381–4388

```
4381
4382 .my-reviews-heading {
4383     display: flex;
4384     margin-bottom: 2rem;
4385     align-items: flex-end;
4386     justify-content: space-between;
4387     gap: 2rem;
4388 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4389–4394

```
4389
4390 .my-reviews-heading h1 {
4391     margin-bottom: 0.4rem;
4392     font-family: var(--serif);
4393     font-size: clamp(3rem, 6vw, 5rem);
4394 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4395–4399

```
4395
4396 .my-reviews-heading p:last-child {
4397     margin: 0;
4398     color: var(--muted);
4399 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4400–4404

```
4400
4401 .my-review-list {
4402     display: grid;
4403     gap: 1rem;
4404 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4405–4415

```
4405
```

```
4406 .my-review-item {
4407     display: grid;
4408     padding: 1.5rem;
4409     grid-template-columns: 210px minmax(0, 1fr) auto;
4410     align-items: start;
4411     gap: 1.5rem;
4412     background: var(--white);
4413     border: 1px solid var(--line);
4414     border-radius: 16px;
4415 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4416–4422

```
4416
4417 .my-review-book {
4418     display: flex;
4419     min-width: 0;
4420     align-items: center;
4421     gap: 0.8rem;
4422 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4423–4435

```
4423
4424 .my-review-book > img,
4425 .my-review-book > span {
4426     display: grid;
4427     width: 48px;
4428     height: 68px;
4429     flex: 0 0 48px;
4430     object-fit: cover;
4431     place-items: center;
4432     color: var(--ochre);
4433     background: var(--paper-deep);
4434     border-radius: 4px;
4435 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4436–4441

```
4436
4437 .my-review-book div {
4438     display: flex;
4439     min-width: 0;
4440     flex-direction: column;
4441 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4442–4451

```
4442
4443 .my-review-book a {
4444     overflow: hidden;
4445     color: var(--forest);
4446     font-family: var(--serif);
4447     font-size: 1.05rem;
4448     font-weight: 700;
4449     text-decoration: none;
4450     text-overflow: ellipsis;
4451 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4452–4456

```
4452
4453 .my-review-book small {
4454     color: var(--muted);
4455     font-size: 0.65rem;
4456 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4457–4461

```
4457
4458 .my-review-content > span {
4459     color: var(--saffron);
4460     font-size: 0.78rem;
4461 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4462–4467

```
4462
4463 .my-review-content h2 {
4464     margin: 0.35rem 0;
4465     font-family: var(--serif);
4466     font-size: 1.35rem;
4467 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

Lines 4468–4473

```
4468
4469 .my-review-content p {
4470     margin: 0;
4471     color: #53645f;
4472     line-height: 1.65;
4473 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4474–4480

```
4474
4475 .my-review-actions {
4476     display: flex;
4477     align-items: flex-end;
4478     flex-direction: column;
4479     gap: 0.65rem;
4480 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4481–4491

```
4481
4482 .my-review-actions button,
4483 .my-review-actions a {
4484     padding: 0;
4485     color: var(--forest);
4486     font-size: 0.7rem;
4487     font-weight: 800;
4488     background: transparent;
4489     border: 0;
4490     text-decoration: none;
4491 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4492–4495

```
4492
4493 .my-review-actions button.danger {
4494     color: #a13d31;
4495 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4496–4501

```
4496
4497 .my-review-edit-form {
4498     display: flex;
4499     flex-direction: column;
4500     gap: 0.65rem;
4501 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4502–4509

```
4502
4503 .my-review-edit-form input,
4504 .my-review-edit-form textarea {
4505     width: 100%;
4506     padding: 0.7rem;
4507     border: 1px solid var(--line);
4508     border-radius: 8px;
4509 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4510–4514

```
4510
4511 .my-review-edit-form textarea {
4512     min-height: 120px;
4513     resize: vertical;
4514 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4515–4520

```
4515
4516 .my-review-edit-form > div:last-child {
4517     display: flex;
4518     align-items: center;
4519     gap: 1rem;
4520 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4521–4526

```
4521
4522 @media (max-width: 991.98px) {
4523     .my-review-item {
4524         grid-template-columns: 180px minmax(0, 1fr);
4525     }
4526
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4527–4532

```
4527     .my-review-actions {
4528         grid-column: 1 / -1;
4529         align-items: center;
4530         flex-direction: row;
4531     }
4532 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4533–4540

```
4533
4534 @media (max-width: 767.98px) {
4535     .community-hero-actions,
4536     .my-reviews-heading {
4537         align-items: stretch;
4538         flex-direction: column;
4539     }
4540
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4541–4544

```
4541     .my-review-item {
4542         grid-template-columns: 1fr;
4543     }
4544
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4545–4548

```
4545 .my-review-actions {  
4546   grid-column: auto;  
4547 }  
4548 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4549–4552

```
4549  
4550 .community-page-hero .inner-page-heading {  
4551   margin-bottom: 0;  
4552 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4553–4556

```
4553  
4554 .community-page-hero .btn {  
4555   flex: 0 0 auto;  
4556 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4557–4565

```
4557  
4558 .community-summary {  
4559   display: grid;  
4560   margin-bottom: 1.5rem;  
4561   grid-template-columns: repeat(4, 1fr);  
4562   overflow: hidden;  
4563   background: var(--forest);  
4564   border-radius: 16px;  
4565 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4566–4574

```
4566
4567 .community-summary > div {
4568     display: flex;
4569     min-height: 110px;
4570     padding: 1.4rem 1.6rem;
4571     flex-direction: column;
4572     justify-content: center;
4573     border-right: 1px solid rgba(255, 255, 255, 0.16);
4574 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4575–4578

```
4575
4576 .community-summary > div:last-child {
4577     border-right: 0;
4578 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4579–4585

```
4579
4580 .community-summary strong {
4581     color: var(--white);
4582     font-family: var(--serif);
4583     font-size: 2rem;
4584     font-weight: 500;
4585 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4586–4590

```
4586
4587 .community-summary span {
4588     color: rgba(255, 255, 255, 0.68);
4589     font-size: 0.82rem;
4590 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4591–4598

```
4591
4592 .community-toolbar {
```



```
4593 display: flex;
4594 margin-bottom: 1.5rem;
4595 align-items: center;
4596 justify-content: space-between;
4597 gap: 1rem;
4598 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4599–4609

```
4599
4600 .community-toolbar > label:first-child {
4601   display: flex;
4602   width: min(100%, 620px);
4603   padding: 0 1rem;
4604   align-items: center;
4605   gap: 0.7rem;
4606   background: var(--white);
4607   border: 1px solid var(--line);
4608   border-radius: 12px;
4609 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4610–4617

```
4610
4611 .community-toolbar input,
4612 .community-toolbar select {
4613   min-height: 52px;
4614   background: transparent;
4615   border: 0;
4616   outline: 0;
4617 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4618–4621

```
4618
4619 .community-toolbar input {
4620   width: 100%;
4621 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4622–4632

```
4622
4623 .community-sort {
4624   display: flex;
4625   padding: 0 0.5rem 0 1rem;
4626   align-items: center;
4627   gap: 0.5rem;
4628   white-space: nowrap;
4629   background: var(--white);
4630   border: 1px solid var(--line);
4631   border-radius: 12px;
4632 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4633–4637

```
4633
4634 .community-sort span {
4635   color: var(--muted);
4636   font-size: 0.78rem;
4637 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4638–4647

```
4638
4639 .community-review-card {
4640   display: flex;
4641   min-height: 430px;
4642   padding: 1.4rem;
4643   flex-direction: column;
4644   background: var(--white);
4645   border: 1px solid var(--line);
4646   border-radius: 16px;
4647 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4648–4658

```
4648
4649 .community-review-book {
4650   display: grid;
4651   margin: -1.4rem -1.4rem 1.4rem;
4652   padding: 1.2rem;
4653   grid-template-columns: 62px minmax(0, 1fr);
4654   align-items: center;
4655   gap: 1rem;
4656   background: var(--paper-deep);
4657   border-radius: 16px 16px 0 0;
```

```
4658 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4659–4667

```
4659
4660 .community-book-cover {
4661     width: 62px;
4662     height: 88px;
4663     object-fit: cover;
4664     background: var(--cream);
4665     border-radius: 5px;
4666     box-shadow: 0 8px 18px rgba(16, 51, 43, 0.12);
4667 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4668–4674

```
4668
4669 .community-book-cover-placeholder {
4670     display: grid;
4671     place-items: center;
4672     color: var(--saffron);
4673     font-size: 1.3rem;
4674 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4675–4681

```
4675
4676 .community-review-book > div {
4677     display: flex;
4678     min-width: 0;
4679     flex-direction: column;
4680     align-items: flex-start;
4681 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4682–4685

```
4682
```

```
4683 .community-review-book .eyebrow {
4684     margin-bottom: 0.35rem;
4685 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4686–4693

```
4686
4687 .community-review-book a {
4688     color: var(--ink);
4689     font-family: var(--serif);
4690     font-size: 1.2rem;
4691     line-height: 1.2;
4692     text-decoration: none;
4693 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4694–4699

```
4694
4695 .community-review-book span {
4696     margin-top: 0.4rem;
4697     color: var(--muted);
4698     font-size: 0.75rem;
4699 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4700–4706

```
4700
4701 .community-review-author {
4702     display: flex;
4703     margin-bottom: 1.25rem;
4704     align-items: center;
4705     gap: 0.8rem;
4706 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4707–4711

```
4707
4708 .community-review-author > div {
4709     display: flex;
4710     flex-direction: column;
4711 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4712–4716

```
4712
4713 .community-review-author > div span {
4714     color: var(--saffron);
4715     font-size: 0.75rem;
4716 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4717–4721

```
4717
4718 .community-review-card h2 {
4719     font-family: var(--serif);
4720     font-size: 1.5rem;
4721 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4722–4726

```
4722
4723 .community-review-card > p:not(.eyebrow) {
4724     color: var(--muted);
4725     line-height: 1.7;
4726 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4727–4737

```
4727
4728 .community-review-link {
4729     display: inline-flex;
4730     margin-top: auto;
4731     align-items: center;
4732     gap: 0.45rem;
4733     color: var(--forest);
4734     font-size: 0.82rem;
4735     font-weight: 800;
4736     text-decoration: none;
4737 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout,

typography, colour, spacing, state or responsive behavior.

- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4738–4746

```
4738
4739 .community-loading {
4740     display: grid;
4741     min-height: 300px;
4742     place-items: center;
4743     align-content: center;
4744     gap: 1rem;
4745     color: var(--muted);
4746 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4747–4751

```
4747
4748 .community-error-state,
4749 .community-no-results {
4750     min-height: 320px;
4751 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4752–4756

```
4752
4753 .community-empty-card {
4754     grid-column: 1 / -1;
4755     min-height: 280px;
4756 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4757–4762

```
4757
4758 @media (max-width: 991.98px) {
4759     .personal-upload-form {
4760         grid-template-columns: 1fr 1fr;
4761     }
4762 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4763–4766

```
4763 .auth-page {  
4764     grid-template-columns: 1fr;  
4765 }  
4766
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4767–4770

```
4767 .auth-story {  
4768     min-height: 420px;  
4769 }  
4770
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4771–4775

```
4771 .cart-layout,  
4772 .checkout-layout {  
4773     grid-template-columns: 1fr;  
4774 }  
4775
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4776–4779

```
4776 .order-summary {  
4777     position: static;  
4778 }  
4779
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4780–4783

```
4780 .community-review-grid {  
4781     grid-template-columns: 1fr 1fr;  
4782 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4784–4788

```

4784 .community-page-hero {
4785     align-items: flex-start;
4786     flex-direction: column;
4787 }
4788

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4789–4792

```

4789 .community-summary {
4790     grid-template-columns: 1fr 1fr;
4791 }
4792

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4793–4796

```

4793 .community-summary > div:nth-child(2) {
4794     border-right: 0;
4795 }
4796

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4797–4800

```

4797 .community-summary > div:nth-child(-n + 2) {
4798     border-bottom: 1px solid rgba(255, 255, 255, 0.16);
4799 }
4800 }

```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4801–4806

```
4801
4802 @media (max-width: 767.98px) {
4803     .inner-page {
4804         padding: 3.5rem 0 5rem;
4805     }
4806
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.

Lines 4807–4810

```
4807     .cart-item {
4808         grid-template-columns: 68px 1fr;
4809     }
4810
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4811–4816

```
4811     .quantity-control,
4812     .cart-item-total {
4813         grid-column: 2;
4814         justify-self: start;
4815     }
4816
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4817–4820

```
4817     .cart-item-total {
4818         align-items: flex-start;
4819     }
4820
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4821–4824

```
4821     .order-row {
```

```
4822     grid-template-columns: 1fr auto;
4823   }
4824
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4825–4828

```
4825   .order-row > .status-pill {
4826     grid-row: 2;
4827   }
4828
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4829–4832

```
4829   .order-row > i {
4830     display: none;
4831   }
4832
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4833–4837

```
4833   .order-detail-heading {
4834     align-items: flex-start;
4835     flex-direction: column;
4836   }
4837
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4838–4843

```
4838   .order-detail-grid,
4839   .review-list,
4840   .community-review-grid {
4841     grid-template-columns: 1fr;
4842   }
4843
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4844–4847

```
4844 .order-journey-card {  
4845     padding: 1.4rem;  
4846 }  
4847
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4848–4852

```
4848 .delivery-timeline {  
4849     grid-template-columns: 1fr;  
4850     gap: 0.7rem;  
4851 }  
4852
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4853–4858

```
4853 .delivery-timeline li {  
4854     display: grid;  
4855     grid-template-columns: 34px 1fr;  
4856     align-items: center;  
4857 }  
4858
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4859–4866

```
4859 .delivery-timeline li::after {  
4860     top: 34px;  
4861     bottom: -0.7rem;  
4862     left: 16px;  
4863     width: 1px;  
4864     height: auto;  
4865 }  
4866
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4867–4870

```
4867 .delivery-timeline li > div {
4868     padding: 0;
4869 }
4870
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4871–4875

```
4871 .community-toolbar {
4872     align-items: stretch;
4873     flex-direction: column;
4874 }
4875
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4876–4879

```
4876 .community-toolbar > label:first-child {
4877     width: 100%;
4878 }
4879
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4880–4883

```
4880 .community-sort {
4881     justify-content: space-between;
4882 }
4883 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4884–4889

```
4884
4885 @media (max-width: 575.98px) {
4886     .personal-upload-form {
4887         grid-template-columns: 1fr;
4888     }
4889 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- The media query changes the layout at a viewport threshold so the same feature remains usable on smaller screens.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4890–4893

```
4890 .auth-panel {  
4891   padding: 3rem 1.3rem;  
4892 }  
4893
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4894–4897

```
4894 .auth-story {  
4895   min-height: 360px;  
4896 }  
4897
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4898–4901

```
4898 .auth-story h1 {  
4899   font-size: 3rem;  
4900 }  
4901
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4902–4905

```
4902 .form-grid {  
4903   grid-template-columns: 1fr;  
4904 }  
4905
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4906–4909

```
4906 .form-grid .span-two {
```

```
4907     grid-column: auto;
4908   }
4909
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4910–4915

```
4910   .checkout-form,
4911   .order-detail-card,
4912   .order-detail-grid > div {
4913     padding: 1.3rem;
4914   }
4915
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4916–4919

```
4916   .review-card {
4917     grid-template-columns: 1fr;
4918   }
4919
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4920–4925

```
4920   .review-heading {
4921     align-items: flex-start;
4922     flex-direction: column;
4923     gap: 1rem;
4924   }
4925
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4926–4929

```
4926   .community-summary {
4927     grid-template-columns: 1fr;
4928   }
4929
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

Lines 4930–4935

```
4930 .community-summary > div {  
4931   min-height: 86px;  
4932   border-right: 0;  
4933   border-bottom: 1px solid rgba(255, 255, 255, 0.16);  
4934 }  
4935
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4936–4939

```
4936 .community-summary > div:last-child {  
4937   border-bottom: 0;  
4938 }  
4939
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.

Lines 4940–4943

```
4940 .review-summary {  
4941   align-items: flex-start;  
4942 }  
4943 }
```

How to read this block

- The selector chooses which rendered elements receive this rule. Properties inside the braces control layout, typography, colour, spacing, state or responsive behavior.
- Grid/Flexbox properties define alignment and responsive distribution without hard-coded element positions.

frontend/src/pages/AdminOrdersPage.jsx

File 86 of 99 · Added or modified · Frontend page

Purpose: Implements administrator order inspection and validated fulfilment actions.

Current file length: 184 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 import { useEffect, useMemo, useState } from 'react'
2 import { Link } from 'react-router-dom'
3 import { getApiErrorMessage } from '../services/api'
4 import { getAdminOrder, getAdminOrders, updateAdminOrderStatus } from '../services/adminOrderService'
5 import { formatPrice } from '../utils/bookPresentation'
6
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 7–14

```
7 const STATUS_FILTERS = ['ALL', 'PLACED', 'CONFIRMED', 'PROCESSING', 'SHIPPED', 'DELIVERED', 'CANCELLED']
8 const NEXT_ACTION = {
9   PLACED: { status: 'CONFIRMED', label: 'Confirm order', icon: 'bi-check2-circle' },
10  CONFIRMED: { status: 'PROCESSING', label: 'Start processing', icon: 'bi-box-seam' },
11  PROCESSING: { status: 'SHIPPED', label: 'Mark as shipped', icon: 'bi-truck' },
12  SHIPPED: { status: 'DELIVERED', label: 'Mark as delivered', icon: 'bi-house-check' },
13 }
14
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 15–18

```
15 function readable(value) {
16   return String(value ?? '').toLowerCase().replaceAll('_', ' ').replace(/^./, (letter) => letter.toUpperCase())
17 }
18
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 19–26

```
19 function formatDate(value, includeTime = false) {
20   const date = new Date(value)
21   if (Number.isNaN(date.getTime())) return 'Date unavailable'
```



```

22     return new Intl.DateTimeFormat('en-IN', includeTime
23       ? { dateStyle: 'medium', timeStyle: 'short' }
24       : { dateStyle: 'medium' }).format(date)
25   }
26

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 27–38

```

27 function AdminOrdersPage() {
28   const [orders, setOrders] = useState([])
29   const [selectedId, setSelectedId] = useState(null)
30   const [detail, setDetail] = useState(null)
31   const [filter, setFilter] = useState('ALL')
32   const [query, setQuery] = useState('')
33   const [loading, setLoading] = useState(true)
34   const [detailLoading, setDetailLoading] = useState(false)
35   const [busy, setBusy] = useState(false)
36   const [error, setError] = useState('')
37   const [notice, setNotice] = useState('')
38

```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 39–50

```

39 async function loadOrders() {
40   setLoading(true)
41   setError('')
42   try {
43     setOrders(await getAdminOrders())
44   } catch (requestError) {
45     setError(getApiErrorMessage(requestError, 'Orders could not be loaded.'))
46   } finally {
47     setLoading(false)
48   }
49 }
50

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 51–59

```

51 useEffect(() => {
52   let active = true
53   getAdminOrders()

```

```

54     .then((data) => { if (active) setOrders(data) })
55     .catch((requestError) => { if (active) setError(getApiErrorMessage(requestError, 'Orders could not be
loaded.')) })
56     .finally(() => { if (active) setLoading(false) })
57     return () => { active = false }
58   }, [])
59

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 60–69

```

60  const visibleOrders = useMemo(() => {
61    const needle = query.trim().toLowerCase()
62    return orders.filter((order) => {
63      if (filter !== 'ALL' && order.orderStatus !== filter) return false
64      if (!needle) return true
65      return [order.orderId, order.customerName, order.customerEmail]
66        .some((value) => String(value ?? '').toLowerCase().includes(needle))
67    })
68  }, [orders, filter, query])
69

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 70–76

```

70  const summary = useMemo(() => ({
71    awaiting: orders.filter((order) => order.orderStatus === 'PLACED').length,
72    active: orders.filter((order) => ['CONFIRMED', 'PROCESSING', 'SHIPPED'].includes(order.orderStatus)).length,
73    delivered: orders.filter((order) => order.orderStatus === 'DELIVERED').length,
74    total: orders.length,
75  })), [orders])
76

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 77–90

```
77  async function selectOrder(orderId) {
78    setSelectedId(orderId)
79    setDetail(null)
80    setDetailLoading(true)
81    setError('')
82    try {
83      setDetail(await getAdminOrder(orderId))
84    } catch (requestError) {
85      setError(getApiErrorMessage(requestError, 'Order details could not be loaded.'))
86    } finally {
87      setDetailLoading(false)
88    }
89  }
90
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 91–104

```
91  async function changeStatus(status) {
92    if (!selectedId) return
93    if (status === 'CANCELLED' && !window.confirm('Cancel this order and restore its reserved inventory?')) return
94    if (status === 'DELIVERED' && !window.confirm('Confirm that this order was delivered? Cash-on-delivery payment
will be marked paid.')) return
95    setBusy(true)
96    setError('')
97    setNotice('')
98    try {
99      const updated = await updateAdminOrderStatus(selectedId, status)
100      setDetail(updated)
101      setOrders((current) => current.map((order) => order.orderId === selectedId
102        ? { ...order, orderStatus: updated.order.orderStatus, updatedAt: new Date().toISOString() }
103        : order))
104      setNotice(`Order #${selectedId} is now ${readable(updated.order.orderStatus)}.`)
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 105–111

```
105     } catch (requestError) {
106         setError(getApiErrorMessage(requestError, 'The order status could not be updated.'))
107     } finally {
108         setBusy(false)
109     }
110 }
111
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 112–115

```
112 const selectedOrder = detail?.order
113 const nextAction = NEXT_ACTION[selectedOrder?.orderStatus]
114 const canCancel = ['PLACED', 'CONFIRMED', 'PROCESSING'].includes(selectedOrder?.orderStatus)
115
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 116–124

```
116 return (
117     <div className="admin-orders-page">
118         <section className="catalogue-admin-hero admin-orders-hero">
119             <div className="container-xl">
120                 <div><p className="eyebrow">Order operations</p><h1>Move every order forward.</h1><p>Confirm new
purchases, prepare dispatches, record delivery, and keep readers informed.</p></div>
121                 <Link className="btn btn-light" to="/admin/books"><i className="bi bi-journals" /> Catalogue
operations</Link>
122             </div>
123         </section>
124     </div>
)
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 125–132

```
125     <section className="container-xl admin-orders-workspace">
126       <div className="catalogue-admin-summary">
127         <div><strong>{summary.awaiting}</strong><span>Awaiting confirmation</span></div>
128         <div><strong>{summary.active}</strong><span>In fulfilment</span></div>
129         <div><strong>{summary.delivered}</strong><span>Delivered</span></div>
130         <div><strong>{summary.total}</strong><span>Total orders</span></div>
131       </div>
132     </section>
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 133–140

```
133     <div className="admin-orders-toolbar">
134       <label><i className="bi bi-search" /><input type="search" value={query} onChange={(event) =>
setQuery(event.target.value)} placeholder="Search order, customer, or email" /></label>
135       <button type="button" onClick={loadOrders} disabled={loading}><i className="bi bi-arrow-clockwise" />
Refresh</button>
136     </div>
137     <div className="admin-order-filters" role="group" aria-label="Filter orders">
138       {STATUS_FILTERS.map((status) => <button className={filter === status ? 'is-active' : ''} key={status}
type="button" onClick={() => setFilter(status)}>{readable(status)}</button>)}
139     </div>
140   </div>
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 141–144

```
141     {error && <div className="admin-feedback admin-feedback-error" role="alert">{error}</div>}
142     {notice && <div className="admin-feedback admin-feedback-success" role="status">{notice}</div>}
143     {loading && <div className="page-loading"><span className="spinner-border" /> Loading orders...</div>}
144   </div>
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 145–157

```
145      {!loading && (
146        <div className="admin-order-layout">
147          <div className="admin-order-list">
148            {visibleOrders.map((order) => (
149              <button className={`admin-order-row${selectedId === order.orderId ? ' is-selected' : ''}`}
type="button" key={order.orderId} onClick={() => selectOrder(order.orderId)}>
150                <div><strong>Order
151                <div><span className={`admin-order-status status-${order.orderStatus.toLowerCase()}`}
>{readable(order.orderStatus)}</span><strong>{formatPrice(order.totalAmount)}</strong><small>{formatDate(order.placedAt)}
</small></div>
152                <i className="bi bi-chevron-right" />
153              </button>
154            )}}
155            {!visibleOrders.length && <div className="catalogue-admin-empty"><i className="bi bi-receipt-
cutoff" /><h2>No matching orders</h2><p>Try another status or search.</p></div>}
156          </div>
157        )}
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 158–171

```
158      <aside className="admin-order-detail">
159        {selectedId && <div className="admin-order-placeholder"><i className="bi bi-box-seam" /><h2>Select an
order</h2><p>Customer, delivery, payment, and confirmation controls will appear here.</p></div>}
160        {detailLoading && <div className="page-loading"><span className="spinner-border" /> Opening
order...</div>}
161        {!detailLoading && selectedOrder && (
162          <>
163            <header><div><p>Order
164            {selectedOrder.orderId}</p><h2>{detail.customerName}</h2><span>{detail.customerEmail}</span></div><span
className={` admin-order-status status-${selectedOrder.orderStatus.toLowerCase()}`}>{readable(selectedOrder.orderStatus)}
</span></header>
165            <div className="admin-order-primary-action">
166              {nextAction ? <button disabled={busy} type="button" onClick={() =>
changeStatus(nextAction.status)}><i className={`bi ${nextAction.icon}`} /> {busy ? 'Updating...' :
nextAction.label}</button> : <p><i className={`bi ${selectedOrder.orderStatus === 'DELIVERED' ? 'bi-check-circle-fill' :
'bi-x-circle-fill'}`} /> This order is {readable(selectedOrder.orderStatus)}</p>}
167              {canCancel && <button className="cancel-order-button" disabled={busy} type="button" onClick={() =>
changeStatus('CANCELLED')}>Cancel order</button>}
168            </div>
169            <section className="admin-order-items"><h3>Books</h3>{selectedOrder.items ?? []}.map((item) =>
<div key={item.orderItemId}><span>{item.coverImageUrl ? <img src={item.coverImageUrl} alt="" /> : <i className="bi bi-
book" /></span><div><strong>{item.bookTitle}</strong><small>{readable(item.bookFormat)} · Qty
{item.quantity}</small></div><strong>{formatPrice(item.subtotal)}</strong></div>)}</section>
170            <div className="admin-order-facts">
171              <section><h3>Deliver to</h3><p>{selectedOrder.shippingAddress?.recipientName}<br
/>{selectedOrder.shippingAddress?.addressLine1}<br />{selectedOrder.shippingAddress?.addressLine2 &&
<>{selectedOrder.shippingAddress.addressLine2}<br /></>}{selectedOrder.shippingAddress?.city},
{selectedOrder.shippingAddress?.state} {selectedOrder.shippingAddress?.postalCode}<br
/>{selectedOrder.shippingAddress?.country}<br />{selectedOrder.shippingAddress?.phone}</p></section>
172            <section><h3>Payment</h3><p>{readable(selectedOrder.paymentMethod)}<br /><span className="admin-
```

```
order-status">{readable(selectedOrder.paymentStatus)}</span></p><strong>Total
{formatPrice(selectedOrder.totalAmount)}</strong><small>Placed {formatDate(selectedOrder.placedAt,
true)}</small></section>
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 172–182

```
172             </div>
173         </>
174     )}
175 </aside>
176 </div>
177 )}
178 </section>
179 </div>
180 )
181 }
182
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 183–184

```
183 export default AdminOrdersPage
184
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/BookDetailsPage.jsx

File 87 of 99 · Added or modified · Frontend page

Purpose: Renders and coordinates the BookDetailsPage React interface.

Current file length: 269 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–14

```
1 import { useEffect, useState } from 'react'
2 import { Link, useLocation, useNavigate, useParams } from 'react-router-dom'
3 import { getBookById } from '../services/bookService'
4 import {
5   formatPrice,
6   getAuthors,
7   getAvailability,
8   getBookPrice,
9   getCoverTone,
10  getPrimaryEdition,
11  getTitleMonogram,
12 } from '../utils/bookPresentation'
13 import useAuth from '../hooks/useAuth'
14 import useReaderData from '../hooks/useReaderData'
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 15–17

```
15 import { getApiErrorMessage } from '../services/api'
16 import ReviewSection from '../components/ReviewSection'
17
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 18–30

```
18 function BookDetailsPage() {
19   const { bookId } = useParams()
20   const [book, setBook] = useState(null)
21   const [loading, setLoading] = useState(true)
22   const [error, setError] = useState('')
23   const { user } = useAuth()
24   const { wishlistBookIds, toggleWishlist, addCartItem } = useReaderData()
25   const navigate = useNavigate()
26   const location = useLocation()
27   const [selectedEditionId, setSelectedEditionId] = useState(null)
28   const [actionBusy, setActionBusy] = useState(false)
29   const [actionMessage, setActionMessage] = useState('')
30 }
```


How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 31–33

```
31  useEffect(() => {
32    let active = true
33  }
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 34–47

```
34  async function loadBook() {
35    try {
36      setLoading(true)
37      setError('')
38      const data = await getBookById(bookId)
39      if (active) {
40        setBook(data)
41        setSelectedEditionId(getPrimaryEdition(data)?.id ?? null)
42      }
43    } catch (requestError) {
44      console.error(requestError)
45      if (active) setError('This book could not be loaded. It may have been removed or the backend may be
unavailable.')
46    } finally {
47      if (active) setLoading(false)
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 48–50

```
48    }
49  }
50
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 51–53

```
51  loadBook()
52  window.scrollTo({ top: 0, behavior: 'auto' })
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 54–58

```
54     return () => {
55         active = false
56     }
57 }, [bookId])
58
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 59–71

```
59     if (loading) {
60         return (
61             <section className="detail-page">
62                 <div className="container-xl detail-loading" aria-label="Loading book details">
63                     <div className="detail-cover-skeleton" />
64                     <div className="detail-copy-skeleton">
65                         <span /><span /><span /><span />
66                     </div>
67                 </div>
68             </section>
69         )
70     }
71
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 72–85

```
72     if (error || !book) {
73         return (
74             <section className="detail-page">
75                 <div className="container-xl">
76                     <div className="catalogue-message catalogue-error">
77                         <span><i className="bi bi-journal-x" /></span>
78                     <div>
79                         <h1>We could not find this book</h1>
80                         <p>{error}</p>

```

```

81         <Link to="/">Return to the catalogue</Link>
82     </div>
83 </div>
84 </div>
85 </section>

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 86–88

```

86     )
87   }
88

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 89–101

```

89   const primaryEdition = getPrimaryEdition(book)
90   const physicalEditions = book.editions?.filter(
91     (edition) => edition.format === 'PAPERBACK' || edition.format === 'HARDCOVER',
92   ) ?? []
93   const selectedEdition = book.editions?.find((edition) => edition.id === selectedEditionId) ?? primaryEdition
94   const availability = getAvailability(book)
95   const selectedAvailable = Boolean(
96     selectedEdition?.inventory?.active
97     && selectedEdition.inventory.stockQuantity > 0
98     && selectedEdition.inventory.availabilityStatus === 'IN_STOCK',
99   )
100   const saved = wishlistBookIds.has(book.id)
101

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 102–107

```

102   function requireReader() {
103     if (user) return true
104     navigate('/login', { state: { from: location } })
105     return false
106   }
107

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 108–120

```
108  async function handleWishlist() {
109    if (!requireReader()) return
110    setActionBusy(true)
111    try {
112      await toggleWishlist(book)
113      setActionMessage(saved ? 'Removed from your wishlist.' : 'Saved to your wishlist.')
114    } catch (requestError) {
115      setActionMessage(getApiErrorMessage(requestError, 'Wishlist could not be updated.'))
116    } finally {
117      setActionBusy(false)
118    }
119  }
120
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 121–133

```
121  async function handleCart() {
122    if (!requireReader() || !selectedEdition?.id) return
123    setActionBusy(true)
124    try {
125      await addCartItem(selectedEdition.id)
126      setActionMessage('Added to your cart.')
127    } catch (requestError) {
128      setActionMessage(getApiErrorMessage(requestError, 'This edition could not be added to your cart.'))
129    } finally {
130      setActionBusy(false)
131    }
132  }
133
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 134–144

```
134     return (  
135         <section className="detail-page">  
136             <div className="container-xl">  
137                 <nav className="detail-breadcrumb" aria-label="Breadcrumb">  
138                     <Link to="/">Home</Link>  
139                     <i className="bi bi-chevron-right" />  
140                     <Link to="/#catalogue">Catalogue</Link>  
141                     <i className="bi bi-chevron-right" />  
142                     <span>{book.title}</span>  
143                 </nav>  
144     )
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 145–158

```
145         <div className="detail-layout">  
146             <div className="detail-cover-column">  
147                 <div className="detail-cover-frame">  
148                     {book.coverImageUrl ? (  
149                         <img src={book.coverImageUrl} alt={`Cover of ${book.title}`} />  
150                     ) : (  
151                         <div className={`fallback-book-cover detail-fallback-cover cover-tone-${getCoverTone(book)}`}>  
152                             <span className="fallback-book-mark">अ</span>  
153                             <strong>{getTitleMonogram(book.title)}</strong>  
154                             <small>{book.title}</small>  
155                         </div>  
156                     )}  
157                 </div>  
158                 <p className="detail-cover-note"><i className="bi bi-shield-check" /> Secure checkout · Reader-first  
support</p>
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 159–167

```
159         </div>  
160     </div>  
161     <div className="detail-content">  
162         <div className="detail-kicker-row">  
163             <span className={`availability-badge availability-${availability.tone}`}>{availability.label}</span>  
164             {book.languageCode && <span>{book.languageCode.toUpperCase()}</span>}  
165             {primaryEdition?.format && <span>{primaryEdition.format}</span>}  
166         </div>  
167     </div>
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props

carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 168–171

```
168      <h1>{book.title}</h1>
169      {book.subtitle} && <p className="detail-subtitle">{book.subtitle}</p>
170      <p className="detail-author">by <strong>{getAuthors(book)}</strong></p>
171
```

How to read this block

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 172–177

```
172      <div className="detail-rating-row">
173        <span className="stars">★★★★★</span>
174        <strong>Reader community</strong>
175        <a href="#reviews">See reviews</a>
176      </div>
177
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 178–182

```
178      <div className="detail-price-row">
179        <strong>{formatPrice(selectedEdition?.inventory?.price ?? getBookPrice(book))}</strong>
180        <span>Inclusive of applicable taxes</span>
181      </div>
182
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 183–196

```
183      <div className="detail-actions">
184        <button
185          className="btn btn-ink detail-cart-button"
186          type="button"
187          disabled={actionBusy || !selectedAvailable}
188          onClick={handleCart}

```

```

189         >
190         <i className="bi bi-bag-plus" />
191         {actionBusy ? 'Please wait...' : 'Add to cart'}
192     </button>
193     <button
194         className={`detail-save-button${saved ? ' is-saved' : ''}`}
195         type="button"
196         aria-pressed={saved}

```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 197–205

```

197         disabled={actionBusy}
198         onClick={handleWishlist}
199     >
200     <i className={`bi ${saved ? 'bi-heart-fill' : 'bi-heart'}`} />
201     {saved ? 'Saved' : 'Save for later'}
202 </button>
203 </div>
204 {actionMessage && <p className="detail-action-message" role="status">{actionMessage}</p>}
205

```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 206–210

```

206     <div className="detail-description">
207         <p className="eyebrow">About this book</p>
208         <p>{book.description || 'A full description for this book will be added by the Akshara editorial team
soon.'}</p>
209     </div>
210

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 211–218

```

211     {book.categories?.length > 0 && (
212         <div className="detail-categories">
213             {book.categories.map((category) => <span key={category.id}>{category.name}</span>)}
214         </div>
215     )}

```

```
216         </div>
217     </div>
218
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 219–226

```
219     <div className="edition-section">
220       <div className="section-heading">
221         <div>
222           <p className="eyebrow">Choose your edition</p>
223           <h2>Available formats</h2>
224         </div>
225       </div>
226
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 227–240

```
227     {physicalEditions.length ? (
228       <div className="edition-grid">
229         {physicalEditions.map((edition) => (
230           <label
231             className={`edition-card edition-choice${selectedEdition?.id === edition.id ? ' is-selected' : ''}
232             key={edition.id}
233           >
234             <div>
235               <span className="edition-icon"><i className="bi bi-book" /></span>
236               <div>
237                 <h3>{edition.editionName || edition.format || 'Book edition'}</h3>
238                 <p>{edition.publisher?.name || 'Publisher unavailable'}</p>
239               </div>
240             </div>

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 241–254

```
241         <dl>
242             <div><dt>Format</dt><dd>{edition.format || '-'}</dd></div>
243             <div><dt>Pages</dt><dd>{edition.pageCount || '-'}</dd></div>
244             <div><dt>ISBN</dt><dd>{edition.isbn13 || edition.isbn10 || '-'}</dd></div>
245             <div><dt>Price</dt><dd>{formatPrice(edition.inventory?.price)}</dd></div>
246         </dl>
247         <input
248             className="edition-radio"
249             type="radio"
250             name="edition"
251             value={edition.id}
252             checked={selectedEdition?.id === edition.id}
253             onChange={() => setSelectedEditionId(edition.id)}
254         />
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 255–267

```
255         </label>
256     )}}
257 </div>
258 ) : (
259     <p className="edition-empty">No physical edition is currently available. Digital editions remain
unavailable until licensed reader access is configured.</p>
260 )}
261 </div>
262 <ReviewSection bookId={book.id} />
263 </div>
264 </section>
265 )
266 }
267
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 268–269

```
268 export default BookDetailsPage
269
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/BookImportPage.jsx

File 88 of 99 · Added or modified · Frontend page

Purpose: Implements the admin hybrid workflow: search, preview an edition, prefill metadata, add commerce data, validate and import; it also provides manual entry.

Current file length: 314 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–11

```
1 import { useMemo, useState } from 'react'
2 import {
3   AVAILABILITY_OPTIONS,
4   BOOK_FORMATS,
5   BOOK_SOURCES,
6   createManualBook,
7   importBook,
8   searchExternalBooks,
9 } from '../services/adminBookService'
10 import { getApiErrorMessage } from '../services/api'
11
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 12–25

```
12 const EMPTY_FORM = {
13   title: '',
14   subtitle: '',
15   description: '',
16   authors: '',
17   publisher: '',
18   languageCode: 'en',
19   pageCount: '',
20   categories: '',
21   coverImageUrl: '',
22   publicationDate: '',
23   editionName: '',
24   isbn10: '',
25   isbn13: '',
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 26–34

```
26   format: 'PAPERBACK',
27   price: '',
28   stockQuantity: '0',
29   availabilityStatus: 'IN_STOCK',
30   sku: '',
31   active: true,
```

```
32   featured: false,
33 }
34
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 35–38

```
35 function cleanIsbn(value = '') {
36   return String(value).replace(/[-\s]/g, '')
37 }
38
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 39–43

```
39 function dateInputValue(value = '') {
40   const match = String(value).match(/^d{4}-d{2}-d{2}/)
41   return match?.[0] ?? ''
42 }
43
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 44–47

```
44 function commaList(value) {
45   return value.split(',').map((item) => item.trim()).filter(Boolean)
46 }
47
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 48–51

```
48 function titleFor(result) {
49   return result.subtitle ? `${result.title}: ${result.subtitle}` : result.title
50 }
51
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 52–65

```
52 function BookMetadataForm({ form, mode, onChange }) {
53   return (
54     <div className="admin-form-sections">
55       <section className="admin-form-section">
56         <div className="admin-section-title"><span>01</span><div><h2>Book metadata</h2><p>{mode === 'import' ?
'Review and correct the metadata before importing.' : 'Enter the bibliographic details for this title.'}</p></div></div>
57         <div className="admin-field-grid">
58           <label className="span-two">Title <input name="title" value={form.title} onChange={onChange}
maxLength="255" required /></label>
59           <label className="span-two">Subtitle <input name="subtitle" value={form.subtitle} onChange={onChange}
maxLength="255" /></label>
60           <label className="span-two">Description <textarea name="description" value={form.description}
onChange={onChange} maxLength="20000" rows="5" /></label>
61           <label>Authors <input name="authors" value={form.authors} onChange={onChange} placeholder="James Clear, ..."
required /></label>
62           <label>Publisher <input name="publisher" value={form.publisher} onChange={onChange} /></label>
63           <label className="span-two">Categories <input name="categories" value={form.categories}
onChange={onChange} placeholder="Self-help, Psychology" required /></label>
64           <label>Language code <input name="languageCode" value={form.languageCode} onChange={onChange}
maxLength="10" placeholder="en" /></label>
65           <label>Cover image URL <input name="coverImageUrl" type="url" value={form.coverImageUrl}
onChange={onChange} placeholder="https://..." /></label>
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 66–68

```
66     </div>
67   </section>
68 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 69–80

```
69     <section className="admin-form-section">
70       <div className="admin-section-title"><span>02</span><div><h2>Edition</h2><p>ISBN identifies this exact
edition and is checked for duplicates by the backend.</p></div></div>
71       <div className="admin-field-grid admin-field-grid-three">
72         <label>ISBN-13 <input name="isbn13" value={form.isbn13} onChange={onChange} inputMode="numeric"
pattern="[0-9]{13}" maxLength="13" /></label>
73         <label>ISBN-10 <input name="isbn10" value={form.isbn10} onChange={onChange} pattern="[0-9]{9}[0-9Xx]"
maxLength="10" /></label>
74         <label>Edition name <input name="editionName" value={form.editionName} onChange={onChange} maxLength="100"
placeholder="First edition" /></label>
75         <label>Publication date <input name="publicationDate" type="date" max={new Date().toISOString().slice(0,
10)} value={form.publicationDate} onChange={onChange} /></label>
76         <label>Page count <input name="pageCount" type="number" min="1" value={form.pageCount} onChange={onChange}
/></label>
77         <label>Format <select name="format" value={form.format} onChange={onChange}>{BOOK_FORMATS.map((option) =>
```

```

<option key={option.value} value={option.value}>{option.label}</option>}}</select><small>Digital editions require
licensed-file delivery and are not enabled yet.</small></label>
78     </div>
79     </section>
80

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 81–94

```

81     <section className="admin-form-section admin-owned-section">
82       <div className="admin-section-title"><span>03</span><div><h2>Akshara catalogue details</h2><p>Pricing,
inventory, visibility, and curation always belong to Akshara.</p></div></div>
83       <div className="admin-field-grid admin-field-grid-three">
84         <label>Selling price (₹) <input name="price" type="number" min="0" step="0.01" value={form.price}
onChange={onChange} required /></label>
85         <label>Stock quantity <input name="stockQuantity" type="number" min="0" step="1"
value={form.stockQuantity} onChange={onChange} required /></label>
86         <label>Availability <select name="availabilityStatus" value={form.availabilityStatus} onChange={onChange}
>{AVAILABILITY_OPTIONS.map((option) => <option key={option.value}
value={option.value}>{option.label}</option>}}</select></label>
87         <label className="span-two">SKU <input name="sku" value={form.sku} onChange={onChange} maxLength="100"
placeholder="AKS-978...-PB" required /></label>
88         <div className="admin-checks">
89           <label><input name="active" type="checkbox" checked={form.active} onChange={onChange} /> Visible in
catalogue</label>
90           <label><input name="featured" type="checkbox" checked={form.featured} onChange={onChange} /> Featured /
curated</label>
91         </div>
92       </div>
93     </section>
94   </div>

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 95–97

```

95   )
96 }

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 98–110

```

98 function BookImportPage() {
99   const [mode, setMode] = useState('import')
100   const [query, setQuery] = useState('')
101   const source = 'OPEN_LIBRARY'
102   const [results, setResults] = useState([])
103   const [selectedIndex, setSelectedIndex] = useState(null)
104   const [form, setForm] = useState(EMPTY_FORM)
105   const [searching, setSearching] = useState(false)
106   const [submitting, setSubmitting] = useState(false)
107   const [searched, setSearch] = useState(false)
108   const [error, setError] = useState('')
109   const [success, setSuccess] = useState('')
110

```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 111–113

```

111   const selected = selectedIndex == null ? null : results[selectedIndex]
112   const resultLabel = useMemo(() => `${results.length} edition${results.length === 1 ? '' : 's'}`, [results.length])
113

```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 114–121

```

114   function switchMode(nextMode) {
115     setMode(nextMode)
116     setForm(EMPTY_FORM)
117     setSelectedIndex(null)
118     setError('')
119     setSuccess('')
120   }
121

```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 122–126

```

122   function updateField(event) {
123     const { name, type, checked, value } = event.target
124     setForm((current) => ({ ...current, [name]: type === 'checkbox' ? checked : value }))
125   }
126

```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 127–140

```
127  async function handleSearch(event) {
128    event.preventDefault()
129    if (!query.trim()) return
130    setSearching(true)
131    setSearched(false)
132    setError('')
133    setSuccess('')
134    try {
135      const nextResults = await searchExternalBooks(query, source)
136      setResults(nextResults)
137      setSelectedIndex(null)
138      setSearched(true)
139    } catch (requestError) {
140      setResults([])
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 141–147

```
141    setSearched(true)
142    setError(getApiErrorMessage(requestError, 'The external catalogue search could not be completed.'))
143  } finally {
144    setSearching(false)
145  }
146  }
147
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 148–161

```
148  function selectEdition(result, index) {
149    setSelectedIndex(index)
150    setError('')
151    setSuccess('')
152    setForm({
153      ...EMPTY_FORM,
154      title: result.title,
155      subtitle: result.subtitle,
156      description: result.description,
157      authors: result.authors.join(', '),
158      publisher: result.publisher,
159      languageCode: result.languageCode || 'en',
160      pageCount: result.pageCount || '',
```



```
161     categories: result.categories.join(', '),
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 162–171

```
162     coverImageUrl: result.coverImageUrl,
163     publicationDate: dateInputValue(result.publicationDate),
164     editionName: result.editionName,
165     isbn10: cleanIsbn(result.isbn10),
166     isbn13: cleanIsbn(result.isbn13),
167     sku: `AKS-${cleanIsbn(result.isbn13 || result.isbn10)}-PB`,
168   })
169   requestAnimationFrame(() => document.querySelector('#catalogue-book-form')?.scrollIntoView({ behavior: 'smooth',
block: 'start' }))
170 }
171
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 172–185

```
172 function importPayload() {
173   return {
174     source,
175     sourceId: selected.sourceId,
176     editionId: selected.editionId,
177     title: form.title.trim(),
178     subtitle: form.subtitle.trim() || null,
179     description: form.description.trim() || null,
180     authors: commaList(form.authors),
181     publisher: form.publisher.trim() || null,
182     languageCode: form.languageCode.trim() || null,
183     pageCount: form.pageCount ? Number(form.pageCount) : null,
184     categories: commaList(form.categories),
185     coverImageUrl: form.coverImageUrl.trim() || null,
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 186–199

```
186     publicationDate: form.publicationDate || null,
187     editionName: form.editionName.trim() || null,
188     isbn10: cleanIsbn(form.isbn10) || null,
189     isbn13: cleanIsbn(form.isbn13) || null,
190     format: form.format,
191     price: Number(form.price),
192     stockQuantity: Number(form.stockQuantity),
193     availabilityStatus: form.availabilityStatus,
194     sku: form.sku.trim(),
195     active: form.active,
196     featured: form.featured,
197   }
198 }
199
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 200–213

```
200  function manualPayload() {
201    return {
202      title: form.title.trim(),
203      subtitle: form.subtitle.trim() || null,
204      description: form.description.trim() || null,
205      coverImageUrl: form.coverImageUrl.trim() || null,
206      languageCode: form.languageCode.trim() || null,
207      authors: commaList(form.authors),
208      publisher: form.publisher.trim() || null,
209      categories: commaList(form.categories),
210      pageCount: form.pageCount ? Number(form.pageCount) : null,
211      publicationDate: form.publicationDate || null,
212      editionName: form.editionName.trim() || null,
213      isbn10: cleanIsbn(form.isbn10) || null,
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 214–224

```
214      isbn13: cleanIsbn(form.isbn13) || null,
215      format: form.format,
216      price: Number(form.price),
217      stockQuantity: Number(form.stockQuantity),
218      availabilityStatus: form.availabilityStatus,
219      sku: form.sku.trim(),
220      active: form.active,
221      featured: form.featured,
222    }
223  }
224
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 225–238

```
225  async function handleSave(event) {
226    event.preventDefault()
227    if (mode === 'import' && !cleanIsbn(form.isbn10) && !cleanIsbn(form.isbn13)) {
228      setError('Add an ISBN-10 or ISBN-13 before saving this edition.')
229      return
230    }
231    setSubmitting(true)
232    setError('')
233    setSuccess('')
234    try {
235      const created = mode === 'import'
236        ? await importBook(importPayload())
237        : await createManualBook(manualPayload())
238      setSuccess({
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-

facing error behavior.

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 239–252

```
239     message: `“${created?.title ?? form.title}” is now in the Akshara catalogue.`,
240     bookId: created?.id,
241   })
242   setForm(EMPTY_FORM)
243   setSelectedIndex(null)
244   if (mode === 'import') setResults([])
245 } catch (requestError) {
246   const duplicateFallback = requestError?.response?.status === 409
247     ? 'That ISBN already exists in the Akshara catalogue. Choose another edition.'
248     : 'The book could not be saved.'
249   setError(getApiErrorMessage(requestError, duplicateFallback))
250 } finally {
251   setSubmitting(false)
252 }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 253–266

```
253   }
254
255   return (
256     <section className="admin-import-page">
257       <header className="admin-import-hero">
258         <div className="container-xl">
259           <div>
260             <p className="eyebrow eyebrow-light">Catalogue administration</p>
261             <h1>Bring the right books<br />onto Akshara’s shelves.</h1>
262             <p>Import trusted metadata, then add the pricing and inventory details only Akshara should own.</p>
263           </div>
264           <div className="admin-import-principle"><i className="bi bi-shield-check" /><span><strong>Metadata
travels.</strong> Commerce data stays with Akshara.</span></div>
265         </div>
266       </header>
267     </section>
268   )
269 }
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 267–273

```
267
268     <div className="container-xl admin-import-workspace">
269       <div className="admin-mode-tabs" role="tablist" aria-label="Add a book">
270         <button type="button" className={mode === 'import' ? 'is-active' : ''} onClick={() =>
switchMode('import')}><i className="bi bi-cloud-arrow-down" /><span>Import metadata<small>Search trusted book
sources</small></span></button>
271         <button type="button" className={mode === 'manual' ? 'is-active' : ''} onClick={() =>
switchMode('manual')}><i className="bi bi-pencil-square" /><span>Manual entry<small>Create a book from
scratch</small></span></button>
272       </div>
273
```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 274–276

```
274     {success && <div className="admin-notice success" role="status"><i className="bi bi-check-circle-fill"
/><span>{success.message}</span><a href={success.bookId ? `/books/${success.bookId}` : '/'}>View book</a></div>}
275     {error && <div className="admin-notice error" role="alert"><i className="bi bi-exclamation-circle-fill"
/><span>{error}</span><button type="button" onClick={() => setError('')} aria-label="Dismiss">x</button></div>}
276
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 277–286

```
277     {mode === 'import' && (
278       <
279         <section className="admin-search-panel">
280           <div><p className="eyebrow">Find a book</p><h2>Search by title, author, or ISBN</h2><p>Results are
 fetched through Akshara's Spring Boot API and are not saved until you import one.</p></div>
281           <form onSubmit={handleSearch} role="search">
282             <div className="admin-search-input"><i className="bi bi-search" /><input value={query}
onChange={(event) => setQuery(event.target.value)} placeholder="Try "Atomic Habits" or 9780735211292" aria-label="Book
search" required /><span className="admin-source-label">Open Library</span><button type="submit" disabled={searching}
>{searching ? <span className="spinner-border spinner-border-sm" /> : 'Search'}</button></div>
283             <small><i className="bi bi-info-circle" /> Metadata is retrieved through Akshara's secured Open
Library provider.</small>
284           </form>
285         </section>
286
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 287–300

```

287         {(results.length > 0 || searched) && (
288             <section className="external-results">
289                 <div className="external-results-heading"><div><p className="eyebrow">External
results</p><h2>{results.length ? `${resultLabel} found` : 'No editions found'}</h2></div>{results.length > 0 &&
<span>Select the exact ISBN and cover</span></div>
290                 {results.length > 0 ? <div className="external-result-grid">{results.map((result, index) => (
291                     <article className={`external-book-card ${selectedIndex === index ? 'is-selected' : ''} $
{result.alreadyImported ? 'is-imported' : ''}`} key={`${result.editionId || result.isbn13 || result.isbn10}-${index}`}>
292                         <div className="external-cover">{result.coverImageUrl ? <img src={result.coverImageUrl}
alt={`Cover of ${result.title}`} /> : <span>⌕<small>No cover</small></span></div>
293                         <div className="external-book-copy"><div className="external-
source"><span>{BOOK_SOURCES.find((item) => item.value === source)?.label}</span>{result.alreadyImported ? <strong>Already
in catalogue</strong> : selectedIndex === index && <i className="bi bi-check-circle-fill" /></div><h3>{titleFor(result)}
</h3><p>{result.authors.join(', ')} || 'Unknown author'}</p><dl><div><dt>ISBN</dt><dd>{result.isbn13 || result.isbn10 ||
'Not supplied'}</dd></div><div><dt>Published</dt><dd>{result.publicationDate ||
'Unknown'}</dd></div><div><dt>Publisher</dt><dd>{result.publisher ||
'Unknown'}</dd></div><div><dt>Pages</dt><dd>{result.pageCount || '-'}</dd></div></dl><button type="button"
disabled={result.alreadyImported} onClick={() => selectEdition(result, index)}>{result.alreadyImported ? 'Duplicate
ISBN' : selectedIndex === index ? 'Edition selected' : 'Select this edition'} <i className="bi bi-arrow-right"
/></button></div>
294                     </article>
295                 )}</div> : <div className="admin-empty-results"><i className="bi bi-journal-x" /><h3>No matching
editions</h3><p>Try an ISBN, fewer words, or the fallback source.</p></div>
296             </section>
297         )}
298     </>
299 )}
300

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 301–312

```

301         {(mode === 'manual' || selected) && (
302             <form id="catalogue-book-form" className="catalogue-book-form" onSubmit={handleSave}>
303                 <div className="catalogue-form-heading"><div><p className="eyebrow">{mode === 'import' ? 'Selected
edition' : 'New catalogue record'}</p><h1>{mode === 'import' ? 'Complete the import' : 'Add a book
manually'}</h1></div>{form.coverImageUrl && <img src={form.coverImageUrl} alt="Selected book cover" /></div>
304                 <BookMetadataForm form={form} mode={mode} onChange={updateField} />
305                 <div className="admin-submit-bar"><div><i className="bi bi-database-check" /><span><strong>Saved
permanently to MySQL</strong><small>The public catalogue will use this Akshara-owned record.</small></span></div><button
className="btn btn-ink" type="submit" disabled={submitting}>{submitting ? <><span className="spinner-border spinner-
border-sm" /> Saving...</> : <>{mode === 'import' ? 'Import into catalogue' : 'Create book'} <i className="bi bi-arrow-

```

```
right" /></></button></div>
306         </form>
307     )}
308 </div>
309 </section>
310 )
311 }
312
```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 313–314

```
313 export default BookImportPage
314
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/CatalogueAdminPage.jsx

File 89 of 99 · Added or modified · Frontend page

Purpose: Implements catalogue search, stock/price/availability editing, additive restock, activation, featured curation and deletion feedback.

Current file length: 297 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–13

```
1 import { useEffect, useMemo, useState } from 'react'
2 import { Link } from 'react-router-dom'
3 import {
4   AVAILABILITY_OPTIONS,
5   deleteCatalogueBook,
6   getAdminBooks,
7   restockEdition,
8   seedCatalogue,
9   updateBookFeatured,
10  updateEditionInventory,
11 } from '../services/adminBookService'
12 import { getApiErrorMessage } from '../services/api'
13
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 14–23

```
14 function inventoryDraft(inventory) {
15   return {
16     price: String(inventory?.price ?? ''),
17     stockQuantity: String(inventory?.stockQuantity ?? 0),
18     availabilityStatus: inventory?.availabilityStatus ?? 'OUT_OF_STOCK',
19     active: Boolean(inventory?.active),
20     restockQuantity: '',
21   }
22 }
23
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 24–33

```
24 function CatalogueAdminPage() {
25   const [books, setBooks] = useState([])
26   const [drafts, setDrafts] = useState({})
27   const [query, setQuery] = useState('')
```

```

28  const [loading, setLoading] = useState(true)
29  const [busy, setBusy] = useState('')
30  const [error, setError] = useState('')
31  const [notice, setNotice] = useState('')
32  const [seeding, setSeeding] = useState(false)
33

```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 34–47

```

34  async function loadCatalogue() {
35    try {
36      setLoading(true)
37      setError('')
38      const data = await getAdminBooks()
39      setBooks(data)
40      setDrafts(Object.fromEntries(data.flatMap((book) =>
41        (book.editions ?? []).map((edition) => [edition.id, inventoryDraft(edition.inventory)])),
42      )))
43    } catch (requestError) {
44      setError(getApiErrorMessage(requestError, 'The catalogue administration data could not be loaded.'))
45    } finally {
46      setLoading(false)
47    }

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 48–52

```

48  }
49
50  useEffect(() => {
51    let active = true
52

```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 53–66

```

53  getAdminBooks()
54    .then((data) => {
55      if (!active) return
56      setBooks(data)
57      setDrafts(Object.fromEntries(data.flatMap((book) =>
58        (book.editions ?? []).map((edition) => [edition.id, inventoryDraft(edition.inventory)])),
59      )))

```



```

60     })
61     .catch((requestError) => {
62       if (active) {
63         setError(getApiErrorMessage(requestError, 'The catalogue administration data could not be loaded.'))
64       }
65     })
66     .finally(() => {

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 67–69

```

67       if (active) setLoading(false)
68     })
69

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 70–74

```

70     return () => {
71       active = false
72     }
73   }, [])
74

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 75–84

```

75   const visibleBooks = useMemo(() => {
76     const needle = query.trim().toLowerCase()
77     if (!needle) return books
78     return books.filter((book) => [
79       book.title,
80       ...(book.authors ?? []).map((author) => author.name),
81       ...(book.editions ?? []).flatMap((edition) => [edition.sku, edition.isbn13, edition.isbn10]),
82     ].some((value) => String(value ?? '').toLowerCase().includes(needle)))
83   }, [books, query])
84

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 85–94

```
85   const summary = useMemo(() => {
86     const editions = books.flatMap((book) => book.editions ?? [])
87     return {
88       books: books.length,
89       editions: editions.length,
90       units: editions.reduce((total, edition) => total + (edition.inventory?.stockQuantity ?? 0), 0),
91       attention: editions.filter((edition) => !edition.inventory?.active || edition.inventory?.stockQuantity ===
0).length,
92     }
93   }, [books])
94
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 95–101

```
95   function updateDraft(editionId, field, value) {
96     setDrafts((current) => ({
97       ...current,
98       [editionId]: { ...current[editionId], [field]: value },
99     }))
100   }
101
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 102–113

```
102   function replaceInventory(editionId, inventory) {
103     setBooks((current) => current.map((book) => ({
104       ...book,
```

```

105     editions: (book.editions ?? []).map((edition) =>
106       edition.id === editionId ? { ...edition, inventory } : edition),
107   ))))
108   setDrafts((current) => ({
109     ...current,
110     [editionId]: { ...inventoryDraft(inventory), restockQuantity: '' },
111   }))
112 }
113

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 114–127

```

114   async function saveInventory(edition) {
115     const draft = drafts[edition.id]
116     try {
117       setBusy(`save-${edition.id}`)
118       setError('')
119       setNotice('')
120       const inventory = await updateEditionInventory(edition.id, {
121         price: Number(draft.price),
122         stockQuantity: Number(draft.stockQuantity),
123         availabilityStatus: draft.availabilityStatus,
124         active: draft.active,
125       })
126       replaceInventory(edition.id, inventory)
127       setNotice(`${edition.sku} inventory was updated.`)

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 128–134

```

128   } catch (requestError) {
129     setError(getApiErrorMessage(requestError, 'Inventory could not be updated.'))
130   } finally {
131     setBusy('')
132   }
133 }
134

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-

facing error behavior.

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 135–148

```
135  async function restock(edition) {
136    const quantity = Number(drafts[edition.id]?.restockQuantity)
137    if (!Number.isInteger(quantity) || quantity <= 0) {
138      setError('Enter a positive whole-number restock quantity.')
139      return
140    }
141    try {
142      setBusy(`restock-${edition.id}`)
143      setError('')
144      setNotice('')
145      const inventory = await restockEdition(edition.id, quantity)
146      replaceInventory(edition.id, inventory)
147      setNotice(`${quantity} units were added to ${edition.sku}.`)
148    } catch (requestError) {
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 149–154

```
149      setError(getApiErrorMessage(requestError, 'The edition could not be restocked.'))
150    } finally {
151      setBusy('')
152    }
153  }
154
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 155–168

```
155  async function toggleFeatured(book) {
156    try {
157      setBusy(`feature-${book.id}`)
158      setError('')
159      const updated = await updateBookFeatured(book.id, !book.featured)
160      setBooks((current) => current.map((item) => item.id === book.id ? updated : item))
161      setNotice(`${book.title} was ${updated.featured ? 'added to' : 'removed from'} the featured shelf.`)
162    } catch (requestError) {
```

```

163     setError(getApiErrorMessage(requestError, 'Featured status could not be updated.'))
164   } finally {
165     setBusy('')
166   }
167 }
168

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 169–174

```

169  async function deleteBook(book) {
170    const confirmed = window.confirm(
171      `Permanently delete "${book.title}" and all of its editions and inventory? This cannot be undone.`
172    )
173    if (!confirmed) return
174

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 175–188

```

175    try {
176      setBusy(`delete-${book.id}`)
177      setError('')
178      setNotice('')
179      await deleteCatalogueBook(book.id)
180      setBooks((current) => current.filter((item) => item.id !== book.id))
181      setDrafts((current) => {
182        const next = { ...current }
183        for (const edition of book.editions ?? []) delete next[edition.id]
184        return next
185      })
186      setNotice(`${book.title} was permanently deleted from the catalogue.`)
187    } catch (requestError) {
188      setError(getApiErrorMessage(requestError, 'The book could not be deleted.'))

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 189–193

```
189     } finally {
190         setBusy('')
191     }
192 }
193
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 194–207

```
194  async function seedStarterCatalogue() {
195      const confirmed = window.confirm(
196          'Import 50 curated books across classics, science, history, philosophy, business, biography, Indian
literature, and children's books? This can take about a minute.',
197      )
198      if (!confirmed) return
199      try {
200          setSeeding(true)
201          setError('')
202          setNotice('Importing the curated starter catalogue...')
203          const result = await seedCatalogue(50)
204          await loadCatalogue()
205          setNotice(`Imported ${result.imported} books. The catalogue now contains ${result.catalogueSizeAfter} books.`)
206      } catch (requestError) {
207          setNotice('')
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 208–213

```
208      setError(getApiErrorMessage(requestError, 'The starter catalogue could not be populated.'))
209  } finally {
210      setSeeding(false)
211  }
212 }
213
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 214–227

```
214   return (
215     <div className="catalogue-admin-page">
216       <section className="catalogue-admin-hero">
217         <div className="container-xl">
218           <div>
219             <p className="eyebrow">Catalogue operations</p>
220             <h1>Manage Akshara's shelves.</h1>
221             <p>Restock editions, adjust price and availability, pause selling, and curate featured books.</p>
222           </div>
223           <div className="catalogue-admin-hero-actions">
224             <Link className="btn btn-light" to="/admin/orders"><i className="bi bi-box-seam" /> Manage orders</Link>
225             <button type="button" className="btn btn-light" onClick={seedStarterCatalogue} disabled={seeding}><i
className="bi bi-collection" /> {seeding ? 'Importing 50 books...' : 'Populate 50 books'}</button>
226             <Link className="btn btn-light" to="/admin/books/import"><i className="bi bi-cloud-arrow-down" /> Import
or add a book</Link>
227           </div>
228         </div>
229       </section>
230     </div>
231   );
232 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 228–230

```
228     </div>
229     </section>
230   </div>
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 231–238

```
231   <section className="container-xl catalogue-admin-workspace">
232     <div className="catalogue-admin-summary" aria-label="Catalogue summary">
233       <div><strong>{summary.books}</strong><span>Books</span></div>
234       <div><strong>{summary.editions}</strong><span>Sellable editions</span></div>
235       <div><strong>{summary.units}</strong><span>Units in stock</span></div>
236       <div><strong>{summary.attention}</strong><span>Need attention</span></div>
237     </div>
238   </section>
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 239–243

```

239         <div className="catalogue-admin-toolbar">
240             <label><i className="bi bi-search" /><input type="search" value={query} onChange={(event) =>
setQuery(event.target.value)} placeholder="Search title, author, ISBN, or SKU" /></label>
241             <button type="button" onClick={loadCatalogue} disabled={loading}><i className="bi bi-arrow-clockwise" />
Refresh</button>
242         </div>
243     
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 244–247

```

244         {error && <div className="admin-feedback admin-feedback-error" role="alert">{error}</div>}
245         {notice && <div className="admin-feedback admin-feedback-success" role="status">{notice}</div>}
246         {loading && <div className="page-loading"><span className="spinner-border" /> Loading catalogue...</div>}
247     
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 248–251

```

248         {!loading && visibleBooks.length === 0 && (
249             <div className="catalogue-admin-empty"><i className="bi bi-journal-x" /><h2>No matching catalogue
records</h2><p>Try another search or import a new book.</p></div>
250         )}
251     
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 252–264

```

252         <div className="catalogue-admin-list">
253             {visibleBooks.map((book) => (
254                 <article className="catalogue-admin-book" key={book.id}>
255                     <header>
256                         <div className="catalogue-admin-cover">{book.coverImageUrl} ? <img src={book.coverImageUrl} alt="" />
: <i className="bi bi-book" />}</div>
257                         <div><p>{(book.authors ?? []).map((author) => author.name).join(', ')} || 'Unknown
author'}</p><h2>{book.title}</h2><span>{book.editions?.length ?? 0} {(book.editions?.length ?? 0) === 1 ? 'edition' :
'editions'}</span></div>
258                         <div className="catalogue-admin-book-actions">
259                             <button type="button" className={book.featured ? 'is-featured' : ''} onClick={() =>
toggleFeatured(book)} disabled={Boolean(busy)}><i className={`bi ${book.featured ? 'bi-star-fill' : 'bi-star'}`} />
{book.featured ? 'Featured' : 'Feature'}</button>
260                             <Link to={`/books/${book.id}`}>Public page <i className="bi bi-arrow-up-right" /></Link>
261                             <button type="button" className="catalogue-delete-button" onClick={() => deleteBook(book)}
disabled={Boolean(busy)}><i className="bi bi-trash3" /> {busy === `delete-${book.id}` ? 'Deleting...' : 'Delete'}</button>
262                         </div>
263                     </header>
264                 </article>

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 265–278

```

265         <div className="catalogue-admin-editions">
266             {(book.editions ?? []).map((edition) => {
267                 const draft = drafts[edition.id] ?? inventoryDraft(edition.inventory)
268                 return (
269                     <section className="catalogue-admin-edition" key={edition.id}>
270                         <div className="edition-identity">
271                             <strong>{edition.format}</strong><span>SKU {edition.sku}</span><span>{edition.isbn13} ? `ISBN
${edition.isbn13}` : 'No ISBN'</span>
272                         </div>
273                         <div className="edition-inventory-fields">
274                             <label>Price (₹)<input type="number" min="0.01" step="0.01" value={draft.price}
onChange={(event) => updateDraft(edition.id, 'price', event.target.value)} /></label>
275                             <label>Total stock<input type="number" min="0" step="1" value={draft.stockQuantity}
onChange={(event) => updateDraft(edition.id, 'stockQuantity', event.target.value)} /></label>
276                             <label>Availability<select value={draft.availabilityStatus} onChange={(event) =>
updateDraft(edition.id, 'availabilityStatus', event.target.value)}>{AVAILABILITY_OPTIONS.map((option) => <option
key={option.value} value={option.value}>{option.label}</option>)}</select></label>
277                             <label className="edition-active-toggle"><input type="checkbox" checked={draft.active}
onChange={(event) => updateDraft(edition.id, 'active', event.target.checked)} /><span>Active for sale</span></label>
278                             <button type="button" className="edition-save-button" onClick={() => saveInventory(edition)}
disabled={Boolean(busy)}>{busy === `save-${edition.id}` ? 'Saving...' : 'Save changes'}</button>

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 279–292

```

279             </div>
280             <div className="edition-restock">
281                 <label><span>Quick restock</span><input type="number" min="1" step="1" placeholder="Units to
add" value={draft.restockQuantity} onChange={(event) => updateDraft(edition.id, 'restockQuantity', event.target.value)}
/></label>
282                 <button type="button" onClick={() => restock(edition)} disabled={Boolean(busy)}><i
className="bi bi-box-arrow-in-down" /> {busy === `restock-${edition.id}` ? 'Adding...' : 'Add stock'}</button>
283             </div>
284         </section>
285     )
286 }}}}
287 </div>
288 </article>
289 )}}
290 </div>
291 </section>
292 </div>

```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 293–295

```

293     )
294 }
295

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 296–297

```

296 export default CatalogueAdminPage
297

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/CataloguePage.jsx

File 90 of 99 · Added or modified · Frontend page

Purpose: Renders and coordinates the CataloguePage React interface.

Current file length: 345 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–7

```
1 import { useEffect, useState } from 'react'
2 import { Link } from 'react-router-dom'
3 import BookCard from '../components/BookCard'
4 import { getBooks } from '../services/bookService'
5 import { getRecentReviews } from '../services/readerService'
6 import useAuth from '../hooks/useAuth'
7
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 8–16

```
8 const categories = [
9   { name: 'Fiction', icon: 'bi-stars' },
10  { name: 'Indian literature', icon: 'bi-feather' },
11  { name: 'Technology', icon: 'bi-cpu' },
12  { name: 'History', icon: 'bi-hourglass-split' },
13  { name: 'Philosophy', icon: 'bi-lightbulb' },
14  { name: 'Children', icon: 'bi-balloon' },
15 ]
16
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 17–24

```
17 const journeySteps = [
18   { number: '01', title: 'Discover', text: 'Find books through ideas, moods, genres, and curated shelves.', href:
19     '#catalogue', action: 'Explore books' },
20   { number: '02', title: 'Evaluate', text: 'Understand a book through details, editions, ratings, and reviews.',
21     href: '#catalogue', action: 'Choose a book' },
22   { number: '03', title: 'Organize', text: 'Shape a personal library with wishlists and reading statuses.', to:
23     '/wishlist', adminTo: '/admin/books', action: 'Open wishlist', adminAction: 'Manage catalogue' },
24   { number: '04', title: 'Connect', text: 'Exchange perspectives through discussions and reader collections.', to:
25     '/community', action: 'Join the community' },
26   { number: '05', title: 'Purchase', text: 'Choose an edition and complete the journey in one place.', to: '/cart',
27     adminTo: '/admin/books', action: 'Open your cart', adminAction: 'Manage inventory' },
28 ]
29
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 25–35

```
25 function CataloguePage() {
26   const { user } = useAuth()
27   const [books, setBooks] = useState([])
28   const [search, setSearch] = useState('')
29   const [appliedQuery, setAppliedQuery] = useState('')
30   const [sort, setSort] = useState('TITLE_ASC')
31   const [total, setTotal] = useState(0)
32   const [loading, setLoading] = useState(true)
33   const [error, setError] = useState('')
34   const [recentReviews, setRecentReviews] = useState([])
35 }
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 36–40

```
36 async function loadBooks(query = '', nextSort = 'TITLE_ASC') {
37   try {
38     setLoading(true)
39     setError('')
40   }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 41–43

```
41   const data = await getBooks({ query, sort: nextSort })
42   const content = Array.isArray(data) ? data : data.content ?? []
43 }
```

How to read this block

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.

Lines 44–54

```
44   setBooks(content)
45   setTotal(Array.isArray(data) ? content.length : data.totalElements ?? content.length)
46   setAppliedQuery(query.trim())
47 } catch (requestError) {
48   console.error(requestError)
49   setError('The catalogue could not be loaded. Confirm that the Akshara backend is running on port 8080.')
50 } finally {
51   setLoading(false)
52 }
53 }
54 }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 55–57

```
55    useEffect(() => {  
56        let active = true  
57    }
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 58–71

```
58    getBooks()  
59        .then((data) => {  
60        if (!active) return  
61        const content = Array.isArray(data) ? data : data.content ?? []  
62        setBooks(content)  
63        setTotal(Array.isArray(data) ? content.length : data.totalElements ?? content.length)  
64    })  
65    .catch((requestError) => {  
66        console.error(requestError)  
67        if (active) {  
68            setError('The catalogue could not be loaded. Confirm that the Akshara backend is running on port 8080.')69        }  
70    })  
71    .finally(() => {
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 72–74

```
72        if (active) setLoading(false)  
73    })  
74
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 75–79

```
75    return () => {  
76        active = false
```

```
77     }
78   }, [])
79
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 80–85

```
80   useEffect(() => {
81     getRecentReviews(3).then(setRecentReviews).catch((requestError) => {
82       console.error('Unable to load community preview', requestError)
83     })
84   }, [])
85
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 86–91

```
86   function handleSubmit(event) {
87     event.preventDefault()
88     loadBooks(search, sort)
89     document.querySelector('#catalogue')?.scrollIntoView({ behavior: 'smooth' })
90   }
91
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 92–97

```
92   function handleCategory(category) {
93     setSearch(category)
94     loadBooks(category, sort)
95     document.querySelector('#catalogue')?.scrollIntoView({ behavior: 'smooth' })
96   }
97
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 98–103

```
98   function handleSort(event) {
99     const nextSort = event.target.value
100    setSort(nextSort)

```

```
101     loadBooks(appliedQuery, nextSort)
102   }
103
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 104–108

```
104   function clearSearch() {
105     setSearch('')
106     loadBooks('', sort)
107   }
108
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 109–114

```
109   return (
110     <>
111       <section className="home-hero">
112         <div className="hero-orb hero-orb-one" aria-hidden="true" />
113         <div className="hero-orb hero-orb-two" aria-hidden="true" />
114
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 115–128

```
115     <div className="container-xl hero-layout">
116       <div className="hero-copy">
117         <p className="eyebrow hero-eyebrow">
118           <span /> A reader-first bookstore
119         </p>
120         <h1>
121           Every book begins<br />
122           with an <em>अक्षर.</em>
123         </h1>
124         <p className="hero-lead">
125           Discover books with depth, understand them through people, and build
126           a reading life that keeps growing.
127         </p>
128
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 129–140

```

129      <form className="hero-search" onSubmit={handleSubmit} role="search">
130        <i className="bi bi-search" aria-hidden="true" />
131        <input
132          type="search"
133          aria-label="Search the book catalogue"
134          placeholder="Search by title, author, or category"
135          value={search}
136          onChange={(event) => setSearch(event.target.value)}
137        />
138        <button type="submit">Explore books</button>
139      </form>
140
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 141–147

```

141      <div className="hero-proof" aria-label="Akshara platform highlights">
142        <span><i className="bi bi-compass" /> Curated discovery</span>
143        <span><i className="bi bi-people" /> Reader community</span>
144        <span><i className="bi bi-bookmark-heart" /> Personal library</span>
145      </div>
146    </div>
147
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 148–161

```

148      <div className="hero-visual" aria-label="A collection of books curated for Akshara readers">
149        <div className="hero-sun" aria-hidden="true" />
150        <div className="hero-book hero-book-one">
151          <span>AKSHARA<br />EDITIONS</span>
152          <strong>The Art of<br />Curiosity</strong>
153          <small>Ideas that open new doors</small>
154        </div>
155        <div className="hero-book hero-book-two">
156          <span>READER'S PICK</span>
157          <strong>Stories<br />from Home</strong>
158          <small>Indian voices · Volume I</small>
159        </div>
160        <div className="hero-book hero-book-three">
161          <span>NEW THOUGHT</span>

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 162–174

```
162         <strong>The Quiet<br />Future</strong>
163         <small>Technology & humanity</small>
164     </div>
165     <div className="hero-quote-card">
166         <i className="bi bi-quote" />
167         <p>A book is not only bought. It is discovered, understood, shared, and remembered.</p>
168         <span>– The Akshara idea</span>
169     </div>
170     <div className="hero-seal"><span>अ</span><small>Begin here</small></div>
171 </div>
172 </div>
173 </section>
174
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 175–184

```
175     <section className="category-strip" aria-labelledby="category-heading">
176         <div className="container-xl">
177             <div className="category-strip-heading">
178                 <div>
179                     <p className="eyebrow">Browse by interest</p>
180                     <h2 id="category-heading">Where does your curiosity lead?</h2>
181                 </div>
182                 <span>Choose a shelf to begin</span>
183             </div>
184
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 185–196

```
185         <div className="category-list">
186             {categories.map((category) => (
187                 <button type="button" key={category.name} onClick={() => handleCategory(category.name)}>
188                     <span><i className={`bi ${category.icon}`} /></span>
189                     {category.name}
190                     <i className="bi bi-arrow-right" />
191                 </button>
192             ))}
193         </div>
194     </div>
195 </section>
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 197–209

```

197     <section className="catalogue-section" id="catalogue" aria-labelledby="catalogue-heading">
198       <div className="container-xl">
199         <div className="section-heading catalogue-heading">
200           <div>
201             <p className="eyebrow">The Akshara shelf</p>
202             <h2 id="catalogue-heading">Books worth finding</h2>
203             <p>
204               {appliedQuery
205                 ? `Showing matches for "${appliedQuery}"`
206                 : 'Browse the latest books added to our growing catalogue.'}
207             </p>
208           </div>
209         </div>

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 210–223

```

210     <div className="catalogue-tools">
211       {!loading && <span className="result-count">{total} {total === 1 ? 'book' : 'books'}</span>}
212       {appliedQuery && (
213         <button className="clear-search-button" type="button" onClick={clearSearch}>
214           Clear search
215         </button>
216       )}
217     <label className="sort-control">
218       <span>Sort</span>
219       <select value={sort} onChange={handleSort}>
220         <option value="TITLE_ASC">Title A-Z</option>
221         <option value="TITLE_DESC">Title Z-A</option>
222         <option value="NEWEST">Newest first</option>
223         <option value="OLDEST">Oldest first</option>

```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 224–228

```
224         </select>
225     </label>
226 </div>
227 </div>
228
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 229–241

```
229     {loading && (
230       <div className="book-grid" aria-label="Loading catalogue">
231         {[0, 1, 2, 3].map((item) => (
232           <div className="book-card-skeleton" key={item}>
233             <span className="skeleton-cover" />
234             <span className="skeleton-line skeleton-line-short" />
235             <span className="skeleton-line" />
236             <span className="skeleton-line skeleton-line-medium" />
237           </div>
238         ))}
239       </div>
240     )}
241
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 242–252

```
242     {error && (
243       <div className="catalogue-message catalogue-error" role="alert">
244         <span><i className="bi bi-plug" /></span>
245         <div>
246           <h3>The shelf is temporarily unavailable</h3>
247           <p>{error}</p>
248           <button type="button" onClick={() => loadBooks(appliedQuery, sort)}>Try again</button>
249         </div>
250       </div>
251     )}
252
```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 253–263

```

253      {!loading && !error && books.length === 0 && (
254        <div className="catalogue-message">
255          <span><i className="bi bi-search" /></span>
256          <div>
257            <h3>No books found for "{appliedQuery}"</h3>
258            <p>Try another title, author, or interest—or return to the full Akshara shelf.</p>
259            <button type="button" onClick={clearSearch}>View all books</button>
260          </div>
261        </div>
262      )}
263    
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 264–271

```

264      {!loading && !error && books.length > 0 && (
265        <div className="book-grid">
266          {books.map((book) => <BookCard book={book} key={book.id} />)}
267        </div>
268      )}
269    </div>
270  </section>
271
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 272–282

```

272    <section className="journey-section" id="journey" aria-labelledby="journey-heading">
273      <div className="container-xl">
274        <div className="journey-intro">
275          <p className="eyebrow eyebrow-light">More than a transaction</p>
276          <h2 id="journey-heading">The complete reader journey,<br />in one thoughtful place.</h2>
277          <p>
278            Akshara connects the moments before and after purchase—the questions,
279            conversations, collections, and discoveries that turn books into a reading life.
280          </p>
281        </div>
282      </div>

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 283–296

```
283     <div className="journey-steps">
284       {journeySteps.map((step) => {
285         const isAdmin = user?.role === 'ADMIN'
286         const destination = isAdmin && step.adminTo ? step.adminTo : step.to
287         const action = isAdmin && step.adminAction ? step.adminAction : step.action
288         return (
289           <article key={step.number} className="journey-step">
290             <span>{step.number}</span>
291             <h3>{step.title}</h3>
292             <p>{step.text}</p>
293             {step.href ? (
294               <a className="journey-step-link" href={step.href}>{action} <i className="bi bi-arrow-right" /></a>
295             ) : (
296               <Link className="journey-step-link" to={destination}>{action} <i className="bi bi-arrow-right"
/></Link>
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 297–304

```
297         )}
298       </article>
299     )
300   }]}
301 </div>
302 </div>
303 </section>
304
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 305–315

```
305     <section className="community-section" id="community" aria-labelledby="community-heading">
306       <div className="container-xl">
307         <div className="section-heading community-heading">
308           <div>
309             <p className="eyebrow">The reading room</p>
310             <h2 id="community-heading">Books become richer in conversation.</h2>
311             <p>Discover the lists, reviews, and discussions taking shape across the Akshara community.</p>
312           </div>
```

```

313         <Link className="btn btn-ink" to="/community">Visit the community <i className="bi bi-arrow-right"
/></Link>
314     </div>
315

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 316–329

```

316     <div className="community-grid">
317         {recentReviews.map((review) => (
318             <article className="community-card" key={review.id}>
319                 <div className="community-card-top">
320                     <span className="reader-avatar">{review.readerName.split(/\s+/).map((part) => part[0]).slice(0,
2).join('')}</span>
321                     <div><strong>{review.readerName}</strong><small>{review.rating}/5 reader review</small></div>
322                     <i className="bi bi-bookmark" />
323                 </div>
324                 <h3>{review.headline || `Thoughts on ${review.bookTitle}`}</h3>
325                 <p>{review.content}</p>
326                 <Link to={`/books/${review.bookId}#reviews`} aria-label={`Read review of ${review.bookTitle}`}><i
className="bi bi-arrow-up-right" /></Link>
327             </article>
328         ))}
329         {!recentReviews.length && (

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 330–343

```

330     <article className="community-card community-empty-card">
331         <div className="community-card-top"><span className="reader-avatar">⌘</span><div><strong>The
reading room</strong><small>Ready for its first voice</small></div></div>
332         <h3>Be the first reader to begin a conversation.</h3>
333         <p>Open any book, add a rating and share what stayed with you.</p>
334         <Link to="/community" aria-label="Visit the community"><i className="bi bi-arrow-up-right" /></Link>
335     </article>
336     )}
337 </div>
338 </div>
339 </section>
340 </>
341 )
342 }
343

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 344–345

```
344 export default CataloguePage
345
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/CommunityPage.jsx

File 91 of 99 · Added or modified · Frontend page

Purpose: Displays recent reviews as a searchable reader community and embeds reply threads.

Current file length: 180 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 import { useEffect, useMemo, useState } from 'react'
2 import { Link } from 'react-router-dom'
3 import { getRecentReviews } from '../services/readerService'
4 import DiscussionThread from '../components/DiscussionThread'
5 import useAuth from '../hooks/useAuth'
6
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 7–10

```
7 function initials(name = 'Reader') {
8   return name.split(/\s+/).filter(Boolean).map((part) => part[0]).slice(0, 2).join('').toUpperCase()
9 }
10
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 11–15

```
11 function reviewDate(value) {
12   if (!value) return 'Recently reviewed'
13   return new Date(value).toLocaleDateString('en-IN', { day: 'numeric', month: 'short', year: 'numeric' })
14 }
15
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 16–23

```
16 function ReviewCover({ review }) {
17   const [failed, setFailed] = useState(false)
18   if (!review.coverImageUrl || failed) {
```

```

19     return <span className="community-book-cover community-book-cover-placeholder" aria-hidden="true"><i
className="bi bi-book" /></span>
20   }
21   return <img className="community-book-cover" src={review.coverImageUrl} alt={`Cover of ${review.bookTitle}`}
onError={() => setFailed(true)} />
22 }
23

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 24–31

```

24 function CommunityPage() {
25   const { user } = useAuth()
26   const [reviews, setReviews] = useState([])
27   const [loading, setLoading] = useState(true)
28   const [error, setError] = useState('')
29   const [query, setQuery] = useState('')
30   const [sort, setSort] = useState('latest')
31

```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 32–44

```

32 async function loadReviews() {
33   try {
34     setLoading(true)
35     setError('')
36     setReviews(await getRecentReviews(20))
37   } catch (requestError) {
38     console.error(requestError)
39     setError('Reader reviews could not be loaded. Please check the backend and try again.')
40   } finally {
41     setLoading(false)
42   }
43 }
44

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 45–58

```
45  useEffect(() => {
46    let active = true
47    getRecentReviews(20)
48      .then((data) => {
49        if (active) setReviews(data)
50      })
51      .catch((requestError) => {
52        console.error(requestError)
53        if (active) setError('Reader reviews could not be loaded. Please check the backend and try again.')
54      })
55      .finally(() => {
56        if (active) setLoading(false)
57      })
58    return () => {
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 59–62

```
59    active = false
60  }
61 }, [])
62
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 63–71

```
63  const summary = useMemo(() => {
64    const books = new Set(reviews.map((review) => review.bookId)).size
65    const readers = new Set(reviews.map((review) => review.userId)).size
66    const average = reviews.length
67      ? reviews.reduce((total, review) => total + review.rating, 0) / reviews.length
68      : 0
69    return { books, readers, average }
70  }, [reviews])
71
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 72–84

```

72  const visibleReviews = useMemo(() => {
73    const normalizedQuery = query.trim().toLowerCase()
74    const filtered = normalizedQuery
75      ? reviews.filter((review) => [review.bookTitle, review.readerName, review.headline, review.content]
76        .some((value) => value?.toLowerCase().includes(normalizedQuery)))
77      : reviews
78    return [...filtered].sort((left, right) => {
79      if (sort === 'highest') return right.rating - left.rating || new Date(right.createdAt) - new
Date(left.createdAt)
80      if (sort === 'oldest') return new Date(left.createdAt) - new Date(right.createdAt)
81      return new Date(right.createdAt) - new Date(left.createdAt)
82    })
83  }, [query, reviews, sort])
84

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 85–96

```

85  return (
86    <section className="inner-page community-page">
87      <div className="container-xl">
88        <header className="community-page-hero">
89          <div className="inner-page-heading">
90            <p className="eyebrow">Community reviews</p>
91            <h1>See what Akshara's readers are reading.</h1>
92            <p>Discover books through honest ratings and reflections shared by the reader community.</p>
93          </div>
94          <div className="community-hero-actions">{user?.role === 'READER' && <Link className="btn btn-outline-dark"
to="/my-reviews">Manage my reviews</Link><Link className="btn btn-link" to="/#catalogue">Browse books to review <i
className="bi bi-arrow-right" /></Link></div>
95        </header>
96      </div>

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 97–103

```
97     <div className="community-summary" aria-label="Community review summary">
98       <div><strong>{reviews.length}</strong><span>Recent reviews</span></div>
99       <div><strong>{summary.books}</strong><span>Books reviewed</span></div>
100      <div><strong>{summary.readers}</strong><span>Reader voices</span></div>
101      <div><strong>{reviews.length ? summary.average.toFixed(1) : '-'}</strong><span>Average rating</span></div>
102    </div>
103
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 104–117

```
104      {!loading && !error && reviews.length > 0 && (
105        <div className="community-toolbar">
106          <label>
107            <i className="bi bi-search" />
108            <input type="search" value={query} onChange={(event) => setQuery(event.target.value)}
placeholder="Search books, readers, or reviews" />
109          </label>
110          <label className="community-sort">
111            <span>Sort by</span>
112            <select value={sort} onChange={(event) => setSort(event.target.value)}>
113              <option value="latest">Latest reviews</option>
114              <option value="highest">Highest rated</option>
115              <option value="oldest">Oldest first</option>
116            </select>
117          </label>

```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 118–120

```
118      </div>
119    )}
120
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 121–131

```
121      {!loading && <div className="community-loading"><span className="spinner-border" /><p>Loading reader reviews...

```

```

</p></div>
122
123      {!loading && error && (
124        <div className="empty-reader-page community-error-state">
125          <i className="bi bi-cloud-slash" />
126          <h2>The reviews are temporarily unavailable</h2>
127          <p>{error}</p>
128          <button type="button" className="btn btn-ink" onClick={loadReviews}>Try again</button>
129        </div>
130      )}
131

```

How to read this block

- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 132–145

```

132      {!loading && !error && reviews.length > 0 && visibleReviews.length > 0 && (
133        <div className="community-review-grid">
134          {visibleReviews.map((review) => (
135            <article className="community-review-card" key={review.id}>
136              <div className="community-review-book">
137                <ReviewCover review={review} />
138                <div>
139                  <p className="eyebrow">Reader review</p>
140                  <Link to={` /books/${review.bookId}`}>{review.bookTitle}</Link>
141                  <span>{reviewDate(review.createdAt)}</span>
142                </div>
143              </div>
144              <div className="community-review-author">
145                <span className="reader-avatar">{initials(review.readerName)}</span>

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 146–156

```

146          <div><strong>{review.readerName}</strong><span aria-label={` ${review.rating} out of 5 stars`}
>{'★'.repeat(review.rating)}{'★'.repeat(5 - review.rating)}</span></div>
147        </div>
148        <h2>{review.headline || 'A reader's perspective'}</h2>
149        <p>{review.content}</p>
150        <Link className="community-review-link" to={` /books/${review.bookId}#reviews`}>View book and reviews
<i className="bi bi-arrow-right" /></Link>
151        <DiscussionThread reviewId={review.id} />
152      </article>
153    )}}
154  </div>
155  )}
156

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 157–165

```
157      {!loading && !error && reviews.length > 0 && visibleReviews.length === 0 && (  
158        <div className="empty-reader-page community-no-results">  
159          <i className="bi bi-search" />  
160          <h2>No reviews match that search</h2>  
161          <p>Try a book title, reader name, or a different phrase.</p>  
162          <button type="button" className="btn btn-ink" onClick={() => setQuery('')}>Clear search</button>  
163        </div>  
164      )}  
165    )}
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 166–178

```
166      {!loading && !error && reviews.length === 0 && (  
167        <div className="empty-reader-page">  
168          <i className="bi bi-chat-quote" />  
169          <h2>Share the first reader review</h2>  
170          <p>Choose a book from the catalogue, open its details, and publish your rating and reflection.</p>  
171          <Link className="btn btn-ink" to="/#catalogue">Find a book to review</Link>  
172        </div>  
173      )}  
174    </div>  
175  </section>  
176 )  
177 }  
178
```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 179–180

```
179 export default CommunityPage  
180
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/MyBooksPage.jsx

File 92 of 99 · Added or modified · Frontend page

Purpose: Combines purchased physical books and private uploads in one authenticated shelf, including secure file open/download/delete actions.

Current file length: 231 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–12

```
1  import { useEffect, useMemo, useState } from 'react'
2  import { Link } from 'react-router-dom'
3  import { getApiErrorMessage } from '../services/api'
4  import {
5    deletePersonalBook,
6    getMyBooks,
7    getPersonalBookFile,
8    getPersonalBooks,
9    uploadPersonalBook,
10 } from '../services/readerService'
11 import { getCoverTone, getTitleMonogram } from '../utils/bookPresentation'
12
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 13–19

```
13 const FILTERS = [
14   { value: 'ALL', label: 'All books' },
15   { value: 'IN_TRANSIT', label: 'On the way' },
16   { value: 'DELIVERED', label: 'On my shelf' },
17   { value: 'UPLOADED', label: 'My uploads' },
18 ]
19
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 20–27

```
20 function statusCopy(status) {
21   if (status === 'DELIVERED') return { label: 'On my shelf', detail: 'Delivered', tone: 'owned' }
22   if (status === 'SHIPPED') return { label: 'On the way', detail: 'Shipped', tone: 'transit' }
23   if (status === 'PROCESSING') return { label: 'Preparing your book', detail: 'Processing', tone: 'transit' }
24   if (status === 'CONFIRMED') return { label: 'Order confirmed', detail: 'Confirmed', tone: 'transit' }
25   return { label: 'Order placed', detail: 'Awaiting confirmation', tone: 'transit' }
26 }
27
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 28–32

```
28 function formatFileSize(bytes) {
29   if (!Number.isFinite(Number(bytes))) return ''
30   return `${(Number(bytes) / 1024 / 1024).toFixed(1)} MB`
31 }
32
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 33–41

```
33 function MyBookCover({ item }) {
34   if (item.coverImageUrl) return <img src={item.coverImageUrl} alt={`Cover of ${item.title}`} />
35   return (
36     <div className={`my-book-fallback cover-tone-${getCoverTone({ id: item.bookId ?? item.id, title: item.title })}}`>
37       <span>✖</span><strong>{getTitleMonogram(item.title)}</strong>
38     </div>
39   )
40 }
41
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 42–52

```
42 function MyBooksPage() {
43   const [books, setBooks] = useState([])
44   const [uploads, setUploads] = useState([])
45   const [filter, setFilter] = useState('ALL')
46   const [loading, setLoading] = useState(true)
47   const [error, setError] = useState('')
48   const [showUpload, setShowUpload] = useState(false)
49   const [uploading, setUploading] = useState(false)
50   const [uploadMessage, setUploadMessage] = useState('')
51   const [busyBookId, setBusyBookId] = useState(null)
52
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 53–66

```
53  async function loadBooks() {
54    setLoading(true)
55    setError('')
56    try {
57      const [purchased, personal] = await Promise.all([getMyBooks(), getPersonalBooks()])
58      setBooks(purchased)
59      setUploads(personal)
60    } catch (requestError) {
61      setError(getApiErrorMessage(requestError, 'Your books could not be loaded.'))
62    } finally {
63      setLoading(false)
64    }
65  }
66
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 67–79

```
67  useEffect(() => {
68    let active = true
69    Promise.all([getMyBooks(), getPersonalBooks()])
70      .then(([purchased, personal]) => {
71        if (!active) return
72        setBooks(purchased)
73        setUploads(personal)
74      })
75      .catch((requestError) => { if (active) setError(getApiErrorMessage(requestError, 'Your books could not be
loaded.')) })
76      .finally(() => { if (active) setLoading(false) })
77    return () => { active = false }
78  }, [])
79
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 80–88

```
80 const visiblePurchased = useMemo(() => books.filter((item) => {
81   if (filter === 'DELIVERED') return item.orderStatus === 'DELIVERED'
82   if (filter === 'IN_TRANSIT') return item.orderStatus !== 'DELIVERED'
83   if (filter === 'UPLOADED') return false
84   return true
85 })), [books, filter])
86 const visibleUploads = filter === 'IN_TRANSIT' || filter === 'DELIVERED' ? [] : uploads
87 const visibleCount = visiblePurchased.length + visibleUploads.length
88
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 89–102

```
89 async function handleUpload(event) {
90   event.preventDefault()
91   const formElement = event.currentTarget
92   const form = new FormData(formElement)
93   const file = form.get('personalFile')
94   setUploading(true)
95   setUploadMessage('')
96   try {
97     const uploaded = await uploadPersonalBook({
98       title: form.get('title'),
99       author: form.get('author'),
100       file,
101     })
102     setUploads((items) => [uploaded, ...items])

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 103–112

```
103   setFilter('UPLOADED')
104   setUploadMessage(`${uploaded.title} was added to My Books.`)
105   formElement.reset()
106 } catch (requestError) {
107   setUploadMessage(getApiErrorMessage(requestError, 'The file could not be uploaded.'))
108 } finally {
109   setUploading(false)
110 }

```

```
111   }  
112
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 113–126

```
113  async function openOrDownload(item, download) {  
114    const previewWindow = !download ? window.open('', '_blank') : null  
115    if (previewWindow) previewWindow.opener = null  
116    setBusyBookId(item.id)  
117    setUploadMessage('')  
118    try {  
119      const blob = await getPersonalBookFile(item.id, download)  
120      const url = URL.createObjectURL(blob)  
121      if (previewWindow) {  
122        previewWindow.location = url  
123      } else {  
124        const anchor = document.createElement('a')  
125        anchor.href = url  
126        anchor.download = item.originalFilename
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 127–139

```
127      document.body.appendChild(anchor)  
128      anchor.click()  
129      anchor.remove()  
130    }  
131    window.setTimeout(() => URL.revokeObjectURL(url), 60000)  
132  } catch (requestError) {  
133    previewWindow?.close()  
134    setUploadMessage(getApiErrorMessage(requestError, 'The personal book could not be opened.'))  
135  } finally {  
136    setBusyBookId(null)  
137  }  
138 }  
139
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 140–153

```

140  async function removeUpload(item) {
141    if (!window.confirm(`Remove "${item.title}" and its file from My Books?`)) return
142    setBusyBookId(item.id)
143    setUploadMessage('')
144    try {
145      await deletePersonalBook(item.id)
146      setUploads((items) => items.filter((book) => book.id !== item.id))
147      setUploadMessage(`${item.title} was removed.`)
148    } catch (requestError) {
149      setUploadMessage(getApiErrorMessage(requestError, 'The personal book could not be removed.'))
150    } finally {
151      setBusyBookId(null)
152    }
153  }

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 154–162

```

154
155  return (
156    <section className="inner-page my-books-page">
157      <div className="container-xl">
158        <header className="my-books-hero">
159          <div><p className="eyebrow">Your personal library</p><h1>My Books</h1><p>Keep purchased books and your own
private reading files together in one thoughtful shelf.</p></div>
160          <div className="my-books-stats"><strong>{books.length + uploads.length}</strong><span>Total
titles</span><strong>{uploads.length}</strong><span>Private uploads</span></div>
161        </header>
162      </div>

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 163–176

```
163         <section className={`personal-upload-panel${showUpload ? ' is-open' : ''}`}>
164             <button className="personal-upload-toggle" type="button" onClick={() => setShowUpload((value) => !value)}
aria-expanded={showUpload}>
165                 <span><i className="bi bi-cloud-arrow-up" /><span><strong>Add your own book</strong><small>Private PDF
or EPUB · maximum 25 MB</small></span></span>
166                 <i className={`bi ${showUpload ? 'bi-chevron-up' : 'bi-chevron-down'}`} />
167             </button>
168             {showUpload && (
169                 <form className="personal-upload-form" onSubmit={handleUpload}>
170                     <label><span>Book title</span><input name="title" maxLength="255" required placeholder="The title
shown on your shelf" /></label>
171                     <label><span>Author <small>optional</small></span><input name="author" maxLength="180"
placeholder="Author or creator" /></label>
172                     <label className="personal-file-field"><span>Book file</span><input name="personalFile" type="file"
accept=".pdf,.epub,application/pdf,application/epub+zip" required /><small>Only upload files you own or are allowed to
use.</small></label>
173                     <button className="btn btn-ink" disabled={uploading} type="submit">{uploading ? <<span
className="spinner-border spinner-border-sm" /> Uploading...</> : 'Add to My Books'}</button>
174                 </form>
175             )}
176             {uploadMessage && <p className="personal-upload-message" role="status">{uploadMessage}</p>}
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 177–183

```
177         </section>
178
179         <div className="my-books-toolbar" role="group" aria-label="Filter your books">
180             {FILTERS.map((item) => <button className={filter === item.value ? 'is-active' : ''} key={item.value}
type="button" onClick={() => setFilter(item.value)}>{item.label}</button>)}
181             <span className="my-books-toolbar-links"><Link to="/my-reviews"><i className="bi bi-chat-quote" /> My
reviews</Link><Link to="/orders"><i className="bi bi-receipt" /> Order history</Link></span>
182         </div>
183
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 184–197

```
184      {loading && <div className="page-loading"><span className="spinner-border" /> Loading your books...</div>}
185      {!loading && error && <div className="catalogue-message catalogue-error"><span><i className="bi bi-
exclamation-circle" /></span><div><h2>We could not open your library</h2><p>{error}</p><button className="btn btn-ink"
type="button" onClick={loadBooks}>Try again</button></div></div>}
186      {!loading && !error && visibleCount > 0 && (
187        <div className="my-books-grid">
188          {visibleUploads.map((item) => (
189            <article className="my-book-card personal-book-card" key={`upload-${item.id}`}>
190              <div className="my-book-cover"><MyBookCover item={item} /><span className="my-book-status status-
uploaded">Private upload</span></div>
191              <div className="my-book-copy">
192                <p>{item.format} · {formatFileSize(item.fileSize)}</p>
193                <h2>{item.title}</h2>
194                <span>{item.author || item.originalFilename}</span>
195                <div className="my-book-progress"><i className="bi bi-shield-lock-fill" /><div><strong>Only
visible to you</strong><small>Stored in your Akshara library</small></div></div>
196                <div className="my-book-actions personal-book-actions">
197                  <button disabled={busyBookId === item.id} type="button" onClick={() => openOrDownload(item,
item.format !== 'PDF')}>{item.format === 'PDF' ? 'Open book' : 'Download EPUB'}</button>
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 198–211

```
198      <button disabled={busyBookId === item.id} type="button" onClick={() => openOrDownload(item,
true)}>Download</button>
199      <button className="danger" disabled={busyBookId === item.id} type="button" onClick={() =>
removeUpload(item)} aria-label={`Remove ${item.title}`}><i className="bi bi-trash" /></button>
200    </div>
201  </div>
202 </article>
203 ))}
204 {visiblePurchased.map((item) => {
205   const status = statusCopy(item.orderStatus)
206   return (
207     <article className="my-book-card" key={`order-${item.orderItemId}`}>
208       <div className="my-book-cover"><MyBookCover item={item} /><span className={`my-book-status status-
${status.tone}`}>{status.label}</span></div>
209       <div className="my-book-copy">
210         <p>{item.format}{item.quantity > 1 ? ` · ${item.quantity} copies` : ''}</p>
211         <h2>{item.title}</h2>
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props

carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 212–225

```
212         <span>{item.publisherName || item.editionName || item.isbn || 'Akshara edition'}</span>
213         <div className="my-book-progress"><i className={`bi ${item.orderStatus === 'DELIVERED' ? 'bi-
check-circle-fill' : 'bi-box-seam'}}` /></div><strong>{status.detail}</strong><small>Order
#{item.orderId}</small></div></div>
214         <div className="my-book-actions">
215             {item.bookId && <Link to={`/books/${item.bookId}`}>{item.orderStatus === 'DELIVERED' ? 'Open
book page' : 'View book'} <i className="bi bi-arrow-right" /></Link>}
216             <Link to={`/orders/${item.orderId}`}>Track order</Link>
217         </div>
218     </div>
219 </article>
220 )
221 }}
222 </div>
223 )}
224 {!loading && !error && visibleCount === 0 && <div className="empty-reader-page"><i className="bi bi-journal-
bookmark" /><h2>{books.length + uploads.length ? 'No books in this view' : 'Your library is waiting'}</h2><p>{books.length
+ uploads.length ? 'Choose another shelf to see your books.' : 'Place an order or upload a private PDF or EPUB to begin.'}
</p>{filter === 'UPLOADED' ? <button className="btn btn-ink" type="button" onClick={() => setShowUpload(true)}>Upload a
book</button> : <Link className="btn btn-ink" to="/#catalogue">Explore books</Link>}</div>}
225 </div>
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 226–229

```
226     </section>
227 )
228 }
229
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 230–231

```
230 export default MyBooksPage
231
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/MyReviewsPage.jsx

File 93 of 99 · Added or modified · Frontend page

Purpose: Lets the signed-in reader find, edit and delete their own reviews.

Current file length: 79 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–5

```
1 import { useEffect, useState } from 'react'
2 import { Link } from 'react-router-dom'
3 import { getApiErrorMessage } from '../services/api'
4 import { deleteReview, getMyReviews, updateReview } from '../services/readerService'
5
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 6–14

```
6 function MyReviewsPage() {
7   const [reviews, setReviews] = useState([])
8   const [loading, setLoading] = useState(true)
9   const [error, setError] = useState('')
10  const [notice, setNotice] = useState('')
11  const [editingId, setEditingId] = useState(null)
12  const [draft, setDraft] = useState({ rating: 5, headline: '', content: '' })
13  const [busy, setBusy] = useState(false)
14}
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 15–23

```
15  useEffect(() => {
16    let active = true
17    getMyReviews()
18      .then((data) => { if (active) setReviews(data) })
19      .catch((requestError) => { if (active) setError(getApiErrorMessage(requestError, 'Your reviews could not be
loaded.')) })
20      .finally(() => { if (active) setLoading(false) })
21    return () => { active = false }
22  }, [])
23}
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns

become DTOs or domain values for the caller.

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 24–30

```
24  function startEditing(review) {
25    setEditingId(review.id)
26    setDraft({ rating: review.rating, headline: review.headline ?? '', content: review.content })
27    setError('')
28    setNotice('')
29  }
30
```

How to read this block

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 31–44

```
31  async function saveReview(event) {
32    event.preventDefault()
33    setBusy(true)
34    setError('')
35    try {
36      const updated = await updateReview(editingId, { ...draft, rating: Number(draft.rating) })
37      setReviews((items) => items.map((review) => review.id === editingId ? updated : review))
38      setEditingId(null)
39      setNotice('Your review was updated.')
40    } catch (requestError) {
41      setError(getApiErrorMessage(requestError, 'Your review could not be updated.'))
42    } finally {
43      setBusy(false)
44    }
45  }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 45–58

```
45  }
46
47  async function removeReview(review) {
48    if (!window.confirm(`Delete your review of "${review.bookTitle}"? Its discussion replies will also be
removed.`)) return
49    setBusy(true)
50    setError('')
```

```

51     try {
52       await deleteReview(review.id)
53       setReviews((items) => items.filter((item) => item.id !== review.id))
54       setNotice('Your review was deleted.')
55     } catch (requestError) {
56       setError(getApiErrorMessage(requestError, 'Your review could not be deleted.'))
57     } finally {
58       setBusy(false)

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 59–61

```

59   }
60   }
61

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 62–75

```

62   return (
63     <section className="inner-page my-reviews-page"><div className="container-xl">
64       <header className="my-reviews-heading"><div><p className="eyebrow">Your reader voice</p><h1>My
Reviews</h1><p>Edit or delete every review you have published from one place.</p></div><Link className="btn btn-ink"
to="/community">Community discussions</Link></header>
65       {error && <p className="form-alert" role="alert">{error}</p>}
66       {notice && <div className="admin-feedback admin-feedback-success" role="status">{notice}</div>}
67       {loading && <div className="page-loading"><span className="spinner-border" /> Loading your reviews...</div>}
68       {!loading && reviews.length > 0 && <div className="my-review-list">{reviews.map((review) => <article
className="my-review-item" key={review.id}>
69         <div className="my-review-book">{review.coverImageUrl ? <img src={review.coverImageUrl} alt={`Cover of $
{review.bookTitle}`} /> : <span><i className="bi bi-book" /></span></div><Link to={` /books/${review.bookId}`}
>{review.bookTitle}</Link><small>Last updated {new Date(review.updatedAt).toLocaleDateString('en-IN', { dateStyle:
'medium' })}</small></div></div>
70         {editingId === review.id ? <form className="my-review-edit-form" onSubmit={saveReview}><div
className="rating-picker">[1, 2, 3, 4, 5].map((rating) => <button className={rating <= draft.rating ? 'is-active' : ''}
type="button" key={rating} onClick={() => setDraft((value) => ({ ...value, rating }))}>★</button>)}</div><input
maxLength="120" placeholder="Headline (optional)" value={draft.headline} onChange={(event) => setDraft((value) =>
({ ...value, headline: event.target.value })}> /><textarea required minLength="10" maxLength="2000" value={draft.content}
onChange={(event) => setDraft((value) => ({ ...value, content: event.target.value })}> /><div><button className="btn btn-ink" disabled={busy} type="submit">Save changes</button><button className="text-button" type="button" onClick={() =>
setEditingId(null)}>Cancel</button></div></form> : <div className="my-review-content"><span>{'★'.repeat(review.rating)}
{'★'.repeat(5 - review.rating)}</span>{review.headline && <h2>{review.headline}</h2><p>{review.content}</p></div>}
71         {editingId !== review.id && <div className="my-review-actions"><button type="button" onClick={() =>
startEditing(review)}><i className="bi bi-pencil" /> Edit</button><button className="danger" disabled={busy} type="button"

```

```

onClick={() => removeReview(review)}><i className="bi bi-trash" /> Delete</button><Link to={`\books/${review.bookId}
#reviews`}>View on book page</Link></div>
72     </article>}}</div>
73     {!loading && !reviews.length && <div className="empty-reader-page"><i className="bi bi-chat-quote" /><h2>You
have not reviewed a book yet</h2><p>Open a book from your shelf or the catalogue to publish your first
reflection.</p><Link className="btn btn-ink" to="/my-books">Go to My Books</Link></div>
74     </div></section>
75     )

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- These JSX event props connect user actions to component functions. Forms prevent default navigation before invoking API logic.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 76–79

```

76 }
77
78 export default MyReviewsPage
79

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/pages/OrdersPage.jsx

File 94 of 99 · Added or modified · Frontend page

Purpose: Renders order history, order details and the physical-delivery journey without implying digital entitlement.

Current file length: 142 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 import { useEffect, useMemo, useState } from 'react'
2 import { Link, useLocation, useParams } from 'react-router-dom'
3 import { getApiErrorMessage } from '../services/api'
4 import { getOrder, getOrders } from '../services/readerService'
5 import { formatPrice } from '../utils/bookPresentation'
6
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 7–12

```
7 const DELIVERY_STEPS = ['PLACED', 'CONFIRMED', 'PROCESSING', 'SHIPPED', 'DELIVERED']
8
9 function readableStatus(value = 'PLACED') {
10   return String(value).replaceAll('_', ' ').toLowerCase().replace(/^../, (letter) => letter.toUpperCase())
11 }
12
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 13–21

```
13 function safeDate(value, includeTime = false) {
14   if (!value) return 'Date unavailable'
15   const date = new Date(value)
16   if (Number.isNaN(date.getTime())) return 'Date unavailable'
17   return date.toLocaleString('en-IN', includeTime
18     ? { dateStyle: 'long', timeStyle: 'short' }
19     : { dateStyle: 'long' })
20 }
21
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 22–25

```
22 function DeliveryTimeline({ status }) {
23   const normalizedStatus = DELIVERY_STEPS.includes(status) ? status : 'PLACED'
24   const currentIndex = DELIVERY_STEPS.indexOf(normalizedStatus)
25 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 26–34

```
26   if (status === 'CANCELLED') {
27     return (
28       <div className="order-cancelled-message">
29         <i className="bi bi-x-circle" />
30         <div><strong>This order was cancelled</strong><span>No further delivery steps will take place.</span></div>
31       </div>
32     )
33   }
34 }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 35–46

```
35   return (
36     <ol className="delivery-timeline" aria-label="Delivery progress">
37       {DELIVERY_STEPS.map((step, index) => (
38         <li className={index < currentIndex ? 'is-complete' : index === currentIndex ? 'is-current' : ''} key={step}
39           <span>{index < currentIndex ? <i className="bi bi-check" /> : index + 1}</span>
40           <div><strong>{readableStatus(step)}</strong><small>{step === 'PLACED' ? 'Order received' : step ===
'DELIVERED' ? 'Books delivered' : 'Pending update'}</small></div>
41         </li>
42       ))}
43     </ol>
44   )
45 }
46 }
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 47–50

```
47 function OrderBook({ item, delivered }) {
48   const isDigital = item.bookFormat === 'PDF' || item.bookFormat === 'EPUB'
49   const bookDestination = item.bookId ? `/books/${item.bookId}` : null
50 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 51–64

```
51   return (
52     <div className="order-book">
53       <span className="order-book-cover">{item.coverImageUrl} <img src={item.coverImageUrl} alt={`Cover of $
{item.bookTitle ?? 'ordered book'}}` /> : '₹'</span>
54       <div>
55         {bookDestination ? <Link className="order-book-title" to={bookDestination}>{item.bookTitle ?? 'Ordered
book'}</Link> : <strong>{item.bookTitle ?? 'Ordered book'}</strong>}
56         <span>{item.editionName || readableStatus(item.bookFormat || 'Edition')}. Qty {item.quantity ?? 1}</span>
57         {isDigital && <small className="order-access-warning"><i className="bi bi-info-circle" /> Digital access is
unavailable for this historical item.</small>}
58         {!isDigital && delivered && bookDestination && <Link className="order-reflection-link" to={`$
{bookDestination}#reviews`}>Review and reflect on this book <i className="bi bi-arrow-right" /></Link>}
59         {!isDigital && !delivered && bookDestination && <Link className="order-reflection-link" to={bookDestination}
>View book details <i className="bi bi-arrow-right" /></Link>}
60       </div>
61       <strong>{formatPrice(item.subtotal)}</strong>
62     </div>
63   )
64 }
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 65–74

```
65
66 function OrdersPage() {
67   const { orderId } = useParams()
68   const location = useLocation()
69   const [data, setData] = useState(null)
70   const [error, setError] = useState('')
71   const [loadedKey, setLoadedKey] = useState('')
72   const [errorKey, setErrorKey] = useState('')
73   const requestKey = orderId || 'order-list'
74 }
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 75–88

```

75  useEffect(() => {
76    let active = true
77    const request = orderId ? getOrder(orderId) : getOrders()
78    request
79      .then((response) => {
80        if (active) {
81          setData(response)
82          setError('')
83          setLoadedKey(orderId || 'order-list')
84        }
85      })
86    .catch((requestError) => {
87      if (active) {
88        setError(getApiErrorMessage(requestError, 'Orders could not be loaded.'))

```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 89–96

```

89    setErrorKey(orderId || 'order-list')
90  }
91  })
92  return () => {
93    active = false
94  }
95  }, [orderId])
96

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 97–103

```

97  const orderItems = useMemo(() => Array.isArray(data?.items) ? data.items : [], [data])
98
99  if (error && errorKey === requestKey) {
100    return <section className="inner-page"><div className="container-xl"><div className="catalogue-message
catalogue-error"><span><i className="bi bi-receipt-cutoff" /></span><div><h1>We could not open this order</h1><p>{error}</p><Link to="/orders">Return to order history</Link></div></div></div></section>
101  }
102  if (!data || loadedKey !== requestKey) return <div className="page-loading"><span className="spinner-border" />

```

```
Loading your orders...</div>
103
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 104–113

```
104   if (!orderId) {
105     const orders = Array.isArray(data) ? data : []
106     return (
107       <section className="inner-page"><div className="container-xl">
108         <div className="inner-page-heading"><p className="eyebrow">Your reading purchases</p><h1>Order
109         history</h1><p>Track physical deliveries and return to books you have purchased.</p></div>
110         {orders.length ? <div className="orders-list">{orders.map((order) => <Link className="order-row"
111         to={` /orders/${order.orderId}`} key={order.orderId}><div><strong>Order
112         #{order.orderId}</strong><span>{safeDate(order.placedAt)}</span></div><span className="status-
113         pill">{readableStatus(order.orderStatus)}</span><strong>{formatPrice(order.totalAmount)}</strong><i className="bi bi-
114         chevron-right" /></Link>}</div> : <div className="empty-reader-page"><i className="bi bi-box-seam" /><h2>No orders
115         yet</h2><p>Your completed purchases will appear here.</p><Link className="btn btn-ink" to="/#catalogue">Explore
116         books</Link></div>}
117       </div></section>
118     )
119   }
120 }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 114–118

```
114   const status = data.orderStatus || 'PLACED'
115   const delivered = status === 'DELIVERED'
116   const address = data.shippingAddress
117   const hasOnlyPhysicalBooks = orderItems.every((item) => item.bookFormat !== 'PDF' && item.bookFormat !== 'EPUB')
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 119–124

```

119   return (
120     <section className="inner-page"><div className="container-xl narrow-page">
121       {location.state?.justPlaced && <div className="success-banner"><i className="bi bi-check-circle-fill"
/><div><strong>Your order is confirmed.</strong><span>Akshara has reserved the physical books in your
cart.</span></div></div>}
122       <div className="order-detail-navigation"><Link className="back-link" to="/orders"><i className="bi bi-arrow-
left" /> All orders</Link><Link className="back-link" to="/my-books"><i className="bi bi-journal-bookmark" /> Go to My
Books</Link></div>
123       <div className="order-detail-heading"><div><p className="eyebrow">Order #{data.orderId ??
orderId}</p><h1>{readableStatus(status)}</h1><p>Placed {safeDate(data.placedAt,
true)}</p></div><strong>{formatPrice(data.totalAmount)}</strong></div>
124

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 125–130

```

125     <section className="order-journey-card">
126       <div className="order-journey-heading"><div><p className="eyebrow">Delivery journey</p><h2>{delivered ?
'Your books have arrived.' : 'Your physical books are on their way.'}</h2></div><span className="status-
pill">{readableStatus(status)}</span></div>
127       <DeliveryTimeline status={status} />
128       {hasOnlyPhysicalBooks && <p className="order-access-note"><i className="bi bi-box-seam" /> This purchase is
for physical editions. It is now saved in <Link to="/my-books">My Books</Link>, where you can follow it from delivery to
your reading shelf.</p>}
129     </section>
130

```

How to read this block

- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 131–140

```

131     <div className="order-detail-card"><h2>Books in this order</h2>{orderItems.length ? orderItems.map((item,
index) => <OrderBook item={item} delivered={delivered} key={item.orderItemId ?? `${item.bookTitle}-${index}`} />) : <p
className="muted-message">No item details are available for this order.</p>}</div>
132
133     <div className="order-detail-grid">

```

```

134         <div><h2>Deliver to</h2>{address ? <p>{address.recipientName}<br />{address.addressLine1}<br
/>{address.addressLine2} && <{address.addressLine2}<br /></>}{address.city}, {address.state} {address.postalCode}<br
/>{address.country}<br />{address.phone}</p> : <p>Shipping details are unavailable.</p></div>
135         <div><h2>Payment</h2><p>{readableStatus(data.paymentMethod || 'Payment method unavailable')}<br /><span
className="status-pill">{readableStatus(data.paymentStatus || 'Pending')}</span></p></div>
136     </div>
137 </div></section>
138 )
139 }
140

```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- CSS classes provide visual styling; ARIA labels/roles provide meaning to assistive technology where native semantics need reinforcement.

Lines 141–142

```

141 export default OrdersPage
142

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/providers/ReaderDataProvider.jsx

File 95 of 99 · Added or modified · Frontend shared state

Purpose: Renders and coordinates the ReaderDataProvider React interface.

Current file length: 106 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–13

```
1 import { useCallback, useEffect, useMemo, useState } from 'react'
2 import ReaderDataContext from '../contexts/readerDataContext'
3 import useAuth from '../hooks/useAuth'
4 import {
5   addCartItem as addCartItemRequest,
6   addToWishlist,
7   getCart,
8   getWishlist,
9   removeCartItem as removeCartItemRequest,
10  removeFromWishlist,
11  updateCartItem as updateCartItemRequest,
12 } from '../services/readerService'
13
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 14–20

```
14 function ReaderDataProvider({ children }) {
15   const { user } = useAuth()
16   const [wishlist, setWishlist] = useState([])
17   const [cart, setCart] = useState(null)
18   const isReader = user?.role === 'READER'
19   const [loading, setLoading] = useState(Boolean(isReader))
20
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 21–31

```
21   const refresh = useCallback(async () => {
22     if (!isReader) return
23     try {
24       const [nextWishlist, nextCart] = await Promise.all([getWishlist(), getCart()])
25       setWishlist(nextWishlist)
26       setCart(nextCart)
27     } finally {
28       setLoading(false)
29     }
30   }, [isReader])
31
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.
- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 32–34

```
32    useEffect(() => {
33        let active = true
34    }
```

How to read this block

- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.

Lines 35–48

```
35    if (!isReader) {
36        Promise.resolve().then(() => {
37            if (!active) return
38            setWishlist([])
39            setCart(null)
40            setLoading(false)
41        })
42    } else {
43        Promise.all([getWishlist(), getCart()])
44            .then([nextWishlist, nextCart]) => {
45            if (!active) return
46            setWishlist(nextWishlist)
47            setCart(nextCart)
48        })
49    }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 49–54

```
49        .catch((error) => console.error('Unable to load reader data', error))
50        .finally(() => {
51            if (active) setLoading(false)
52        })
53    }
54 }
```

How to read this block

- The try/catch boundary converts low-level failures into an Akshara-specific outcome and protects cleanup or user-facing error behavior.

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 55–57

```
55     return () => { active = false }
56   }, [isReader])
57
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 58–65

```
58   const toggleWishlist = useCallback(async (book) => {
59     const existing = wishlist.find((item) => item.book.id === book.id)
60     if (existing) {
61       await removeFromWishlist(book.id)
62       setWishlist((items) => items.filter((item) => item.book.id !== book.id))
63       return false
64     }
65
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 66–70

```
66     const item = await addToWishlist(book.id)
67     setWishlist((items) => [item, ...items])
68     return true
69   }, [wishlist])
70
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.

- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 71–76

```
71  const addCartItem = useCallback(async (bookEditionId, quantity) => {
72    const nextCart = await addCartItemRequest(bookEditionId, quantity)
73    setCart(nextCart)
74    return nextCart
75  }, [])
76
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 77–82

```
77  const updateCartItem = useCallback(async (itemId, quantity) => {
78    const nextCart = await updateCartItemRequest(itemId, quantity)
79    setCart(nextCart)
80    return nextCart
81  }, [])
82
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 83–88

```
83  const removeCartItem = useCallback(async (itemId) => {
84    const nextCart = await removeCartItemRequest(itemId)
85    setCart(nextCart)
86    return nextCart
87  }, [])
88
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 89–101

```
89   const value = useMemo(() => ({
90     wishlist,
91     wishlistBookIds: new Set(wishlist.map((item) => item.book.id)),
92     cart,
93     loading,
94     refresh,
95     toggleWishlist,
96     addCartItem,
97     updateCartItem,
98     removeCartItem,
99     clearCartState: () => setCart(null),
100   })), [wishlist, cart, loading, refresh, toggleWishlist, addCartItem, updateCartItem, removeCartItem])
101
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- React hooks hold component state, synchronize with external effects, or memoize derived values. Hook order must stay stable across renders.
- A React state setter schedules a re-render; updater callbacks are used when the next value depends on the previous value.

Lines 102–104

```
102   return <ReaderDataContext.Provider value={value}>{children}</ReaderDataContext.Provider>
103 }
104
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.

Lines 105–106

```
105 export default ReaderDataProvider
106
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

frontend/src/services/adminBookService.js

File 96 of 99 · Added or modified · Frontend API client

Purpose: Normalizes backend/external edition shapes and contains all frontend calls for import and catalogue administration.

Current file length: 107 lines. The listing below is the code at commit `de9d4fc`; line numbers are learning aids and may move after future edits.

Lines 1–6

```
1 import api from './api'
2
3 export const BOOK_SOURCES = [
4   { value: 'OPEN_LIBRARY', label: 'Open Library' },
5 ]
6
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.

Lines 7–11

```
7 export const BOOK_FORMATS = [
8   { value: 'PAPERBACK', label: 'Paperback' },
9   { value: 'HARDCOVER', label: 'Hardcover' },
10 ]
11
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 12–17

```
12 export const AVAILABILITY_OPTIONS = [
13   { value: 'IN_STOCK', label: 'Available' },
14   { value: 'OUT_OF_STOCK', label: 'Out of stock' },
15   { value: 'UNAVAILABLE', label: 'Unavailable' },
16 ]
17
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 18–22

```
18 function unwrapResults(data) {
19   if (Array.isArray(data)) return data
20   return data?.results ?? data?.items ?? data?.content ?? []
21 }
22
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 23–30

```

23 function toList(value) {
24   if (Array.isArray(value)) {
25     return value.map((item) => typeof item === 'string' ? item : item?.name).filter(Boolean)
26   }
27   if (typeof value === 'string') return value.split(',').map((item) => item.trim()).filter(Boolean)
28   return []
29 }
30

```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 31–44

```

31 function normalizeEdition(item, parent = {}) {
32   const isbn = item.isbn ?? item.isbn13 ?? item.isbn10 ?? ''
33   return {
34     sourceId: item.sourceId ?? item.externalId ?? item.key ?? parent.sourceId ?? parent.externalId ?? parent.key ??
35     editionId: item.editionId ?? item.externalEditionId ?? item.key ?? '',
36     title: item.title ?? parent.title ?? '',
37     subtitle: item.subtitle ?? parent.subtitle ?? '',
38     description: item.description ?? parent.description ?? '',
39     authors: toList(item.authors ?? item.authorNames ?? parent.authors ?? parent.authorNames),
40     publisher: item.publisher ?? item.publisherName ?? parent.publisher ?? parent.publisherName ?? '',
41     languageCode: item.languageCode ?? item.language ?? parent.languageCode ?? parent.language ?? '',
42     pageCount: item.pageCount ?? item.numberOfPages ?? parent.pageCount ?? parent.numberOfPages ?? '',
43     categories: toList(item.categories ?? item.subjects ?? parent.categories ?? parent.subjects),
44     coverImageUrl: item.coverImageUrl ?? item.coverUrl ?? item.thumbnailUrl ?? parent.coverImageUrl ??
parent.coverUrl ?? parent.thumbnailUrl ?? '',

```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 45–52

```

45   publicationDate: item.publicationDate ?? item.publishedDate ?? item.publishDate ?? parent.publicationDate ??
parent.publishedDate ?? parent.publishDate ?? '',
46   isbn10: item.isbn10 ?? (String(isbn).replace(/[-\s]/g, '').length === 10 ? isbn : ''),
47   isbn13: item.isbn13 ?? (String(isbn).replace(/[-\s]/g, '').length === 13 ? isbn : ''),
48   editionName: item.editionName ?? item.name ?? '',
49   alreadyImported: Boolean(item.alreadyImported ?? item.imported ?? item.existsInCatalogue ??
parent.alreadyImported ?? parent.imported),
50 }
51 }
52

```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 53–65

```
53 export async function searchExternalBooks(query, source = 'OPEN_LIBRARY') {
54   const response = await api.get('/admin/book-import/search', {
55     params: { query: query.trim(), source },
56     timeout: 30000,
57   })
58   return unwrapResults(response.data).flatMap((item) => {
59     const editions = item.editions ?? item.variants
60     return Array.isArray(editions) && editions.length
61       ? editions.map((edition) => normalizeEdition(edition, item))
62       : [normalizeEdition(item)]
63   })
64 }
65
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 66–70

```
66 export async function importBook(payload) {
67   const response = await api.post('/admin/book-import', payload)
68   return response.data
69 }
70
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 71–75

```
71 export async function createManualBook(payload) {
72   const response = await api.post('/admin/books', payload)
73   return response.data
74 }
75
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 76–80

```
76 export async function getAdminBooks() {
77   const response = await api.get('/admin/books')
78   return Array.isArray(response.data) ? response.data : []
79 }
80
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 81–85

```
81 export async function updateEditionInventory(editionId, payload) {
82   const response = await api.put(`/admin/editions/${editionId}/inventory`, payload)
83   return response.data
84 }
85
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 86–90

```
86 export async function restockEdition(editionId, quantity) {
87   const response = await api.post(`/admin/editions/${editionId}/inventory/restock`, { quantity })
88   return response.data
89 }
90
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic

metadata.

Lines 91–95

```
91 export async function updateBookFeatured(bookId, featured) {
92   const response = await api.patch(`/admin/books/${bookId}/featured`, { featured })
93   return response.data
94 }
95
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 96–99

```
96 export async function deleteCatalogueBook(bookId) {
97   await api.delete(`/admin/books/${bookId}`)
98 }
99
```

How to read this block

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 100–107

```
100 export async function seedCatalogue(count = 50) {
101   const response = await api.post('/admin/book-import/seed', null, {
102     params: { count },
103     timeout: 180000,
104   })
105   return response.data
106 }
107
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

frontend/src/services/adminOrderService.js

File 97 of 99 · Added or modified · Frontend API client

Purpose: Contains the frontend HTTP operations for listing, opening and advancing admin orders.

Current file length: 17 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–7

```
1 import api from './api'
2
3 export async function getAdminOrders() {
4   const response = await api.get('/admin/orders')
5   return Array.isArray(response.data) ? response.data : []
6 }
7
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 8–12

```
8 export async function getAdminOrder(orderId) {
9   const response = await api.get(`/admin/orders/${orderId}`)
10  return response.data
11 }
12
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 13–17

```
13 export async function updateAdminOrderStatus(orderId, status) {
14   const response = await api.patch(`/admin/orders/${orderId}/status`, { status })
15   return response.data
16 }
17
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns

become DTOs or domain values for the caller.

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

frontend/src/services/readerService.js

File 98 of 99 · Added or modified · Frontend API client

Purpose: Contains reader-facing review, discussion, library and personal-upload API operations.

Current file length: 131 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–7

```
1 import api from './api'
2
3 export async function getWishlist() {
4   const response = await api.get('/wishlist')
5   return response.data
6 }
7
```

How to read this block

- These imports reveal the dependencies used in this section. Java imports add type names only; they do not create objects or execute code.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 8–12

```
8 export async function addToWishlist(bookId) {
9   const response = await api.post(`/wishlist/${bookId}`)
10  return response.data
11 }
12
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 13–16

```
13 export async function removeFromWishlist(bookId) {
14   await api.delete(`/wishlist/${bookId}`)
15 }
16
```

How to read this block

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state

should be updated around the awaited operation.

- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 17–21

```
17 export async function getCart() {  
18   const response = await api.get('/cart')  
19   return response.data  
20 }  
21
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 22–26

```
22 export async function addCartItem(bookEditionId, quantity = 1) {  
23   const response = await api.post('/cart/items', { bookEditionId, quantity })  
24   return response.data  
25 }  
26
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 27–31

```
27 export async function updateCartItem(itemId, quantity) {  
28   const response = await api.patch(`/cart/items/${itemId}`, { quantity })  
29   return response.data  
30 }  
31
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 32–36

```
32 export async function removeCartItem(itemId) {
33   const response = await api.delete(`/cart/items/${itemId}`)
34   return response.data
35 }
36
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 37–41

```
37 export async function submitCheckout(shippingAddress) {
38   const response = await api.post('/checkout', { shippingAddress })
39   return response.data
40 }
41
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 42–46

```
42 export async function getOrders() {
43   const response = await api.get('/orders')
44   return response.data
45 }
46
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 47–51

```
47 export async function getOrder(orderId) {
48   const response = await api.get(`/orders/${orderId}`)
49   return response.data
50 }
51
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 52–56

```
52 export async function getMyBooks() {  
53   const response = await api.get('/library')  
54   return response.data  
55 }  
56
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 57–61

```
57 export async function getPersonalBooks() {  
58   const response = await api.get('/library/uploads')  
59   return response.data  
60 }  
61
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 62–70

```
62 export async function uploadPersonalBook({ title, author, file }) {  
63   const form = new FormData()  
64   form.append('title', title)  
65   if (author) form.append('author', author)  
66   form.append('file', file)  
67   const response = await api.post('/library/uploads', form, { timeout: 60000 })  
68   return response.data  
69 }  
70
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as

the gate that must be true to enter its block.

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.
- This browser API handles confirmation, secure blob viewing/downloading or multipart upload data; temporary object URLs are revoked after use.

Lines 71–79

```
71 export async function getPersonalBookFile(bookId, download = false) {
72   const response = await api.get(`/library/uploads/${bookId}/file`, {
73     params: { disposition: download ? 'attachment' : 'inline' },
74     responseType: 'blob',
75     timeout: 60000,
76   })
77   return response.data
78 }
79
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 80–83

```
80 export async function deletePersonalBook(bookId) {
81   await api.delete(`/library/uploads/${bookId}`)
82 }
83
```

How to read this block

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 84–88

```
84 export async function getBookReviews(bookId) {
85   const response = await api.get(`/public/books/${bookId}/reviews`)
86   return response.data
87 }
88
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 89–93

```
89 export async function getRecentReviews(limit = 6) {
90   const response = await api.get('/public/reviews/recent', { params: { limit } })
91   return response.data
92 }
93
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 94–98

```
94 export async function createReview(review) {
95   const response = await api.post('/reviews', review)
96   return response.data
97 }
98
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 99–103

```
99 export async function updateReview(reviewId, review) {
100   const response = await api.put(`/reviews/${reviewId}`, review)
101   return response.data
102 }
103
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 104–107

```
104 export async function deleteReview(reviewId) {
105   await api.delete(`/reviews/${reviewId}`)
106 }
107
```

How to read this block

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 108–112

```
108 export async function getMyReviews() {
109   const response = await api.get('/reviews/me')
110   return response.data
111 }
112
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 113–117

```
113 export async function getReviewReplies(reviewId) {
114   const response = await api.get(`/public/reviews/${reviewId}/replies`)
115   return response.data
116 }
117
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 118–122

```
118 export async function createReviewReply(reviewId, content) {
119   const response = await api.post(`/reviews/${reviewId}/replies`, { content })
120   return response.data
121 }
122
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 123–127

```
123 export async function updateReviewReply(replyId, content) {  
124   const response = await api.put(`/reviews/replies/${replyId}`, { content })  
125   return response.data  
126 }  
127
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

Lines 128–131

```
128 export async function deleteReviewReply(replyId) {  
129   await api.delete(`/reviews/replies/${replyId}`)  
130 }  
131
```

How to read this block

- The asynchronous flow waits for HTTP/file work without blocking the browser. Loading, success and error state should be updated around the awaited operation.
- The frontend calls Akshara's `/api` boundary through the shared Axios client, which adds JWT authorization and standard timeout/error behavior.

frontend/src/utils/bookPresentation.js

File 99 of 99 · Added or modified · Project support

Purpose: Provides JavaScript support logic for project support.

Current file length: 63 lines. The listing below is the code at commit [de9d4fc](#); line numbers are learning aids and may move after future edits.

Lines 1–5

```
1  const coverTones = ['navy', 'forest', 'saffron', 'clay', 'plum', 'teal']
2
3  export function getPrimaryEdition(book) {
4    if (!book?.editions?.length) return null
5  }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 6–9

```
6  const physicalEditions = book.editions.filter(
7    (edition) => edition.format === 'PAPERBACK' || edition.format === 'HARDCOVER',
8  )
9
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.

Lines 10–19

```
10  return (
11    physicalEditions.find(
12      (edition) => edition.inventory?.active && edition.inventory?.stockQuantity > 0,
13    ) ??
14    physicalEditions.find((edition) => edition.inventory?.active) ??
15    physicalEditions[0] ??
16    null
17  )
18 }
19
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- This JSX describes the rendered interface. Conditions choose loading/error/empty/content states, while props carry data and event handlers into elements or child components.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 20–23

```
20 export function getBookPrice(book) {  
21   return getPrimaryEdition(book)?.inventory?.price ?? null  
22 }  
23
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 24–26

```
24 export function formatPrice(value) {  
25   if (value == null) return 'Price unavailable'  
26 }
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 27–33

```
27   return new Intl.NumberFormat('en-IN', {  
28     style: 'currency',  
29     currency: 'INR',  
30     maximumFractionDigits: 0,  
31   }).format(Number(value))  
32 }  
33
```

How to read this block

- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 34–37

```
34 export function getAuthors(book) {  
35   return book?.authors?.map((author) => author.name).filter(Boolean).join(', ') || 'Author unavailable'  
36 }  
37
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method’s result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 38–40

```
38 export function getAvailability(book) {  
39   const inventory = getPrimaryEdition(book)?.inventory  
40 }
```

How to read this block

- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 41–45

```
41 if (!inventory?.active) return { label: 'Coming soon', tone: 'muted' }  
42 if (inventory.stockQuantity > 0) return { label: 'In stock', tone: 'available' }  
43 return { label: 'Out of stock', tone: 'unavailable' }  
44 }  
45
```

How to read this block

- Conditional branches enforce invariants or select the correct behavior for the current state; read each condition as the gate that must be true to enter its block.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.
- These values are Akshara-owned commerce data. They are deliberately kept separate from external bibliographic metadata.

Lines 46–50

```
46 export function getCoverTone(book) {  
47   const seed = String(book?.id ?? book?.title ?? 'akshara')  
48   .split('')  
49   .reduce((total, character) => total + character.charCodeAt(0), 0)  
50 }
```

How to read this block

- This block completes the surrounding declaration or expression. Read it with the preceding block because formatting has split one logical construct across line ranges.

Lines 51–53

```
51 return coverTones[seed % coverTones.length]  
52 }  
53
```

How to read this block

- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.

Lines 54–63

```
54 export function getTitleMonogram(title = 'Akshara') {  
55   return title  
56   .split(/\s+/)  
57   .filter(Boolean)  
58   .slice(0, 3)  
59   .map((word) => word[0])  
60 }
```

```
60     .join('')
61     .toUpperCase()
62 }
63
```

How to read this block

- The loop/stream transforms a collection step by step. Filtering removes unsuitable values, mapping changes shape, and collection creates the final immutable/list result.
- The return statement produces the method's result. Controller returns become HTTP responses; service returns become DTOs or domain values for the caller.